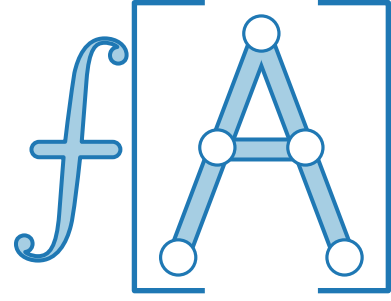
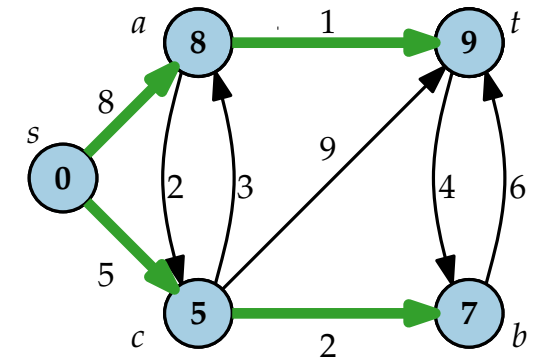
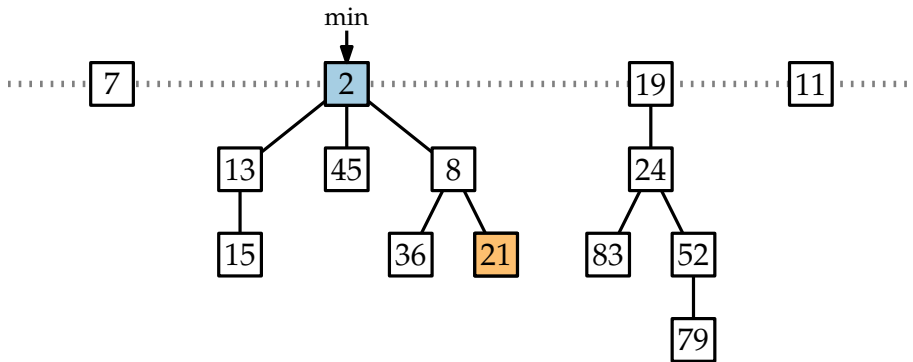




Fortgeschrittene Algorithmen

18. Vorlesung Datenstrukturen IV: Fibonacci-Heaps

Teil I: OPERATIONEN



Prioritätsschlange



Abstrakter Datentyp. PRIORITÄTSSCHLANGE

verwaltet Elemente einer Menge Q ,
wobei jedes Element $x \in Q$ eine Priorität $x.\text{key}$ hat.

(Wir benutzen Q , um sowohl
die Menge, als auch die
Prioritätsschlange selber zu
bezeichnen)

Operation	Funktionalität
$Q.\text{INSERT}(x)$	$Q = Q \cup \{x\}$
$Q.\text{MIN}()$	liefere $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{EXTRACTMIN}()$	liefere und lösche $x \in Q$ mit $x.\text{key} = \min\{y.\text{key} \mid y \in Q\}$
$Q.\text{DECREASEKEY}(x, p)$	Setze $x.\text{key} = p$, wobei $p < x.\text{key}$
$Q.\text{UNION}(Q')$	$Q = Q \cup Q'$
$Q.\text{DELETE}(x)$	$Q.\text{DECREASEKEY}(x, -\infty); Q.\text{EXTRACTMIN}()$

Überblick



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

* amortisiert

Operation	ARRAY	LISTE	HEAP	FIBONACCIHEAP
$Q.\text{INSERT}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)^*$
$Q.\text{MIN}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{EXTRACTMIN}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)^*$
$Q.\text{DECREASEKEY}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)^*$
$Q.\text{UNION}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$	$\Theta(1)^*$
DIJKSTRA	$\Theta(n^2 + m)$	$\Theta(n^2 + m)$	$\Theta((n + m) \log n)$	$\Theta(n \log n + m)$

Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

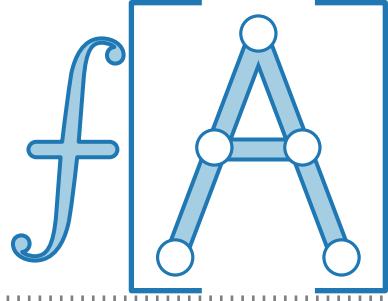
Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten

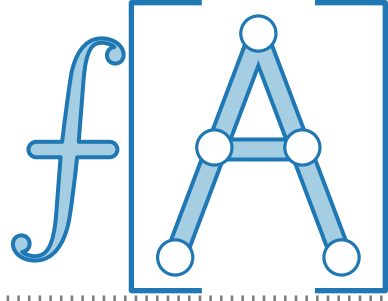
Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten

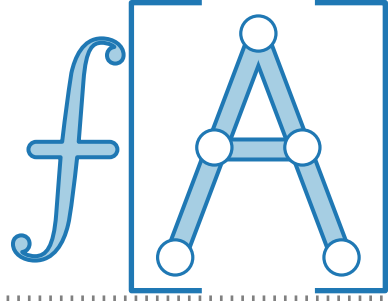
Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten

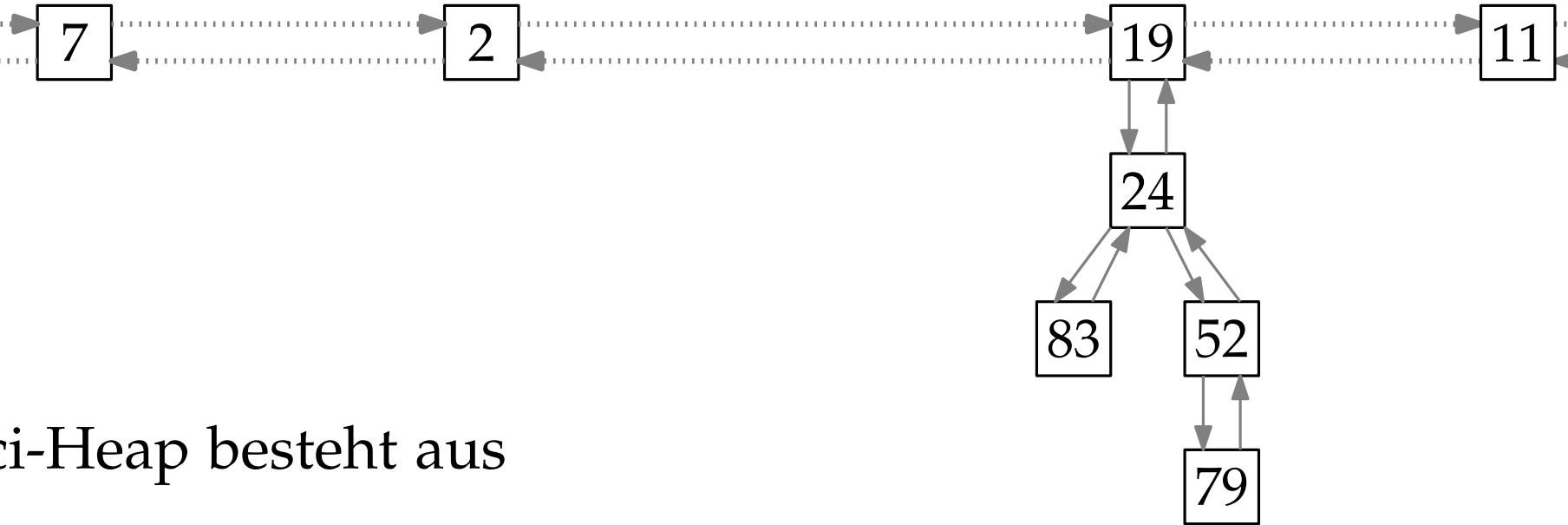
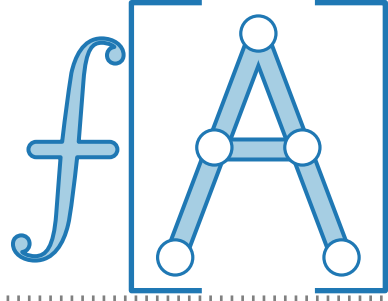
Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)

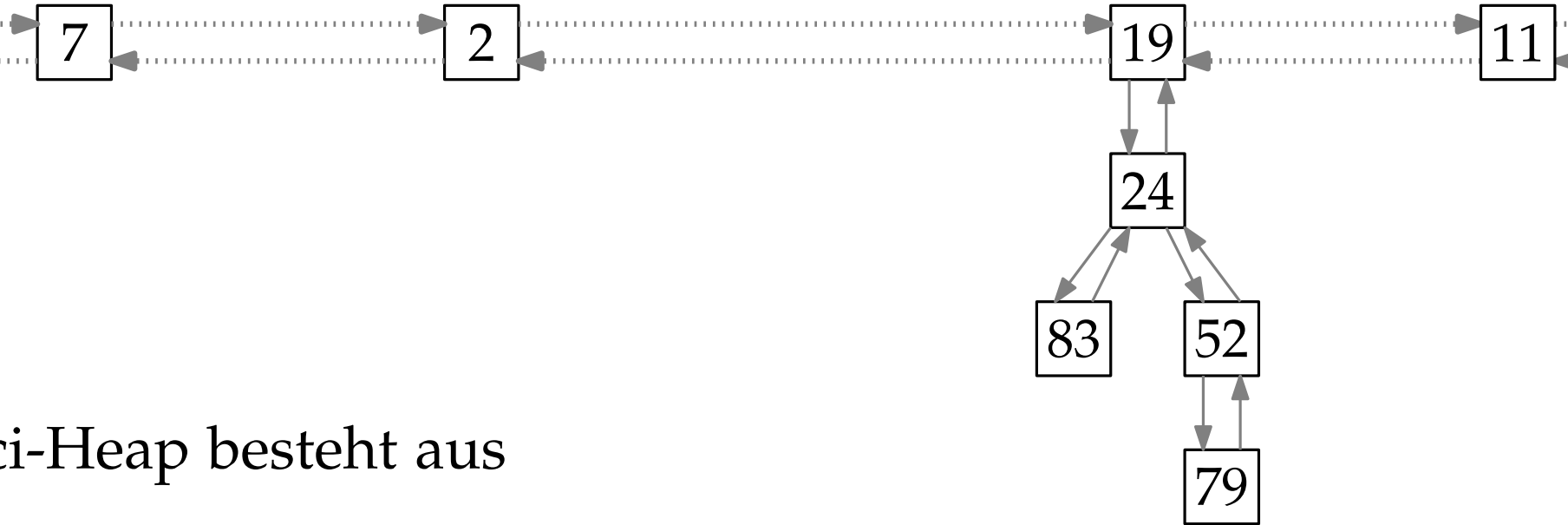
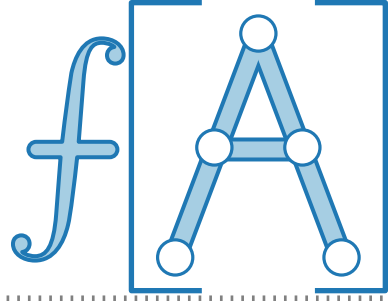
Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)

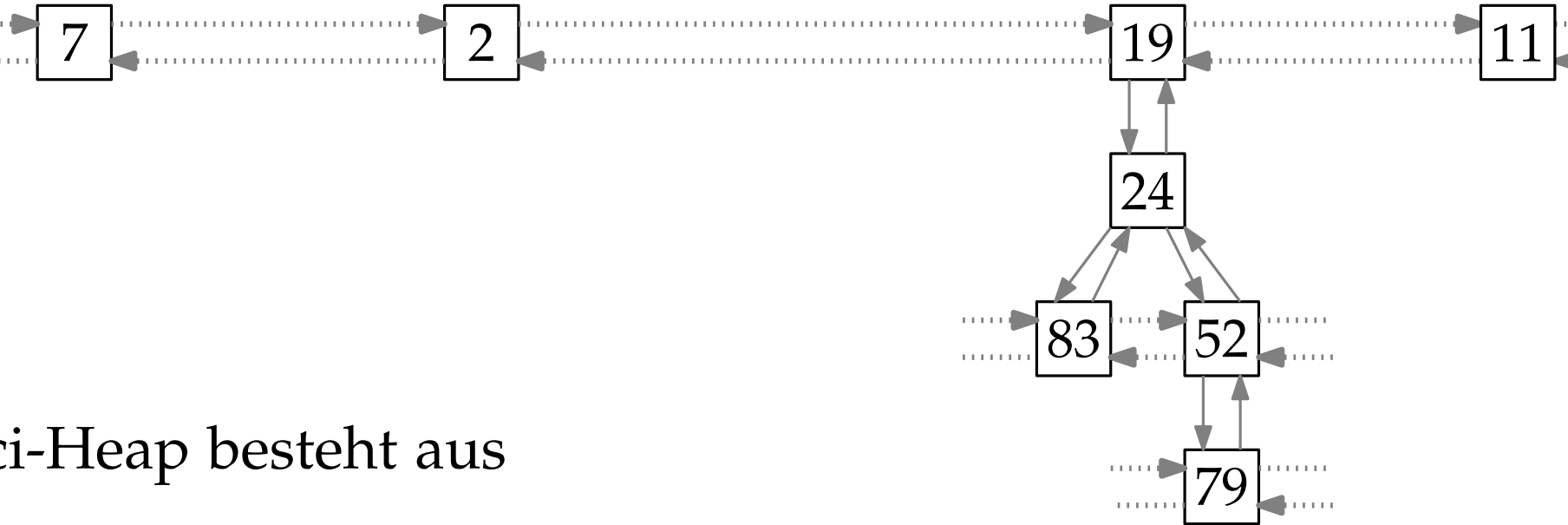
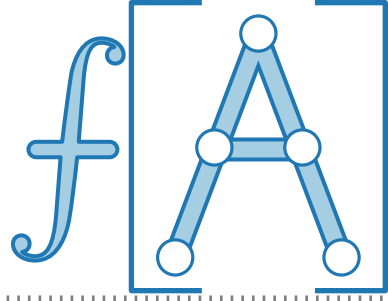
Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten

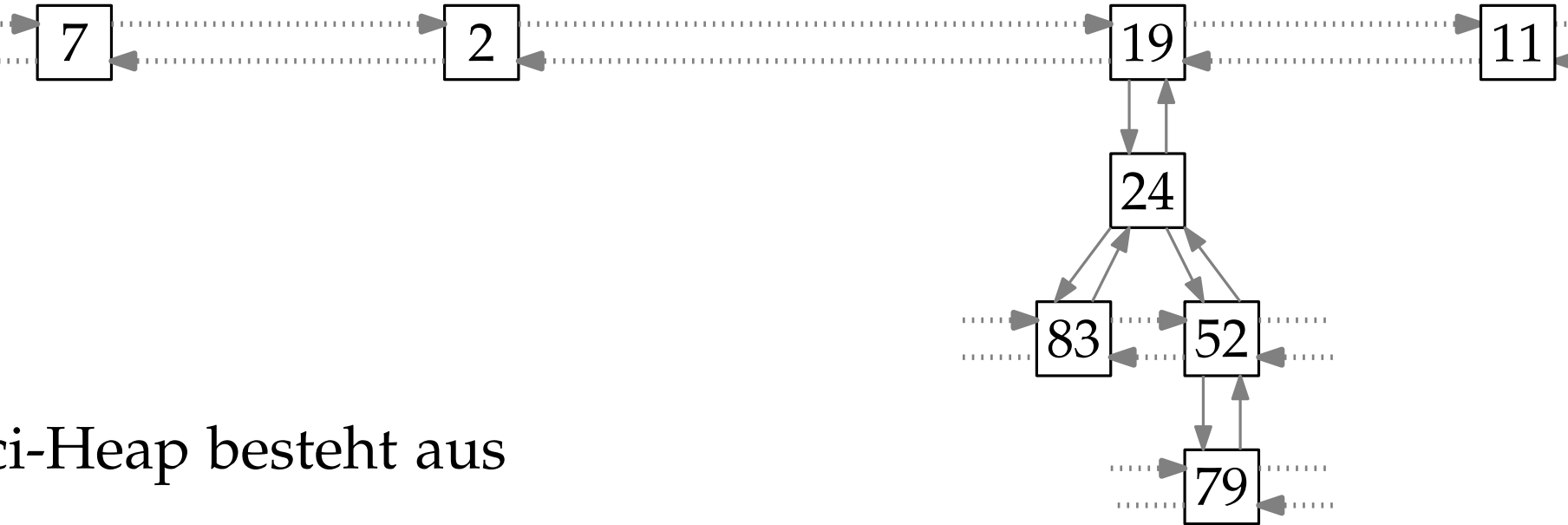
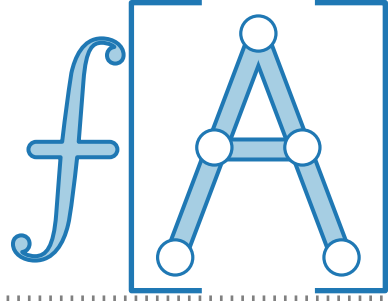
Fibonacci-Heaps



Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten

Fibonacci-Heaps

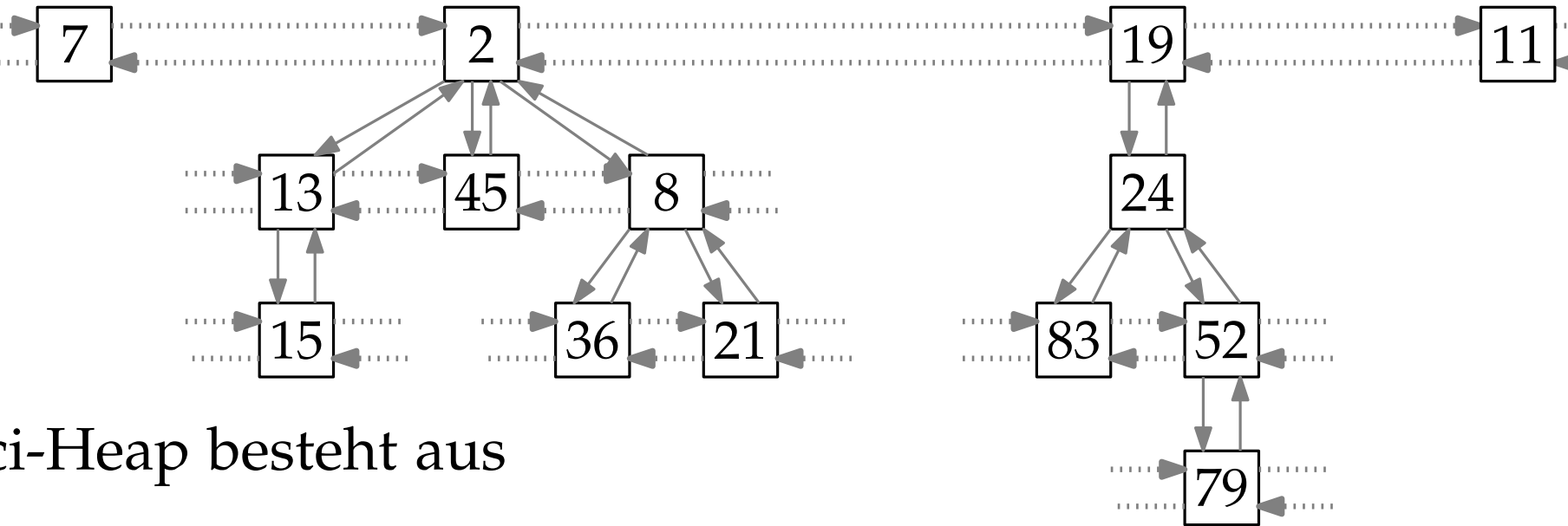
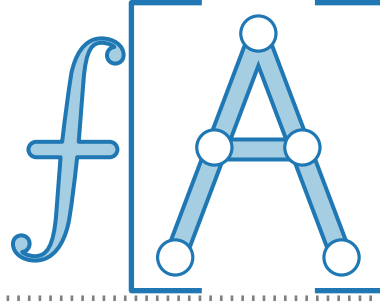


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

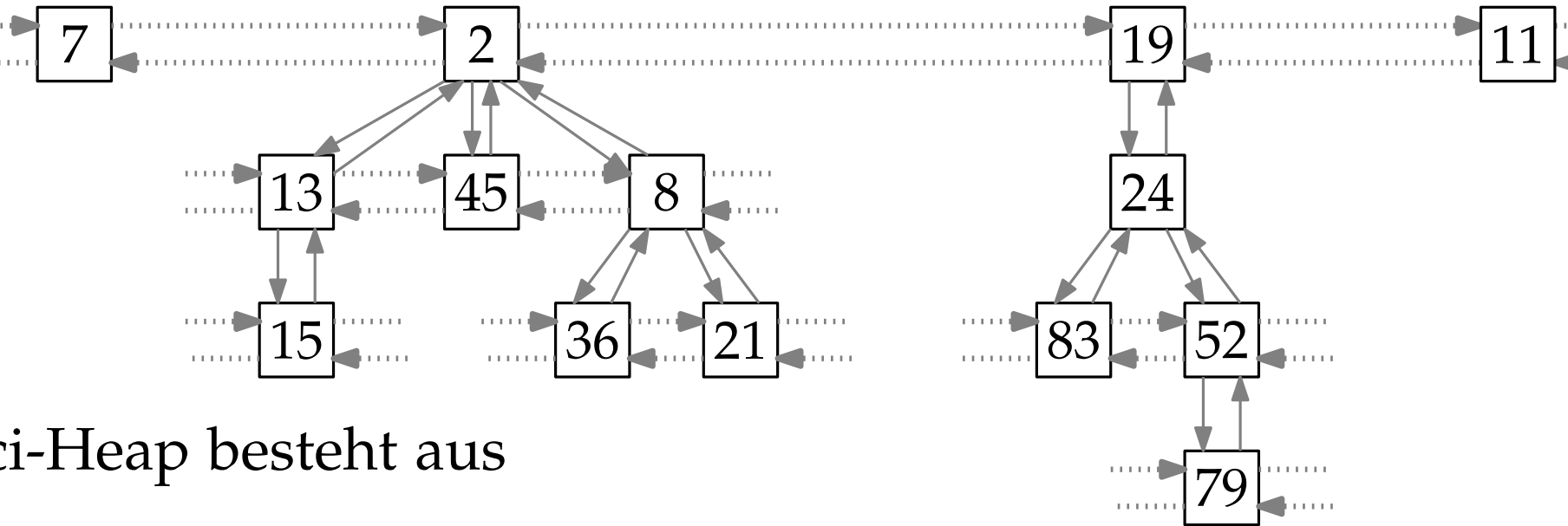
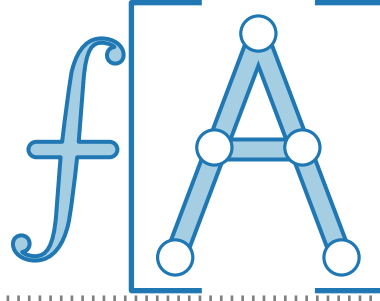


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

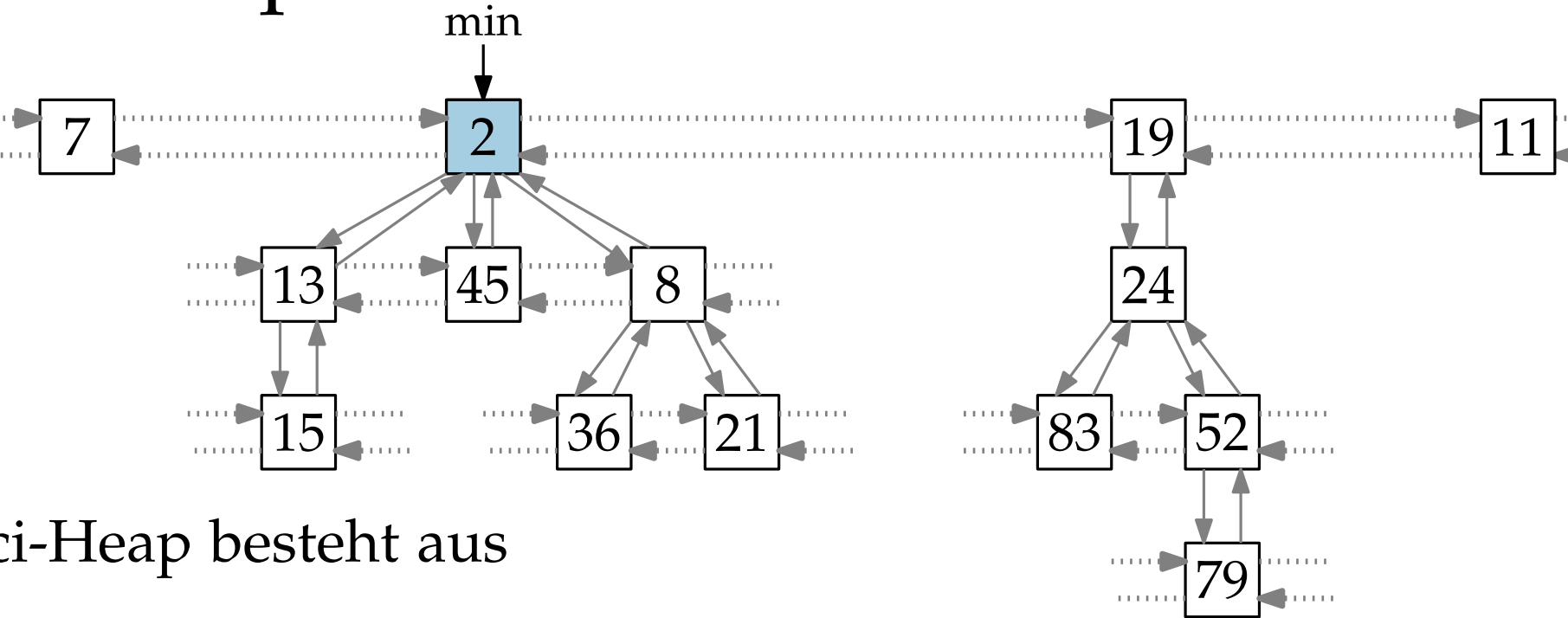
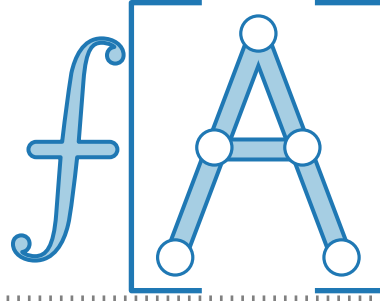


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten
- Ein Zeiger auf den Knoten mit minimalem Schlüssel

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

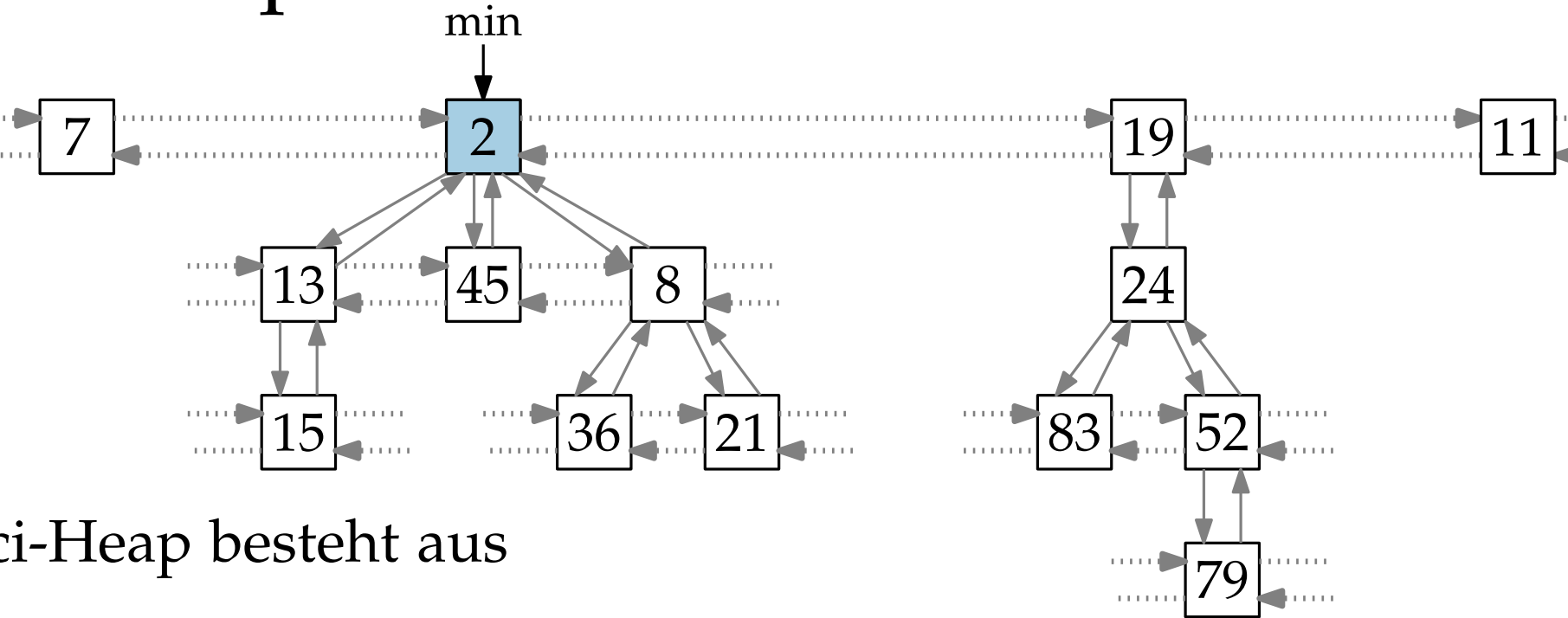
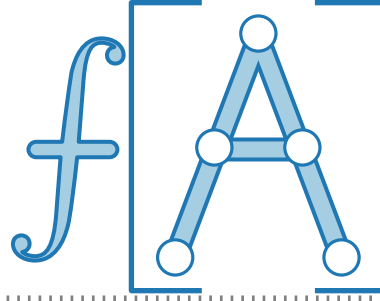


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten
- Ein Zeiger auf den Knoten mit minimalem Schlüssel

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

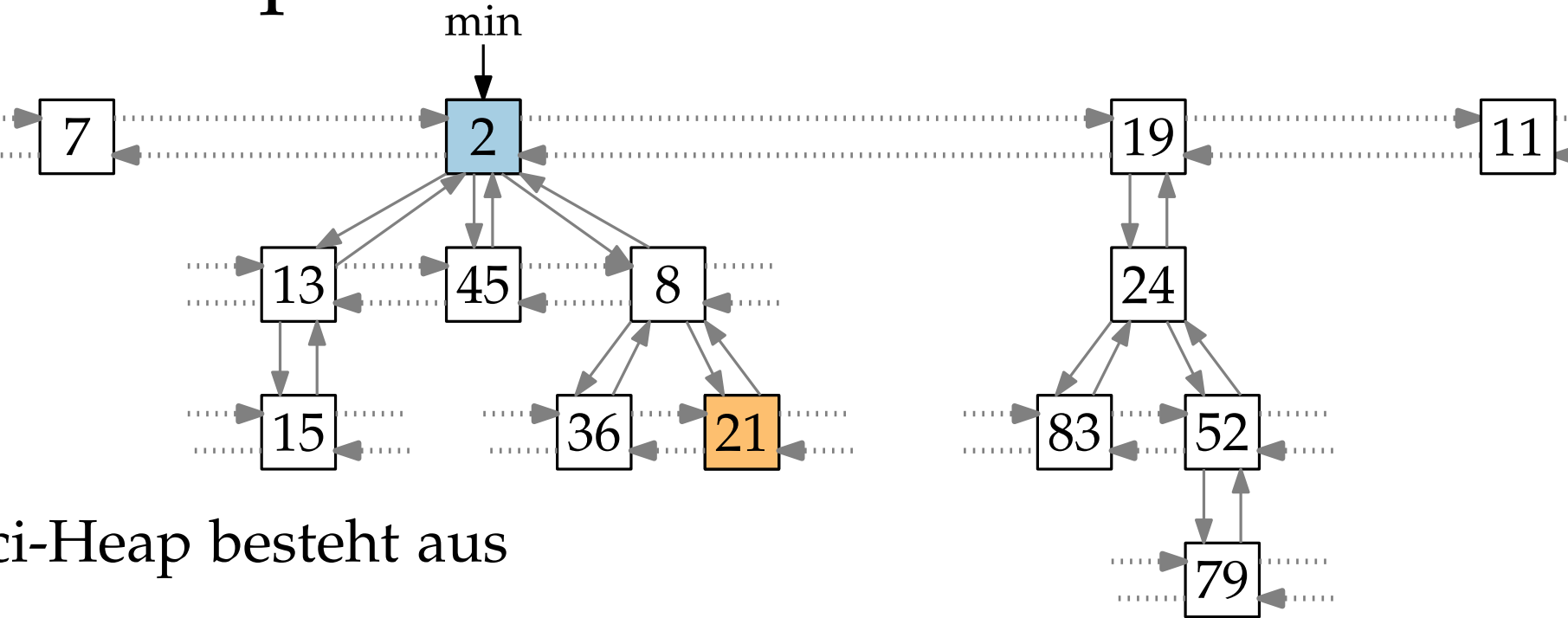
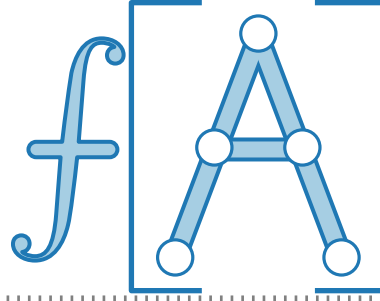


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten
- Ein Zeiger auf den Knoten mit minimalem Schlüssel
- Manche Knoten sind **markiert**

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

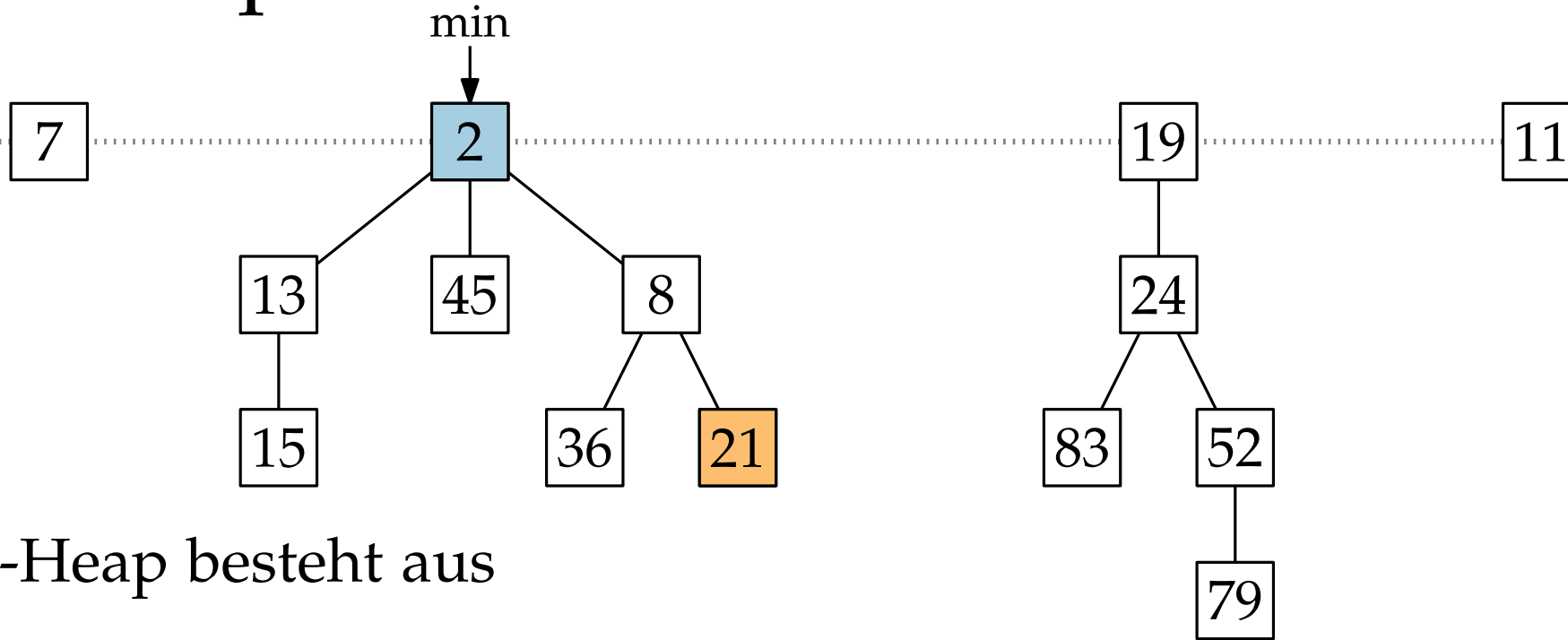


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten
- Ein Zeiger auf den Knoten mit minimalem Schlüssel
- Manche Knoten sind **markiert**

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

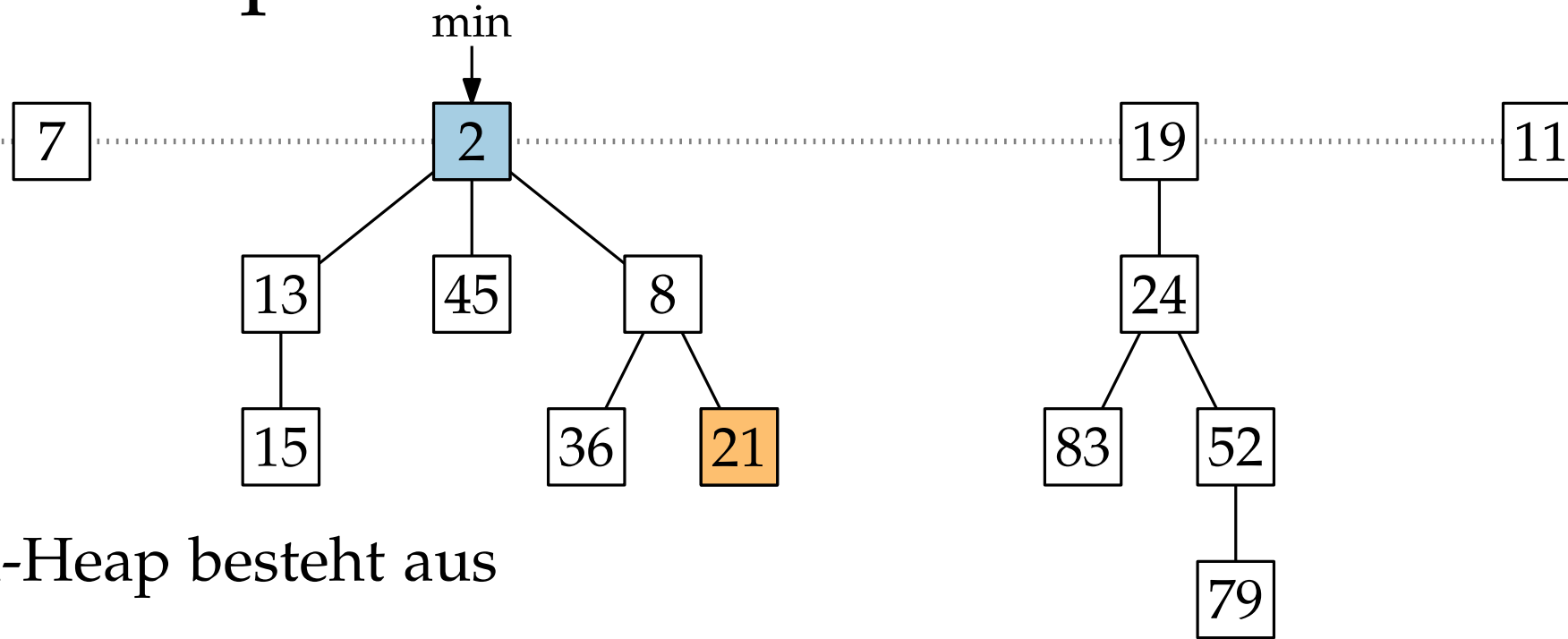


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten
- Ein Zeiger auf den Knoten mit minimalem Schlüssel
- Manche Knoten sind **markiert**

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

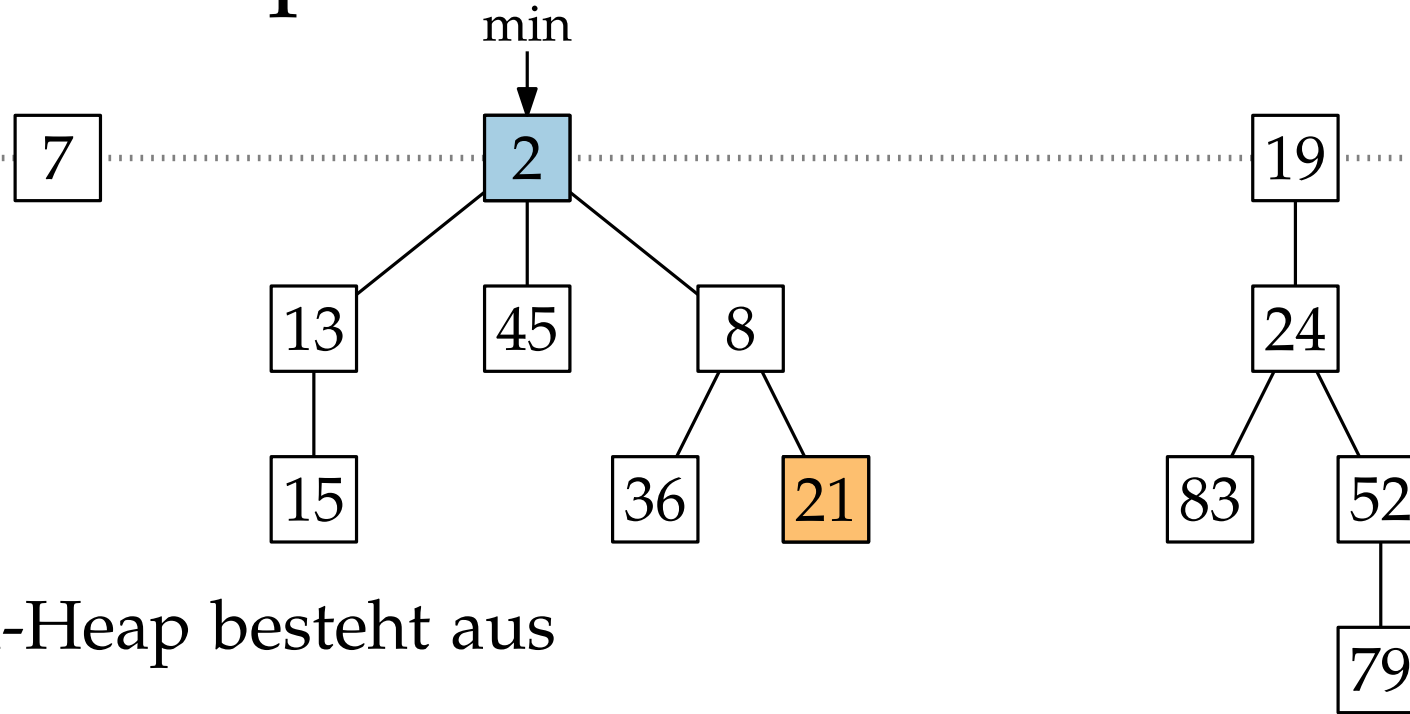


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten
- Ein Zeiger auf den Knoten mit minimalem Schlüssel
- Manche Knoten sind **markiert**
- Ein Fibonacci-Heap heißt **normiert**, wenn alle Wurzelknoten unterschiedlichen Grad haben

im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Fibonacci-Heaps

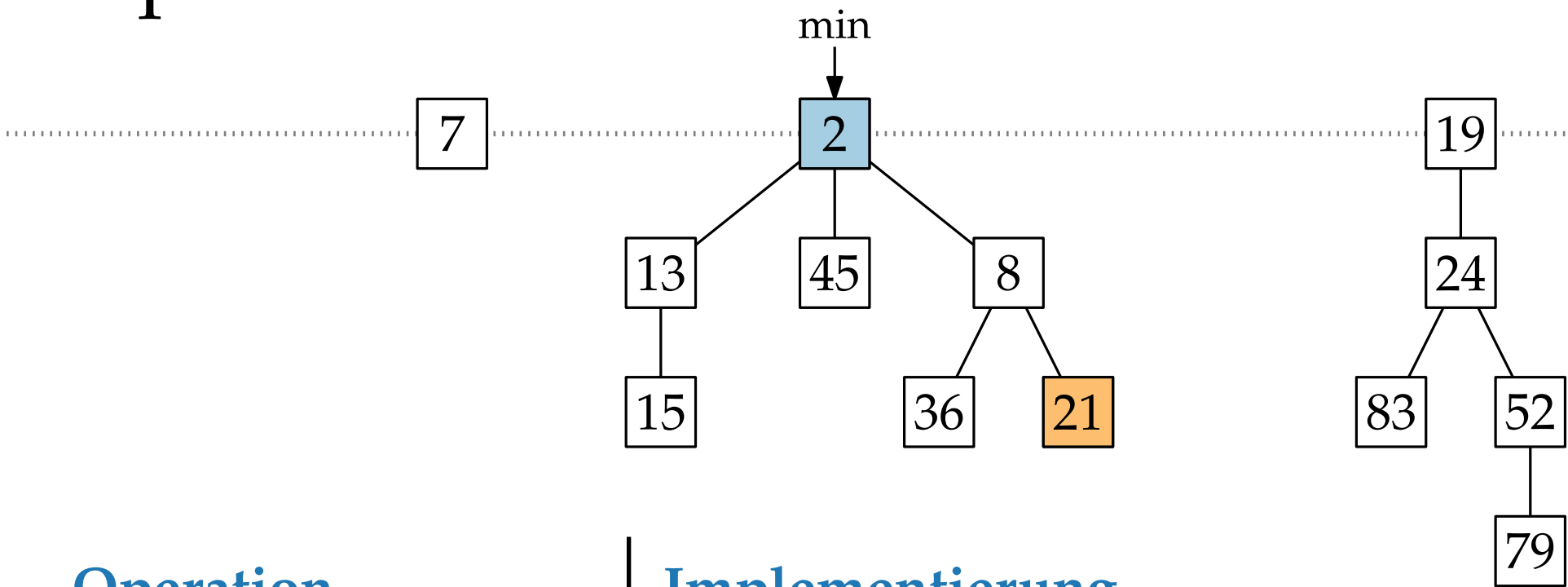
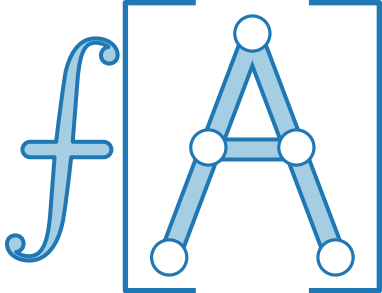


Ein Fibonacci-Heap besteht aus

- Eine doppelt verkettete zirkuläre **Wurzelliste** von Wurzelknoten
- Ein **minheapgeordneter Baum** an jedem Wurzelknoten
(Wert jedes Knoten nicht größer als Werte der Kinder)
mit doppelt verketteter zirkulärer Liste für Geschwisterknoten
- Ein Zeiger auf den Knoten mit minimalem Schlüssel
- Manche Knoten sind **markiert**
- Ein Fibonacci-Heap heißt **normiert**, wenn alle Wurzelknoten unterschiedlichen Grad haben

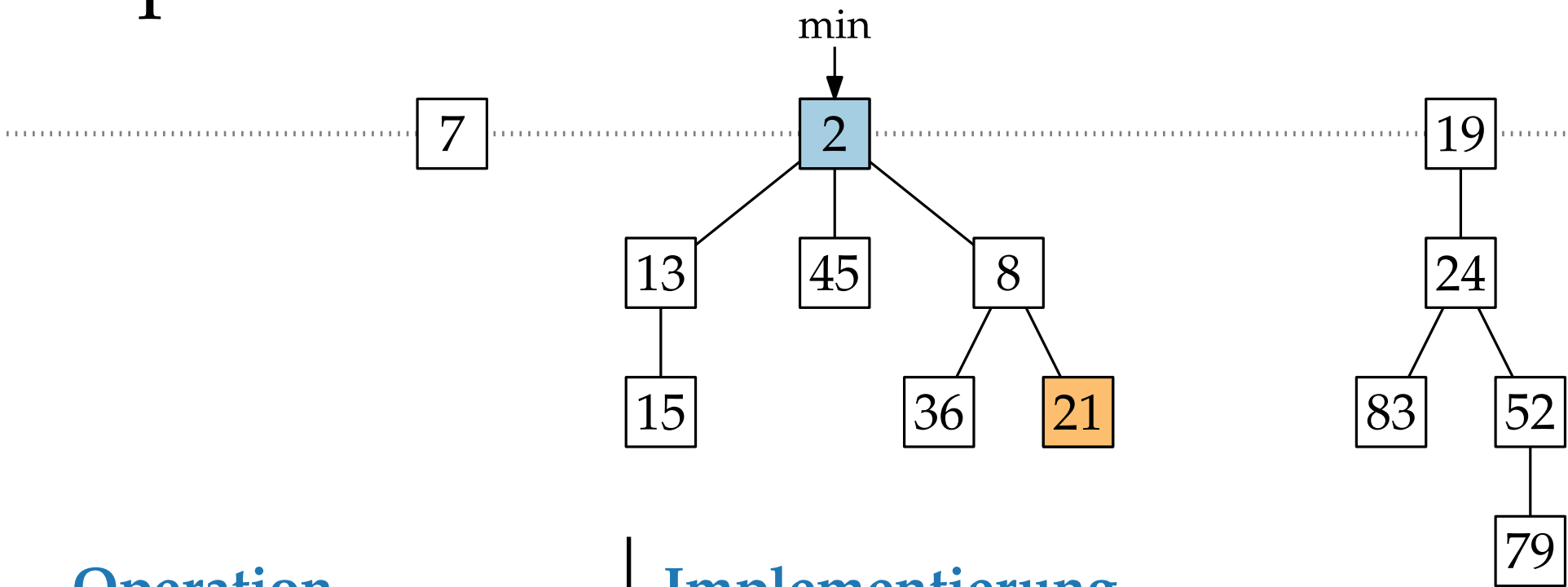
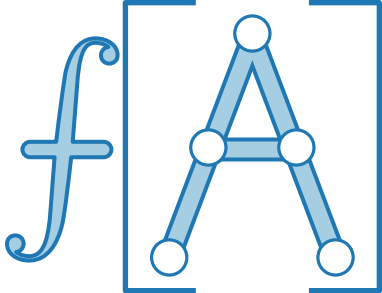
im Gegensatz zu Heaps
können Knoten mehr als
2 Kinder haben!

Operationen



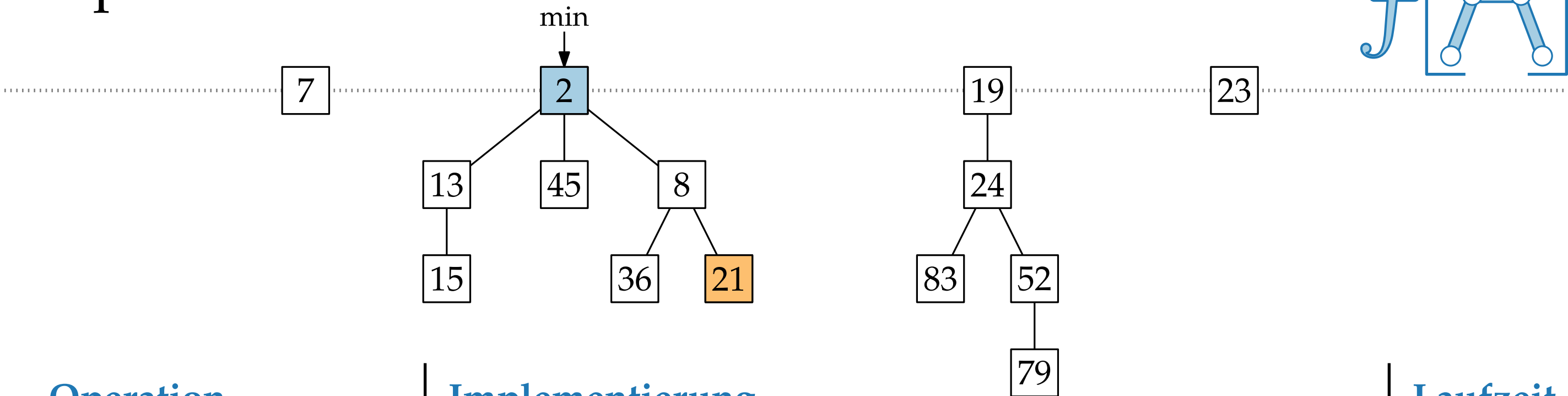
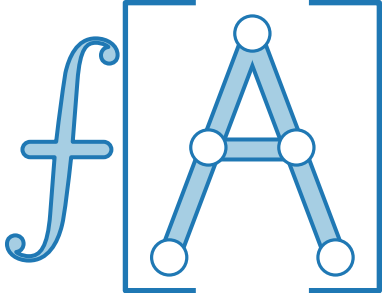
Operation	Implementierung	Laufzeit
$Q.\text{INSERT}(x)$		$\Theta(1)$
$Q.\text{MIN}()$		
$Q.\text{EXTRACTMIN}()$		
$Q.\text{DECREASEKEY}(x, p)$		
$Q.\text{UNION}(Q')$		

Operationen



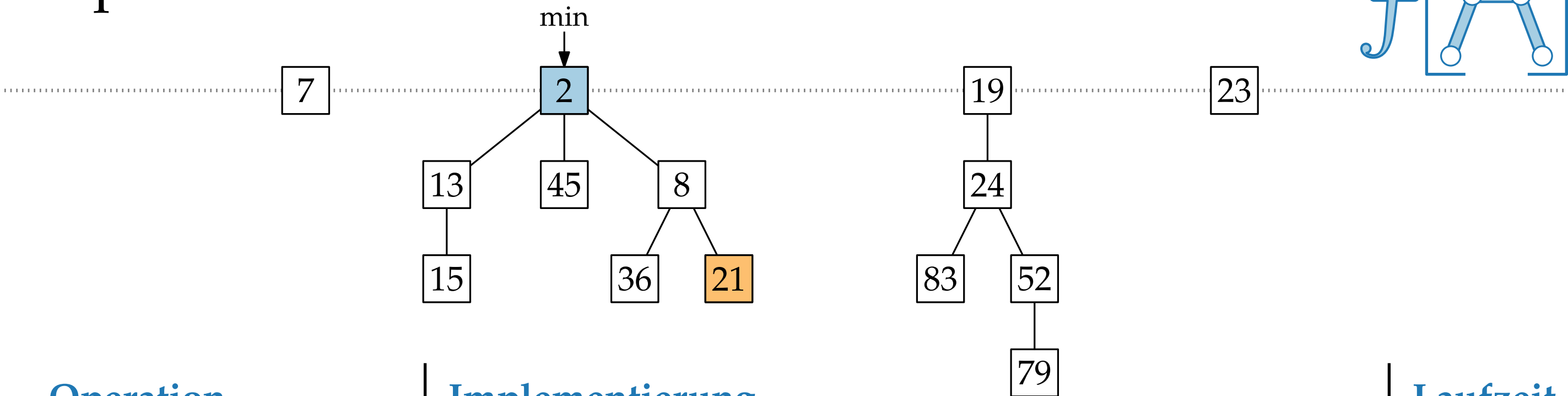
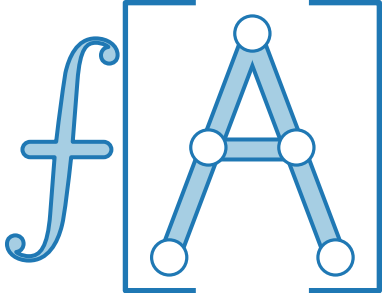
Operation	Implementierung	Laufzeit
$Q.\text{INSERT}(x)$	Füge x in Wurzelliste ein, update Pointer	$\Theta(1)$
$Q.\text{MIN}()$		
$Q.\text{EXTRACTMIN}()$		
$Q.\text{DECREASEKEY}(x, p)$		
$Q.\text{UNION}(Q')$		

Operationen



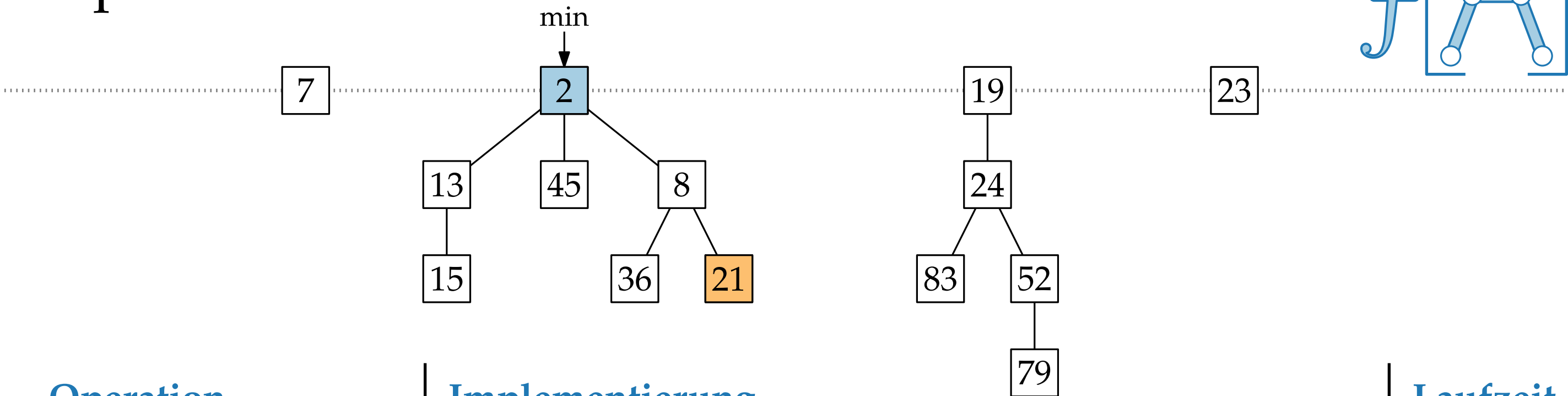
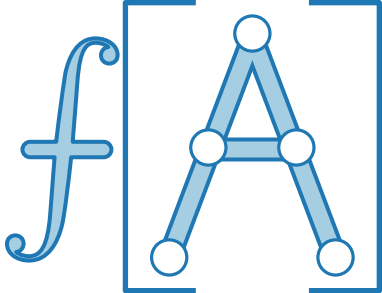
Operation	Implementierung	Laufzeit
$Q.\text{INSERT}(x)$	Füge x in Wurzelliste ein, update Pointer	$\Theta(1)$
$Q.\text{MIN}()$		
$Q.\text{EXTRACTMIN}()$		
$Q.\text{DECREASEKEY}(x, p)$		
$Q.\text{UNION}(Q')$		

Operationen



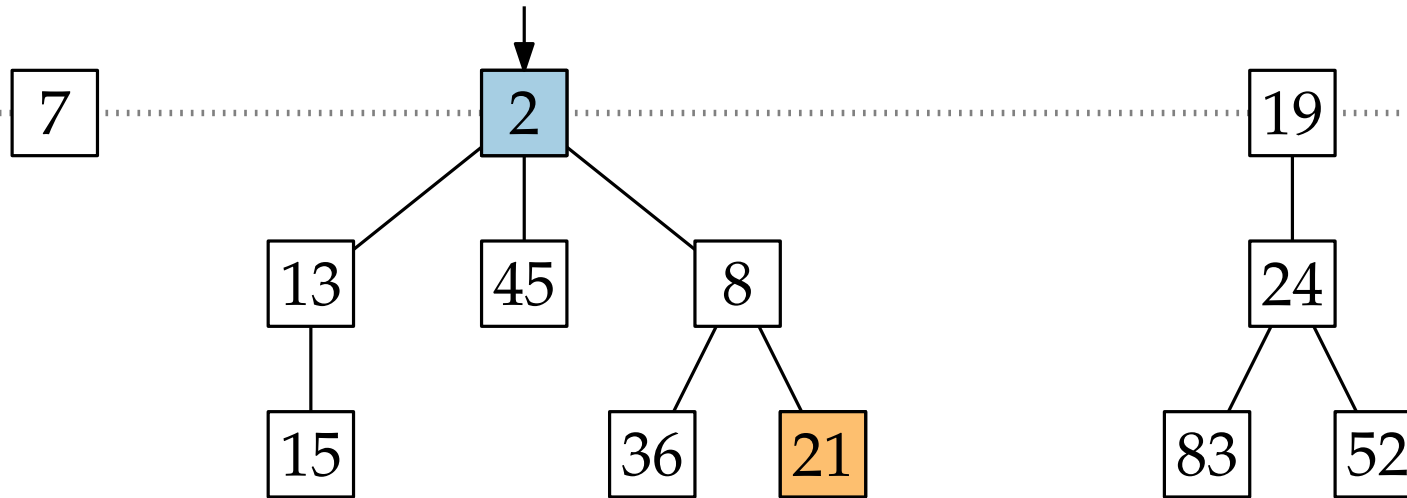
Operation	Implementierung	Laufzeit
$Q.\text{INSERT}(x)$	Füge x in Wurzelliste ein, update Pointer	$\Theta(1)$
$Q.\text{MIN}()$	Benutze Pointer	
$Q.\text{EXTRACTMIN}()$		
$Q.\text{DECREASEKEY}(x, p)$		
$Q.\text{UNION}(Q')$		

Operationen

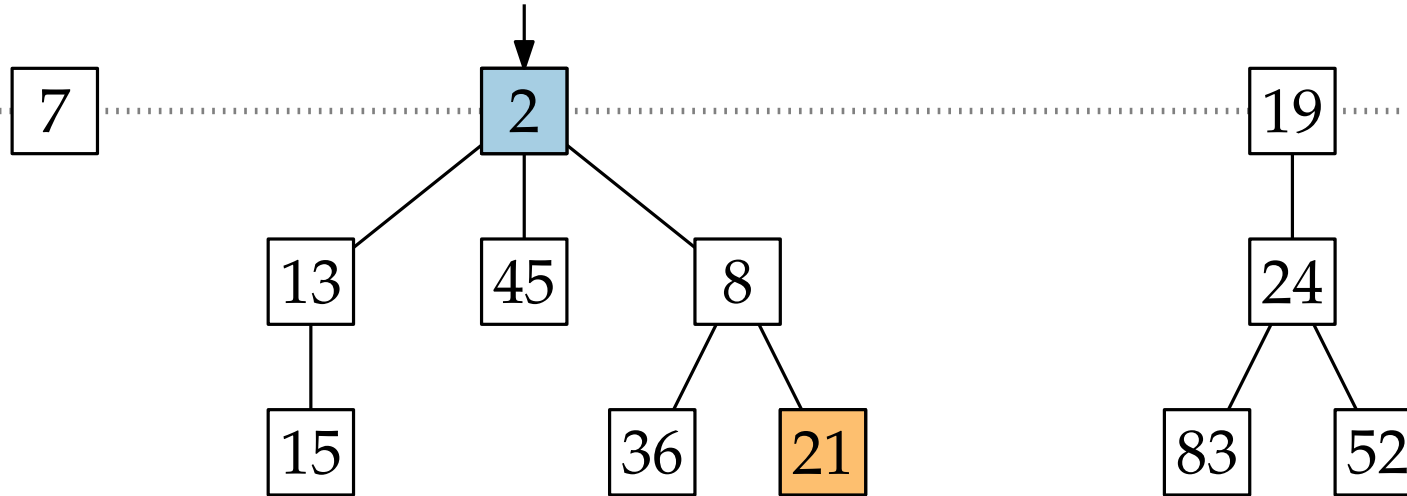


Operation	Implementierung	Laufzeit
$Q.\text{INSERT}(x)$	Füge x in Wurzelliste ein, update Pointer	$\Theta(1)$
$Q.\text{MIN}()$	Benutze Pointer	$\Theta(1)$
$Q.\text{EXTRACTMIN}()$		
$Q.\text{DECREASEKEY}(x, p)$		
$Q.\text{UNION}(Q')$		

Manipulation von Bäumen

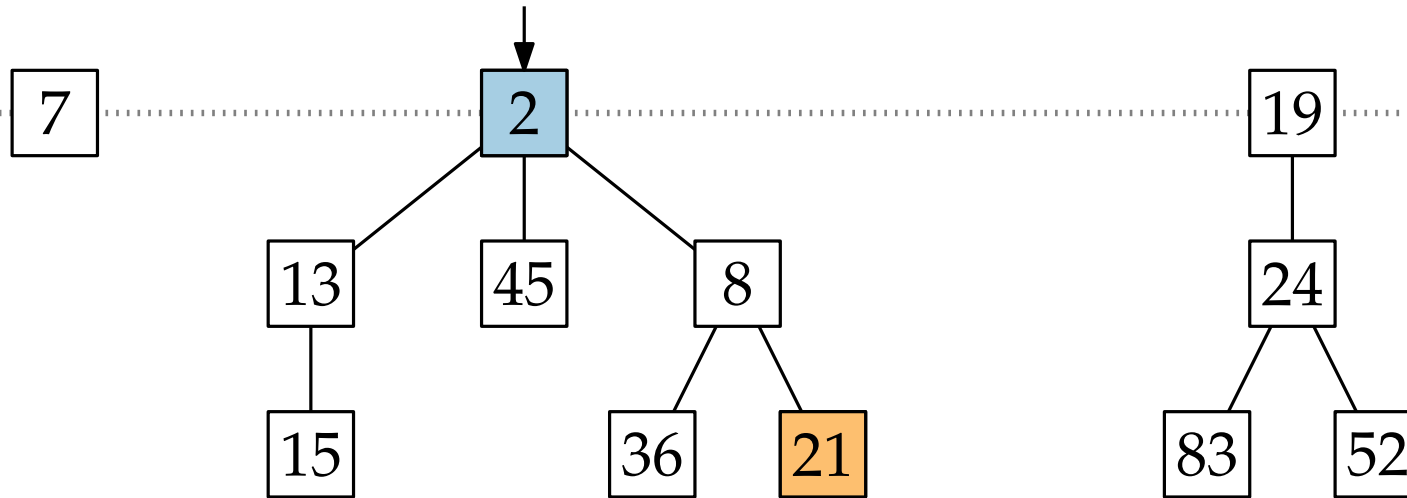


Manipulation von Bäumen



Drei Basis-Methoden zur Manipulation von Bäumen in Fibonacci-Heaps:

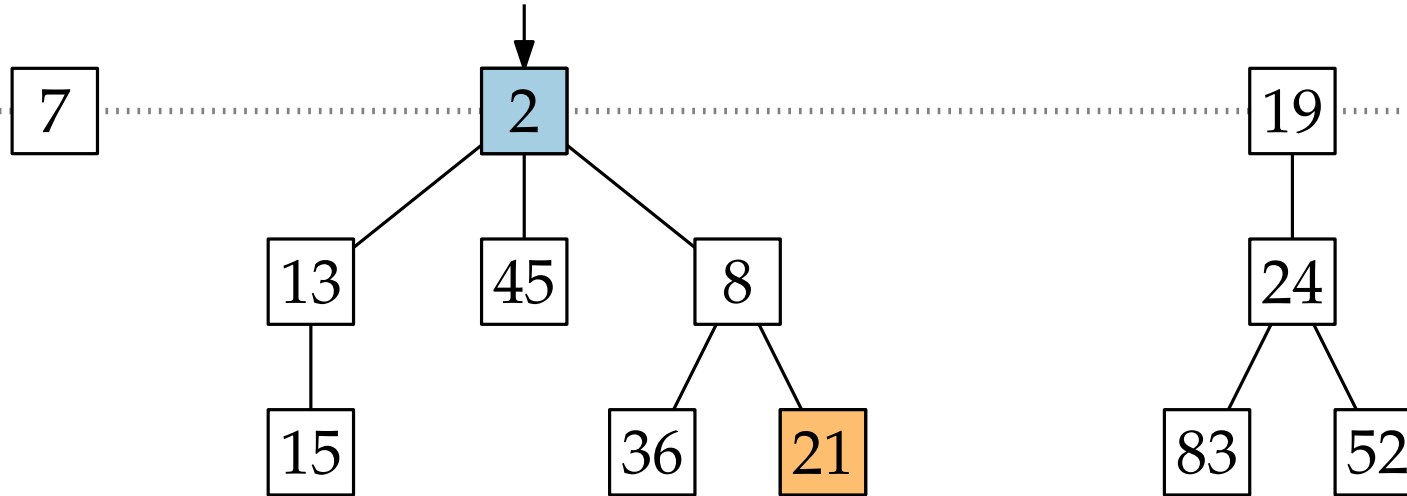
Manipulation von Bäumen



Drei Basis-Methoden zur Manipulation von Bäumen in Fibonacci-Heaps:

$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

Manipulation von Bäumen



Drei Basis-Methoden zur Manipulation von Bäumen in Fibonacci-Heaps:

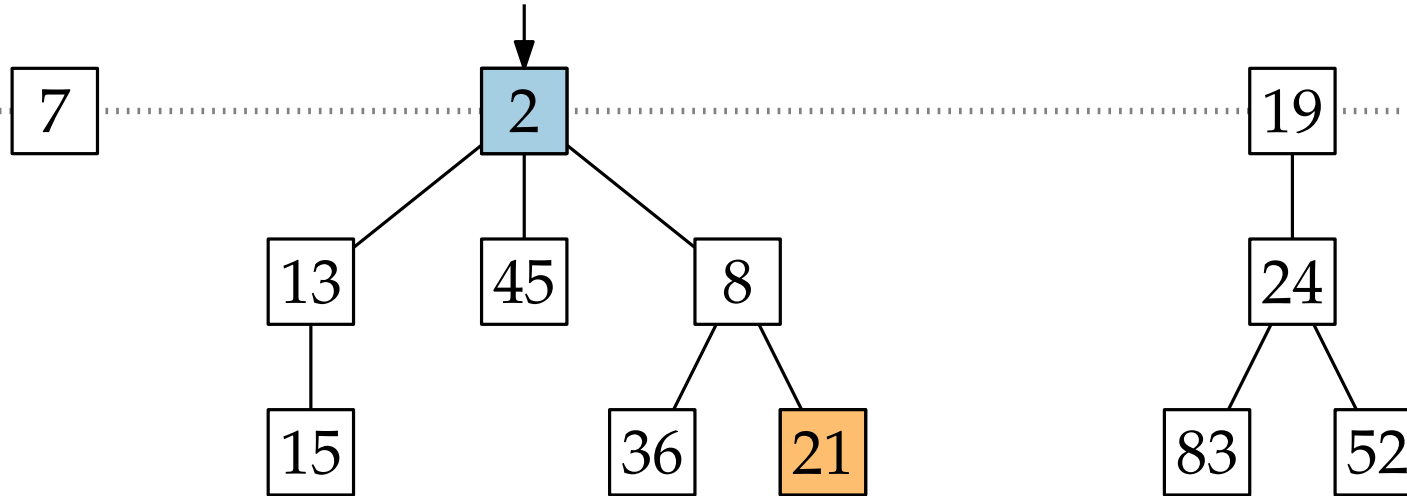
$\text{LINK}(r, r')$ **Wachstum** von Bäumen.

Zwei Bäume werden zu einem Neuen verbunden.

$\text{CUT}(x)$ **Beschneiden** von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

Manipulation von Bäumen



Drei Basis-Methoden zur Manipulation von Bäumen in Fibonacci-Heaps:

$\text{LINK}(r, r')$ **Wachstum** von Bäumen.

Zwei Bäume werden zu einem Neuen verbunden.

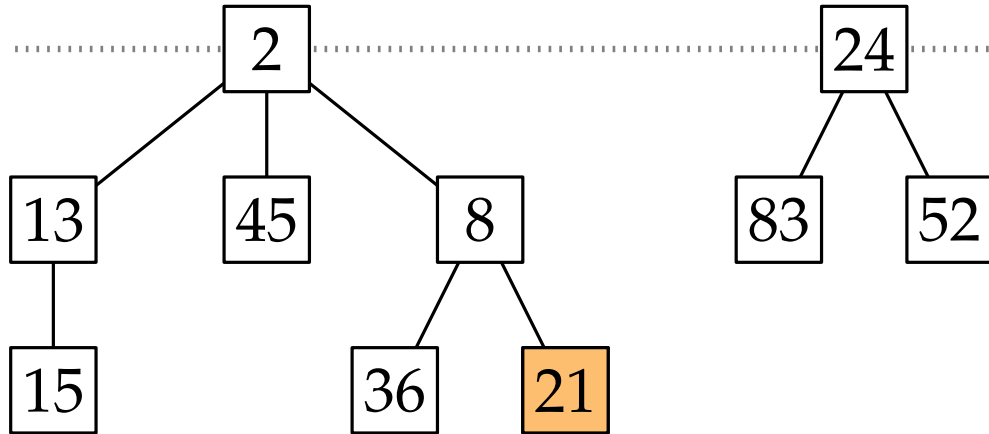
$\text{CUT}(x)$ **Beschneiden** von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

$\text{REMOVE}(r)$ **Spalten** von Bäumen an der Wurzel.

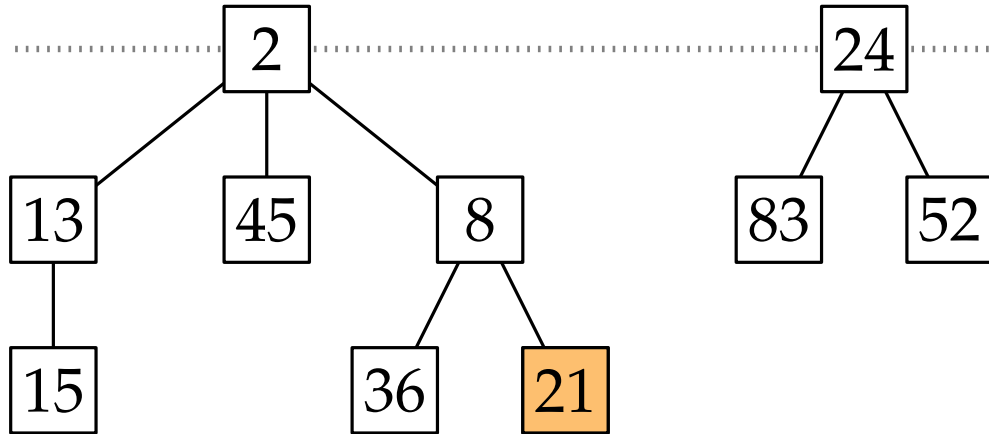
Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

Baummanipulation: $\text{LINK}(r, r')$



$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

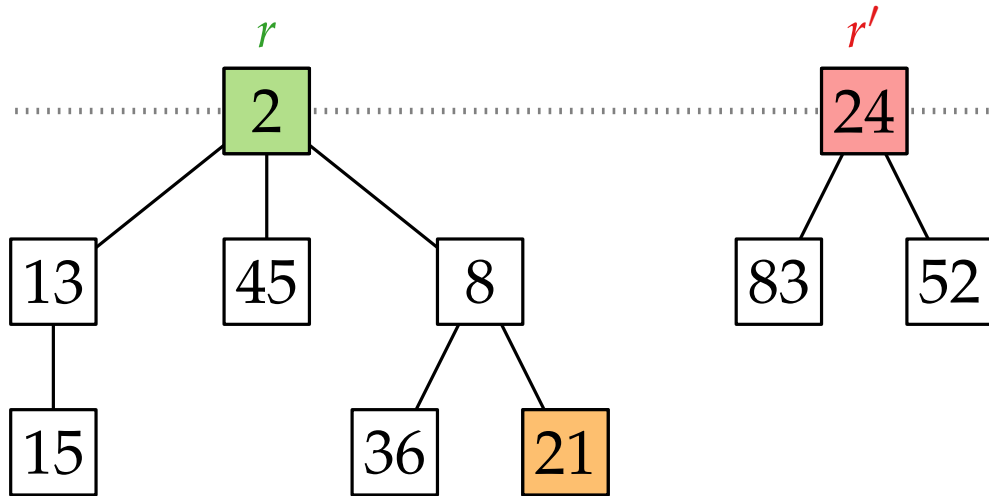
Baummanipulation: $\text{LINK}(r, r')$



$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

■ Seien r, r' Wurzeln der Bäume mit $r.\text{key} \leq r'.\text{key}$

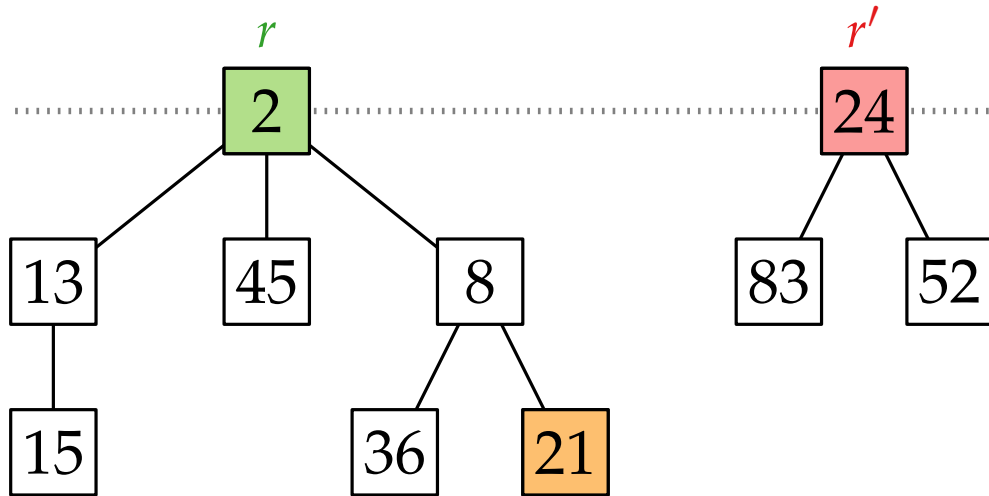
Baummanipulation: $\text{LINK}(r, r')$



$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

■ Seien r, r' Wurzeln der Bäume mit $r.\text{key} \leq r'.\text{key}$

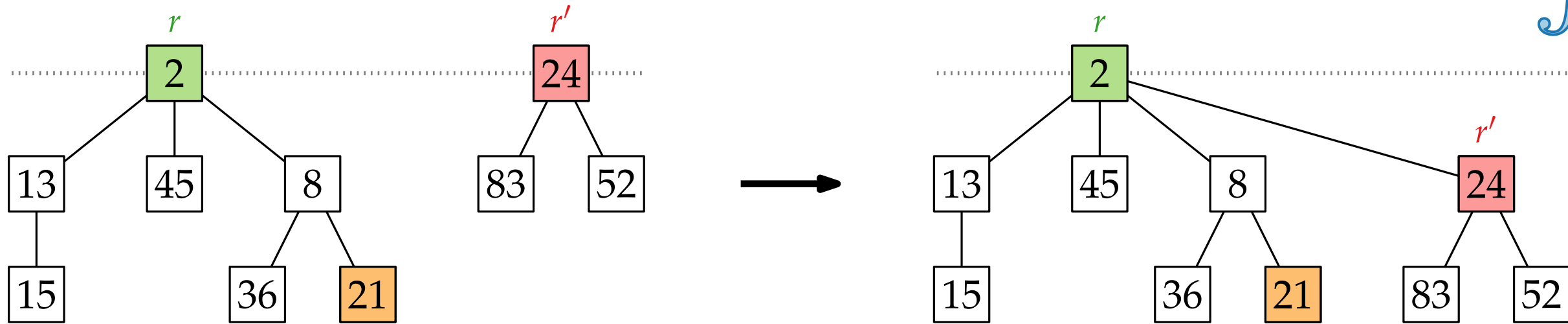
Baummanipulation: $\text{LINK}(r, r')$



$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

- Seien r, r' Wurzeln der Bäume mit $r.\text{key} \leq r'.\text{key}$
- Hänge r' als neues Kind an r an

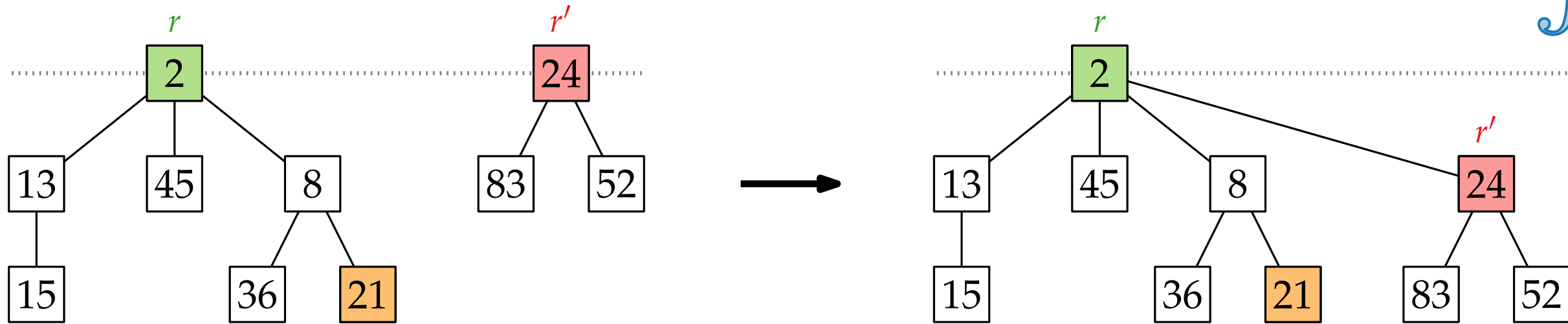
Baummanipulation: $\text{LINK}(r, r')$



$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

- Seien r, r' Wurzeln der Bäume mit $r.\text{key} \leq r'.\text{key}$
- Hänge r' als neues Kind an r an

Baummanipulation: $\text{LINK}(r, r')$

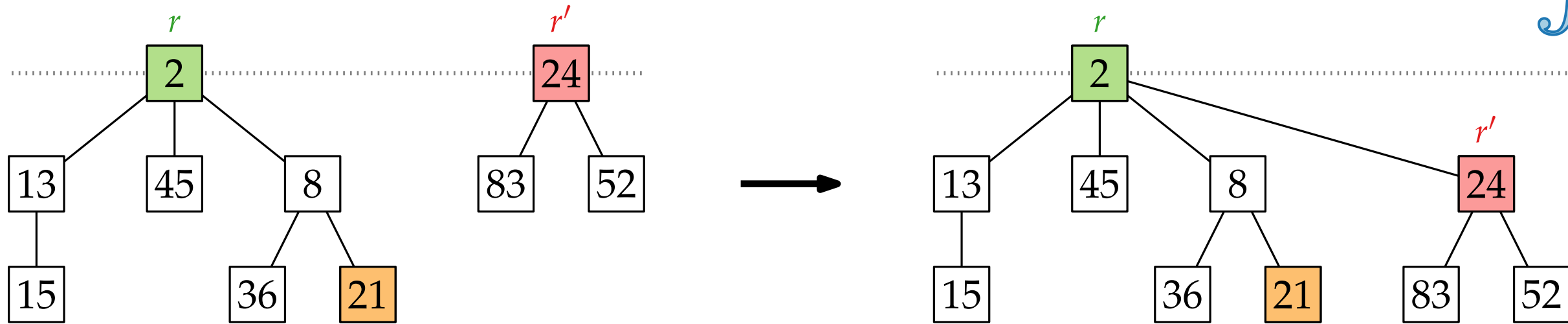


$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

- Seien r, r' Wurzeln der Bäume mit $r.\text{key} \leq r'.\text{key}$
- Hänge r' als neues Kind an r an

Laufzeit.

Baummanipulation: $\text{LINK}(r, r')$

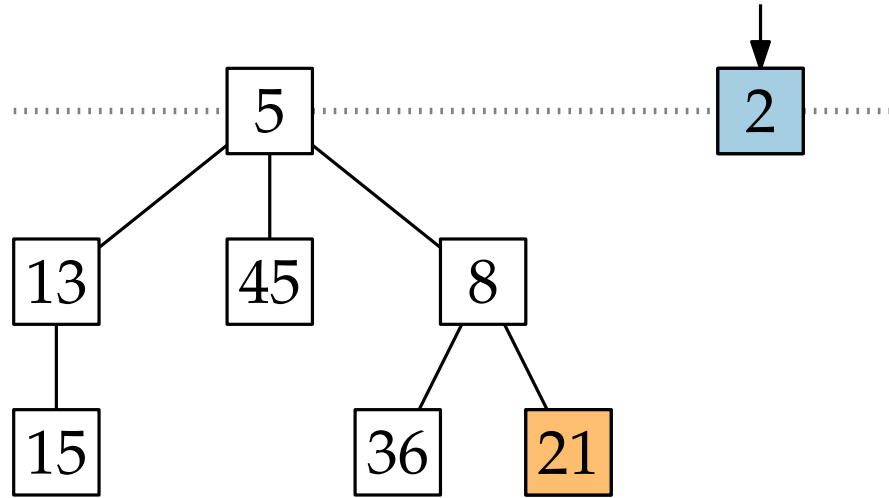


$\text{LINK}(r, r')$ **Wachstum** von Bäumen.
Zwei Bäume werden zu einem Neuen verbunden.

- Seien r, r' Wurzeln der Bäume mit $r.\text{key} \leq r'.\text{key}$
- Hänge r' als neues Kind an r an

Laufzeit. $\Theta(1)$

Baummanipulation: $\text{CUT}(x)$

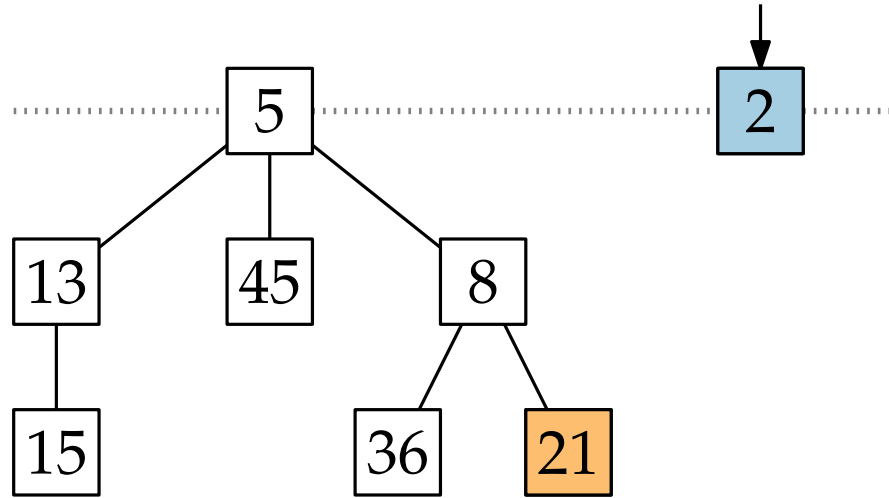


$\text{CUT}(x)$

Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

Baummanipulation: $\text{CUT}(x)$



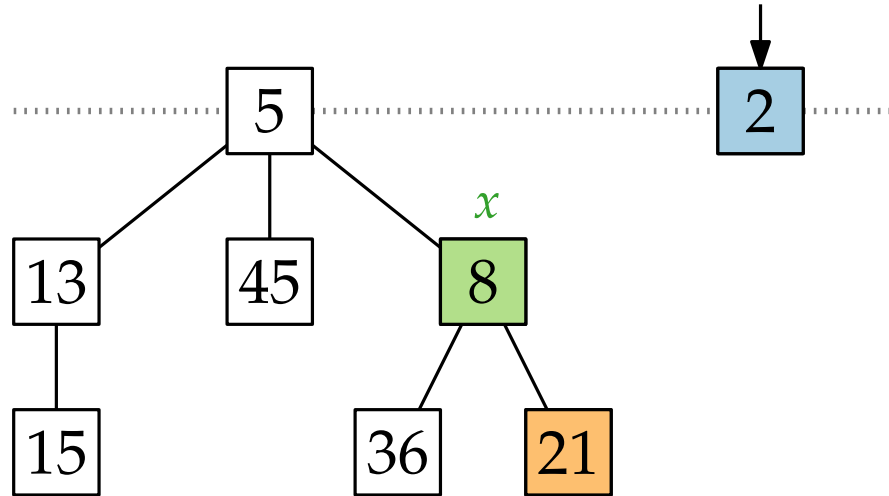
$\text{CUT}(x)$

Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

- Sei x Knoten, der nicht in Wurzelliste steht

Baummanipulation: $\text{CUT}(x)$



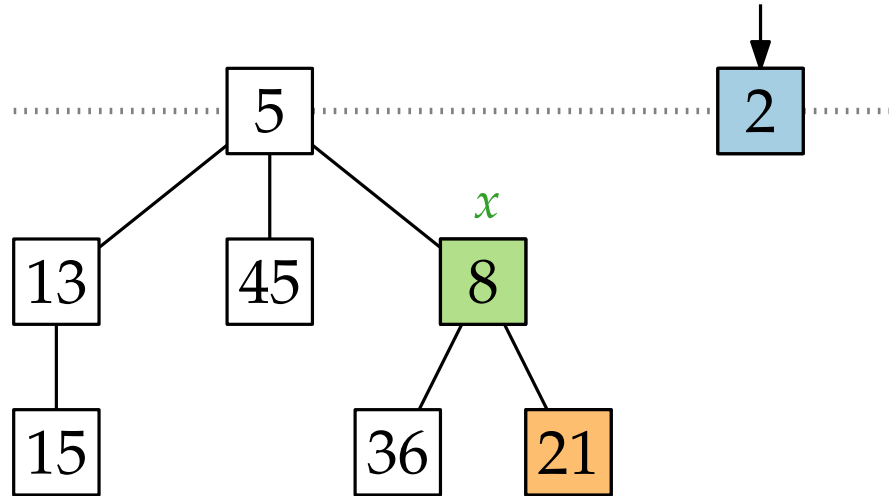
$\text{CUT}(x)$

Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

- Sei x Knoten, der nicht in Wurzelliste steht

Baummanipulation: $\text{CUT}(x)$



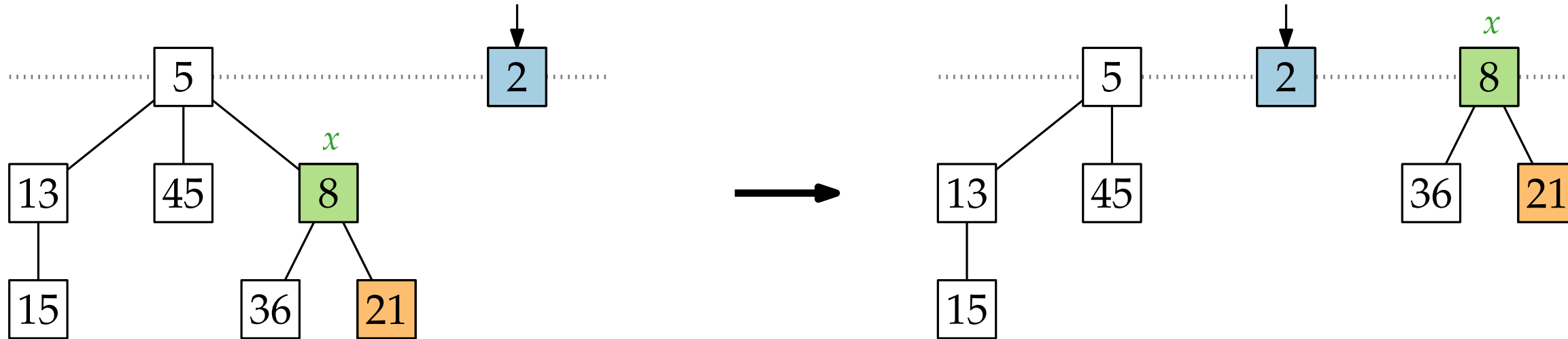
$\text{CUT}(x)$

Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

- Sei x Knoten, der nicht in Wurzelliste steht
- Trenne x von seinem Elternknoten ab und füge x rechts von $Q.\text{MIN}()$ in Wurzelliste ein

Baummanipulation: $\text{CUT}(x)$



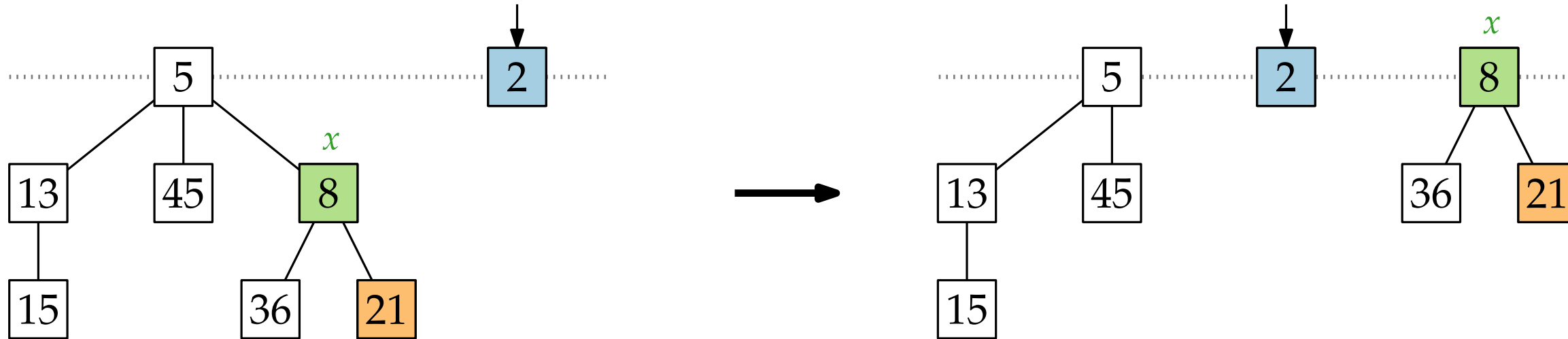
$\text{CUT}(x)$

Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

- Sei x Knoten, der nicht in Wurzelliste steht
- Trenne x von seinem Elternknoten ab und füge x rechts von $Q.\text{MIN}()$ in Wurzelliste ein

Baummanipulation: $\text{CUT}(x)$



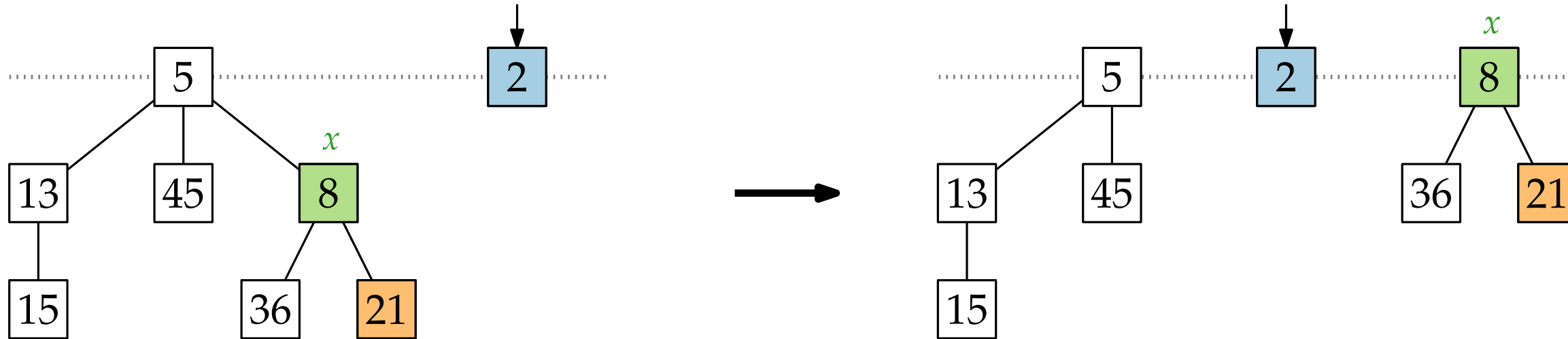
$\text{CUT}(x)$

Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

- Sei x Knoten, der nicht in Wurzelliste steht
- Trenne x von seinem Elternknoten ab und füge x rechts von $Q.\text{MIN}()$ in Wurzelliste ein
- Falls (ehemaliger) Elternknoten y von x nicht in Wurzelliste, **markiere** y

Baummanipulation: $\text{CUT}(x)$



$\text{CUT}(x)$

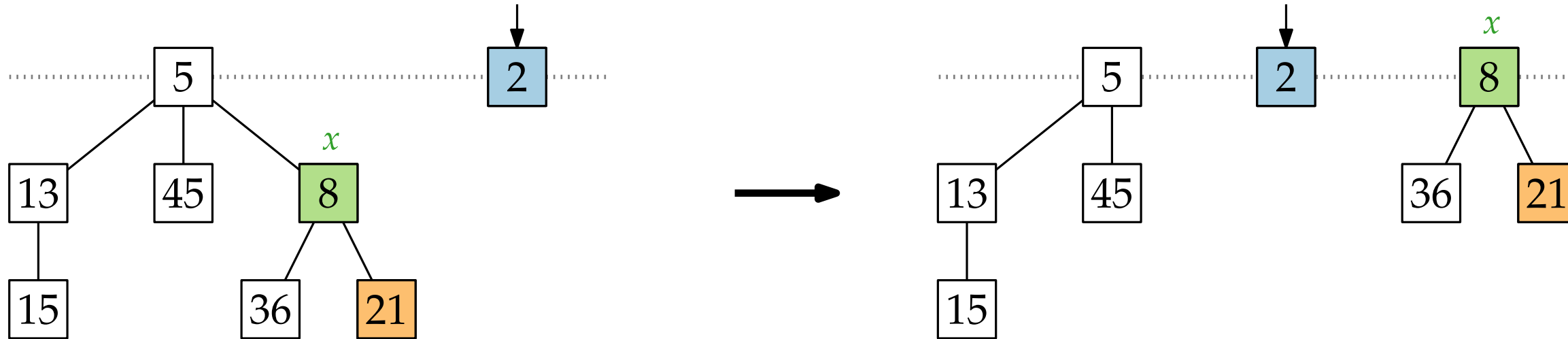
Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

- Sei x Knoten, der nicht in Wurzelliste steht
- Trenne x von seinem Elternknoten ab und füge x rechts von $Q.\text{MIN}()$ in Wurzelliste ein
- Falls (ehemaliger) Elternknoten y von x nicht in Wurzelliste, **markiere** y

Laufzeit.

Baummanipulation: $\text{CUT}(x)$



$\text{CUT}(x)$

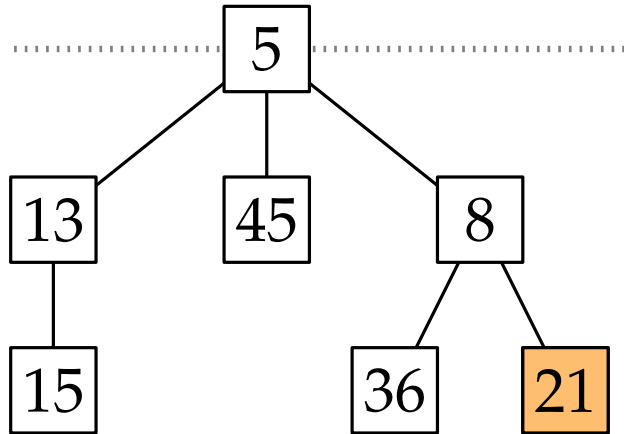
Beschneiden von Bäumen.

Ein Teilbaum wird aus einem Baum herausgetrennt und als neuer Baum in die Wurzelliste eingefügt.

- Sei x Knoten, der nicht in Wurzelliste steht
- Trenne x von seinem Elternknoten ab und füge x rechts von $Q.\text{MIN}()$ in Wurzelliste ein
- Falls (ehemaliger) Elternknoten y von x nicht in Wurzelliste, **markiere** y

Laufzeit. $\Theta(1)$

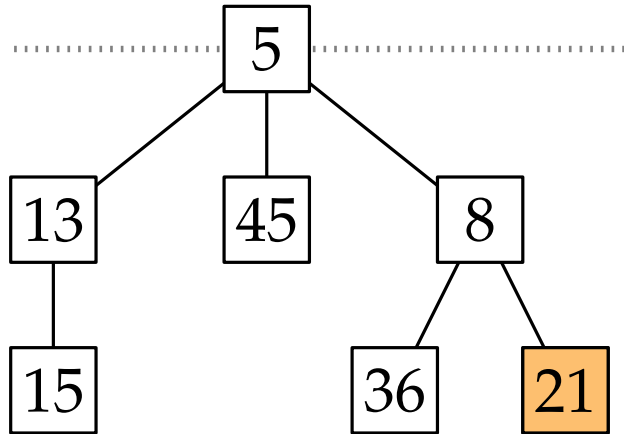
Baummanipulation: REMOVE(r)



REMOVE(r) **Spalten** von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

Baummanipulation: REMOVE(r)

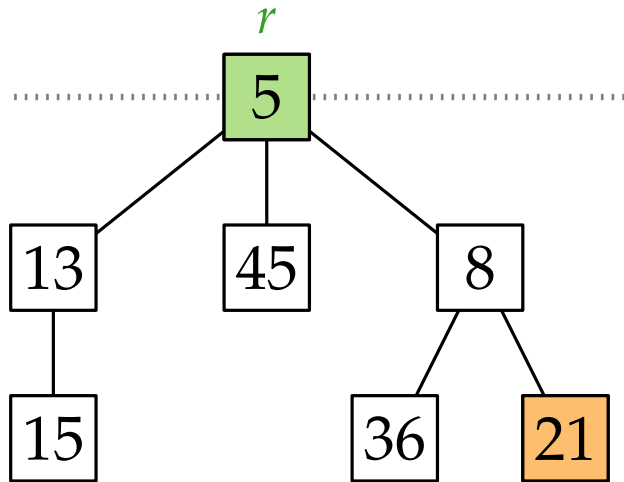


REMOVE(r) **Spalten** von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

■ Sei r Wurzel in Wurzelliste

Baummanipulation: REMOVE(r)

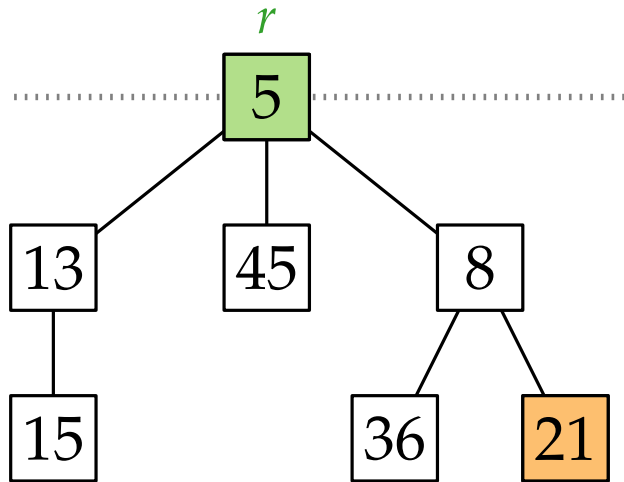


REMOVE(r) **Spalten** von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

■ Sei r Wurzel in Wurzelliste

Baummanipulation: REMOVE(r)

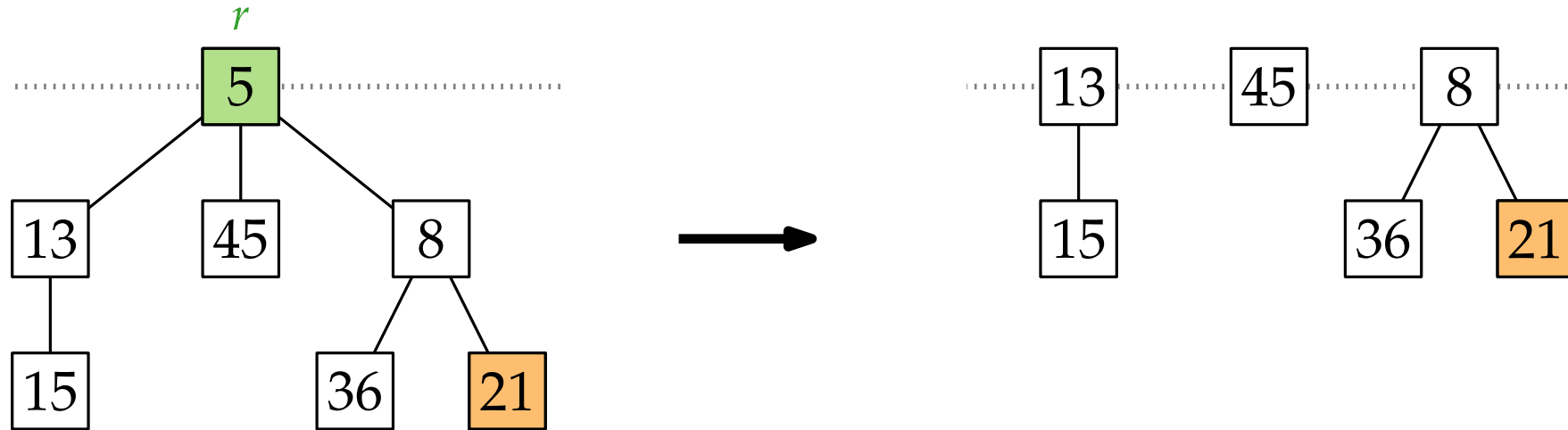


REMOVE(r) Spalten von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

- Sei r Wurzel in Wurzelliste
- Lösche r aus Wurzelliste und füge Kinder von r in Wurzelliste ein

Baummanipulation: $\text{REMOVE}(r)$

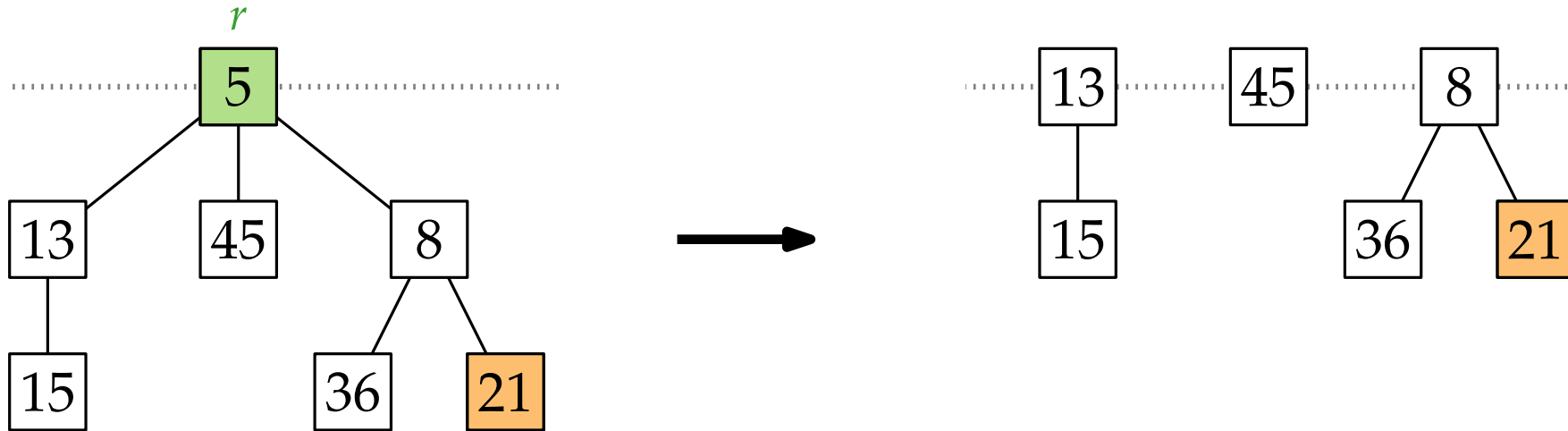


$\text{REMOVE}(r)$ **Spalten** von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

- Sei r Wurzel in Wurzelliste
- Lösche r aus Wurzelliste und füge Kinder von r in Wurzelliste ein

Baummanipulation: $\text{REMOVE}(r)$



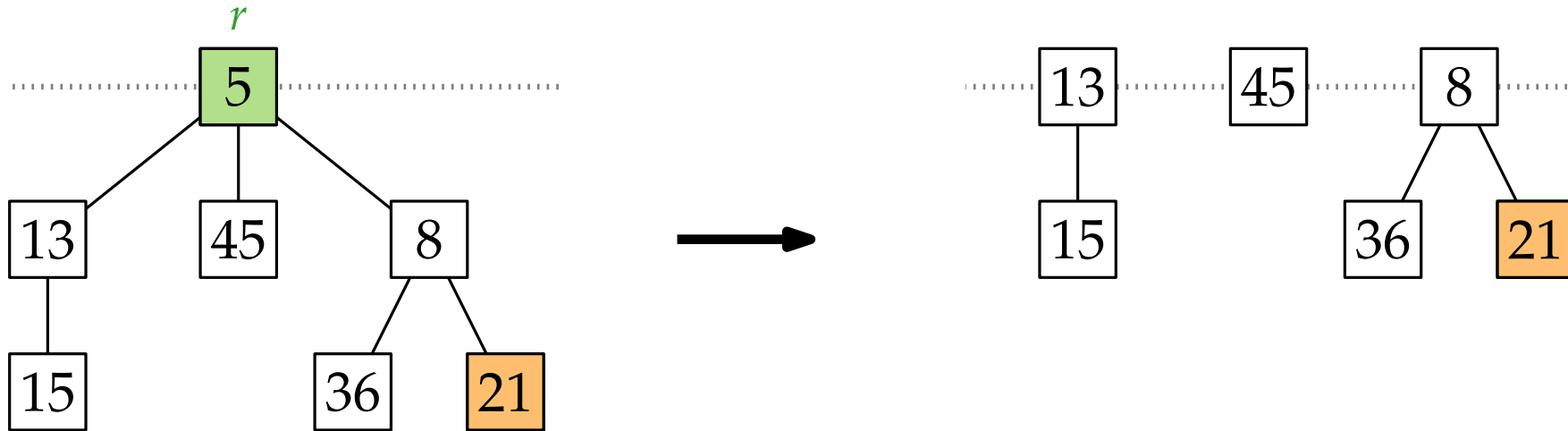
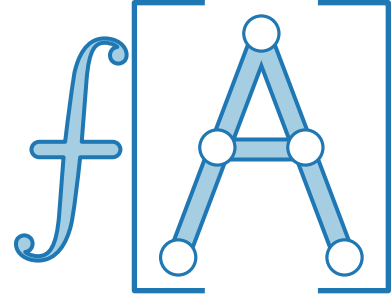
$\text{REMOVE}(r)$ **Spalten** von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

- Sei r Wurzel in Wurzelliste
- Lösche r aus Wurzelliste und füge Kinder von r in Wurzelliste ein

Laufzeit.

Baummanipulation: REMOVE(r)



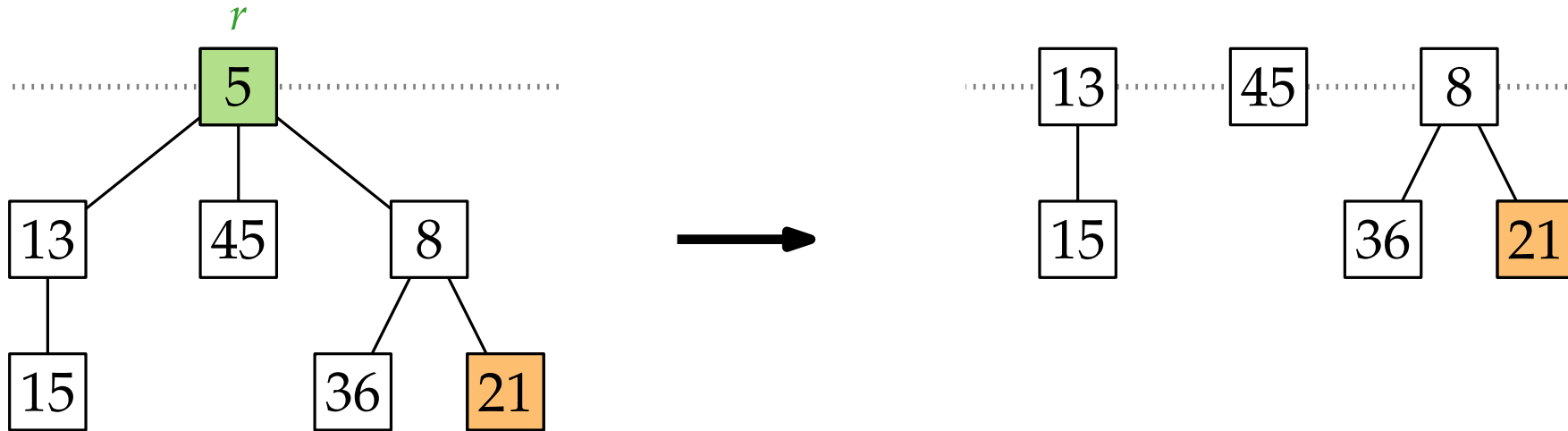
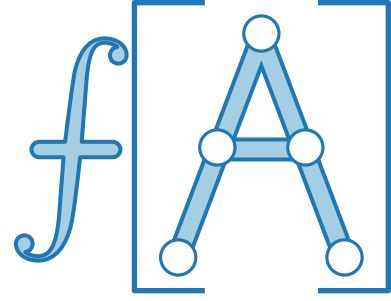
REMOVE(r) Spalten von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

- Sei r Wurzel in Wurzelliste
- Lösche r aus Wurzelliste und füge Kinder von r in Wurzelliste ein
 - ➔ Müssen nur 4 Zeiger umhängen

Laufzeit.

Baummanipulation: REMOVE(r)



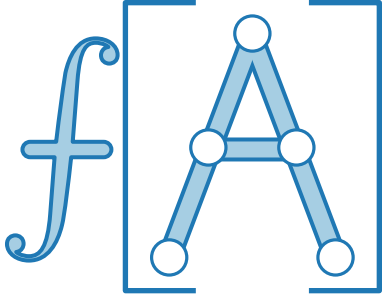
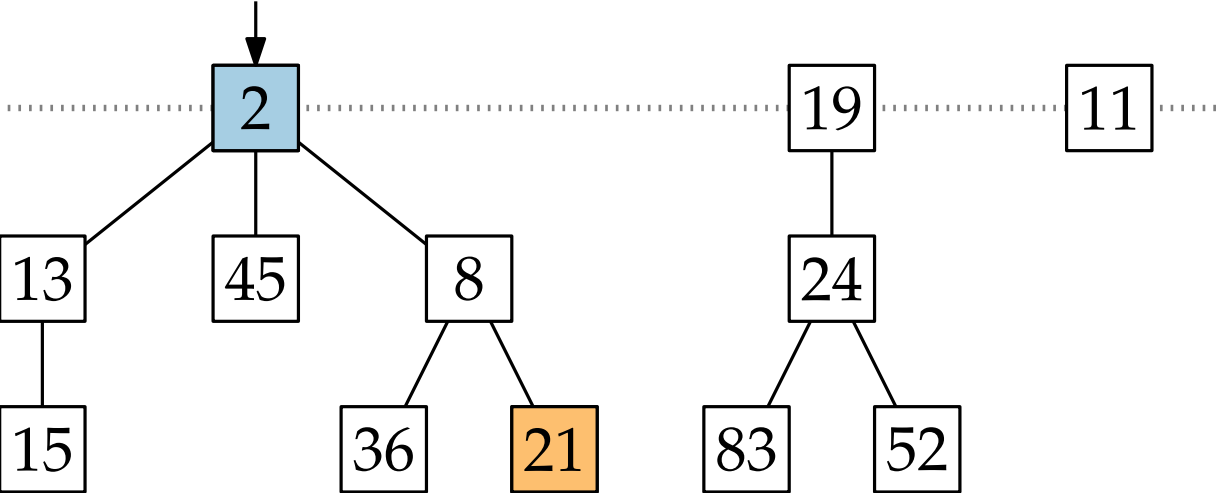
REMOVE(r) **Spalten** von Bäumen an der Wurzel.

Entfernt die Wurzel eines Baums und fügt die Kinder der Wurzel als neue Bäume in die Wurzelliste ein.

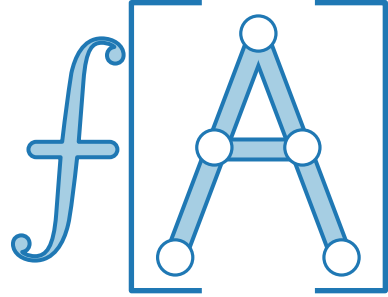
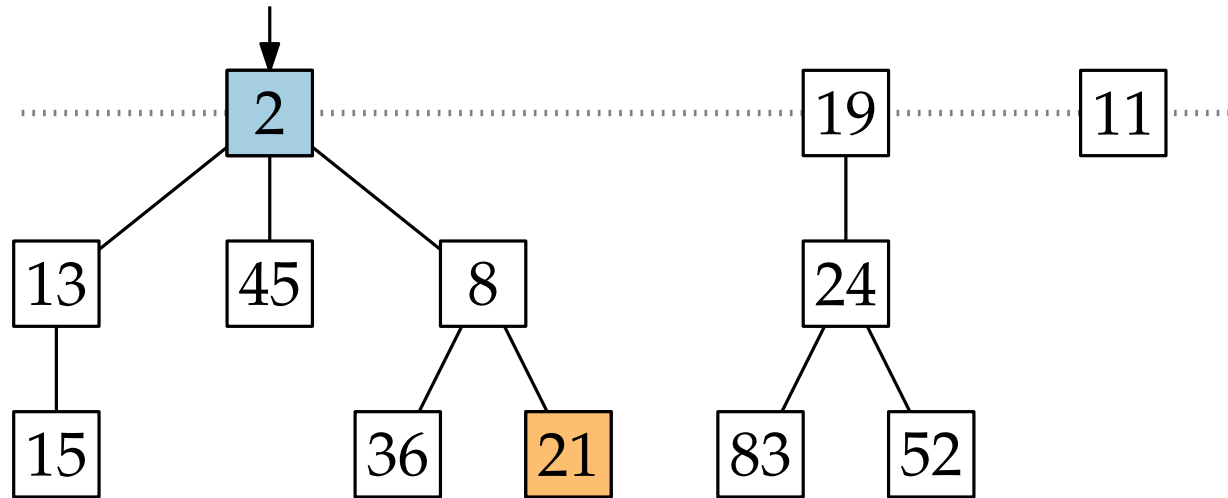
- Sei r Wurzel in Wurzelliste
- Lösche r aus Wurzelliste und füge Kinder von r in Wurzelliste ein
 - ➔ Müssen nur 4 Zeiger umhängen

Laufzeit. $\Theta(1)$

Operation: EXTRACTMIN()

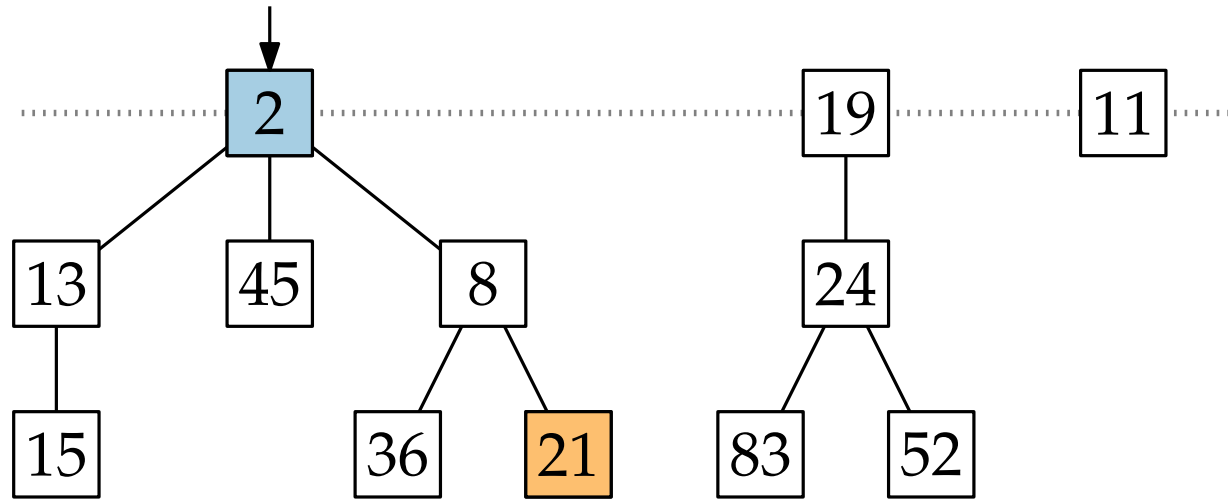


Operation: EXTRACTMIN()



Q.EXTRACTMIN()

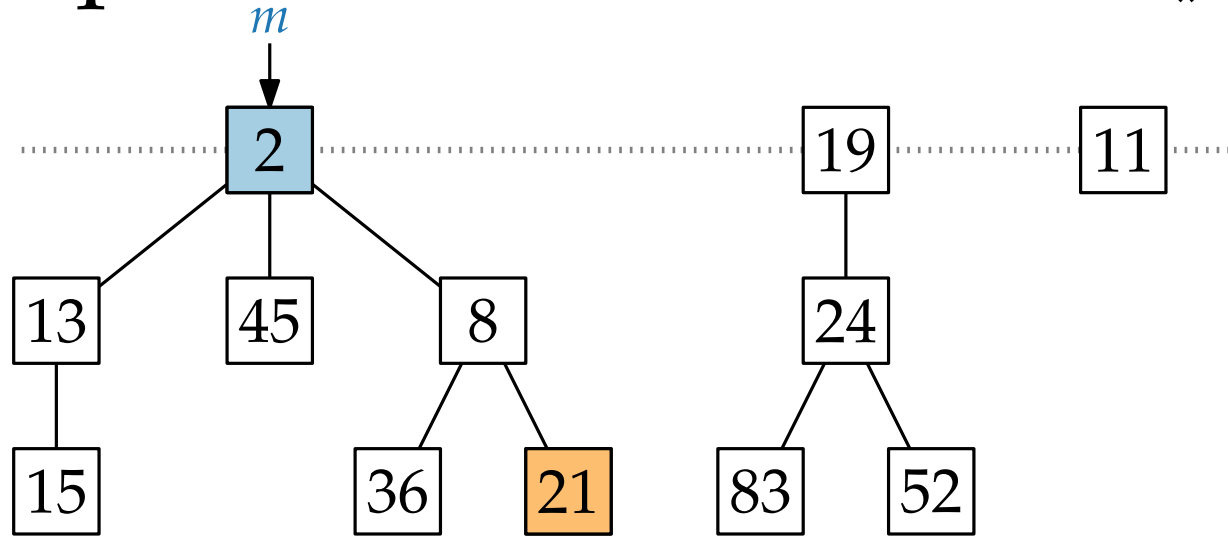
Operation: EXTRACTMIN()



$Q.EXTRACTMIN()$

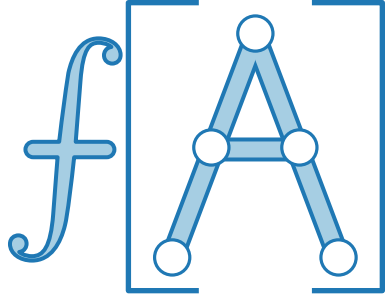
$m = Q.MIN()$

Operation: EXTRACTMIN()

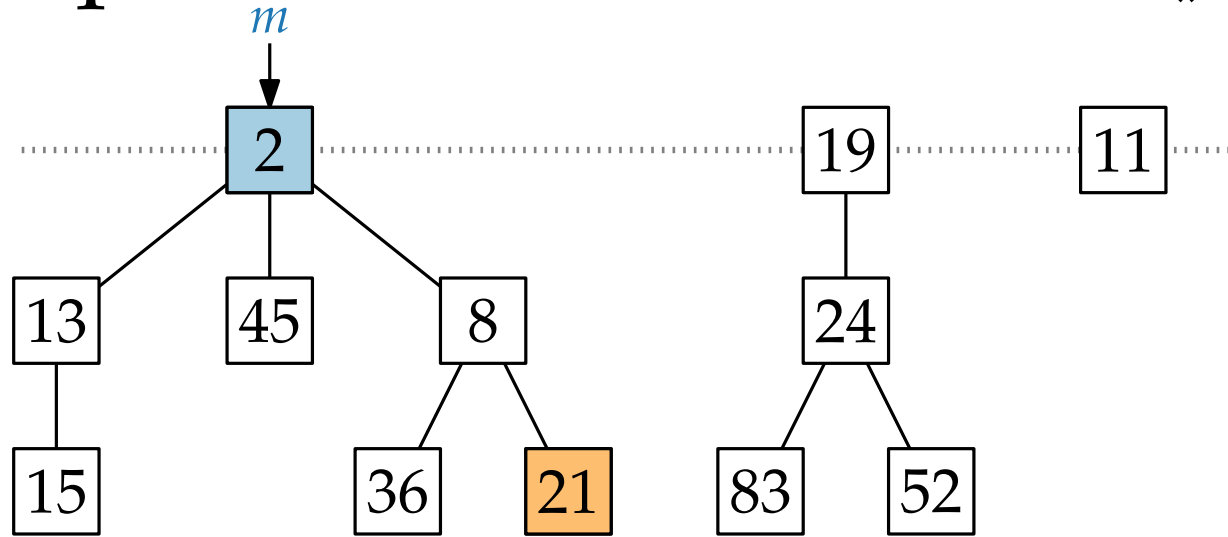


$Q.EXTRACTMIN()$

$m = Q.MIN()$



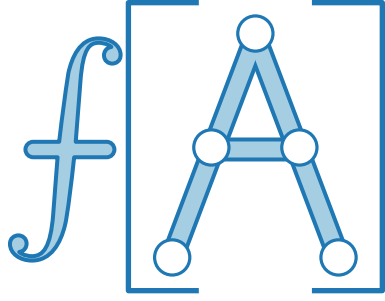
Operation: EXTRACTMIN()



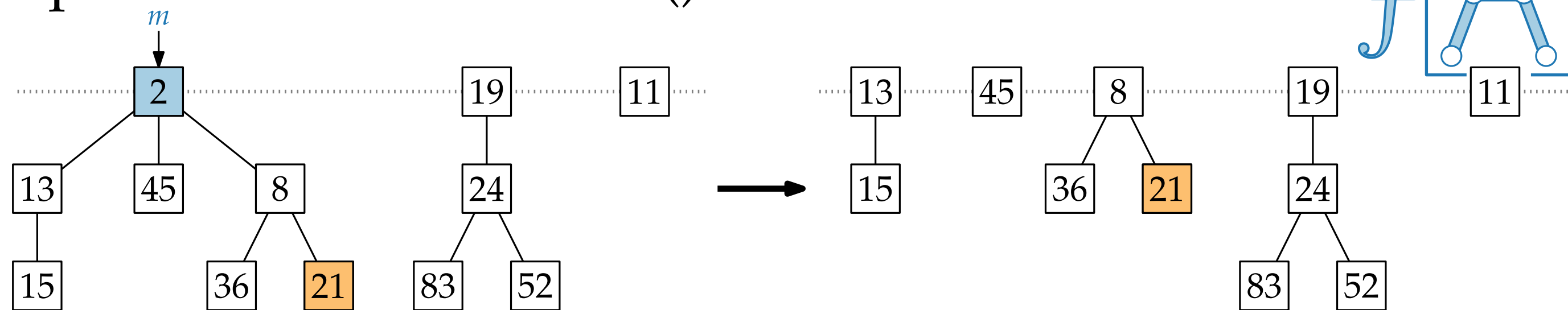
Q.EXTRACTMIN()

$m = Q.MIN()$

Q.REMOVE(m)



Operation: EXTRACTMIN()

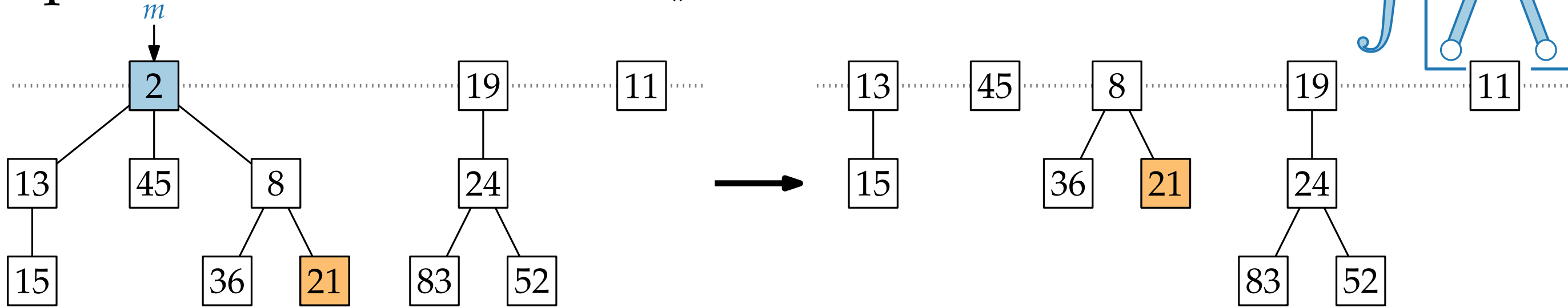


Q.EXTRACTMIN()

$m = Q.MIN()$

Q.REMOVE(m)

Operation: EXTRACTMIN()



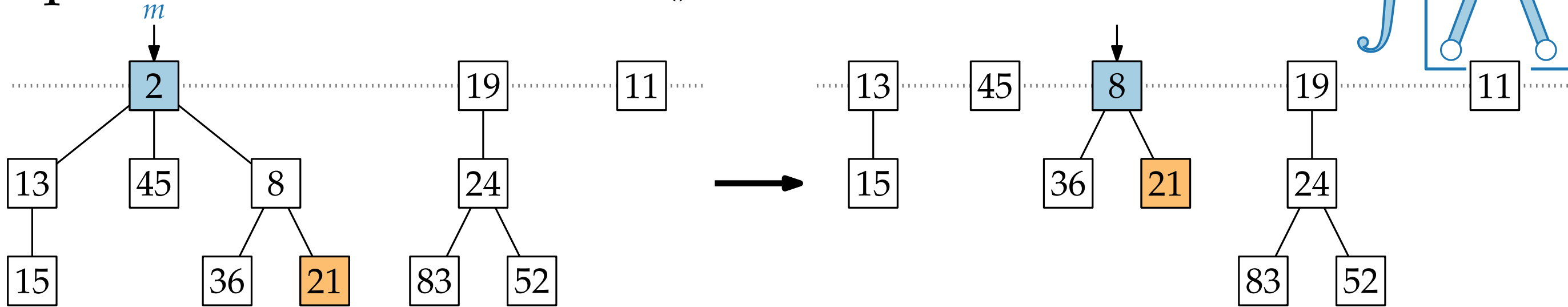
$Q.EXTRACTMIN()$

$m = Q.MIN()$

$Q.REMOVE(m)$

Finde neues Minimum, aktualisiere Pointer

Operation: EXTRACTMIN()



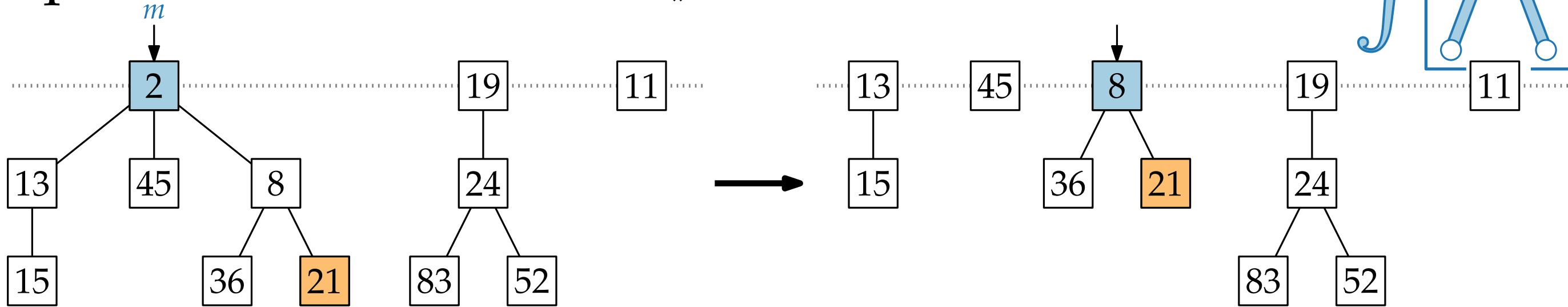
$Q.EXTRACTMIN()$

$m = Q.MIN()$

$Q.REMOVE(m)$

Finde neues Minimum, aktualisiere Pointer

Operation: EXTRACTMIN()



Q.EXTRACTMIN()

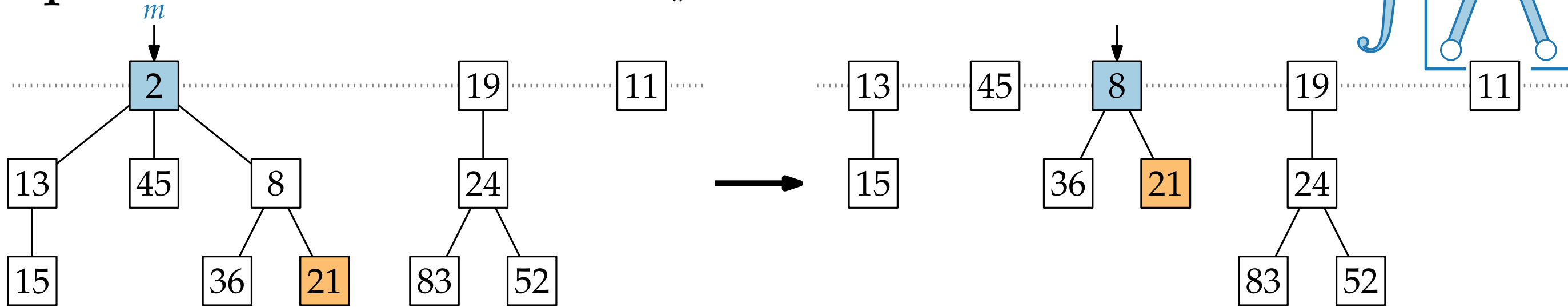
$m = Q.MIN()$

Q.REMOVE(m)

Finde neues Minimum, aktualisiere Pointer

return m

Operation: EXTRACTMIN()



Q.EXTRACTMIN()

$m = Q.MIN()$

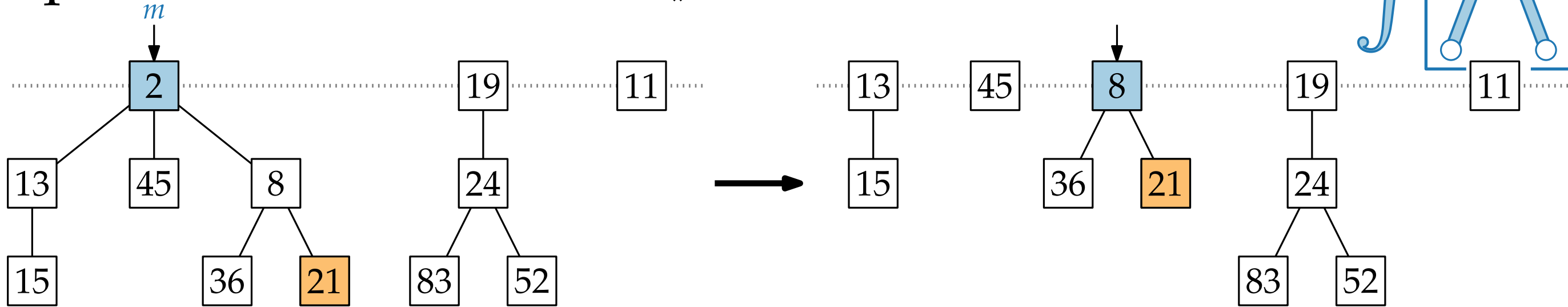
Q.REMOVE(m)

Q.CLEANUP()

Finde neues Minimum, aktualisiere Pointer

return m

Operation: EXTRACTMIN()



Q.EXTRACTMIN()

$m = Q.MIN()$

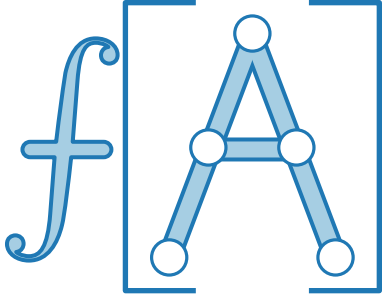
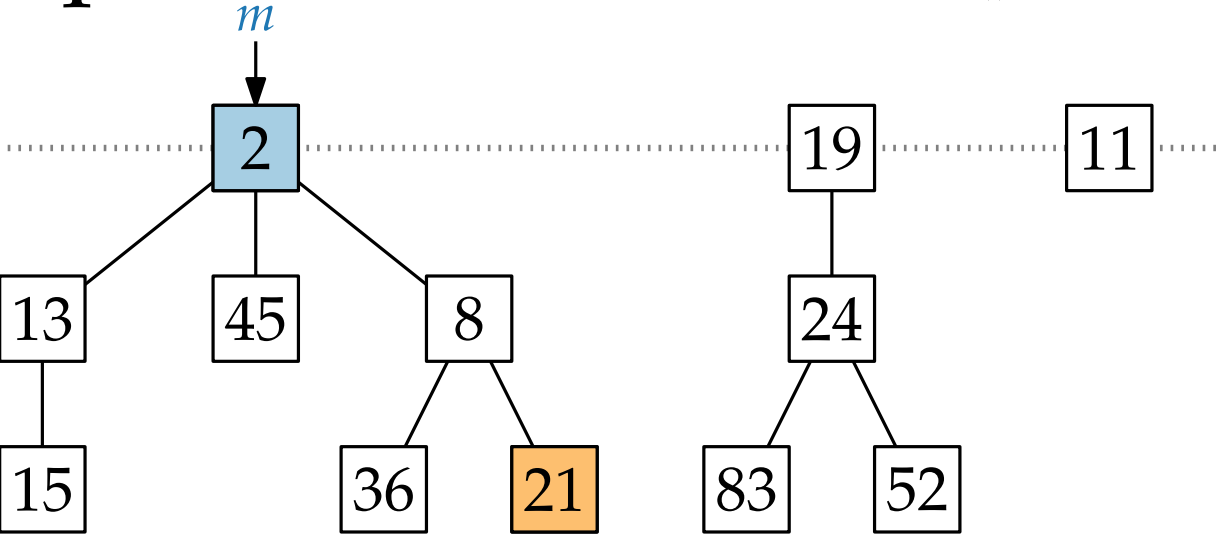
Q.REMOVE(m)

Q.CLEANUP()?

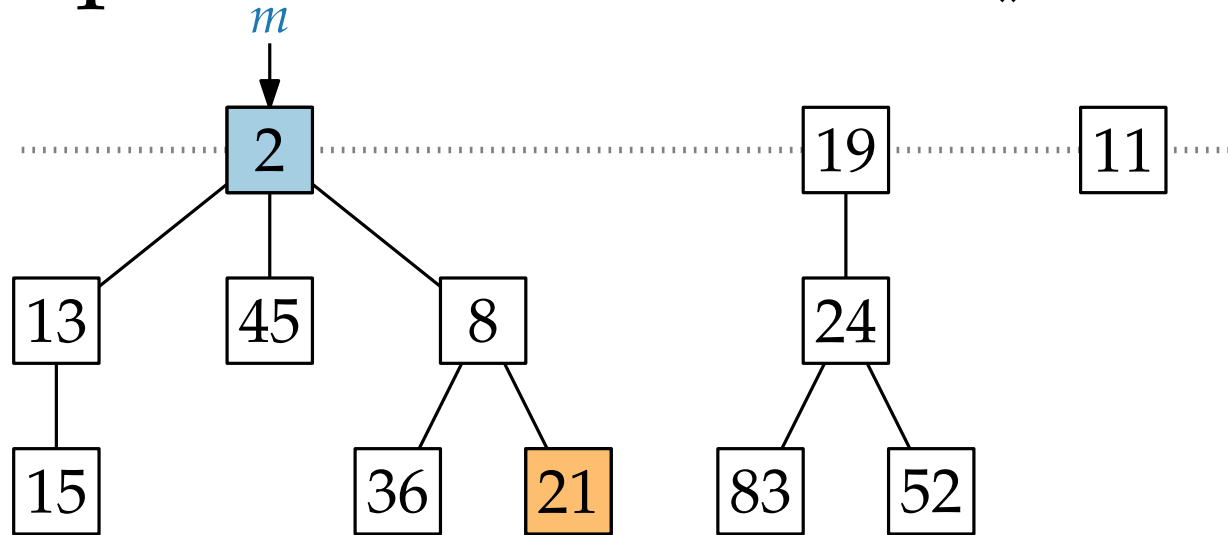
Finde neues Minimum, aktualisiere Pointer

return m

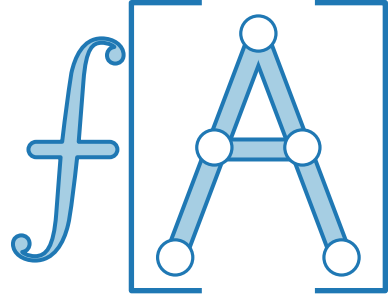
Operation: CLEANUP()



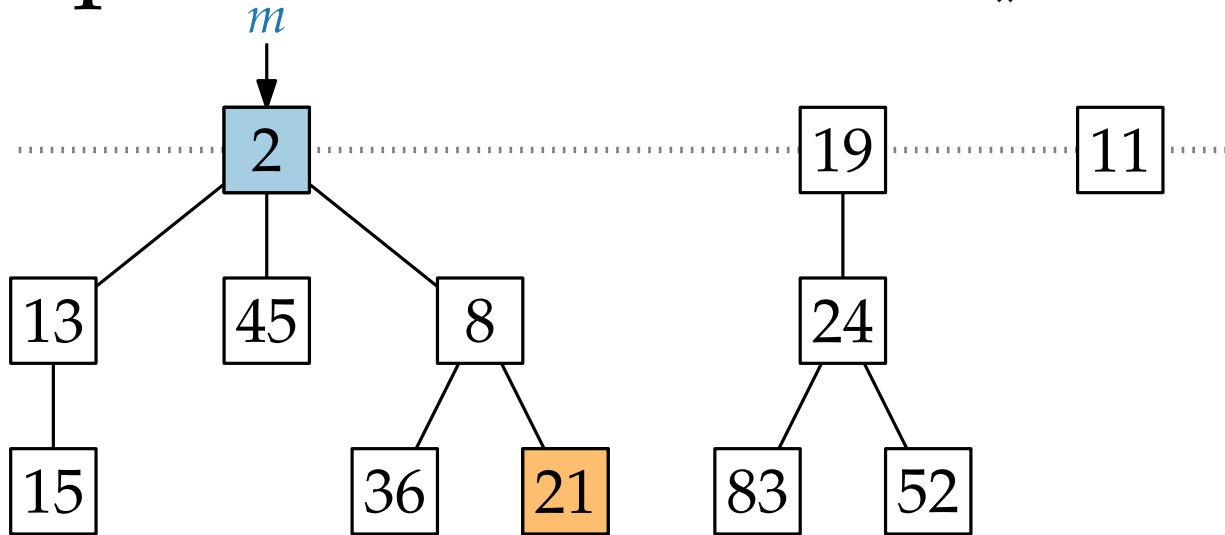
Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

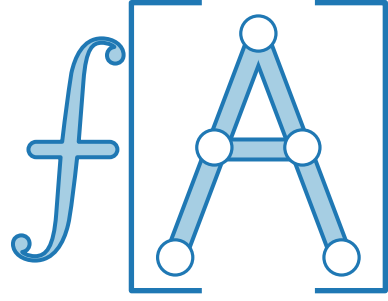


Operation: CLEANUP()

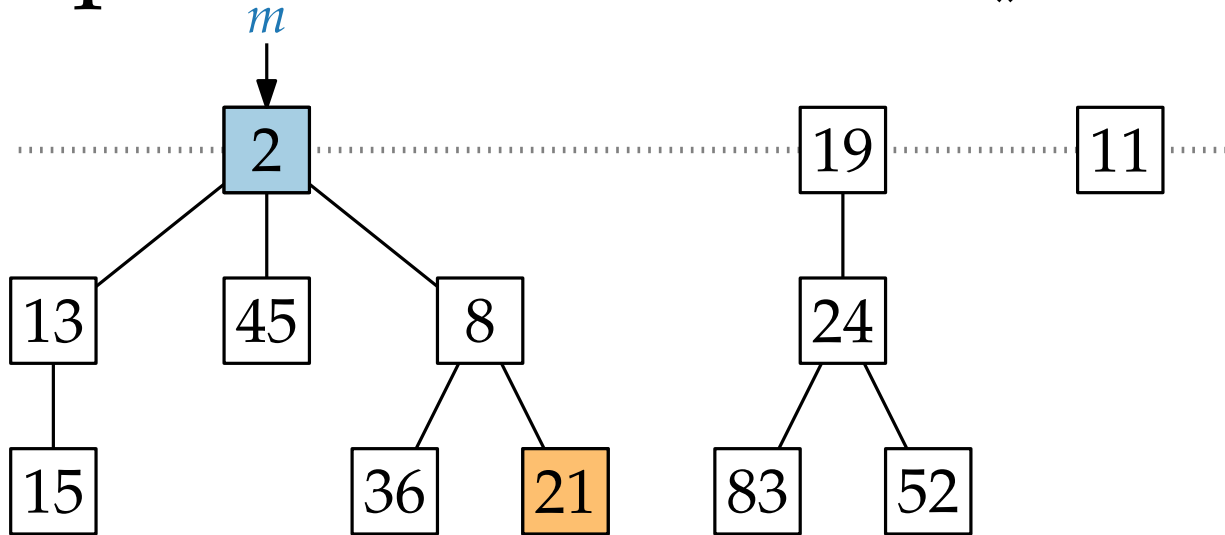


normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad



Operation: CLEANUP()

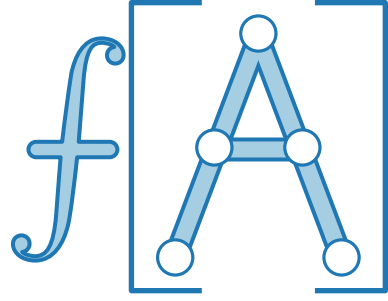


normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

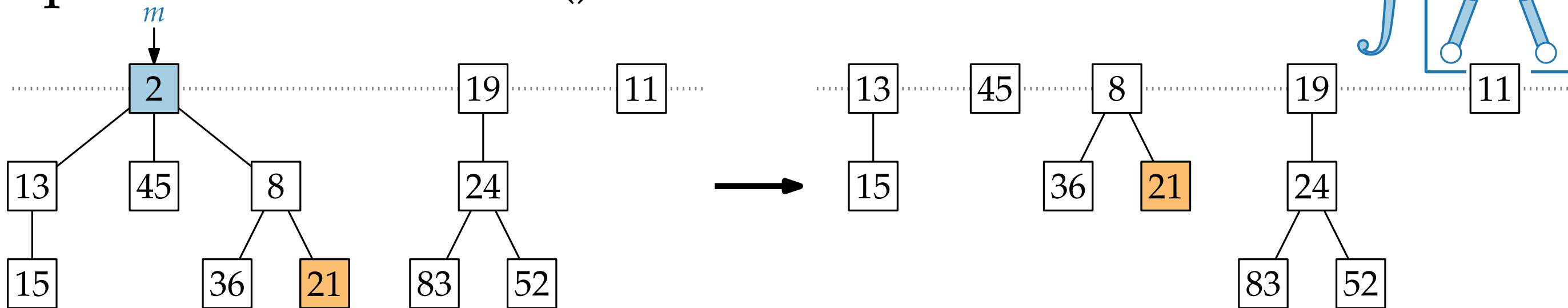
Speicher Array **degArray** für Wurzelknoten nach Grad

degArray

0	11
1	19
2	
3	2



Operation: CLEANUP()



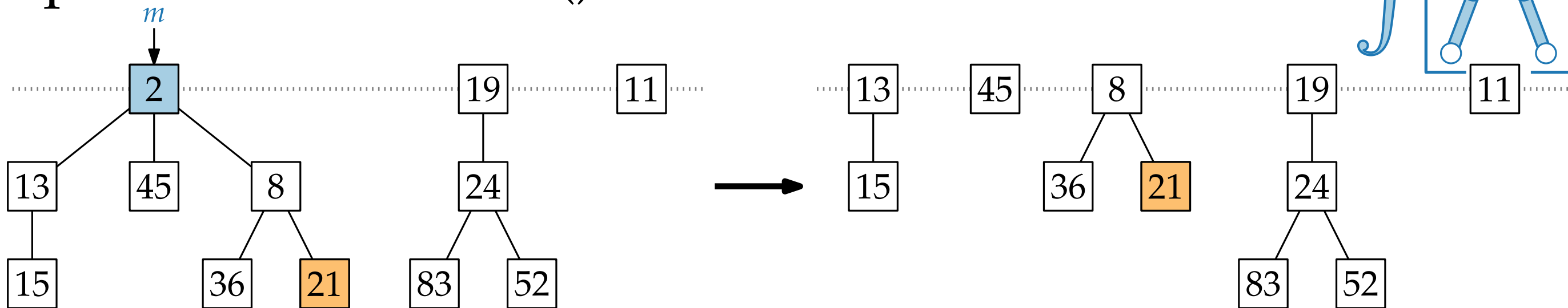
normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

degArray

0	11
1	19
2	
3	2

Operation: CLEANUP()



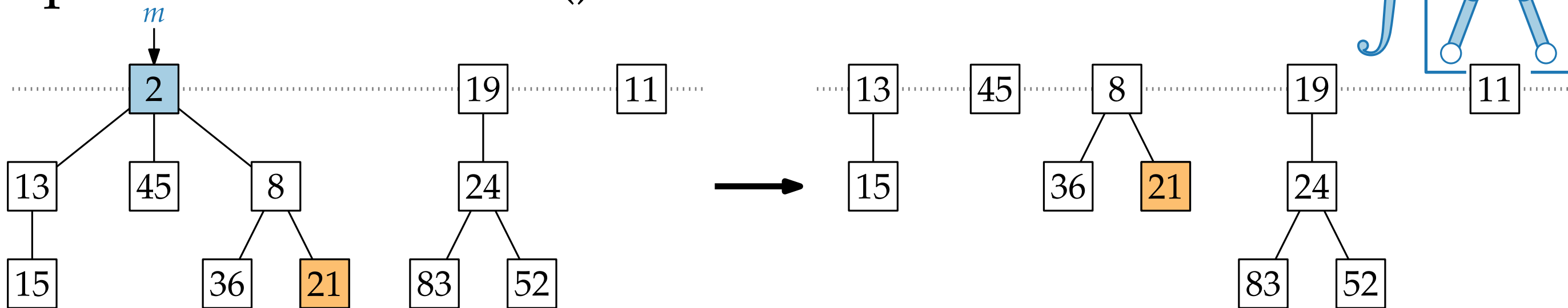
normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

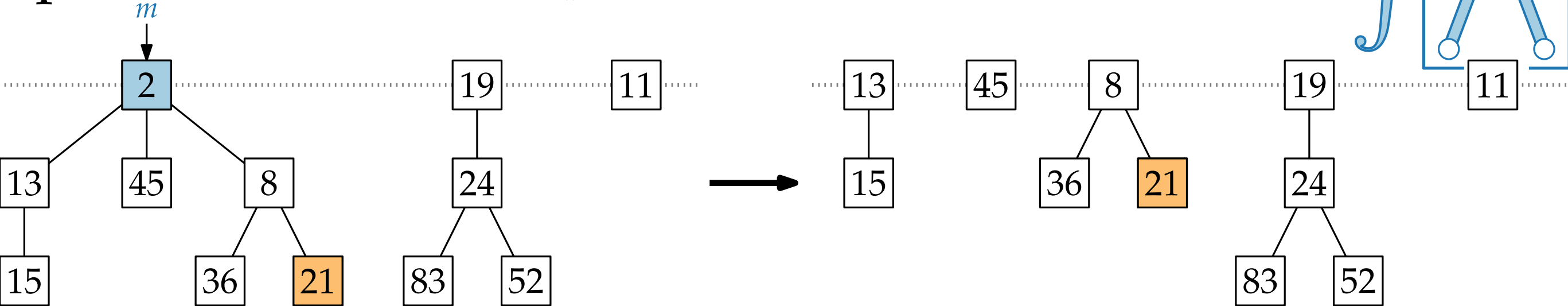
Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

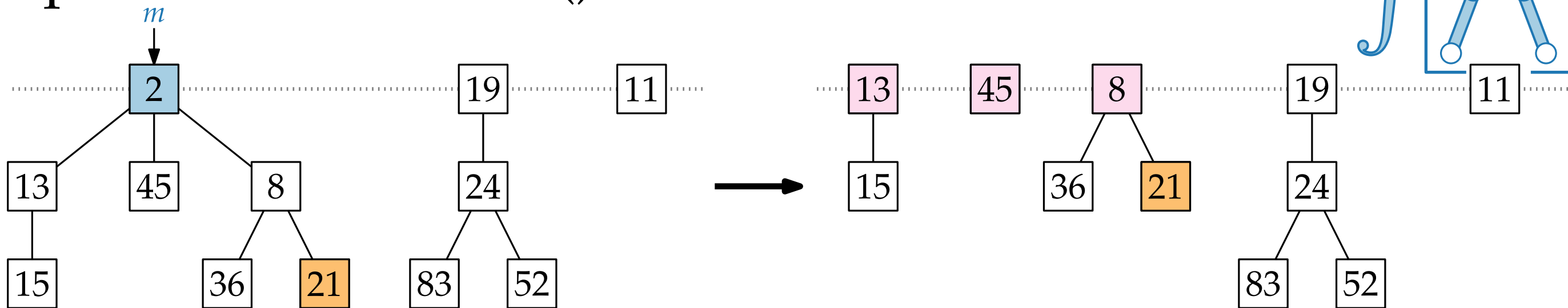
R = neue Knoten in Wurzelliste

(seit letztem CLEANUP)

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

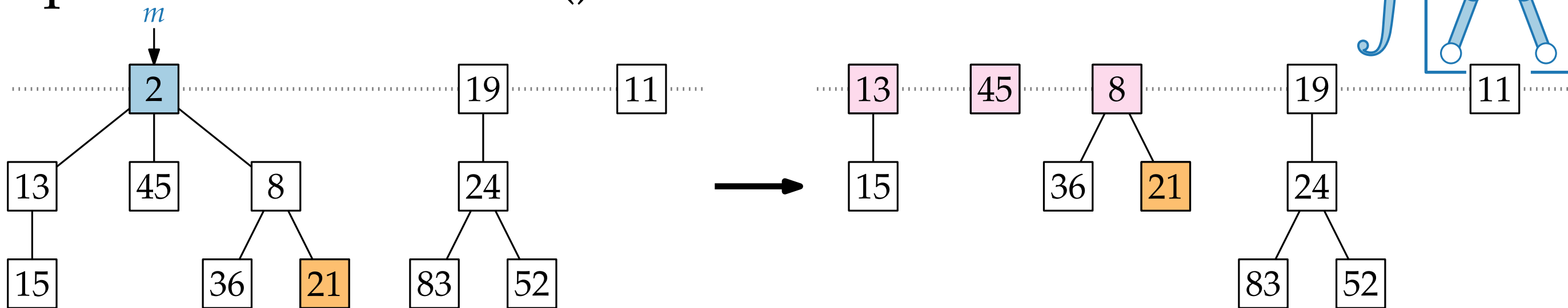
R = neue Knoten in Wurzelliste

(seit letztem CLEANUP)

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

R = neue Knoten in Wurzelliste

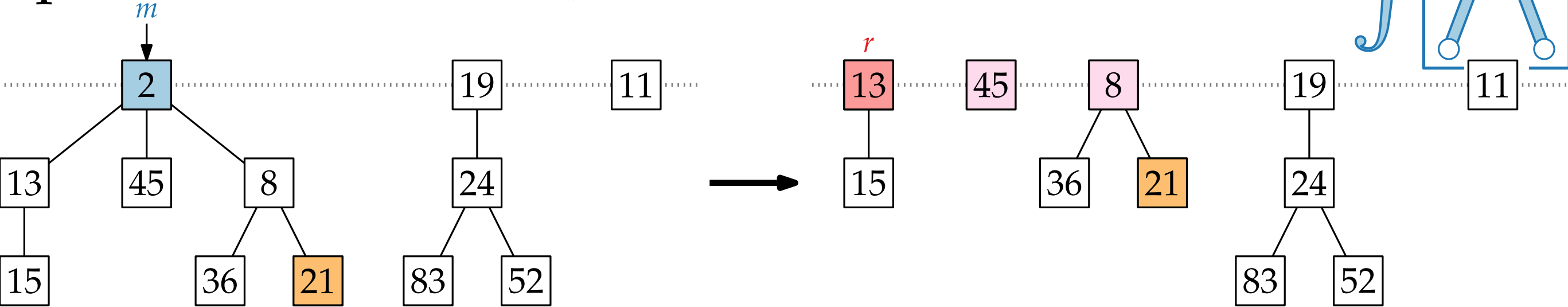
foreach *r* ∈ *R* do

(seit letztem CLEANUP)

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

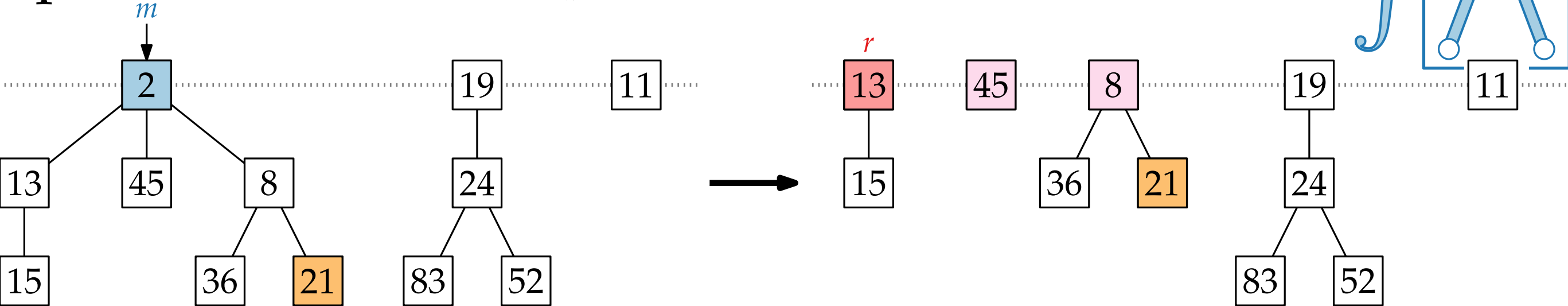
Speicher Array **degArray** für Wurzelknoten nach Grad

```
Q.CLEANUP()  
  R = neue Knoten in Wurzelliste  
  foreach r ∈ R do  
    (seit letztem CLEANUP)
```

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

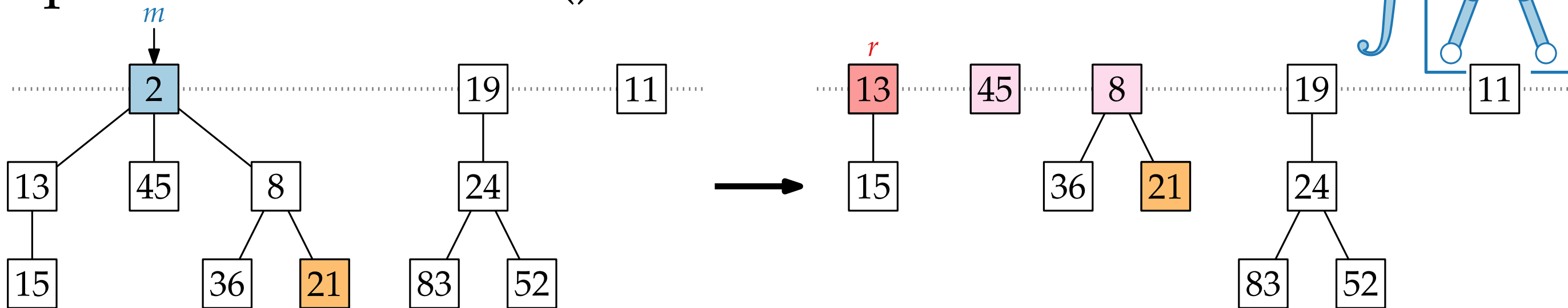
```
Q.CLEANUP()  
  R = neue Knoten in Wurzelliste  
  foreach r ∈ R do  
    while degArray(deg(r)) ≠ nil do  
      [ ]
```

(seit letztem CLEANUP)

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

R = neue Knoten in Wurzelliste

foreach *r* ∈ *R* do

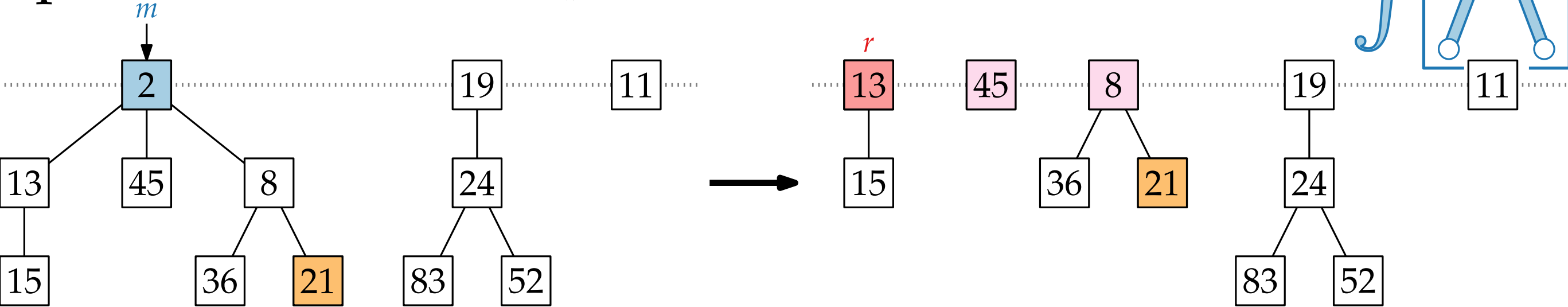
 while **degArray**(deg(*r*)) ≠ nil do

(seit letztem CLEANUP)

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

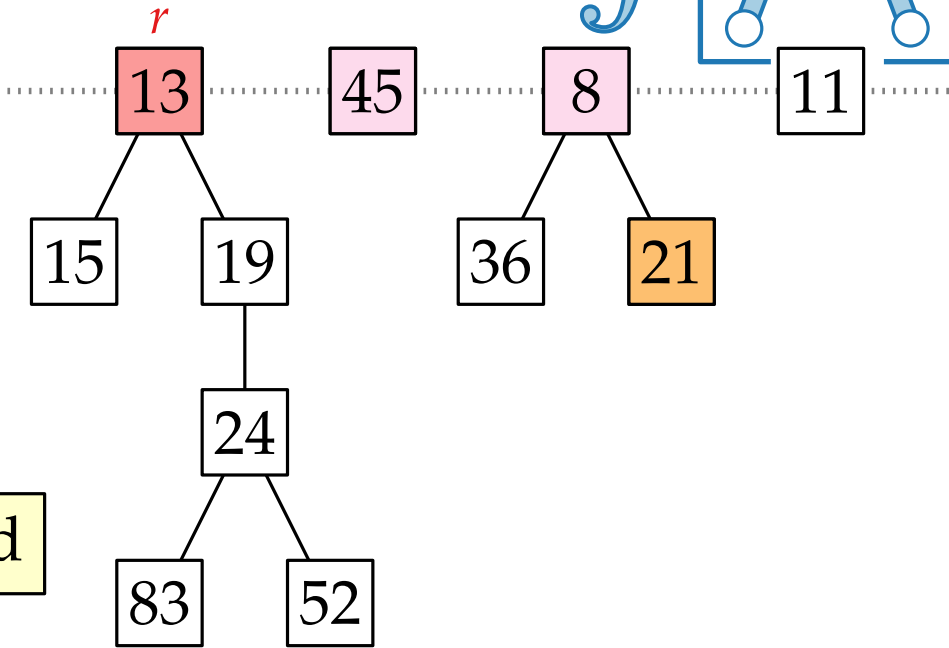
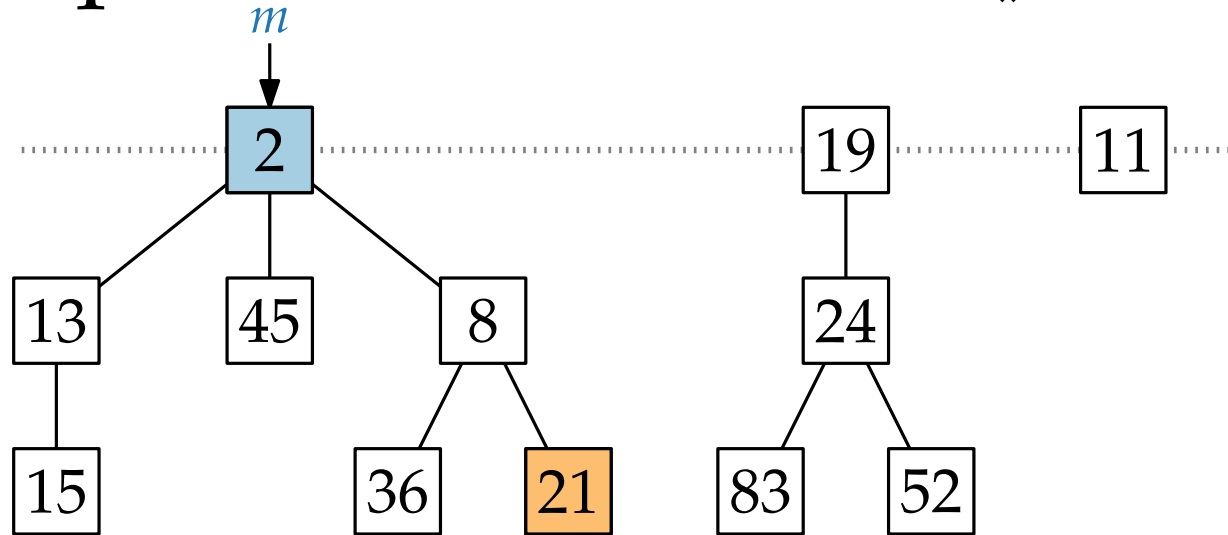
```
Q.CLEANUP()  
  R = neue Knoten in Wurzelliste  
  foreach r ∈ R do  
    while degArray(deg(r)) ≠ nil do  
      r = LINK(r, degArray(deg(r)))
```

(seit letztem CLEANUP)

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

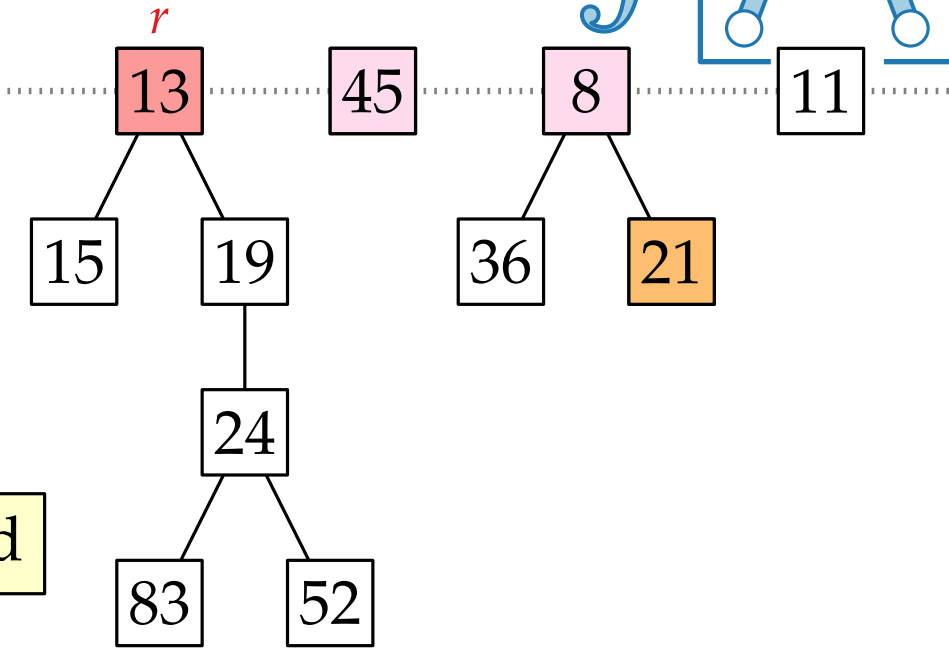
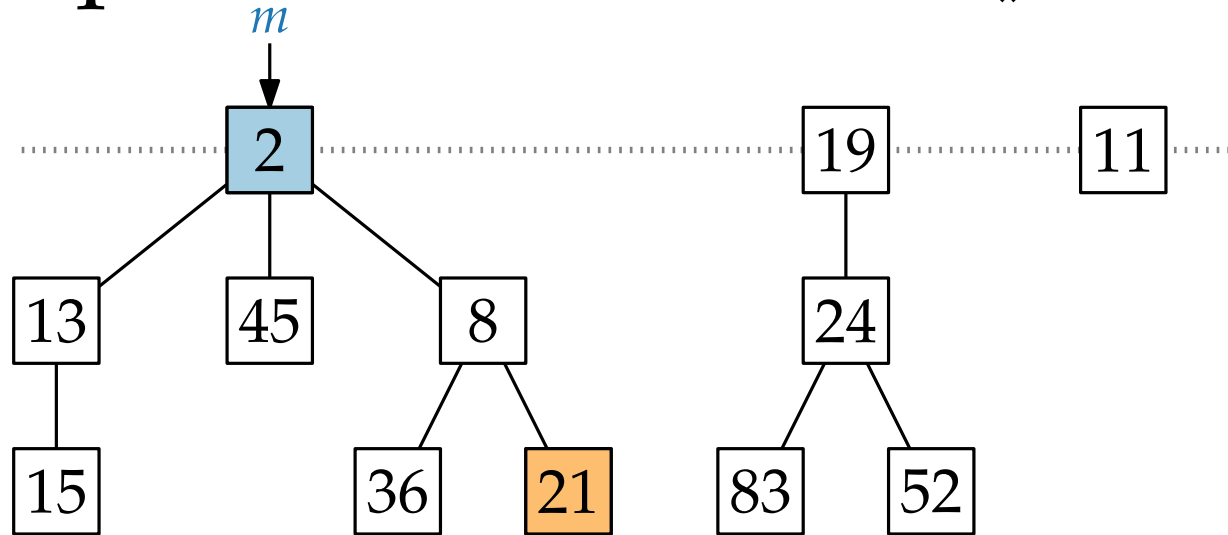
 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

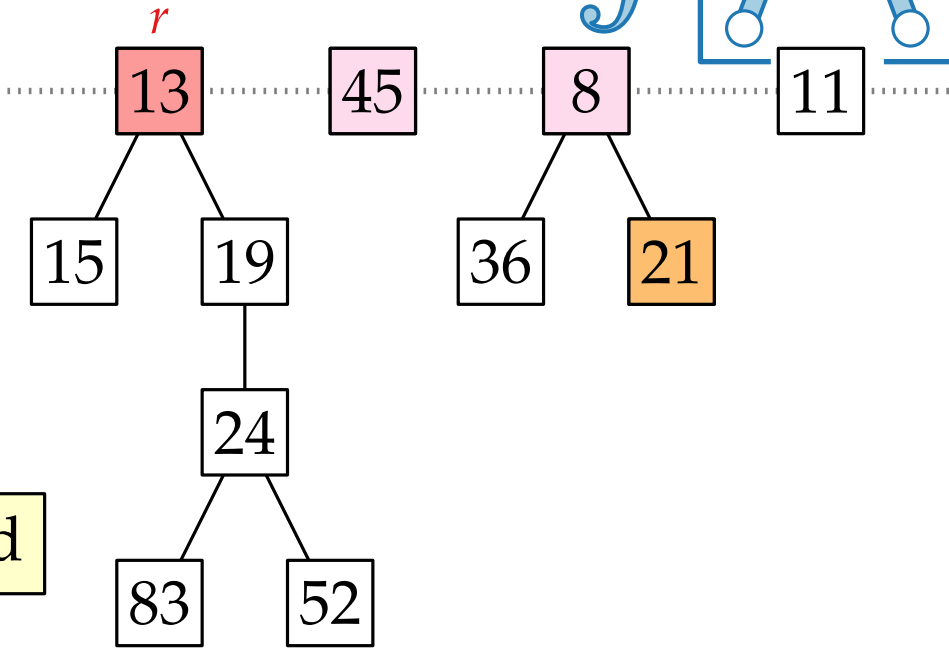
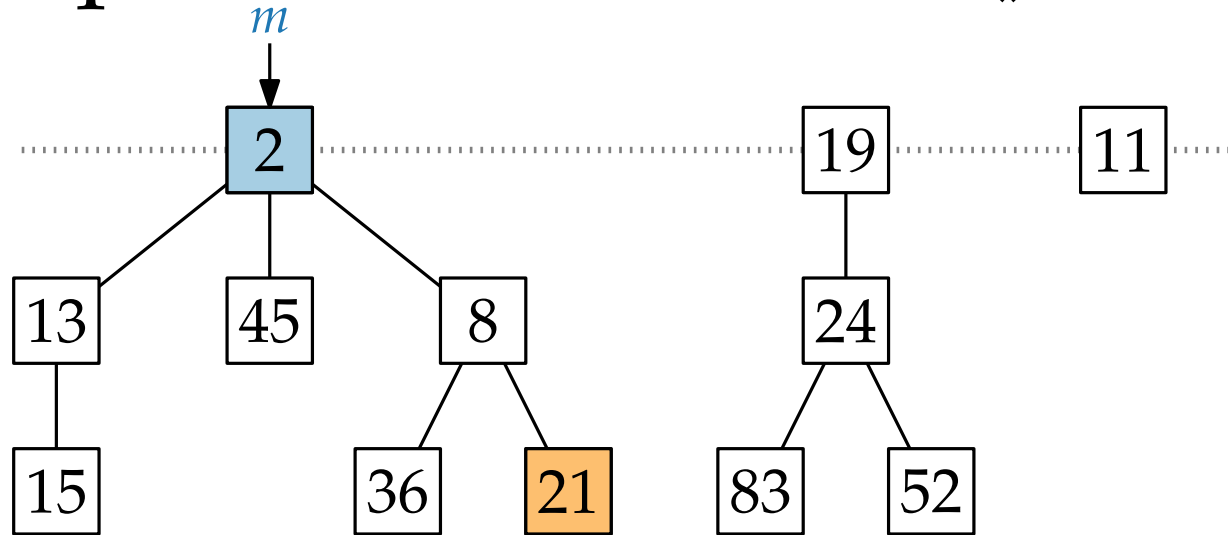
$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray(deg(r) - 1) = nil

degArray

0	11
1	19
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

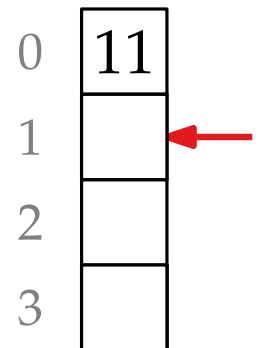
foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

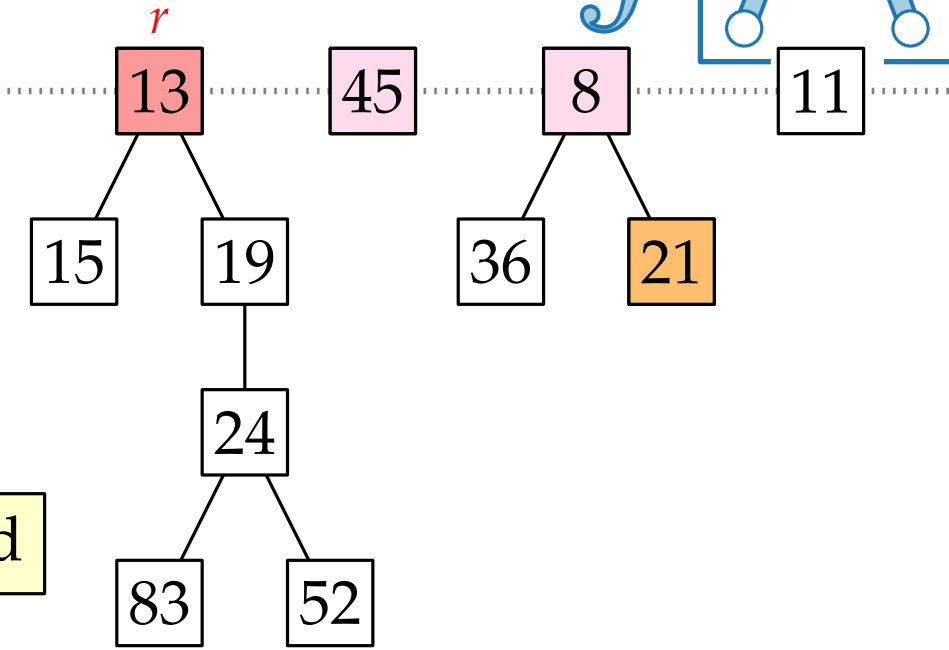
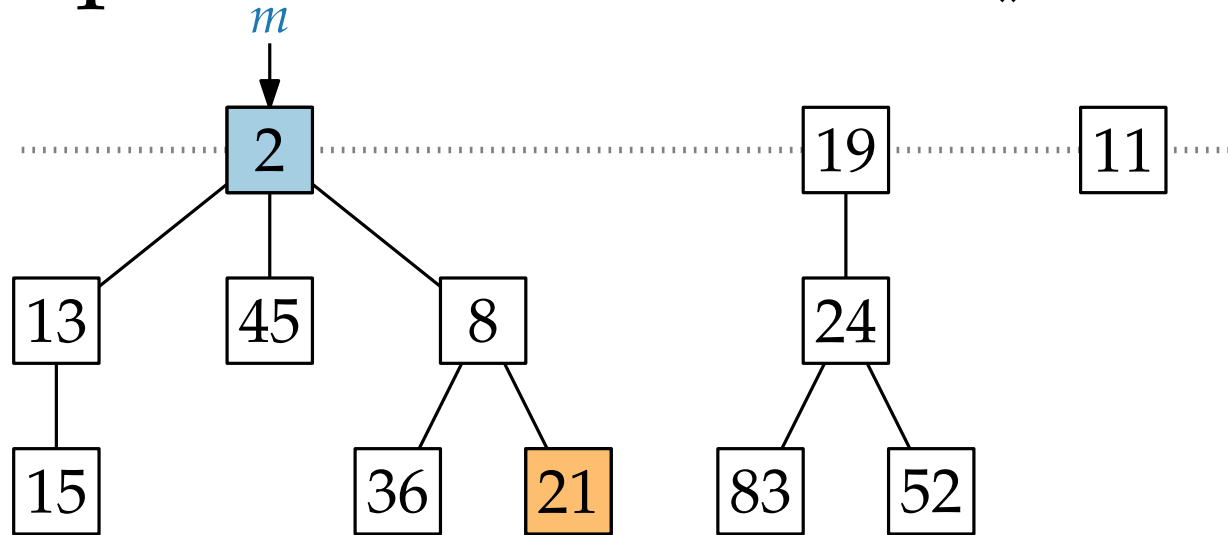
$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray(deg(r) - 1) = nil

degArray



Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

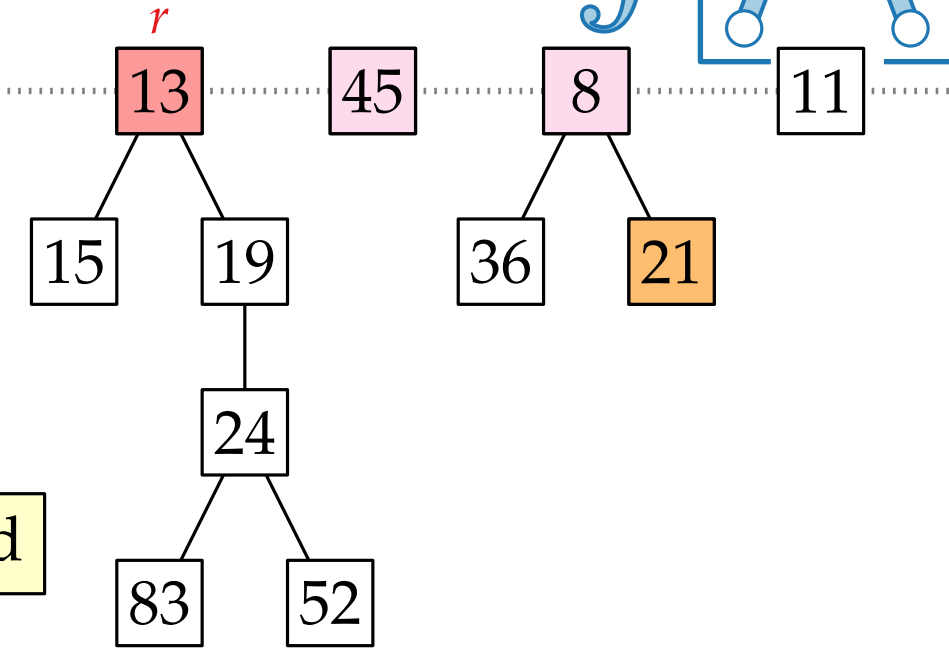
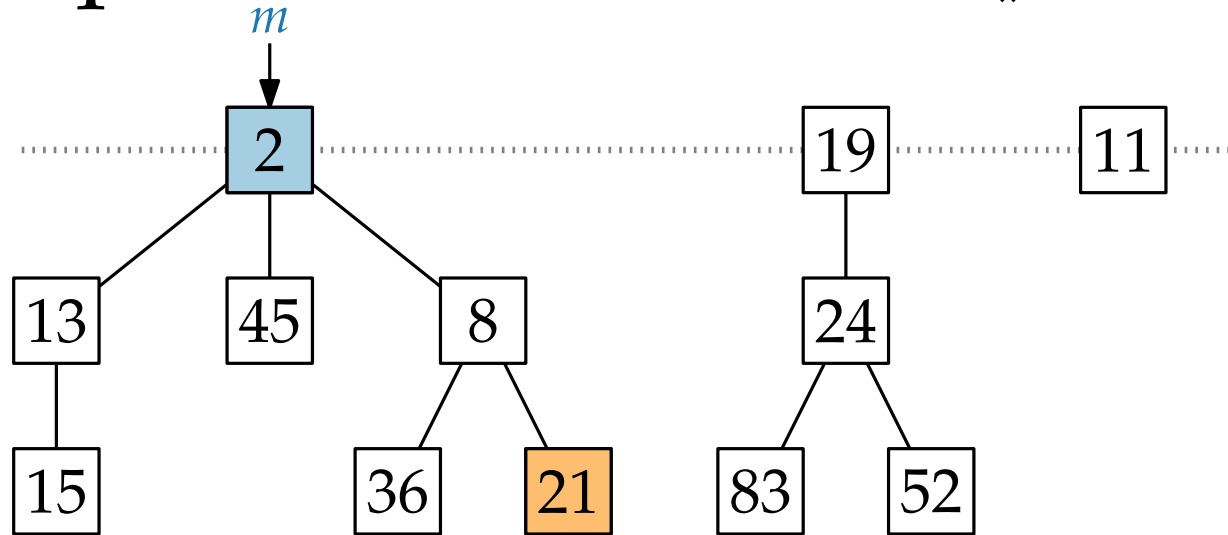
$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray(deg(r) - 1) = nil

degArray

0	11
1	
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

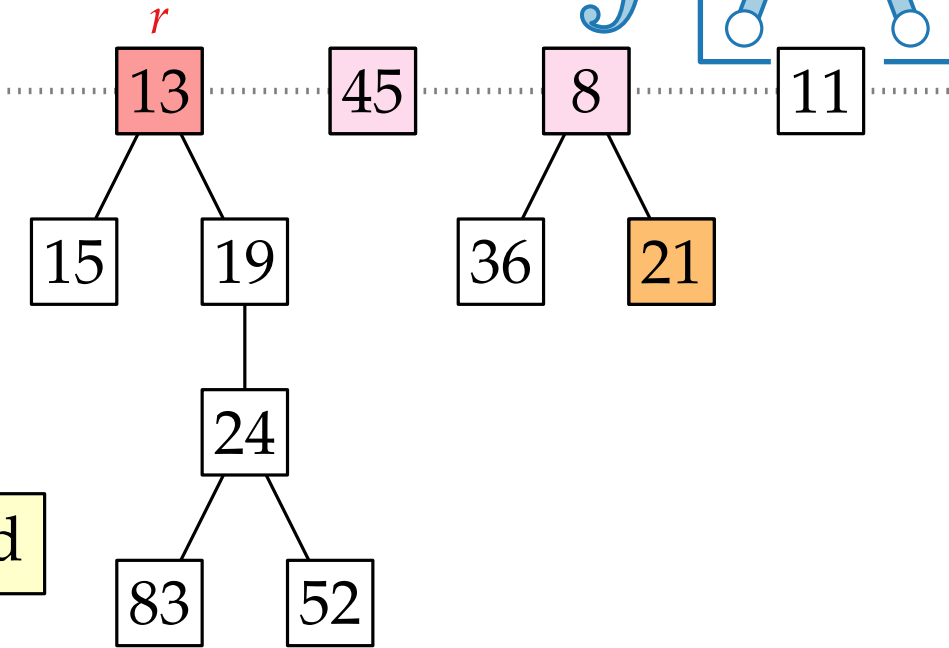
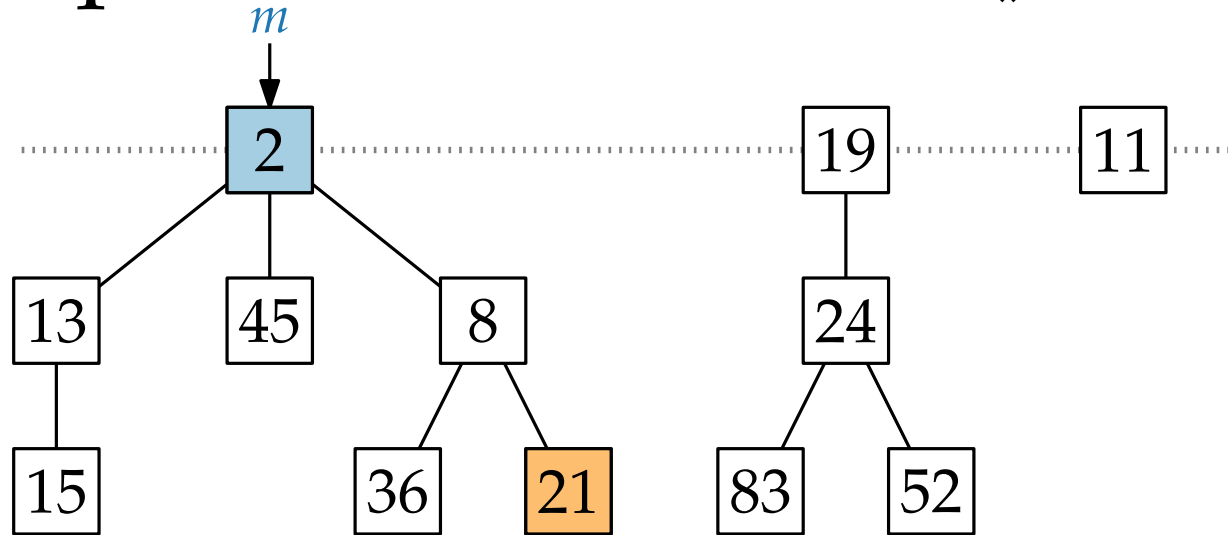
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	11
1	
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

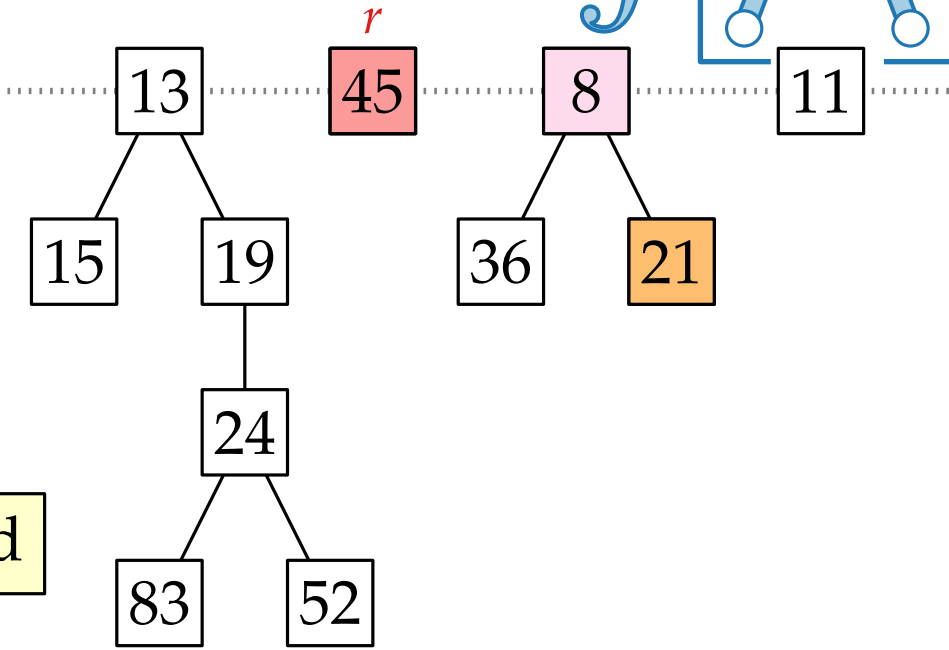
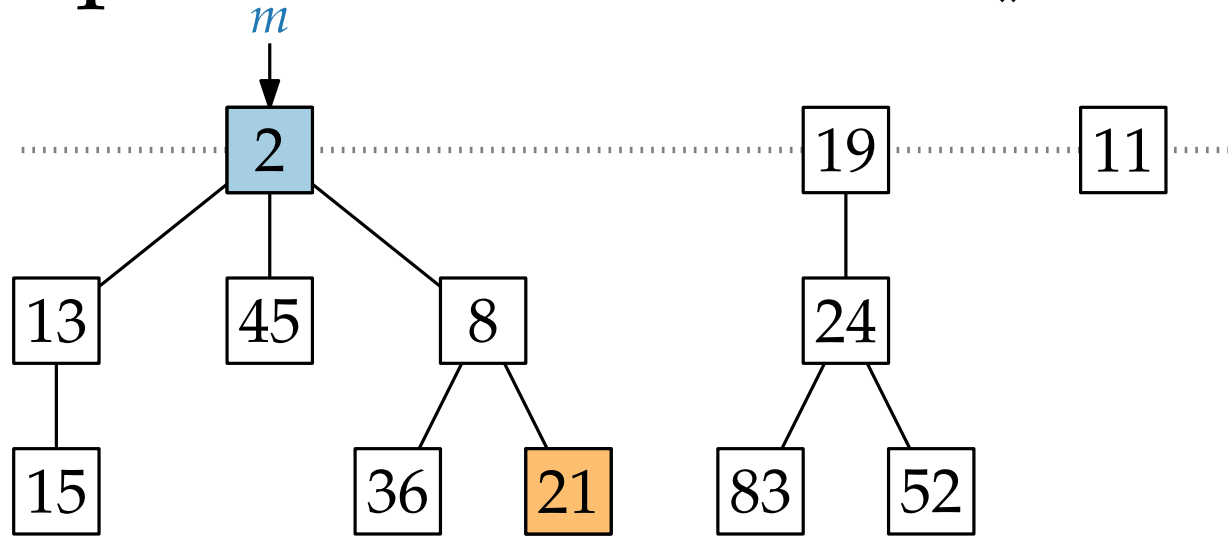
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	11
1	
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

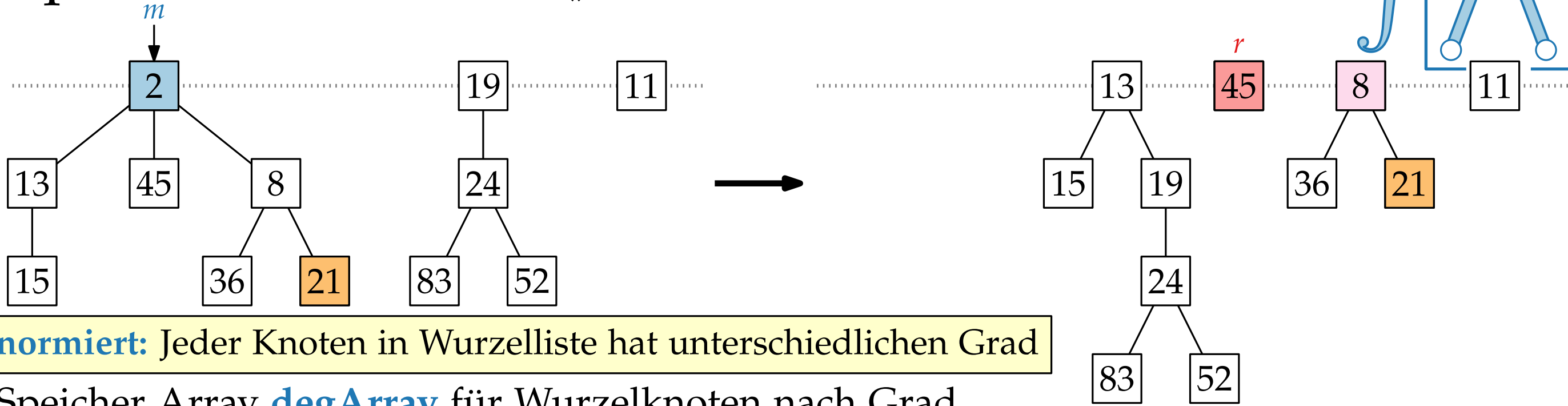
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	11
1	
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

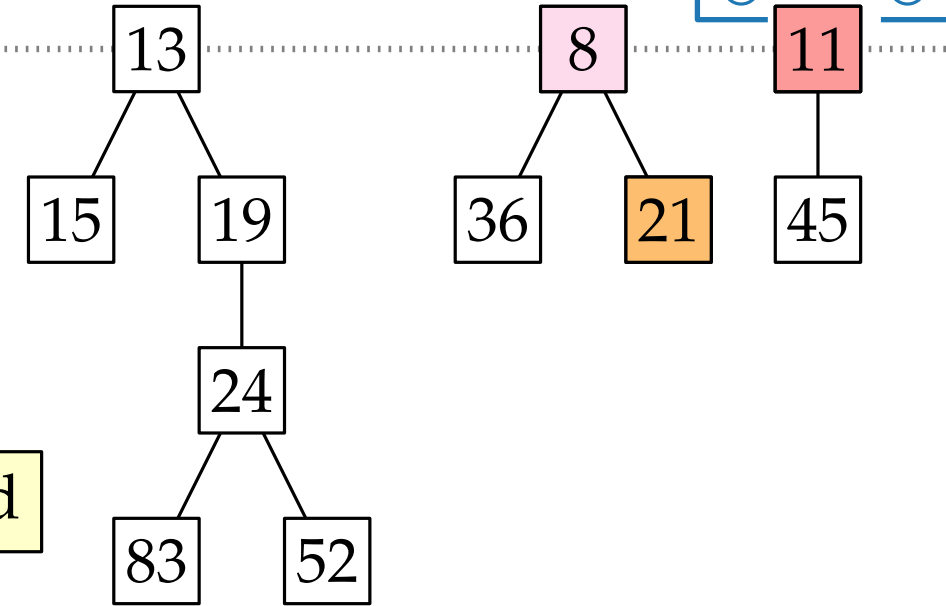
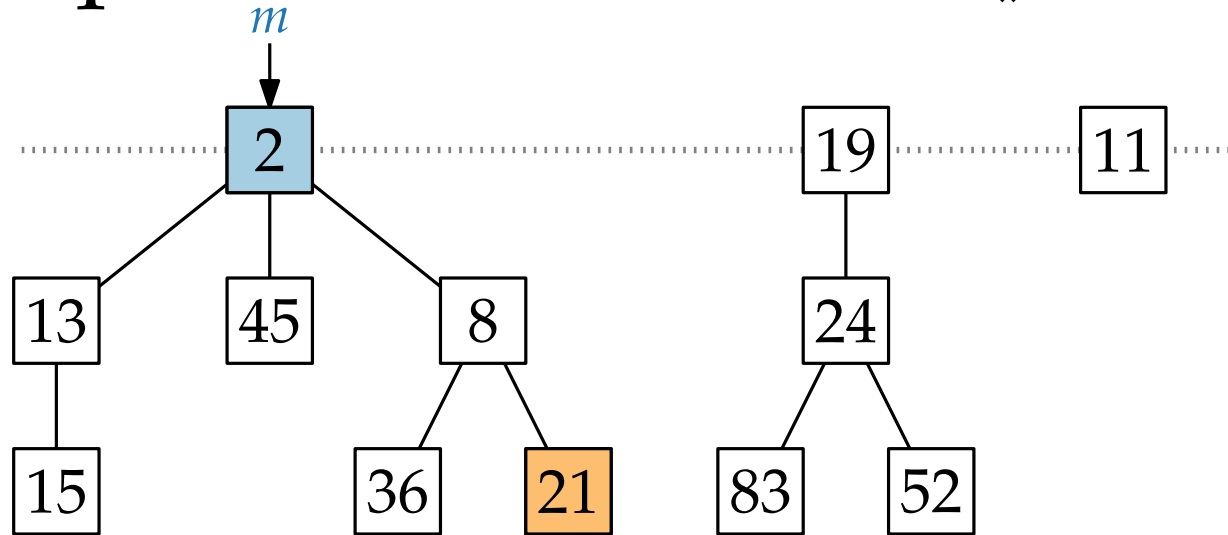
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	11
1	
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

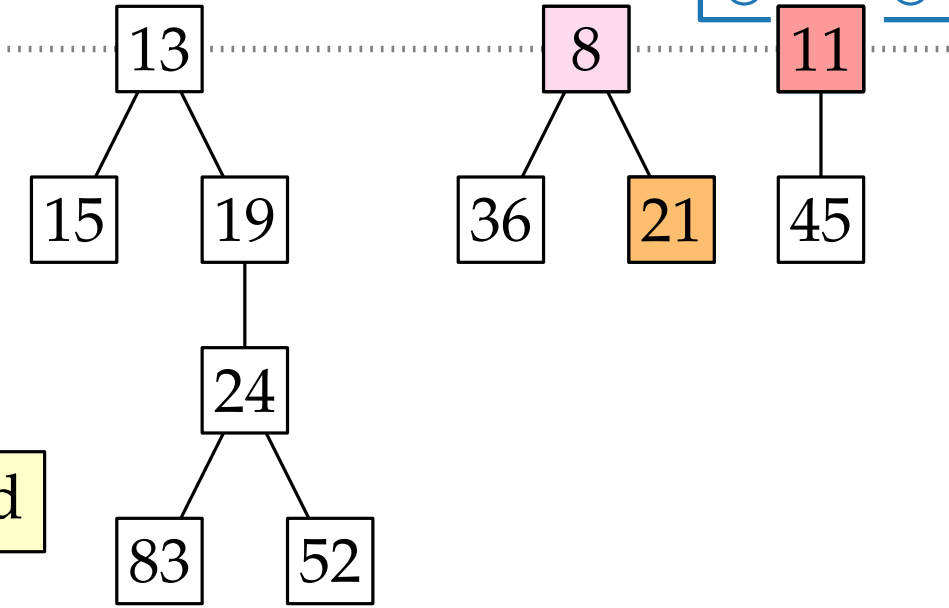
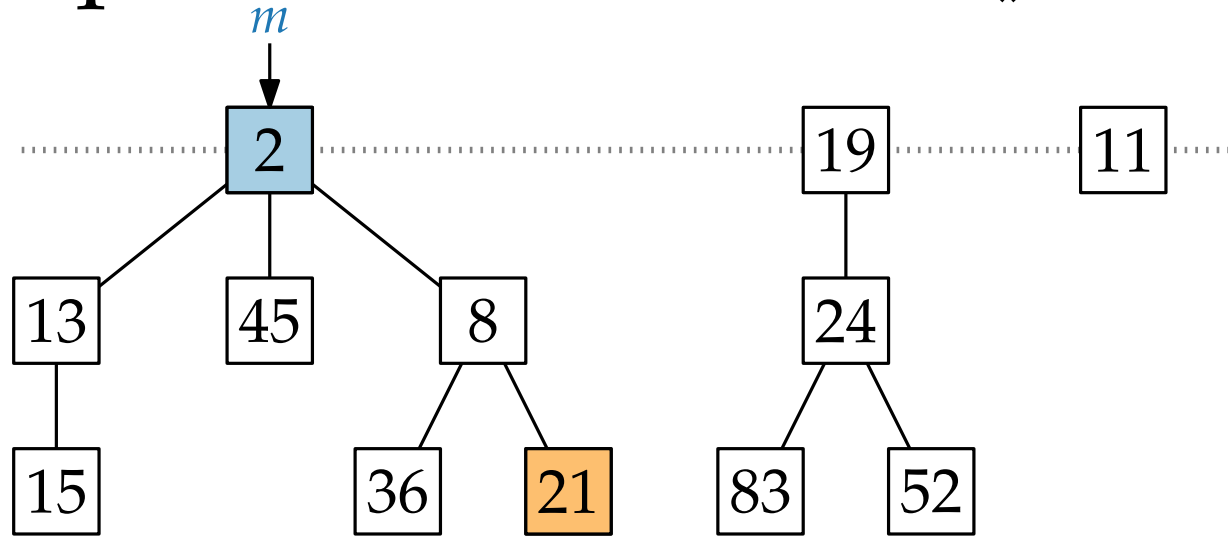
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	11
1	
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

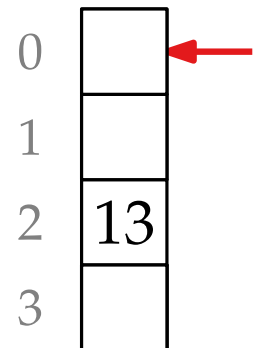
 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

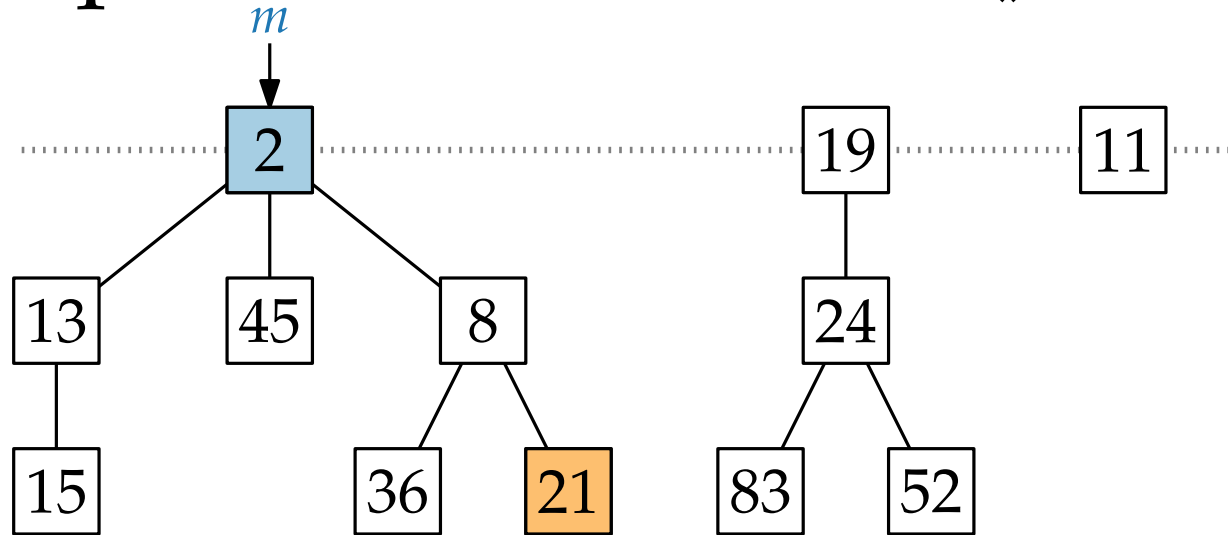
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray



Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

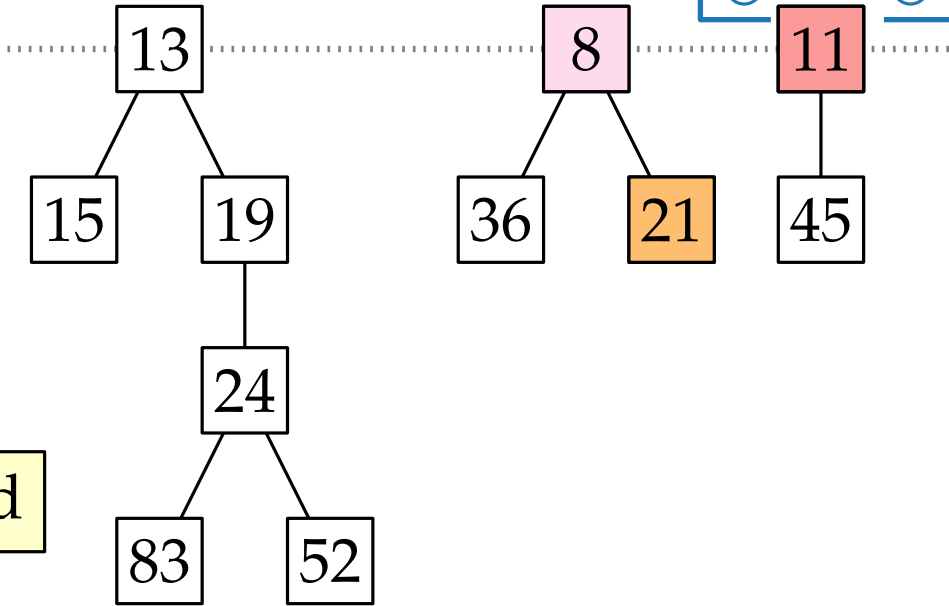
foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

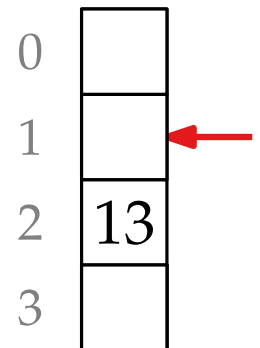
$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray(deg(r) - 1) = nil

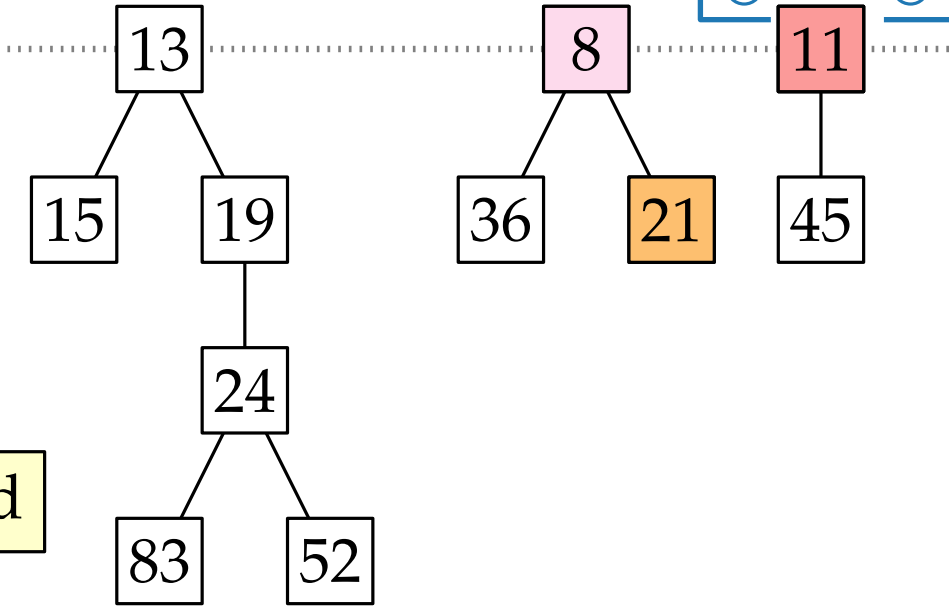
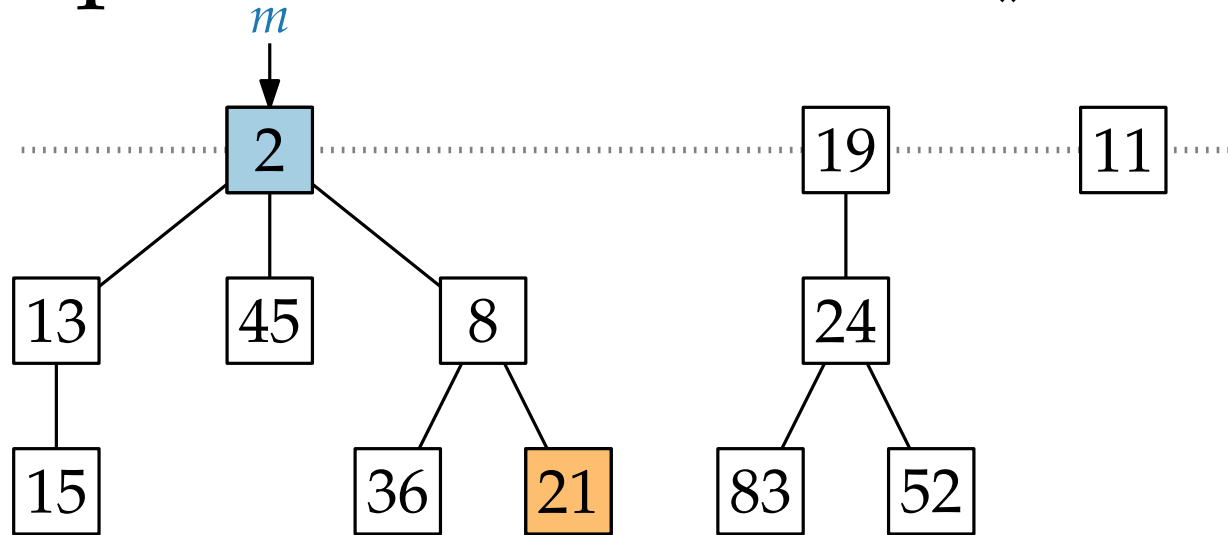
degArray(deg(r)) = r



degArray



Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

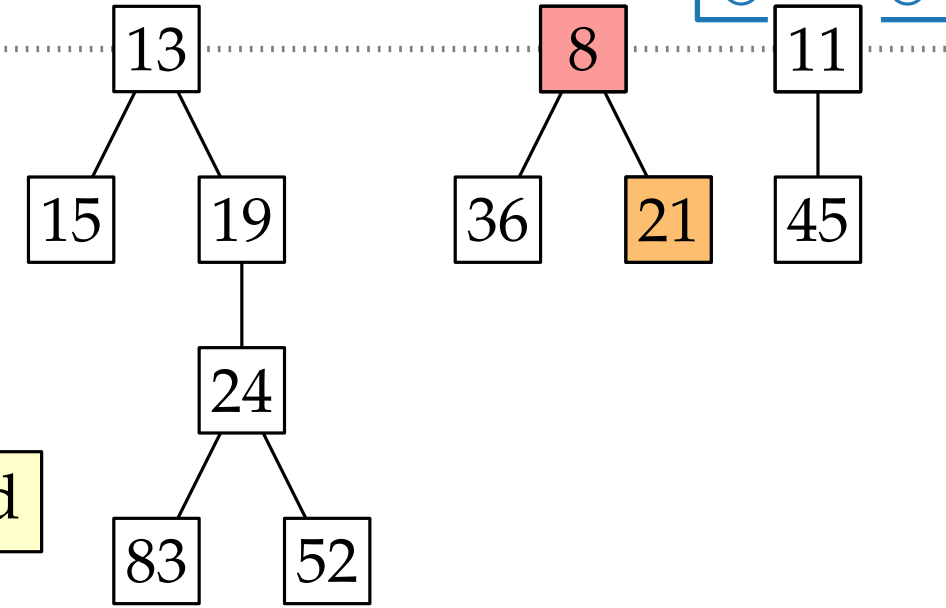
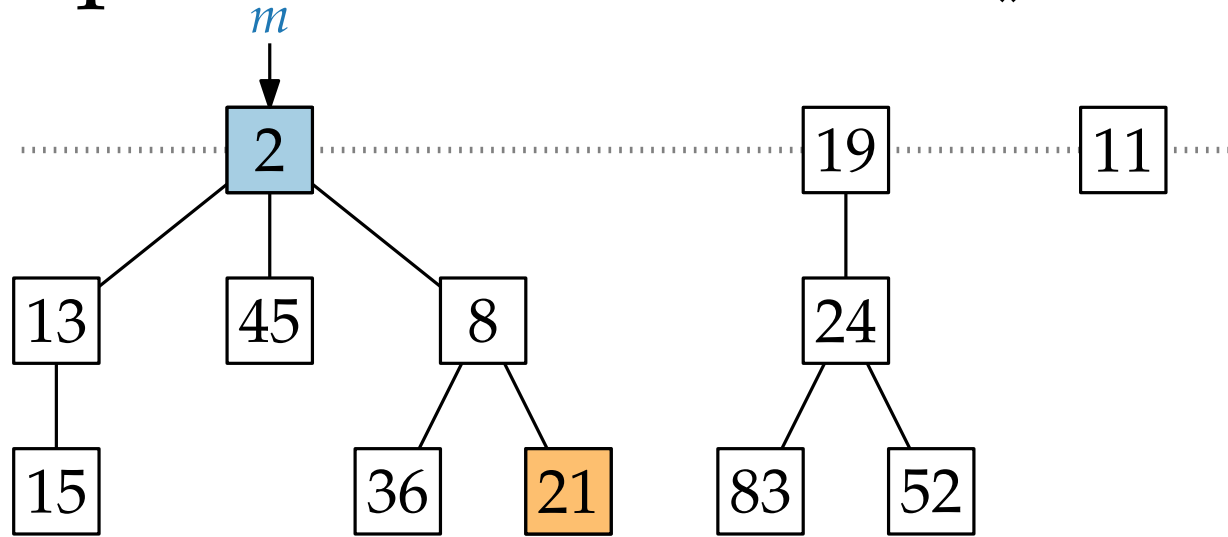
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	
1	11
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

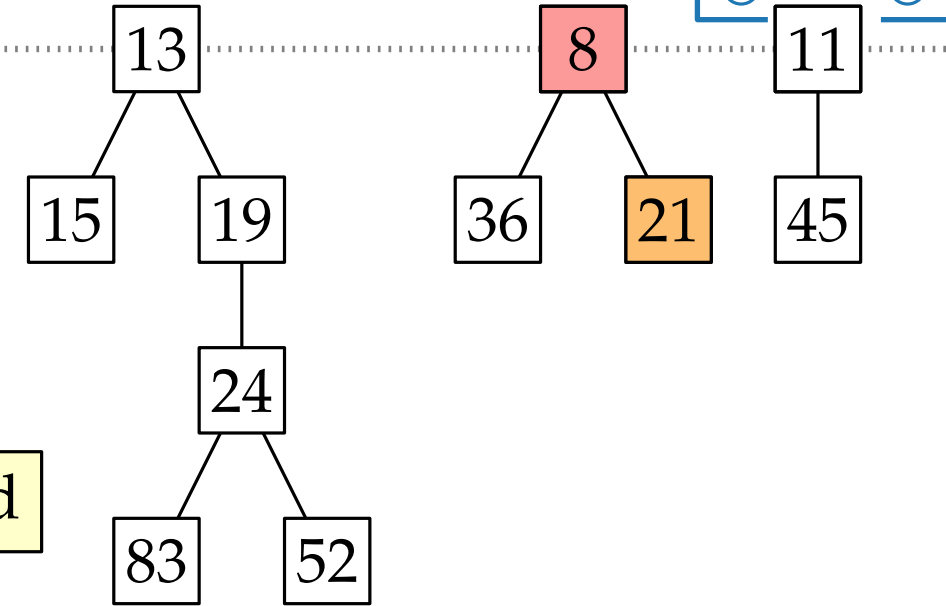
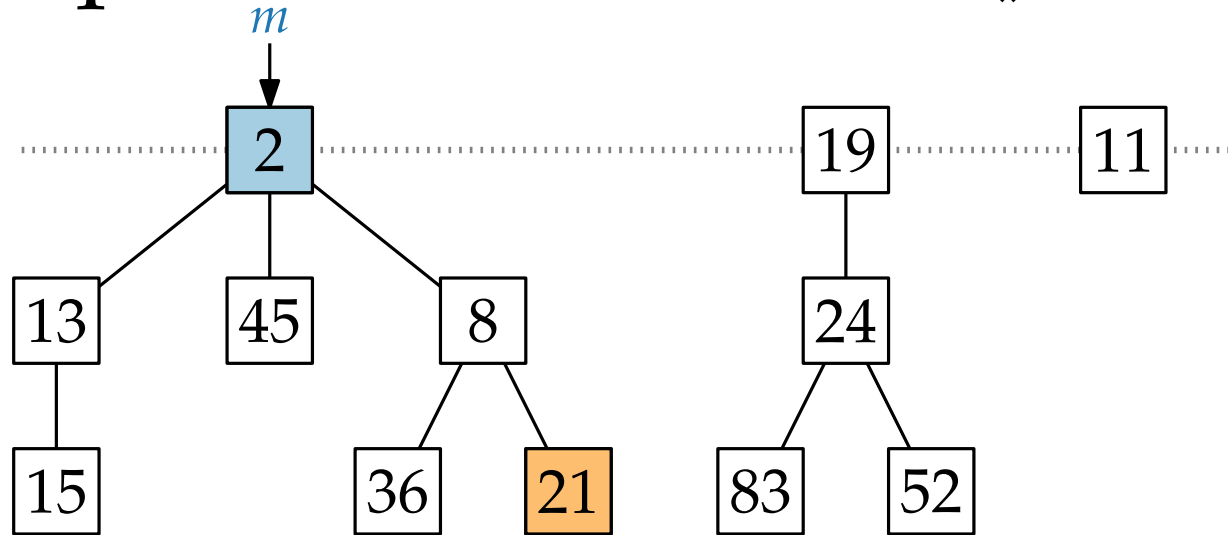
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	
1	11
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

(seit letztem CLEANUP)

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

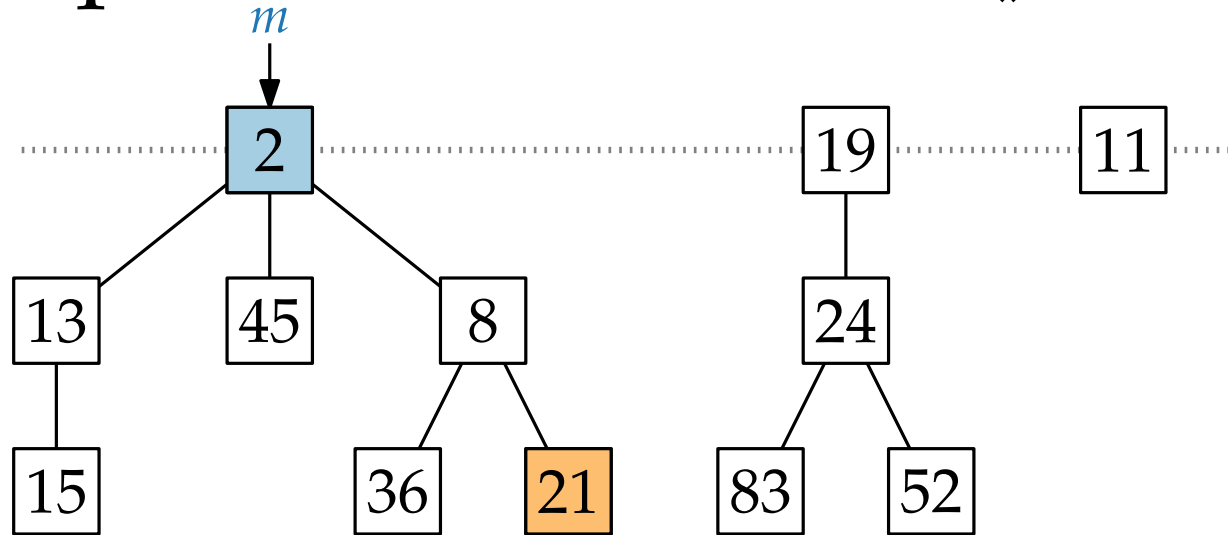
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

degArray

0	
1	11
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

R = neue Knoten in Wurzelliste

(seit letztem CLEANUP)

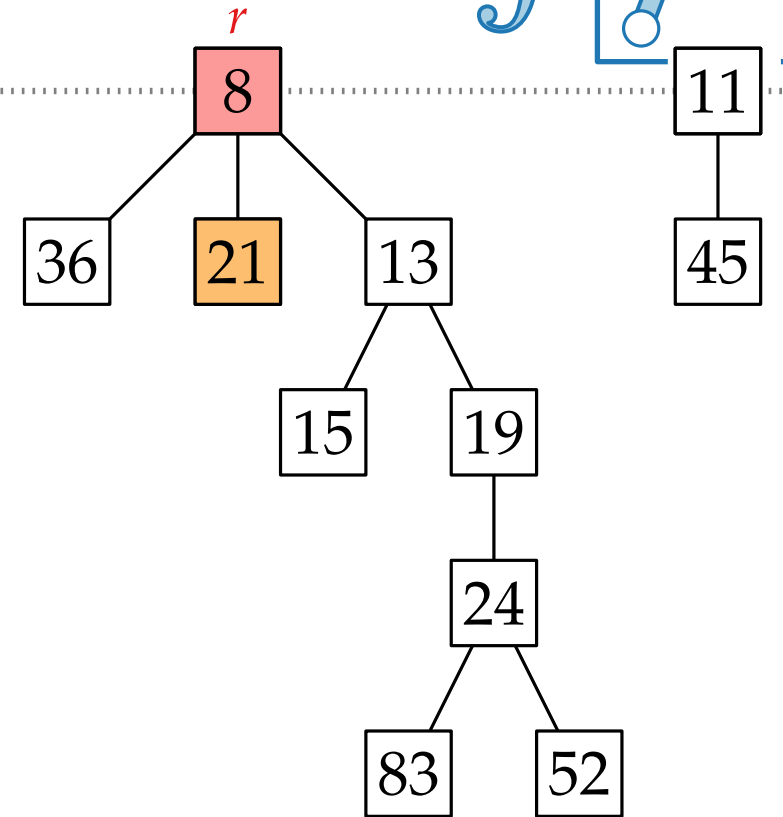
foreach *r* ∈ *R* do

 while **degArray**(deg(*r*)) ≠ nil do

r = **LINK**(*r*, **degArray**(deg(*r*)))

degArray(deg(*r*) - 1) = nil

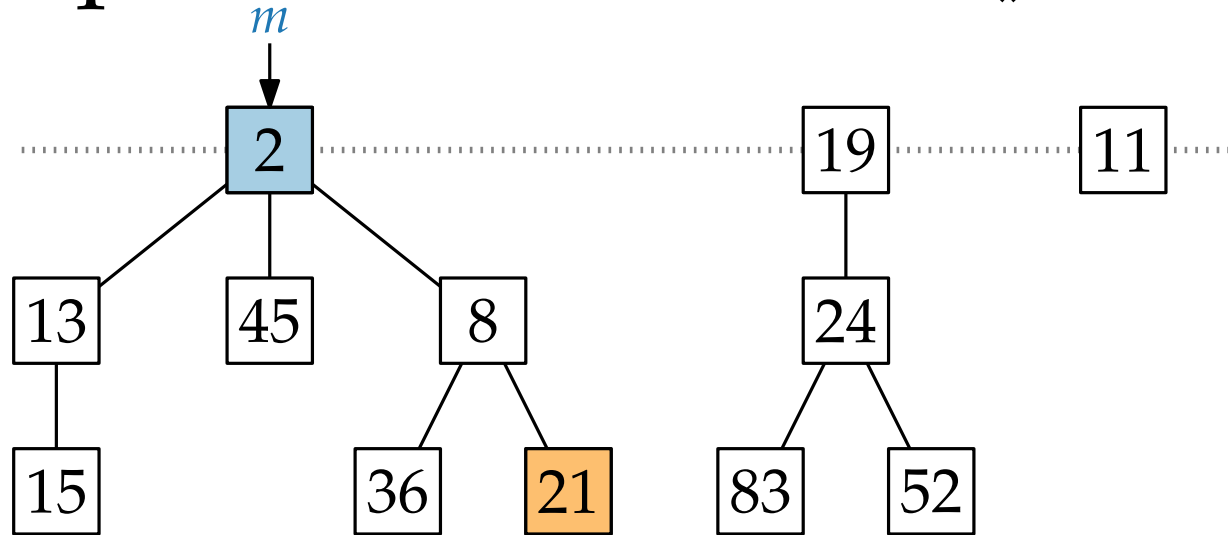
degArray(deg(*r*)) = *r*



degArray

0	
1	11
2	13
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

R = neue Knoten in Wurzelliste

(seit letztem CLEANUP)

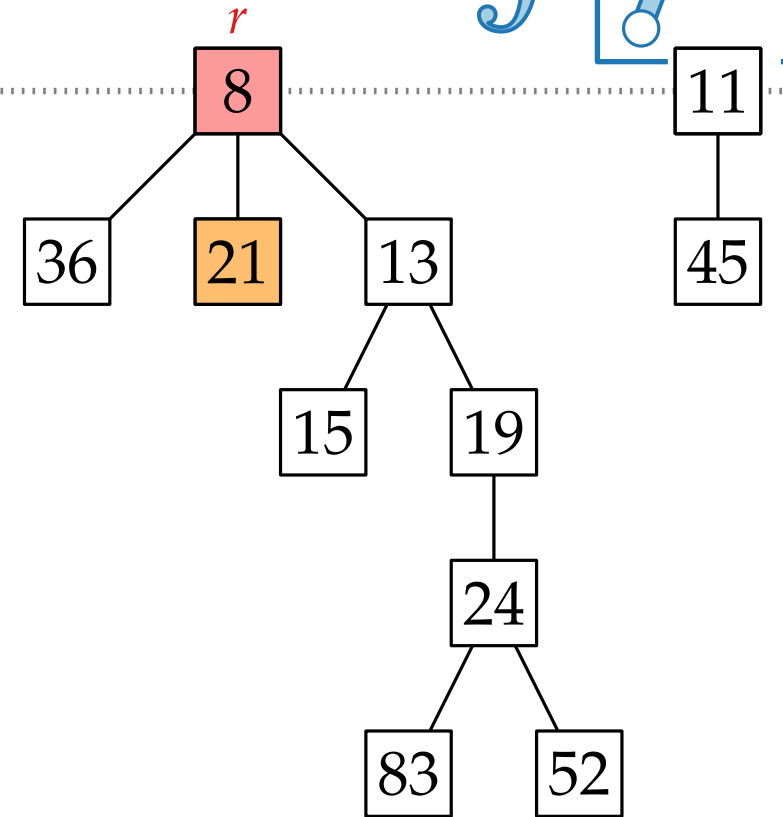
foreach *r* ∈ *R* do

 while **degArray**(deg(*r*)) ≠ nil do

r = **LINK**(*r*, **degArray**(deg(*r*)))

degArray(deg(*r*) - 1) = nil

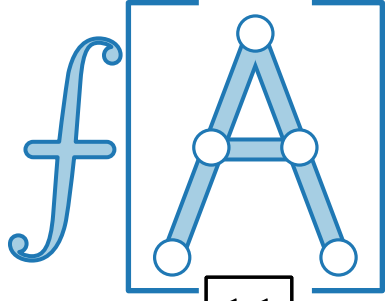
degArray(deg(*r*)) = *r*



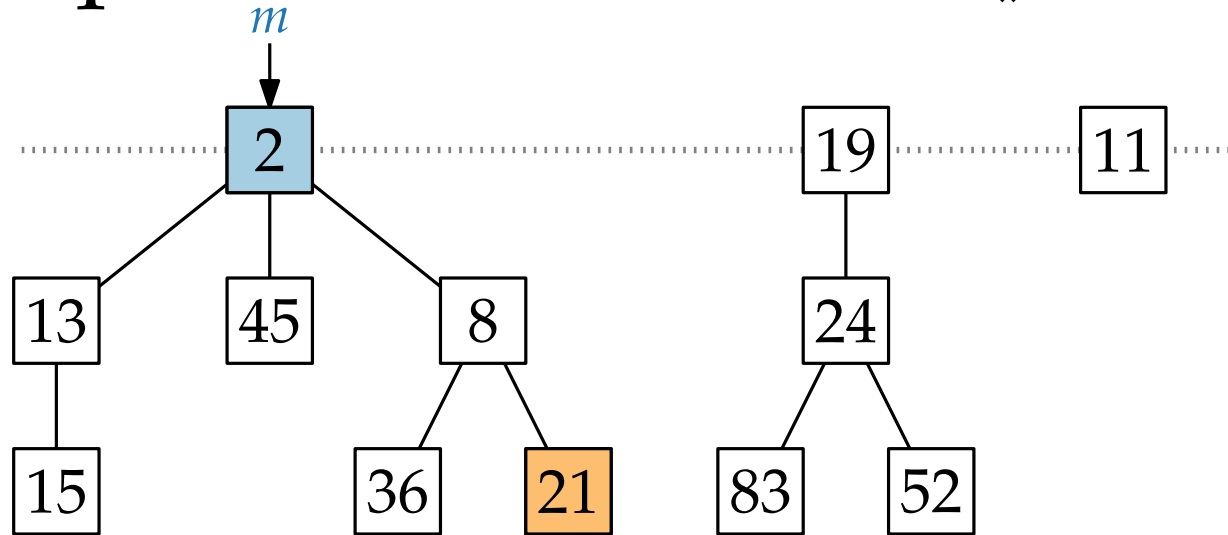
degArray

0	
1	11
2	
3	

A red arrow points to the empty cell at index 2.



Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

R = neue Knoten in Wurzelliste

(seit letztem CLEANUP)

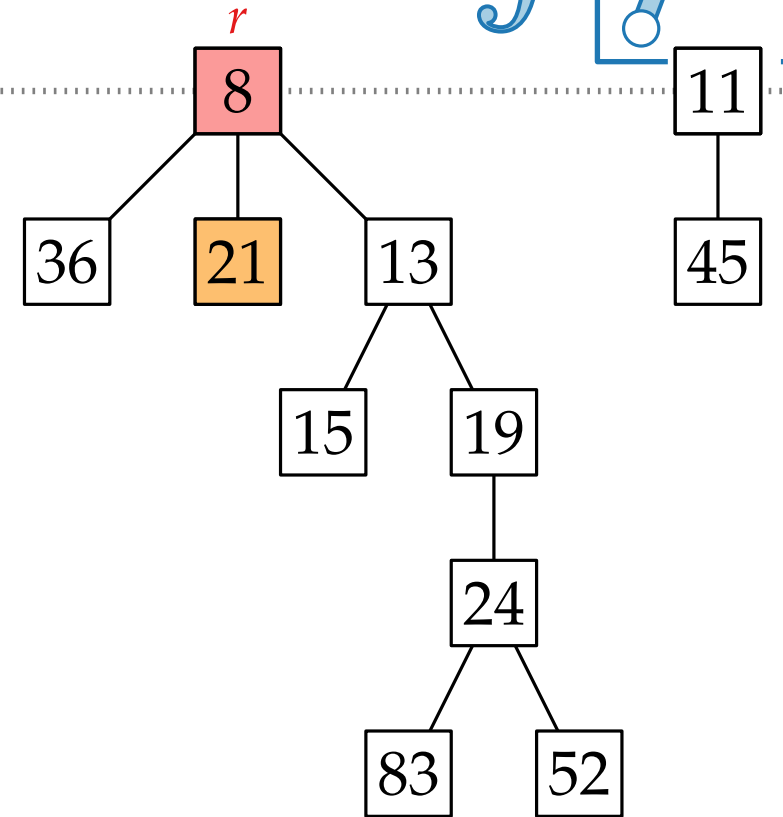
foreach $r \in R$ do

while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray(deg(r) - 1) = nil

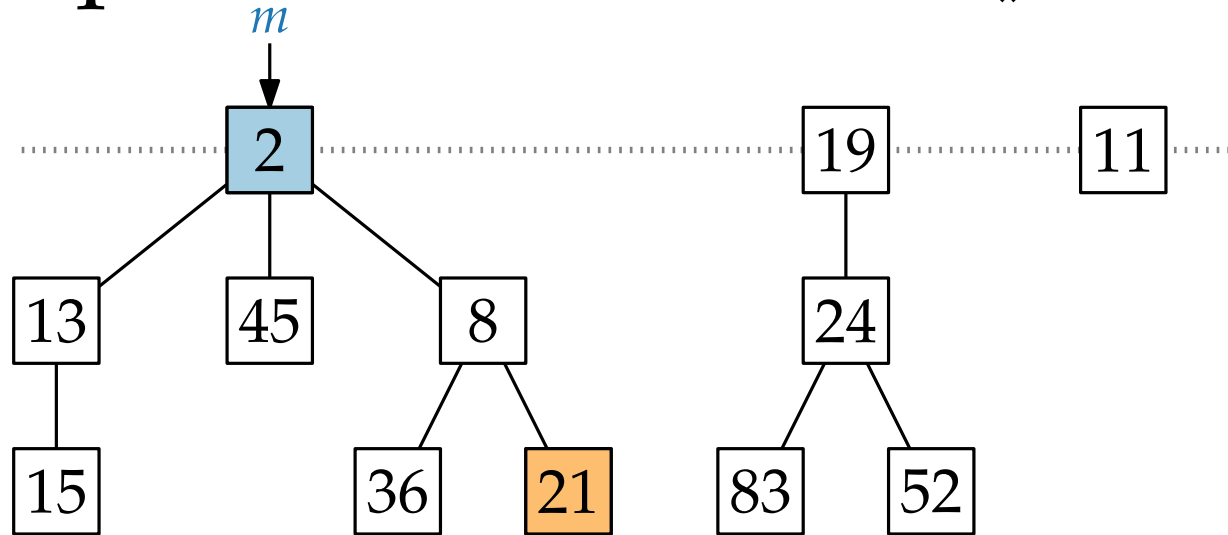
degArray(deg(r)) = r



degArray

0	
1	11
2	
3	

Operation: CLEANUP()



normiert: Jeder Knoten in Wurzelliste hat unterschiedlichen Grad

Speicher Array **degArray** für Wurzelknoten nach Grad

Q.CLEANUP()

R = neue Knoten in Wurzelliste

(seit letztem CLEANUP)

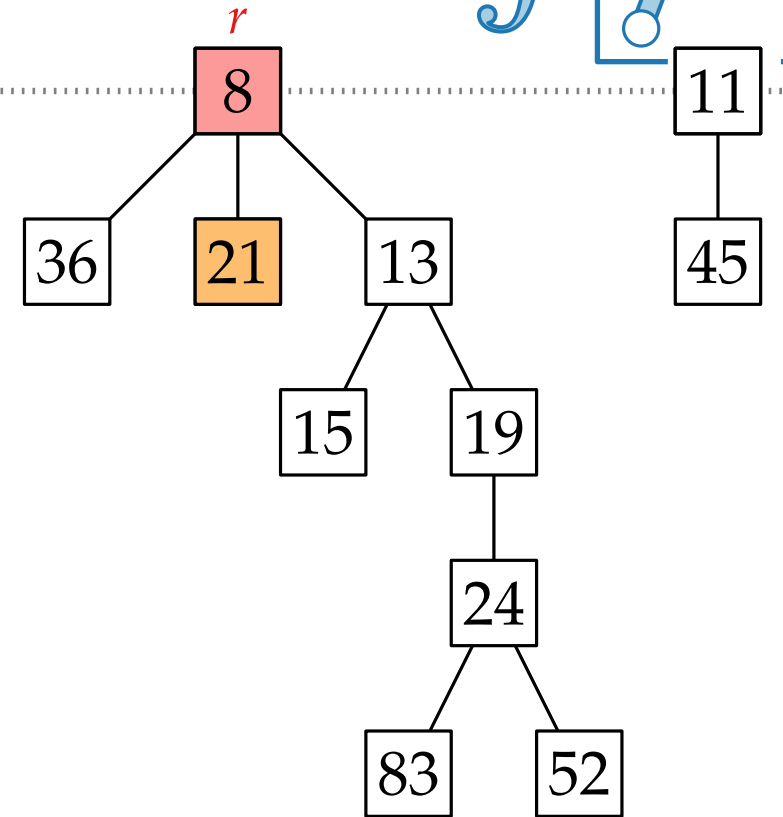
foreach $r \in R$ do

while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray(deg(r) - 1) = nil

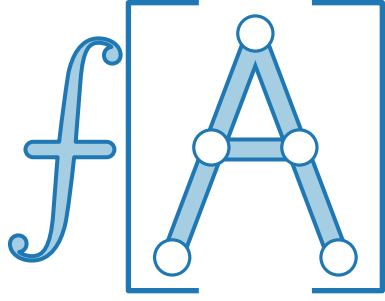
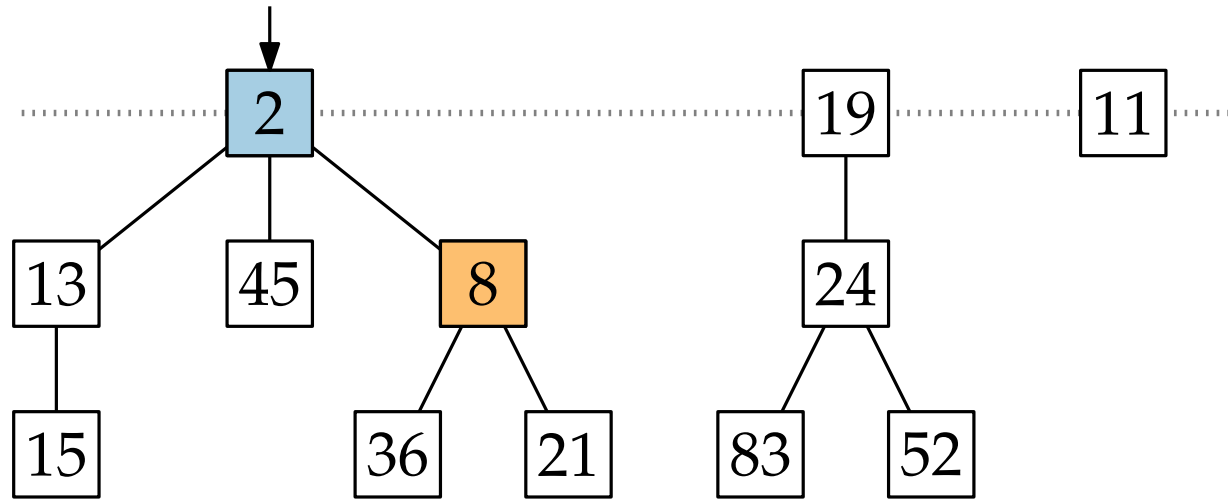
degArray(deg(r)) = r



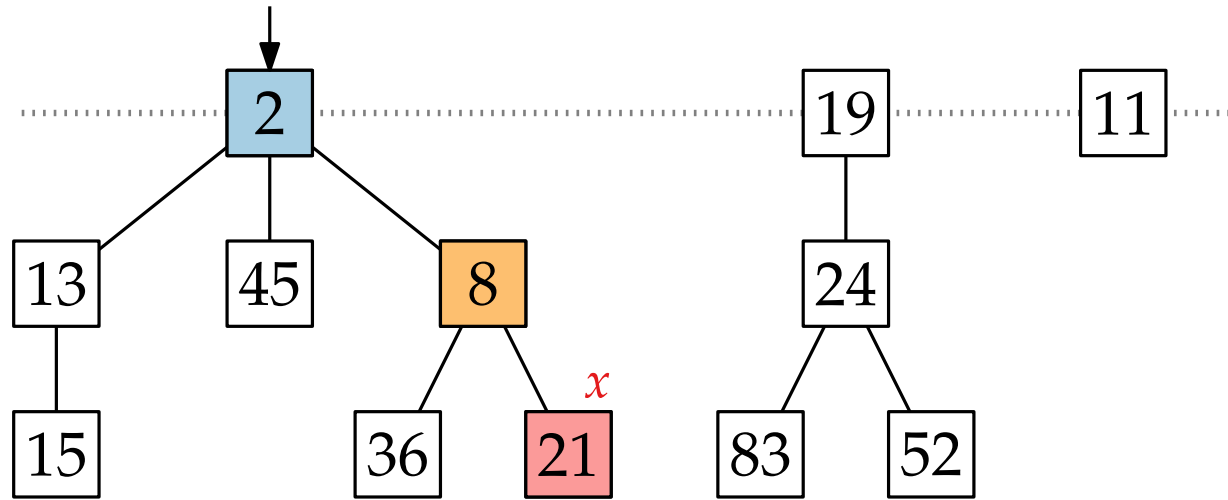
degArray

0	
1	11
2	
3	8

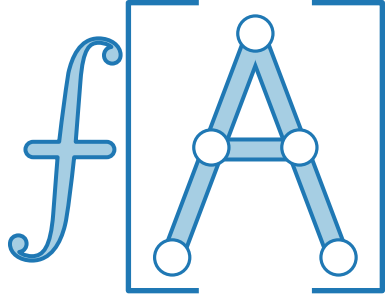
Operation: $\text{DECREASEKEY}(x, p)$



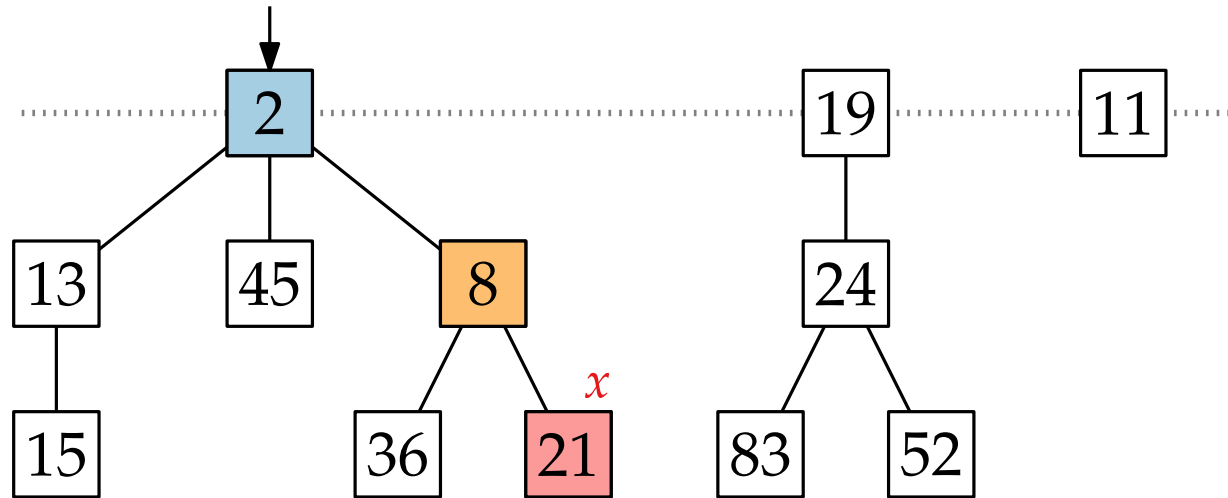
Operation: $\text{DECREASEKEY}(x, p)$



$Q.\text{DECREASEKEY}(x, p)$

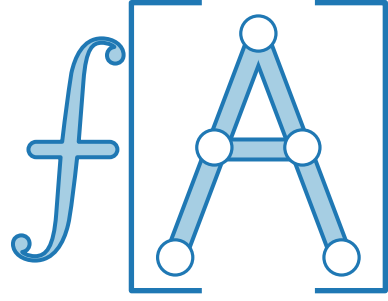


Operation: DECREASEKEY(x, p)

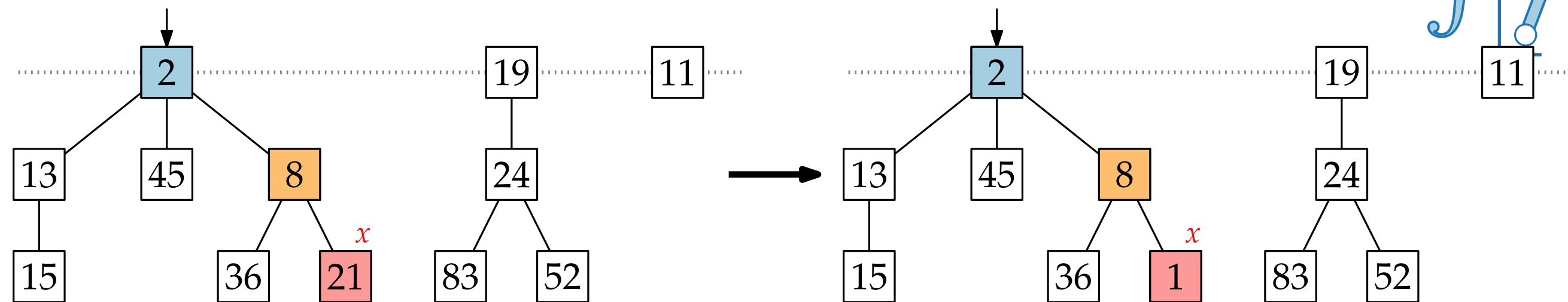
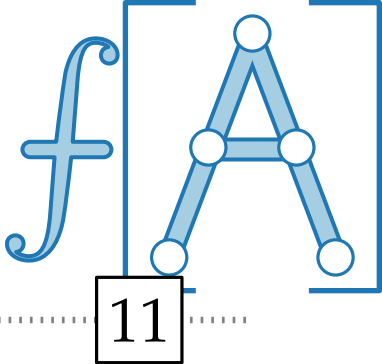


Q.DECREASEKEY(x, p)

$x.key = p$



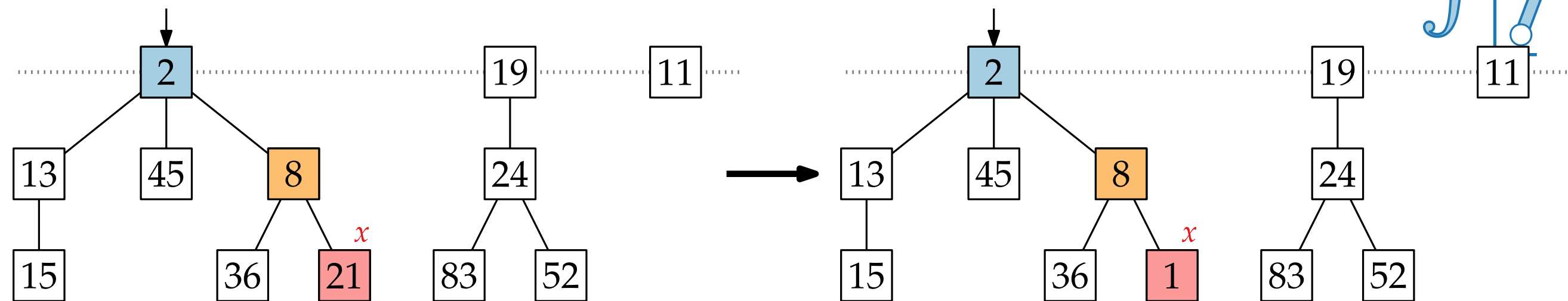
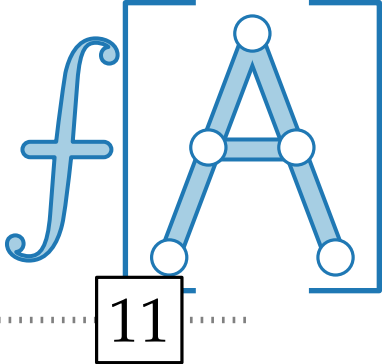
Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

Operation: DECREASEKEY(x, p)

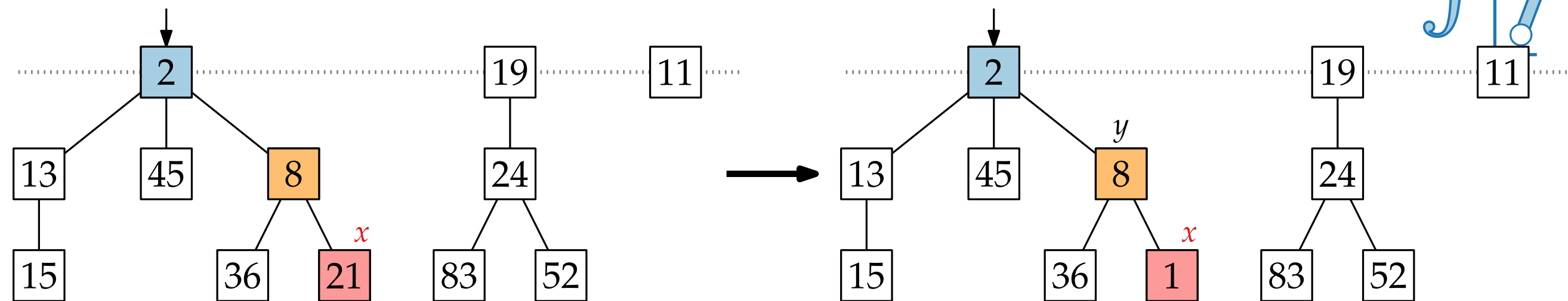
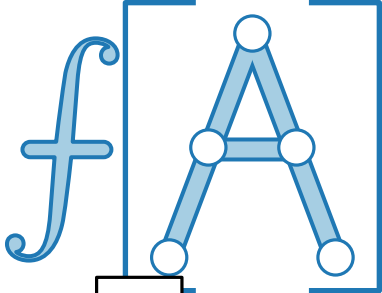


Q.DECREASEKEY(x, p)

x .key = p

$y = x$.parent

Operation: DECREASEKEY(x, p)

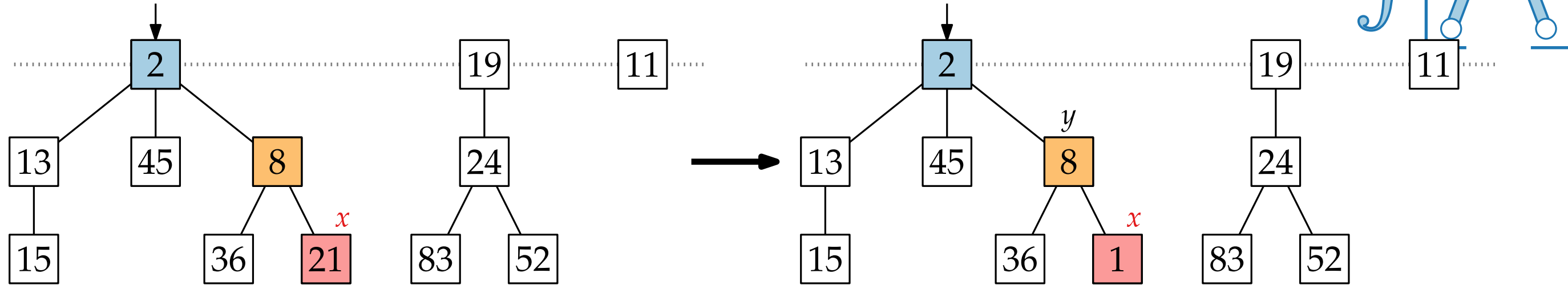


Q.DECREASEKEY(x, p)

x .key = p

$y = x$.parent

Operation: DECREASEKEY(x, p)



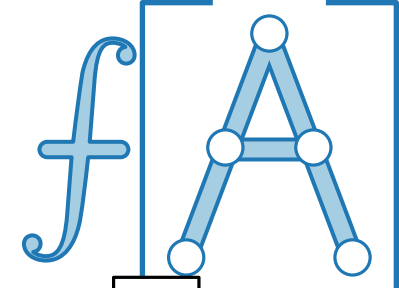
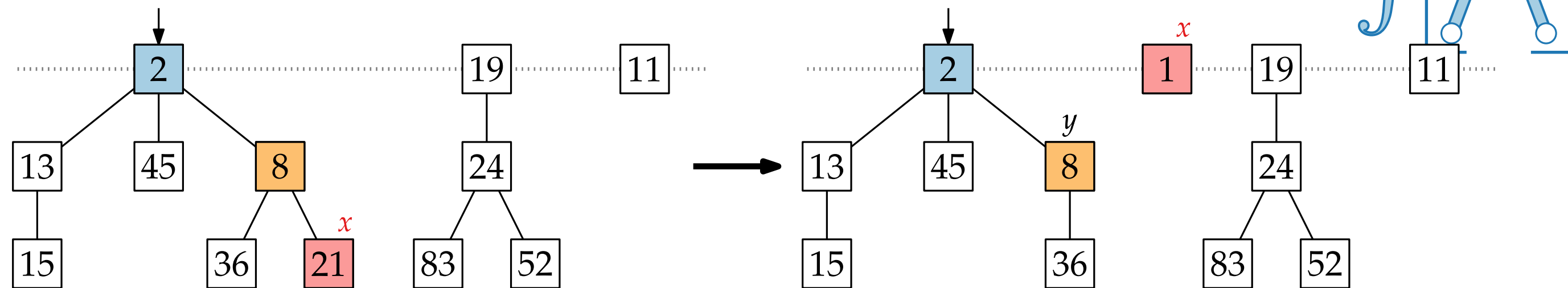
Q.DECREASEKEY(x, p)

x .key = p

$y = x$.parent

Q.CUT(x)

Operation: DECREASEKEY(x, p)



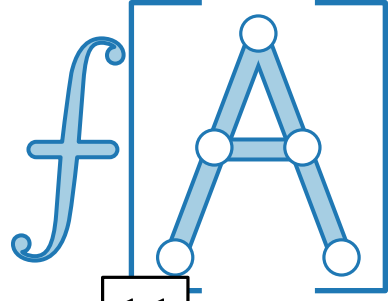
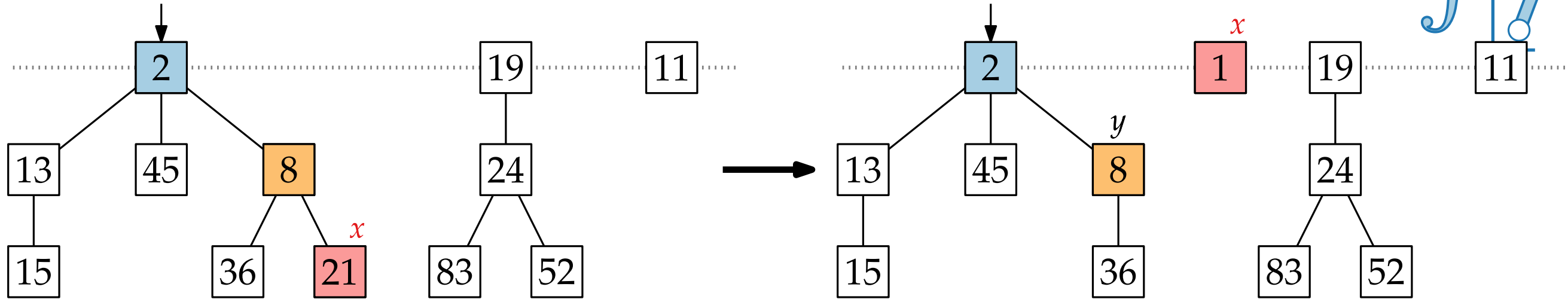
Q.DECREASEKEY(x, p)

x .key = p

$y = x$.parent

Q.CUT(x)

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

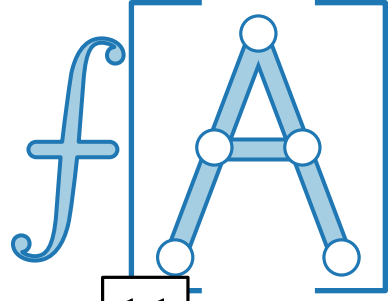
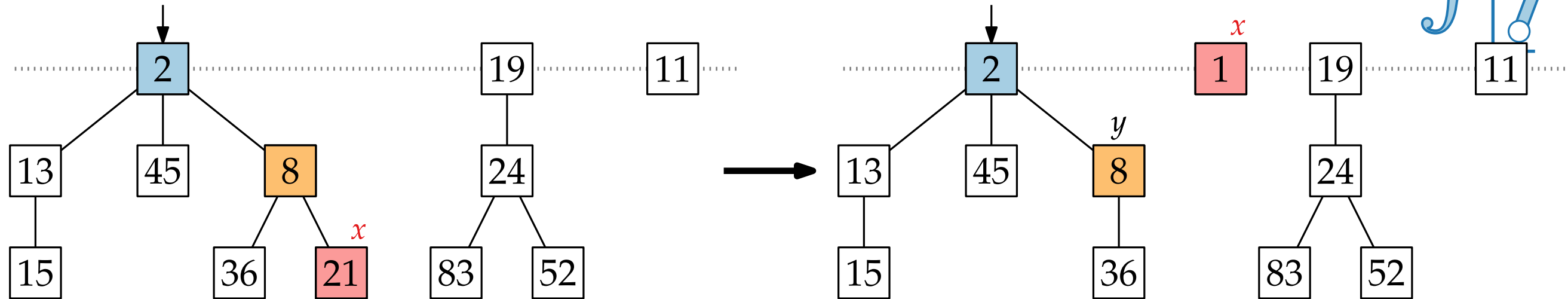
$y = x$.parent

Q.CUT(x)

while y ist markiert do

|

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

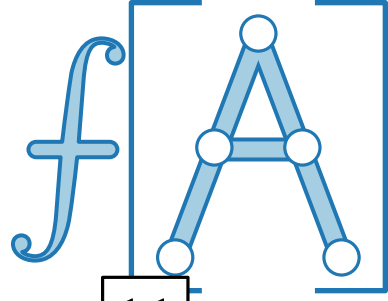
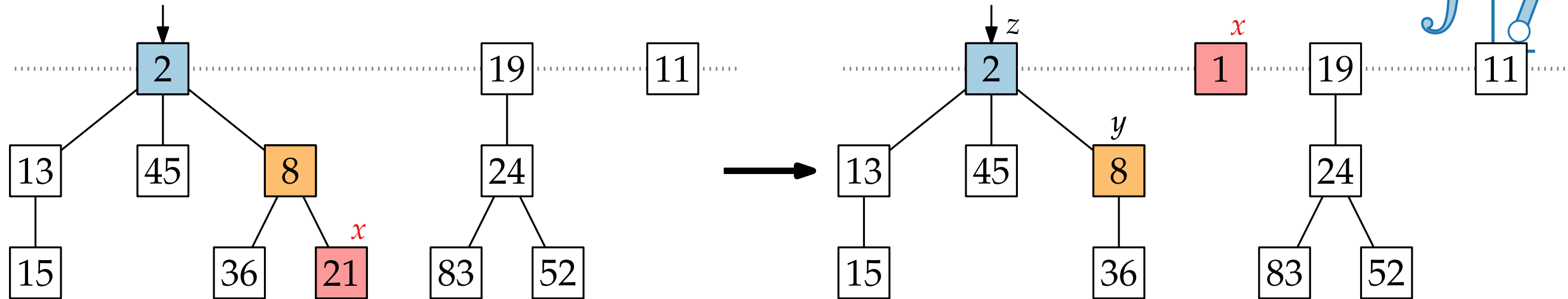
$y = x$.parent

Q.CUT(x)

while y ist markiert do

$z = y$.parent;

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

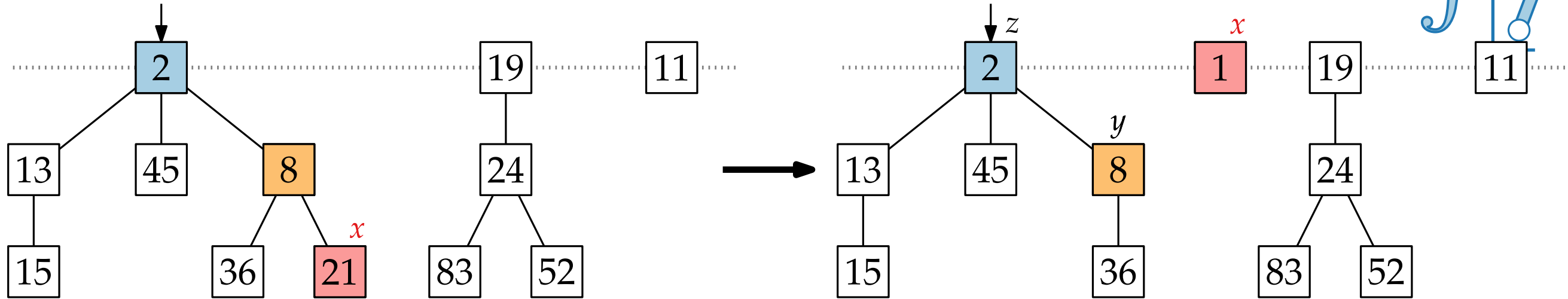
$y = x$.parent

Q.CUT(x)

while y ist markiert do

$z = y$.parent;

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

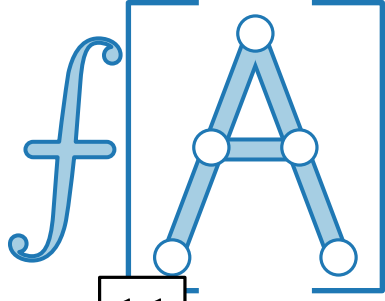
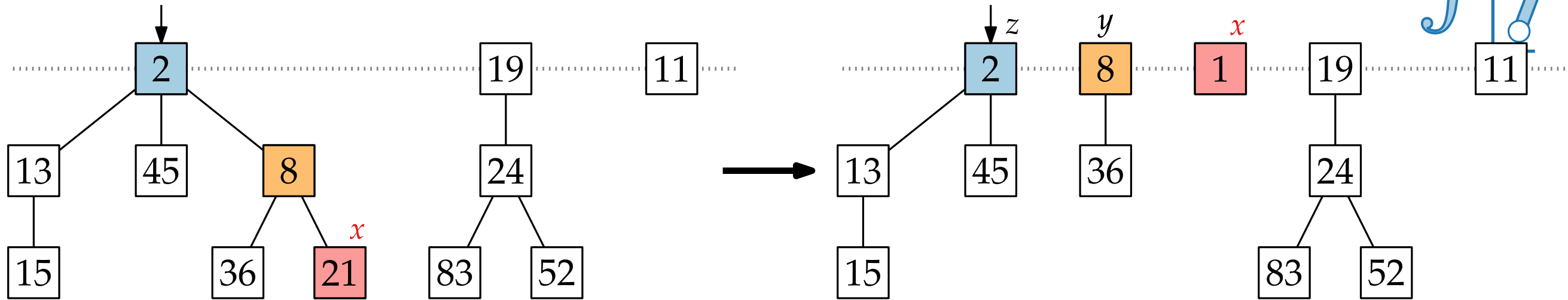
$y = x$.parent

Q.CUT(x)

while y ist markiert do

$z = y$.parent; Q.CUT(y)

Operation: DECREASEKEY(x, p)



$Q.$ DECREASEKEY(x, p)

$x.$ key = p

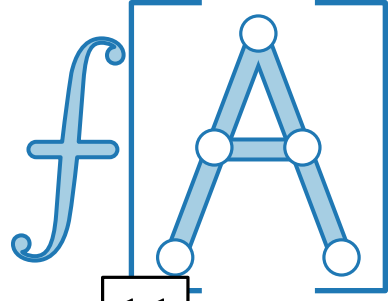
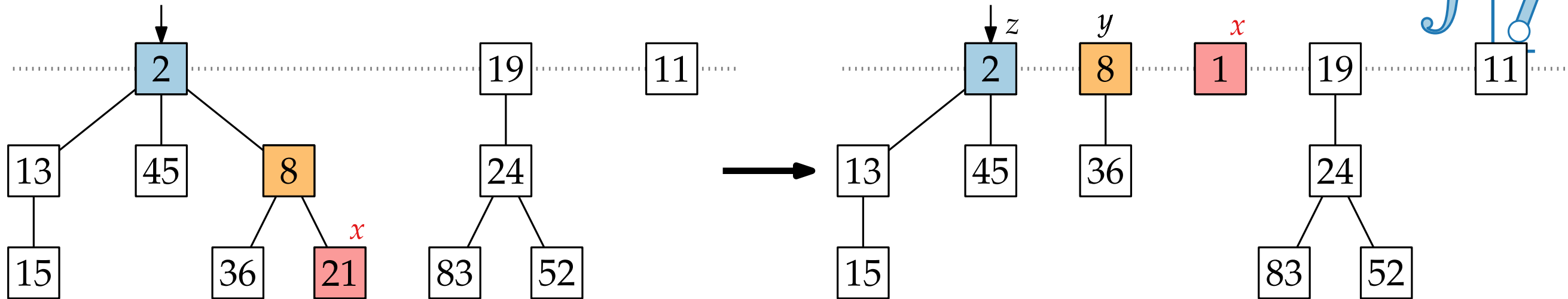
$y = x.$ parent

$Q.$ CUT(x)

while y ist **markiert** **do**

$z = y.$ parent; $Q.$ CUT(y)

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

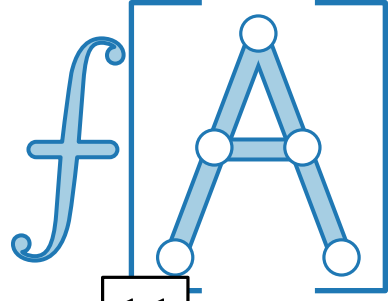
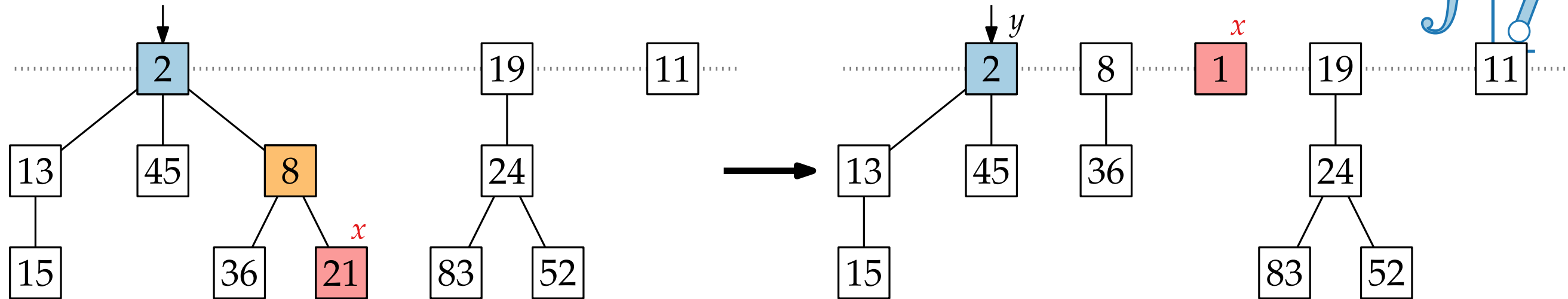
$y = x$.parent

Q.CUT(x)

while y ist **markiert** do

$z = y$.parent; Q.CUT(y)
 entmarkiere y ; $y = z$

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

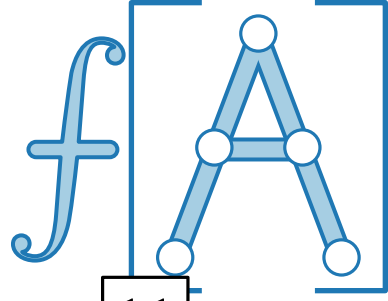
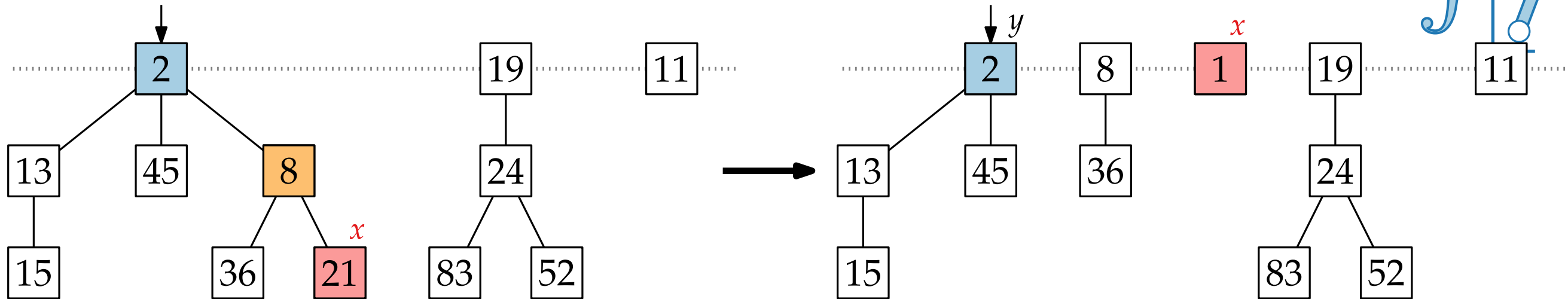
$y = x$.parent

Q.CUT(x)

while y ist **markiert** do

$z = y$.parent; Q.CUT(y)
 entmarkiere y ; $y = z$

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

$y = x$.parent

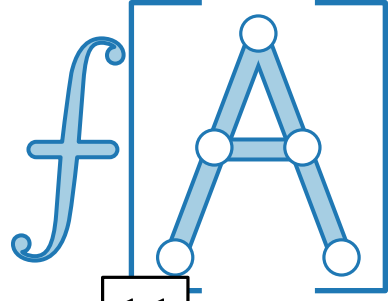
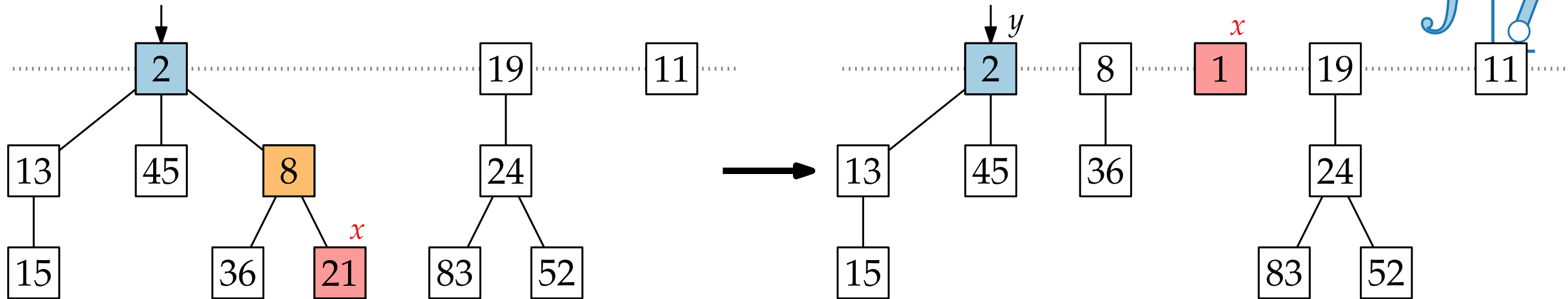
Q.CUT(x)

while y ist markiert do

$z = y$.parent; Q.CUT(y)
 entmarkiere y ; $y = z$

if y nicht in Wurzelliste then markiere y

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

$y = x$.parent

Q.CUT(x)

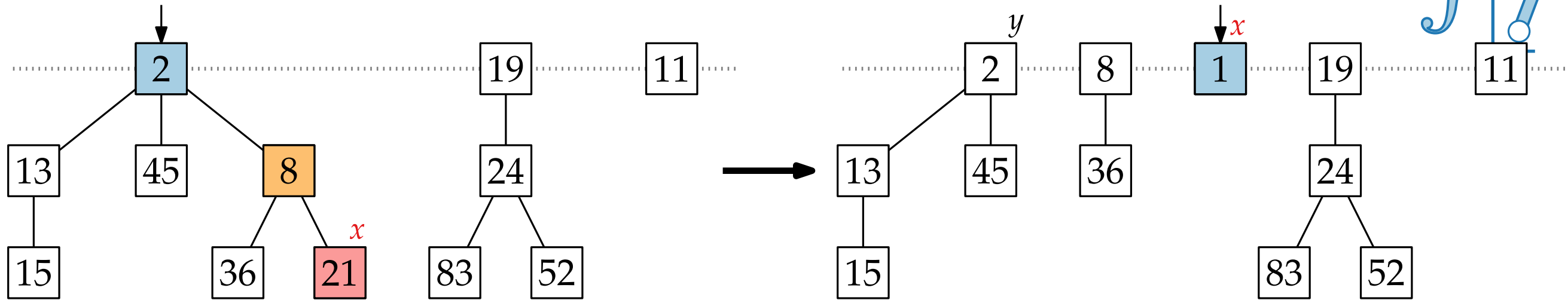
while y ist **markiert** do

$z = y$.parent; Q.CUT(y)
 entmarkiere y ; $y = z$

if y nicht in Wurzelliste then **markiere** y

if $p < Q$.MIN() then setze min Pointer auf x

Operation: DECREASEKEY(x, p)



Q.DECREASEKEY(x, p)

x .key = p

$y = x$.parent

Q.CUT(x)

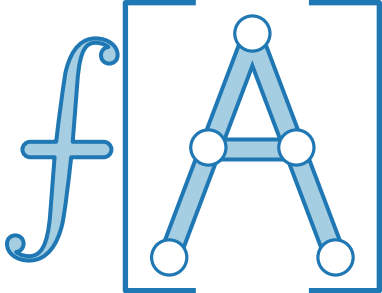
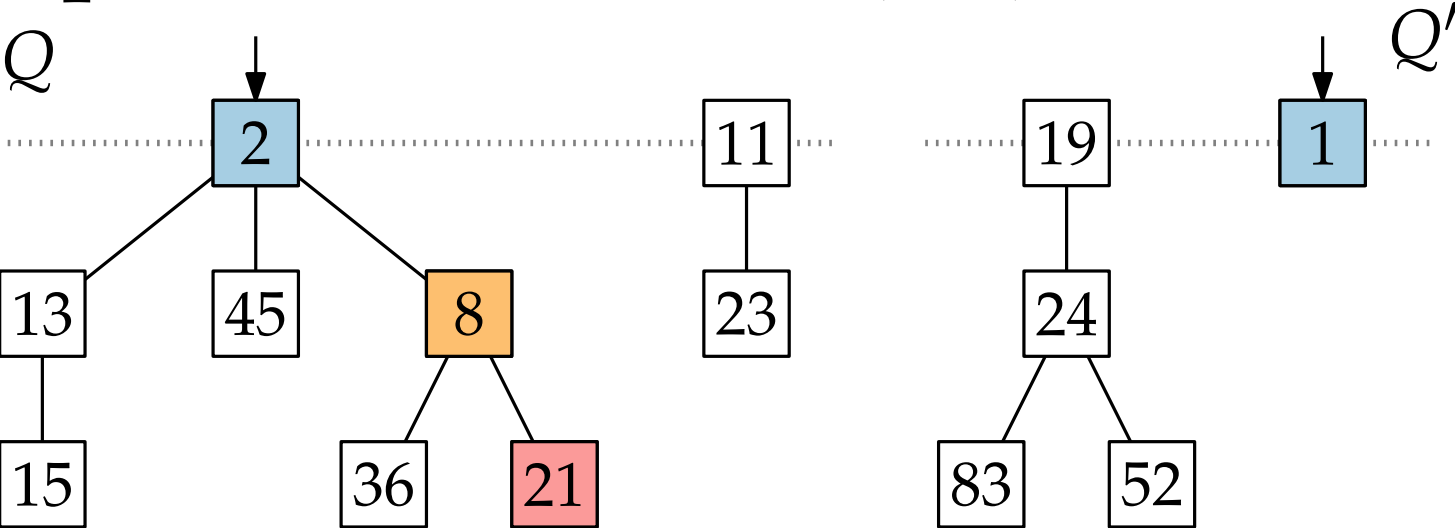
while y ist **markiert** do

$z = y$.parent; Q.CUT(y)
 entmarkiere y ; $y = z$

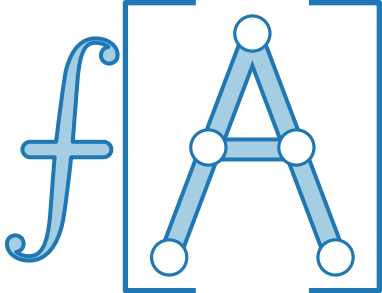
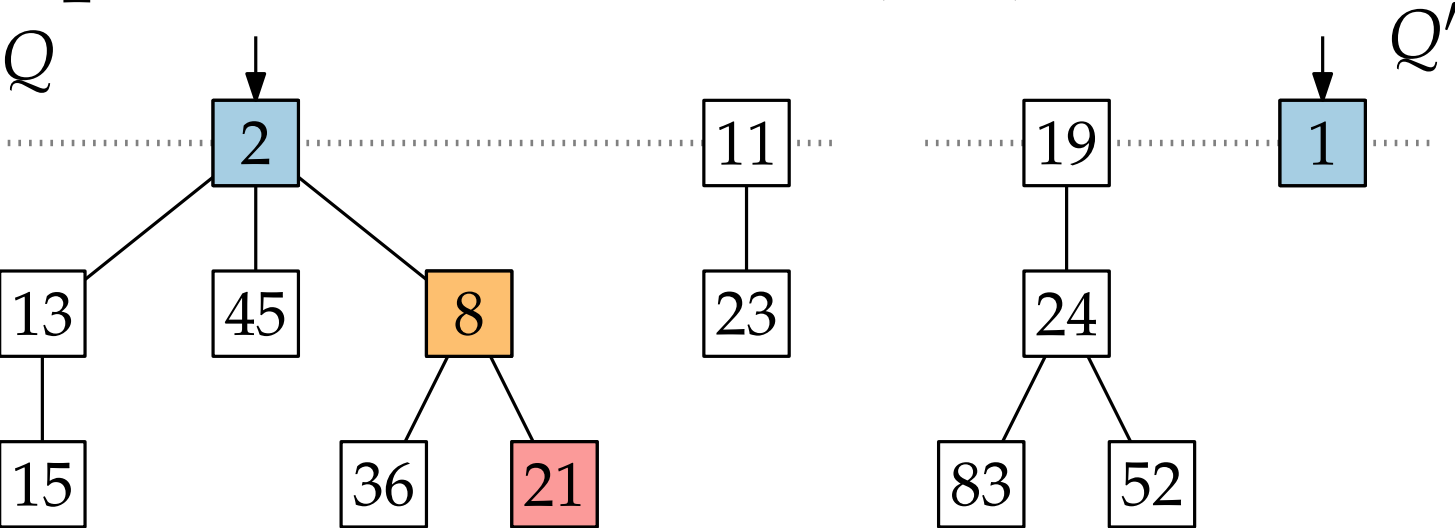
if y nicht in Wurzelliste then **markiere** y

if $p < Q$.MIN() then setze min Pointer auf x

Operation: UNION(Q')

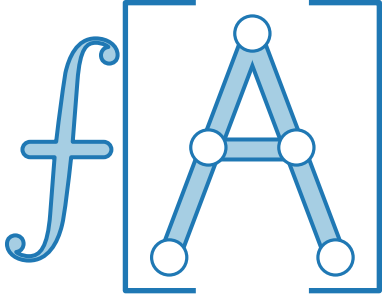
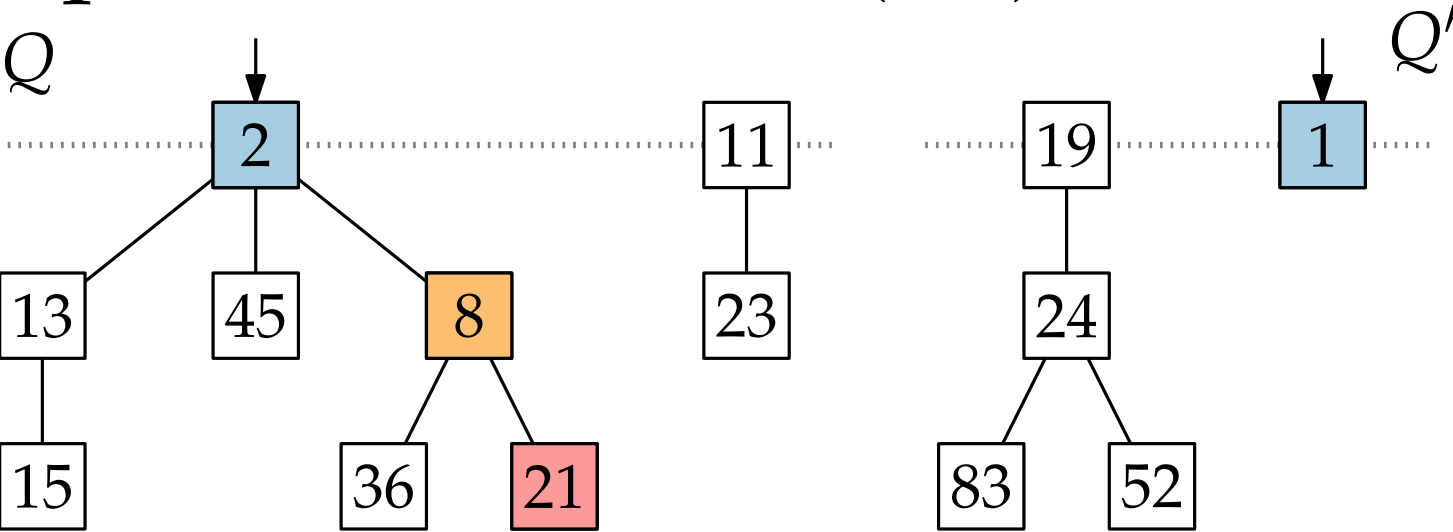


Operation: UNION(Q')



$Q.UNION(Q')$

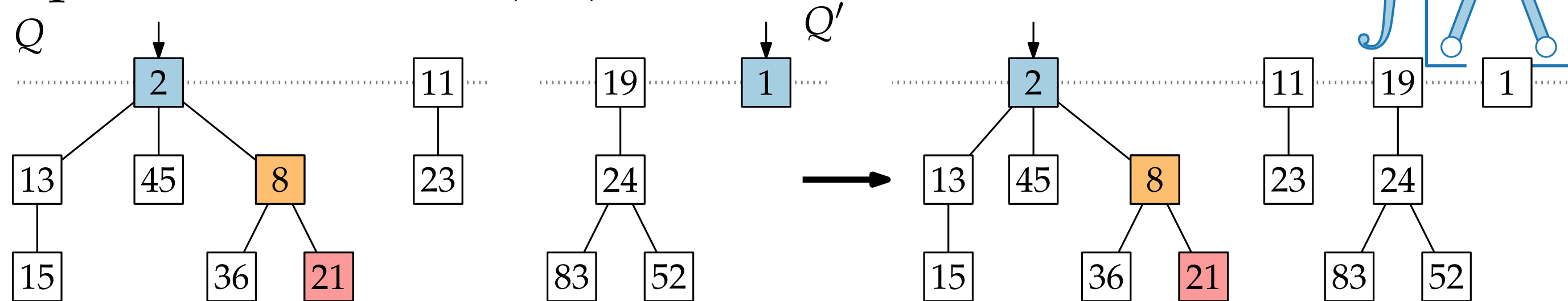
Operation: UNION(Q')



$Q.UNION(Q')$

Hänge Wurzelliste von Q' an Q

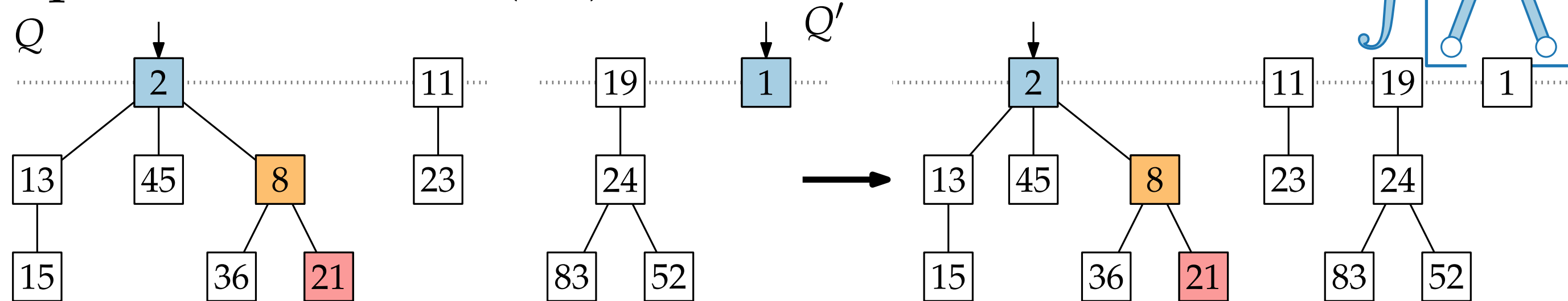
Operation: UNION(Q')



$Q.UNION(Q')$

Hänge Wurzelliste von Q' an Q

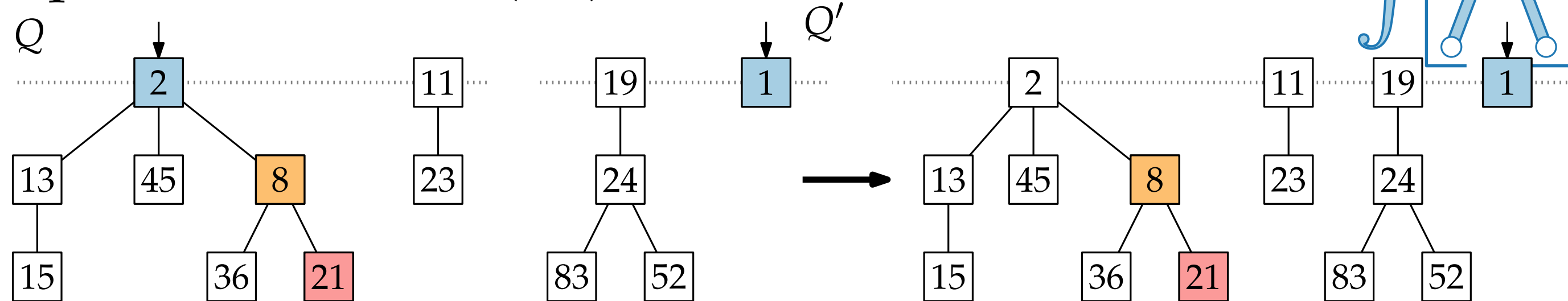
Operation: UNION(Q')



$Q.UNION(Q')$

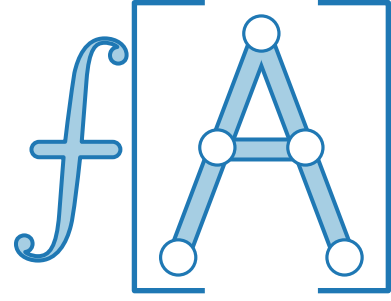
Hänge Wurzelliste von Q' an Q
Aktualisiere min Pointer

Operation: UNION(Q')



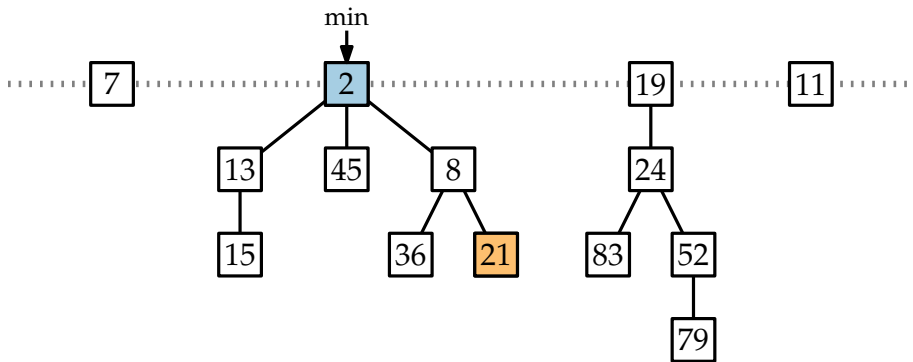
$Q.UNION(Q')$

Hänge Wurzelliste von Q' an Q
Aktualisiere min Pointer

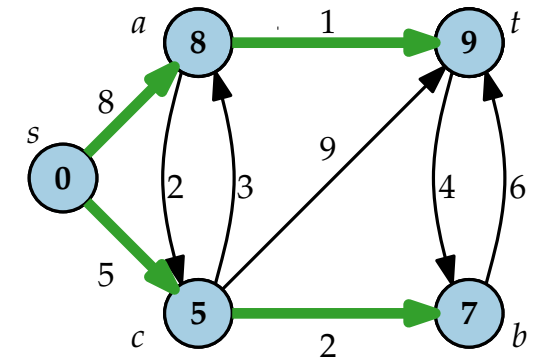


Fortgeschrittene Algorithmen

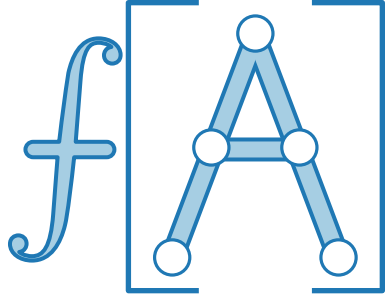
18. Vorlesung Datenstrukturen IV: Fibonacci-Heaps



Teil II: BAUMGRÖSSEN

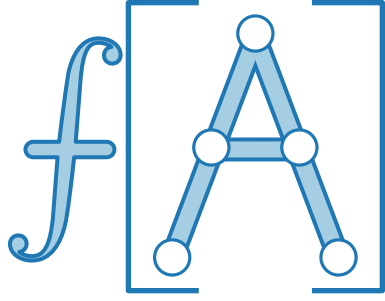


Fibonacci?



Fibonacci?

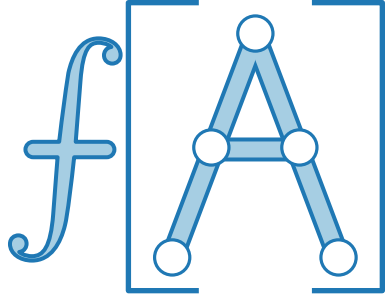
Erinnerung. Fibonacci Zahlen F



Fibonacci?

Erinnerung. Fibonacci Zahlen F

■ $F_0 = 0$



Fibonacci?



Erinnerung. Fibonacci Zahlen F

- $F_0 = 0$
- $F_1 = 1$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$ ■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1}$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

$$\blacksquare F_0 = 0 \quad \blacksquare F_1 = 1$$

$$\blacksquare F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i$$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

$$\blacksquare F_0 = 0 \quad \blacksquare F_1 = 1$$

$$\blacksquare F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1. Sei x ein Knoten mit $k = \deg(x)$.

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

1) $\deg(c_1) \geq 0$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

1) $\deg(c_1) \geq 0$

2) $\deg(c_i) \geq i - 2$ für $i = 2, \dots, k$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

1) $\deg(c_1) \geq 0$

2) $\deg(c_i) \geq i - 2$ für $i = 2, \dots, k$

Beweis.

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

$$\blacksquare F_0 = 0$$

$$\blacksquare F_1 = 1$$

$$\blacksquare F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

$$1) \deg(c_1) \geq 0$$

$$2) \deg(c_i) \geq i - 2 \text{ für } i = 2, \dots, k$$

Beweis. 1) klar.

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

1) $\deg(c_1) \geq 0$

2) $\deg(c_i) \geq i - 2$ für $i = 2, \dots, k$

Beweis.

1) klar.

2) ■ Als c_i Kind von x wurde, war $\deg(x) \geq i - 1$.

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

1) $\deg(c_1) \geq 0$

2) $\deg(c_i) \geq i - 2$ für $i = 2, \dots, k$

Beweis.

1) klar.

2) ■ Als c_i Kind von x wurde, war $\deg(x) \geq i - 1$.

■ **LINK** verknüpft immer Knoten vom gleichen Grad, also $\deg(c_i) \geq i - 1$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

$$\blacksquare F_0 = 0$$

$$\blacksquare F_1 = 1$$

$$\blacksquare F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

$$1) \deg(c_1) \geq 0$$

$$2) \deg(c_i) \geq i - 2 \text{ für } i = 2, \dots, k$$

Beweis.

1) klar.

2) $\blacksquare$ Als c_i Kind von x wurde, war $\deg(x) \geq i - 1$.

$\blacksquare$ **LINK** verknüpft immer Knoten vom gleichen Grad, also $\deg(c_i) \geq i - 1$

$\blacksquare$ Seitdem hat c_i max. ein Kind durch **CUT** verloren: beim ersten Mal wird c_i **markiert**, beim zweiten Mal mit abgeschnitten

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

1) $\deg(c_1) \geq 0$

2) $\deg(c_i) \geq i - 2$ für $i = 2, \dots, k$

Beweis.

1) klar.

2) ■ Als c_i Kind von x wurde, war $\deg(x) \geq i - 1$.

■ **LINK** verknüpft immer Knoten vom gleichen Grad, also $\deg(c_i) \geq i - 1$

■ Seitdem hat c_i max. ein Kind durch **CUT** verloren: beim ersten Mal wird c_i **markiert**, beim zweiten Mal mit abgeschnitten $\Rightarrow \deg(c_i) \geq i - 2$

Fibonacci?



Erinnerung.

Fibonacci Zahlen F

$$\blacksquare F_0 = 0$$

$$\blacksquare F_1 = 1$$

$$\blacksquare F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$$

Lemma 1.

Sei x ein Knoten mit $k = \deg(x)$. Seien $c_1, \dots, c_k$ die Kinder von x in der Reihenfolge, in der sie (mit **LINK**) zu x angehängt wurden. Dann gilt:

$$1) \deg(c_1) \geq 0$$

$$2) \deg(c_i) \geq i - 2 \text{ für } i = 2, \dots, k$$

Beweis.

1) klar.

2) $\blacksquare$ Als c_i Kind von x wurde, war $\deg(x) \geq i - 1$.

$\blacksquare$ **LINK** verknüpft immer Knoten vom gleichen Grad, also $\deg(c_i) \geq i - 1$

$\blacksquare$ Seitdem hat c_i max. ein Kind durch **CUT** verloren: beim ersten Mal wird c_i **markiert**, beim zweiten Mal mit abgeschnitten $\Rightarrow \deg(c_i) \geq i - 2$ $\square$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

$$\blacksquare F_0 = 0 \quad \blacksquare F_1 = 1$$

$$\blacksquare F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

$$\blacksquare F_0 = 0 \quad \blacksquare F_1 = 1$$

$$\blacksquare F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

- Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ • ■ $s_1 = 2$ •

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 4$ •

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 4$ • ?

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

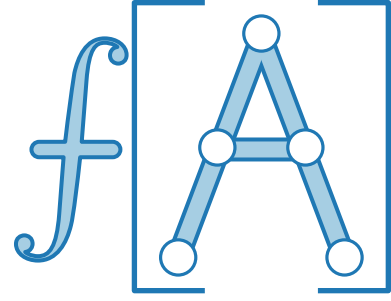
■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

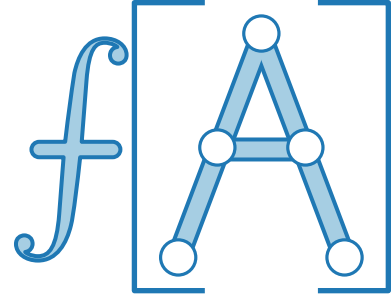
■ $s_2 = 3$ •

■ $s_3 = 5$ •

■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

→ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

→ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

■ **Lemma 1** $\Rightarrow \deg(c_1) = 0, \deg(c_i) = i - 2$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

➔ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

■ **Lemma 1** $\Rightarrow \deg(c_1) = 0, \deg(c_i) = i - 2$

➔ $\text{size}(x)$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

→ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

■ **Lemma 1** $\Rightarrow \deg(c_1) = 0, \deg(c_i) = i - 2$

→ $\text{size}(x) \geq s_k$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

→ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

■ **Lemma 1** $\Rightarrow \deg(c_1) = 0, \deg(c_i) = i - 2$

→ $\text{size}(x) \geq s_k = \text{size}(y) :$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

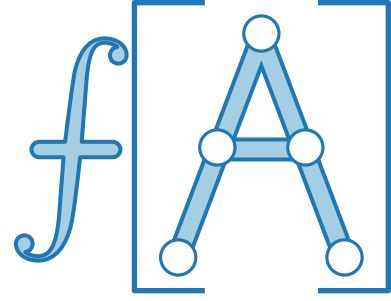
■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

→ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

■ **Lemma 1** $\Rightarrow \deg(c_1) = 0, \deg(c_i) = i - 2$

→ $\text{size}(x) \geq s_k = \text{size}(y) = 1 + s_0 + \sum_{i=2}^k s_{i-2}$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •

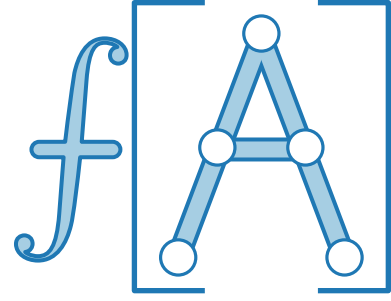
■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

→ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

■ **Lemma 1** $\Rightarrow \deg(c_1) = 0, \deg(c_i) = i - 2$ (Induktion)

→ $\text{size}(x) \geq s_k = \text{size}(y) = 1 + s_0 + \sum_{i=2}^k s_{i-2} = 1 + 1 + \sum_{i=2}^k F_i$

Baumgrößen



Erinnerung.

Fibonacci Zahlen F

■ $F_0 = 0$

■ $F_1 = 1$

■ $F_{k+2} = F_k + F_{k+1} = 1 + \sum_{i=1}^k F_i \geq 1.618^k$

Lemma 2.

Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:

$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Beweis.

■ Sei $s_k = \min\{\text{size}(y) \mid \deg(y) = k\}$,
also die kleinstmögliche Größe eines Baums mit Wurzelgrad k .

■ $s_0 = 1$ •

■ $s_1 = 2$ •

■ $s_2 = 3$ •

■ $s_3 = 5$ •



■ Sei y ein Knoten $\deg(y) = k$, $\text{size}(y) = s_k$ und Kindern $c_1, \dots, c_k$

→ $s_k = \text{size}(y) = 1 + \sum_{i=1}^k \text{size}(c_i)$

■ **Lemma 1** $\Rightarrow \deg(c_1) = 0, \deg(c_i) = i - 2$ (Induktion)

→ $\text{size}(x) \geq s_k = \text{size}(y) = 1 + s_0 + \sum_{i=2}^k s_{i-2} = 1 + 1 + \sum_{i=2}^k F_i = F_{k+2}$



Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz.
$$\text{maxdeg}(Q) \leq 2 \log n$$

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz.
$$\text{maxdeg}(Q) \leq 2 \log n$$

Beweis.

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz.
$$\text{maxdeg}(Q) \leq 2 \log n$$

Beweis. ■ Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz. $\text{maxdeg}(Q) \leq 2 \log n$

Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\rightarrow n$

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz. $\text{maxdeg}(Q) \leq 2 \log n$

Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\Rightarrow n \geq \text{size}(x)$

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz. $\text{maxdeg}(Q) \leq 2 \log n$

Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\rightarrow n \geq \text{size}(x) \geq 1.618^k$

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
 $\text{size}(x) \geq F_{k+2} \geq 1.618^k$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz. $\text{maxdeg}(Q) \leq 2 \log n$

Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\Rightarrow n \geq \text{size}(x) \geq 1.618^k$
 $\Rightarrow k$

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz. $\text{maxdeg}(Q) \leq 2 \log n$

Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\rightarrow n \geq \text{size}(x) \geq 1.618^k$
 $\rightarrow k \leq \log_{1.618} n$

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
$$\text{size}(x) \geq F_{k+2} \geq 1.618^k$$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz.
$$\text{maxdeg}(Q) \leq 2 \log n$$

Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\rightarrow n \geq \text{size}(x) \geq 1.618^k$
 $\rightarrow k \leq \log_{1.618} n \leq 2 \log n$

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
 $\text{size}(x) \geq F_{k+2} \geq 1.618^k$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz. $\text{maxdeg}(Q) \leq 2 \log n$

Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\rightarrow n \geq \text{size}(x) \geq 1.618^k$
 $\rightarrow k \leq \log_{1.618} n \leq 2 \log n$ □

Maximaler Knotengrad



Lemma 2. Sei x ein Knoten mit $k = \deg(x)$. Sei $\text{size}(x)$ die Anzahl der Knoten im Teilbaum von x . Dann gilt:
 $\text{size}(x) \geq F_{k+2} \geq 1.618^k$

Sei $\text{maxdeg}(Q)$ der maximale Grad unter allen Wurzelknoten von Q .

Satz. $\text{maxdeg}(Q) \leq 2 \log n$

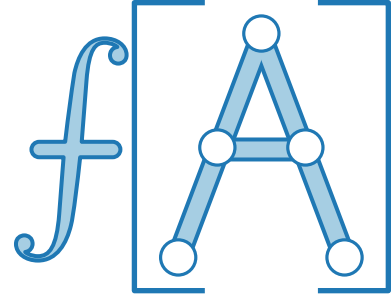
Beweis.

- Sei x ein Wurzelknoten mit $k = \deg(x) = \text{maxdeg}(Q)$.
- **Lemma 2** $\rightarrow n \geq \text{size}(x) \geq 1.618^k$
 $\rightarrow k \leq \log_{1.618} n \leq 2 \log n$ $\square$

Korollar. Die Anzahl der Wurzelknoten in einem **normierten** Fibonacci-Heap ist höchstens $2 \log n$.

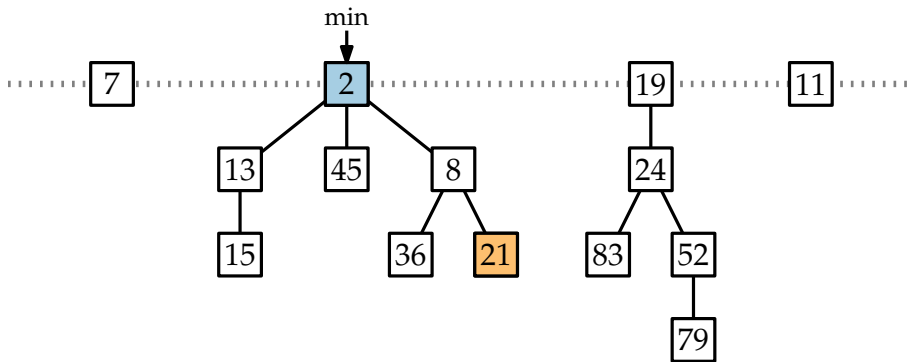
degArray

0	
1	11
2	
3	8

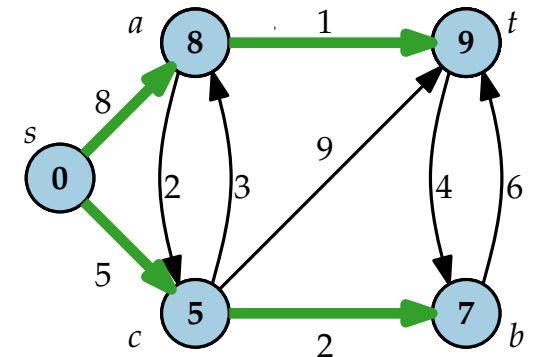


Fortgeschrittene Algorithmen

18. Vorlesung Datenstrukturen IV: Fibonacci-Heaps

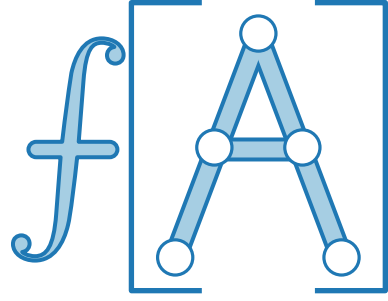


Teil III: LAUFZEITEN



Amortisierte Analyse: INSERT

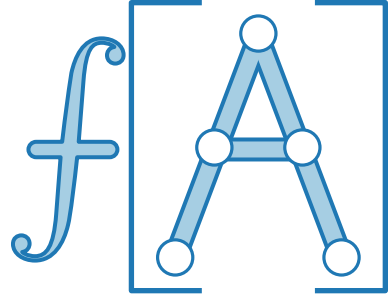
Buchhaltermethode zur Analyse der Kosten der Operationen



Amortisierte Analyse: INSERT

Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt



Amortisierte Analyse: INSERT



Buchhaltermethode zur Analyse der Kosten der Operationen

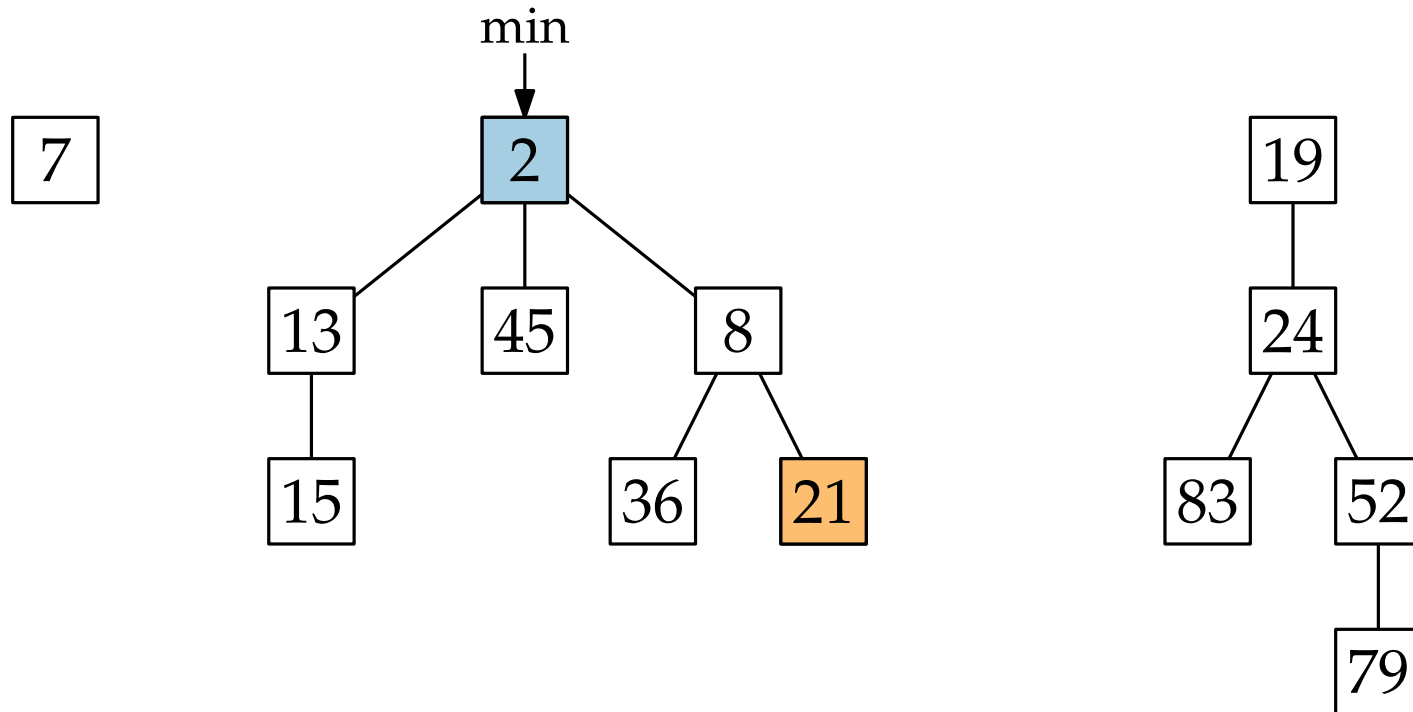
- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 

Amortisierte Analyse: INSERT



Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 

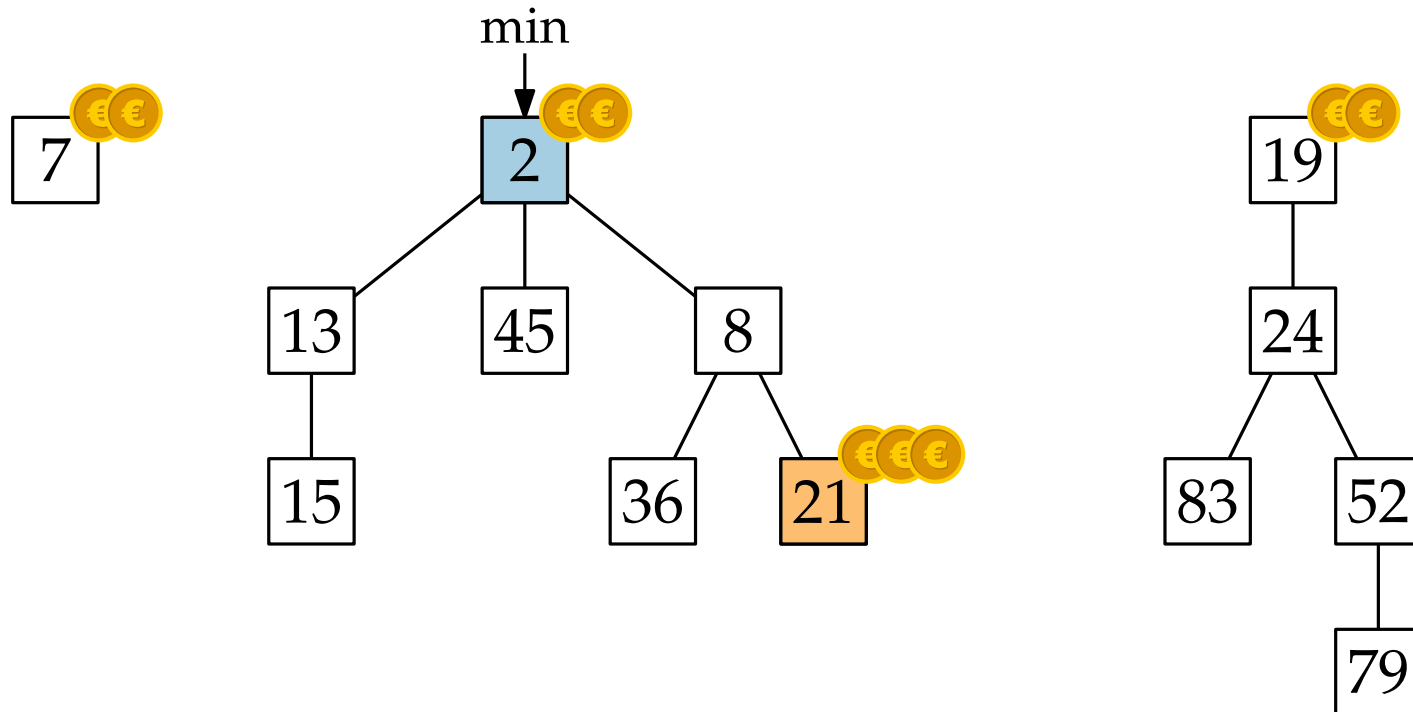


Amortisierte Analyse: INSERT



Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 

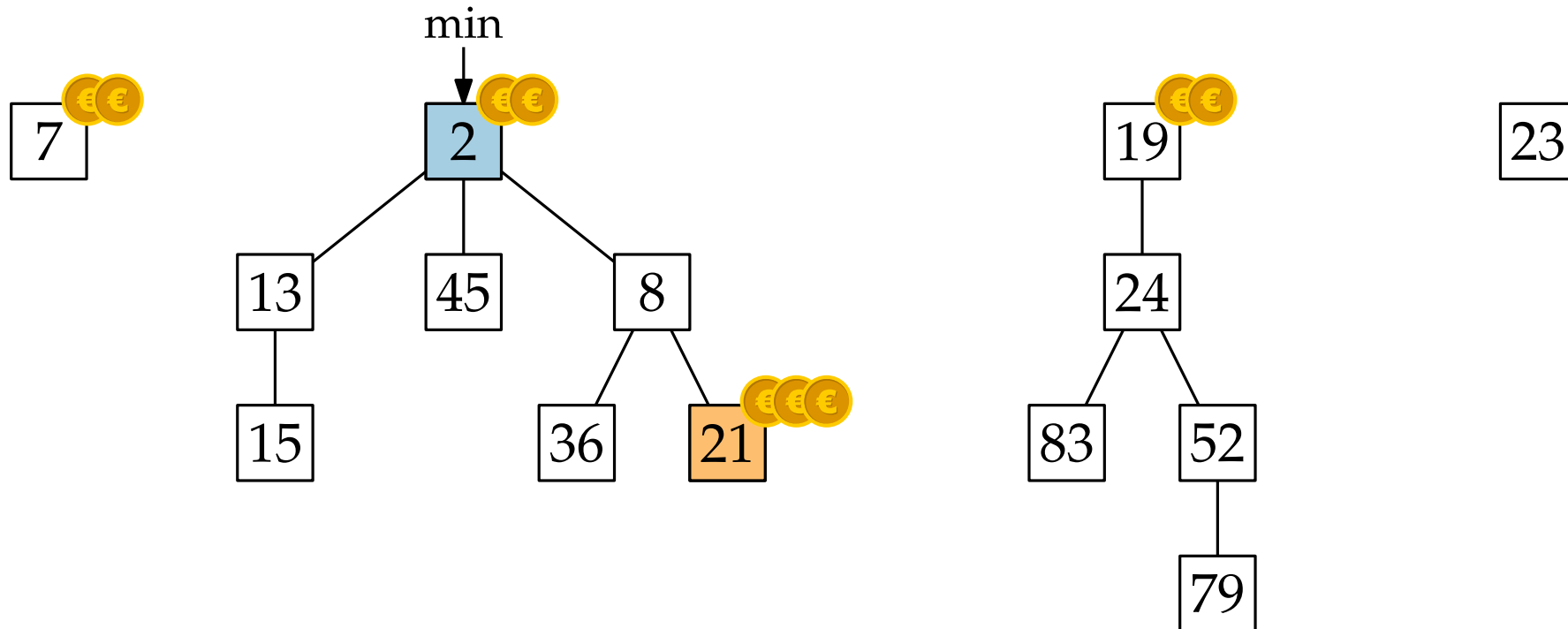


Amortisierte Analyse: INSERT



Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 

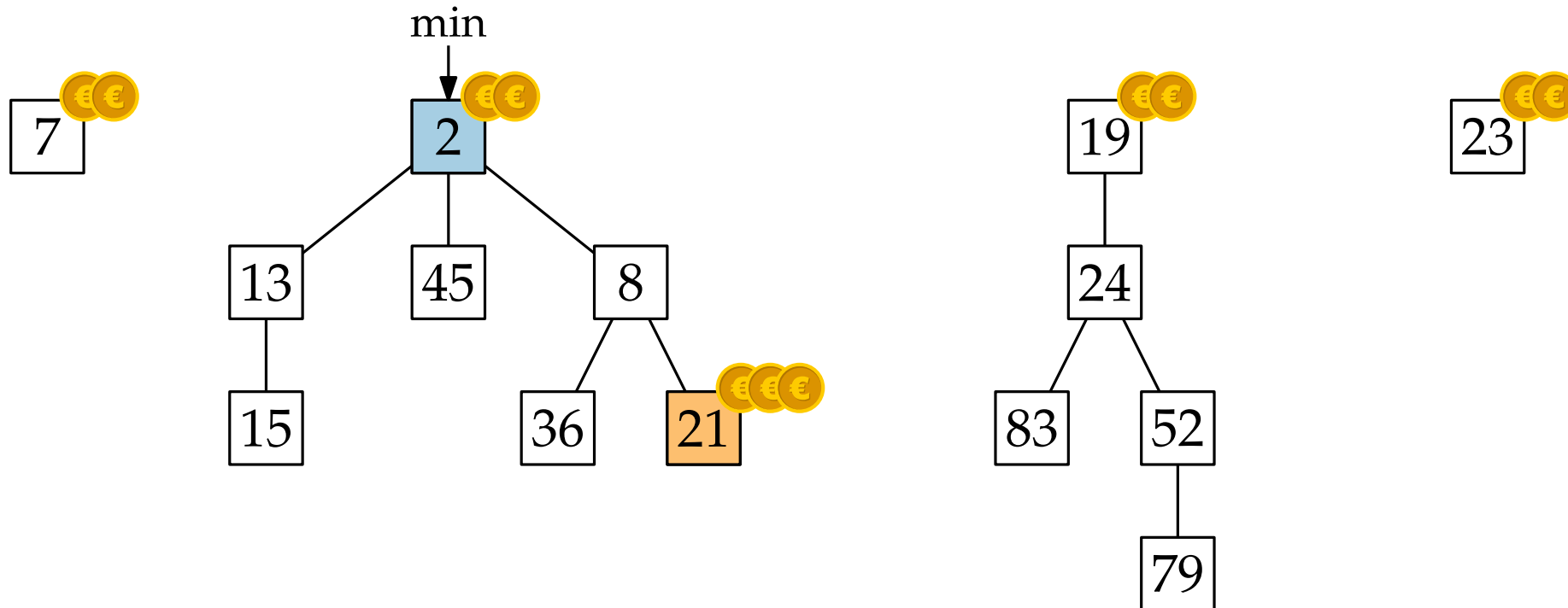


Amortisierte Analyse: INSERT



Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 

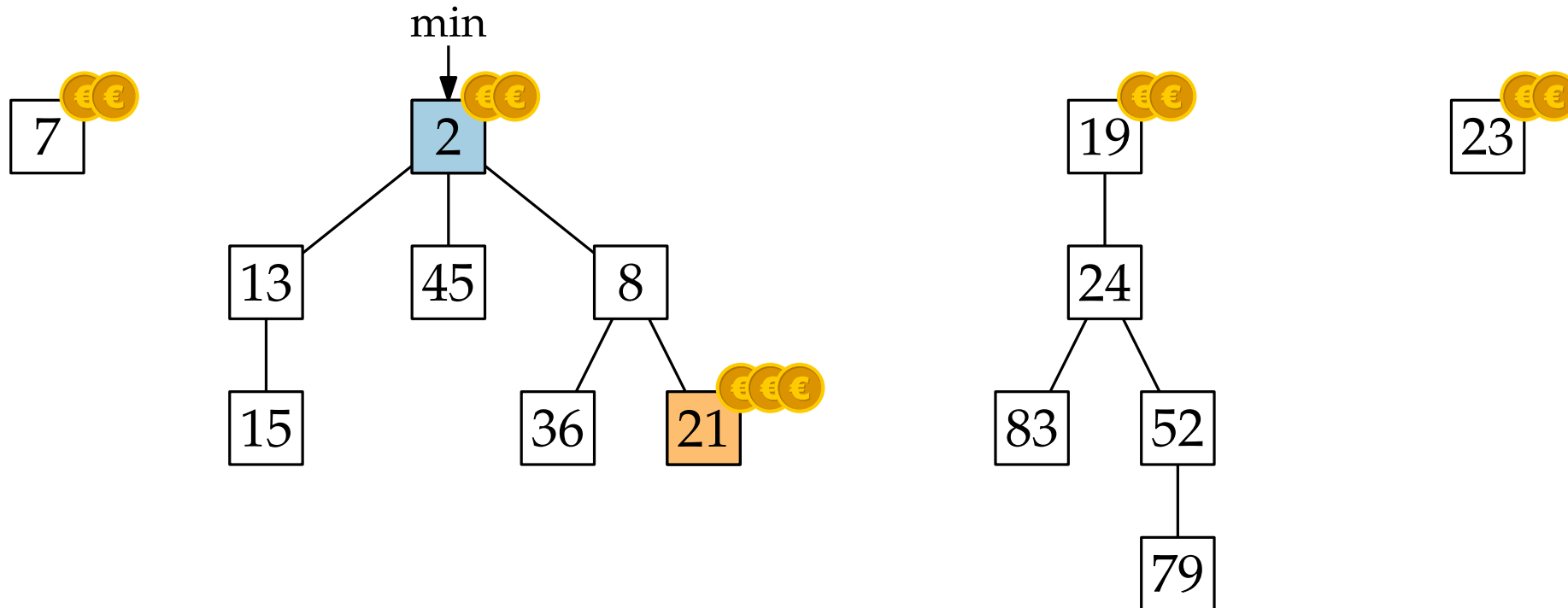


Amortisierte Analyse: INSERT



Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 
- Tatsächliche Kosten: $\mathcal{O}(1)$ (Einfügen + min-Pointer umsetzen)

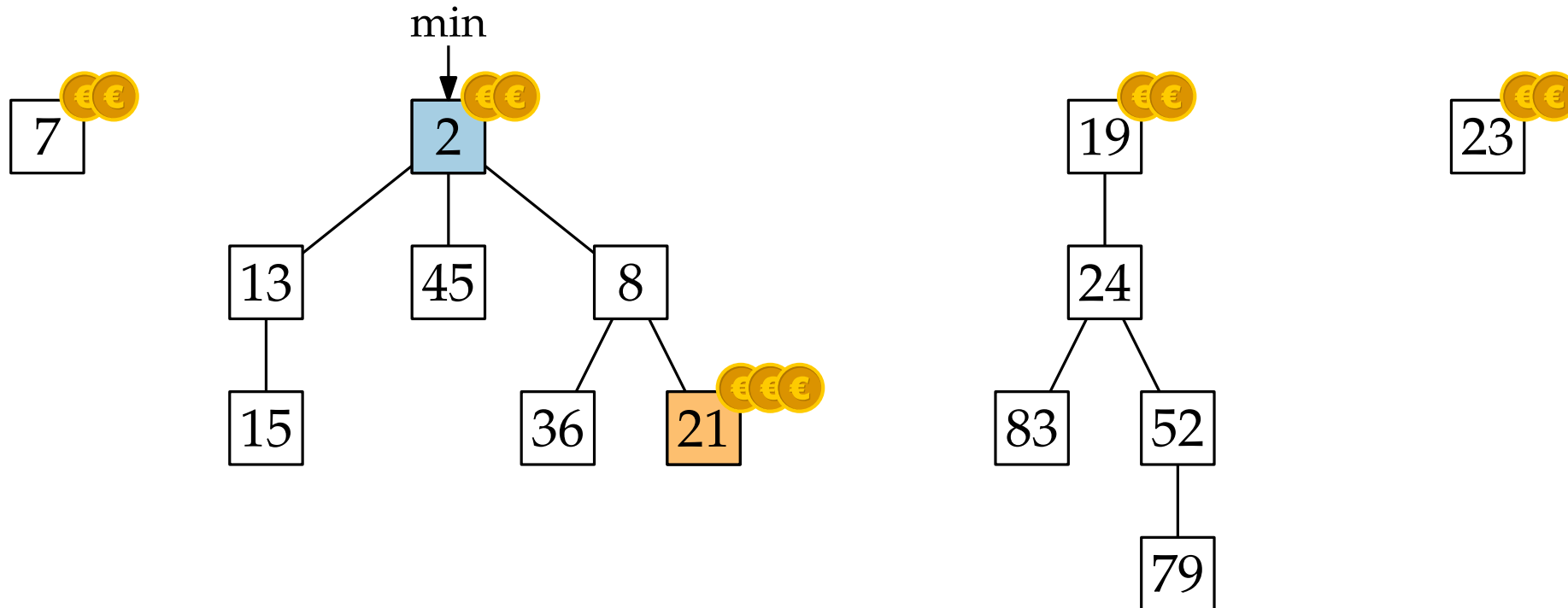


Amortisierte Analyse: INSERT



Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 
- Tatsächliche Kosten: $\mathcal{O}(1)$ (Einfügen + min-Pointer umsetzen)
- Amortisierte Kosten: $\mathcal{O}(1) + 2$ (Tatsächliche Kosten +  für neuen Wurzelknoten)



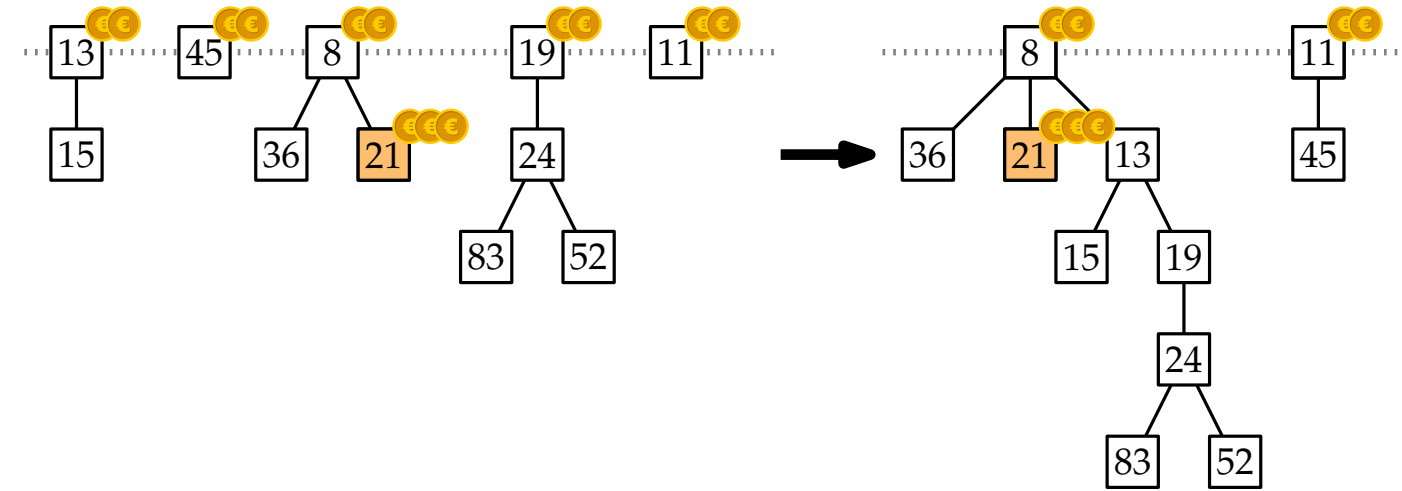
Amortisierte Analyse: CLEANUP



Buchhaltermethode zur Analyse der Kosten der Operationen

- Jeder **markierte Knoten** bekommt 
- Jeder Knoten in der Wurzelliste bekommt 

Amortisierte Analyse: CLEANUP



Q.CLEANUP()

R = neue Knoten in Wurzelliste

foreach *r* ∈ *R* do

 while **degArray**(deg(*r*)) ≠ nil do

r = **LINK**(*r*, **degArray**(deg(*r*)))

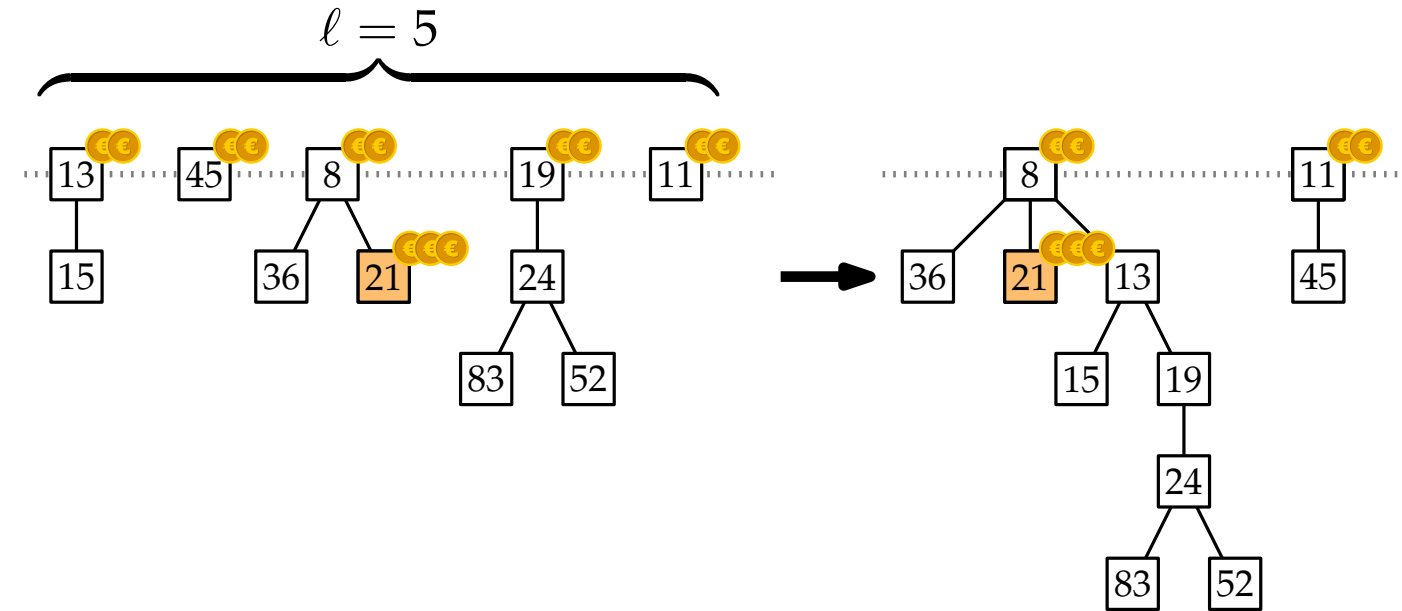
degArray(deg(*r*) - 1) = nil

degArray(deg(*r*)) = *r*

Amortisierte Analyse: CLEANUP



- ℓ Knoten zu Beginn in Wurzelliste



Q.CLEANUP()

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

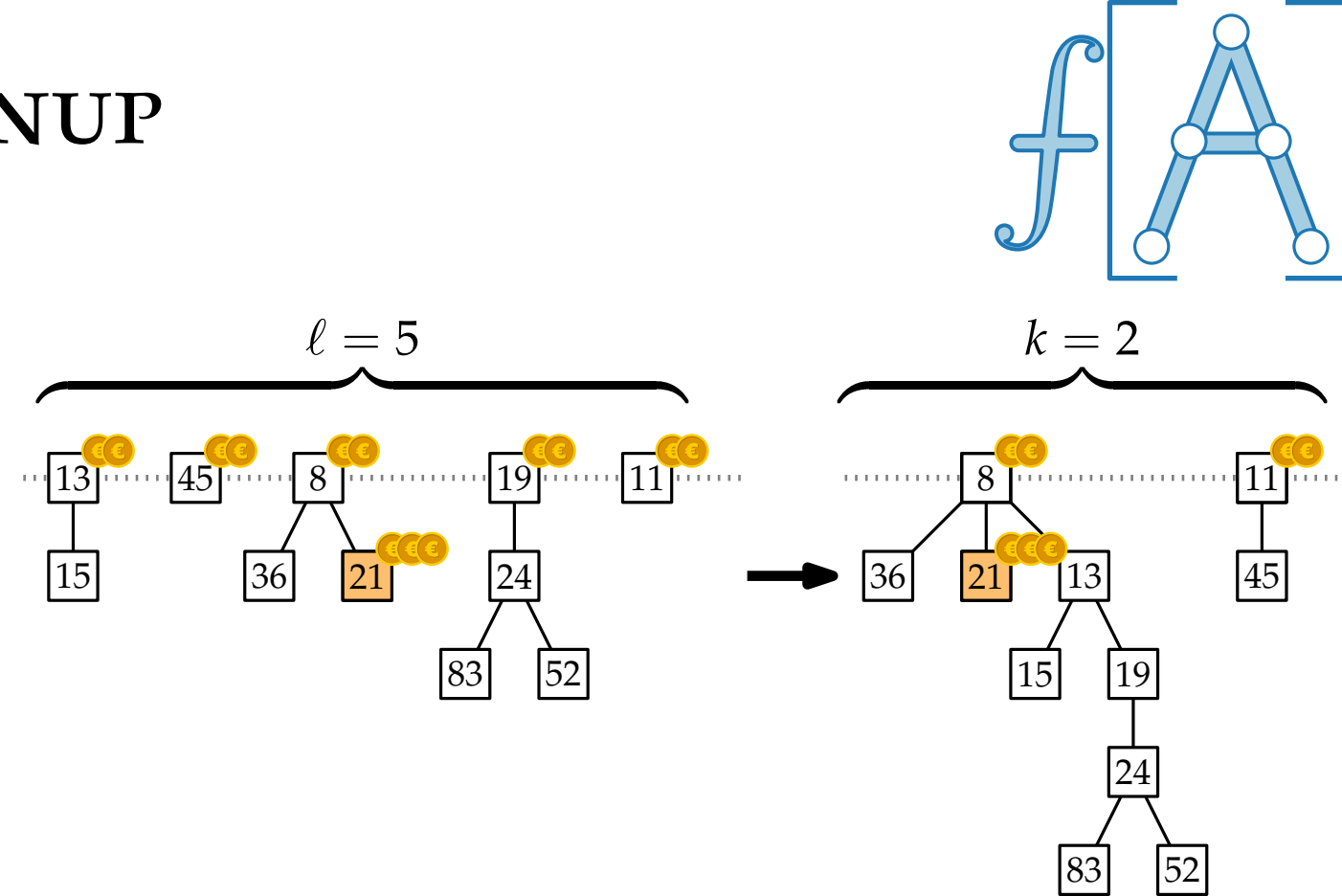
$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste



Q.CLEANUP()

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while **degArray**(deg(r)) $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

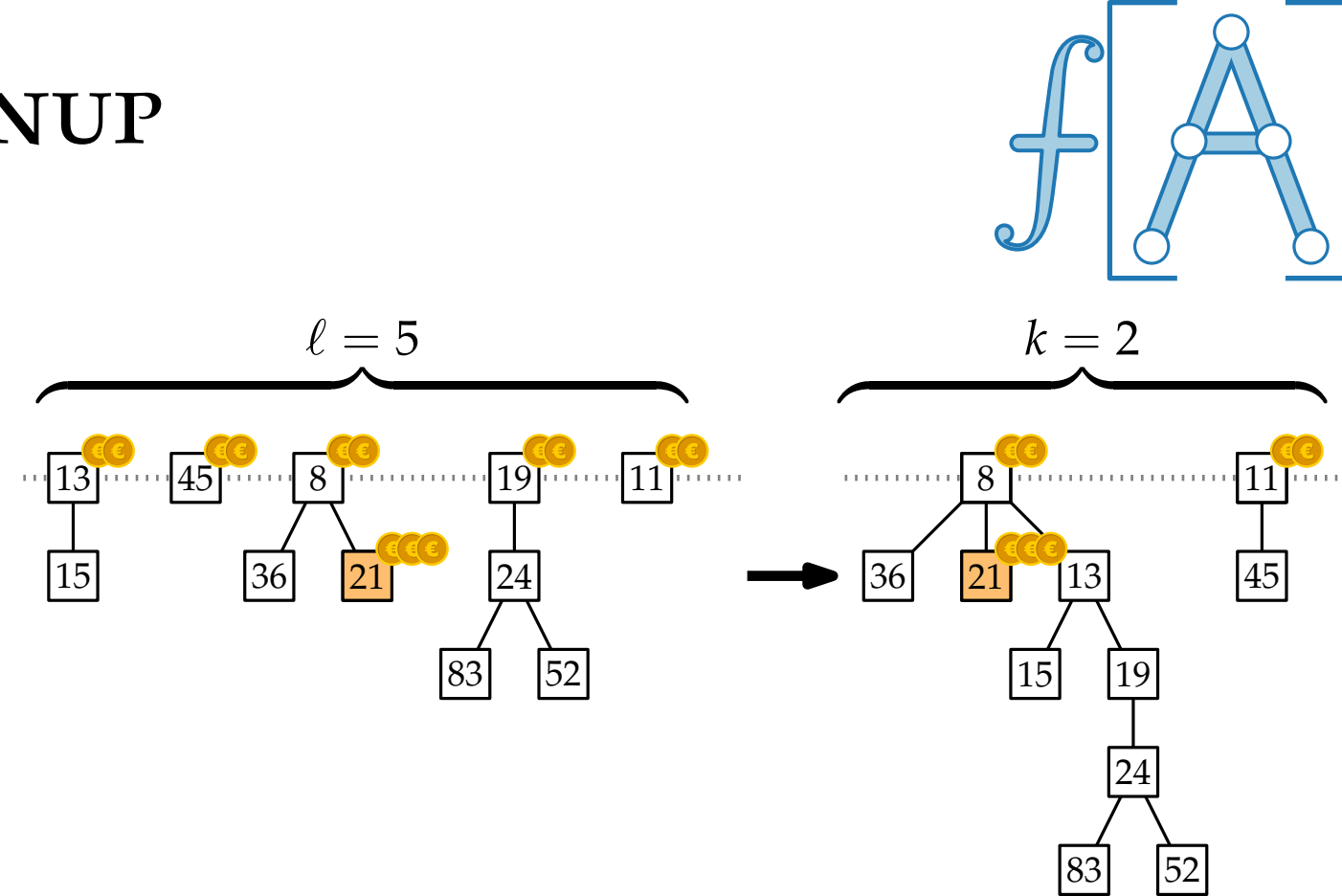
degArray(deg(r) - 1) = nil

degArray(deg(r)) = r

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

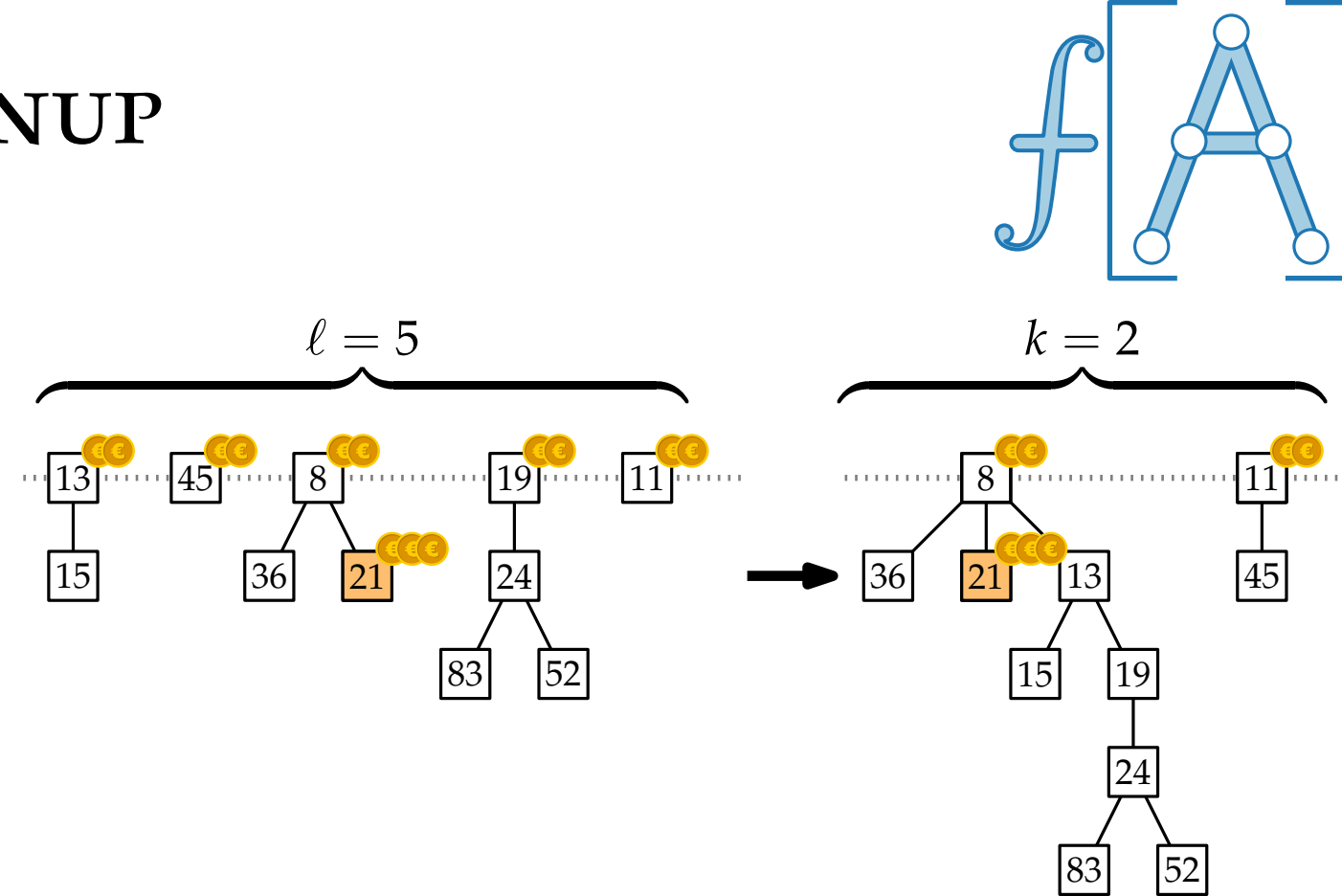
`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq$ `nil` do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

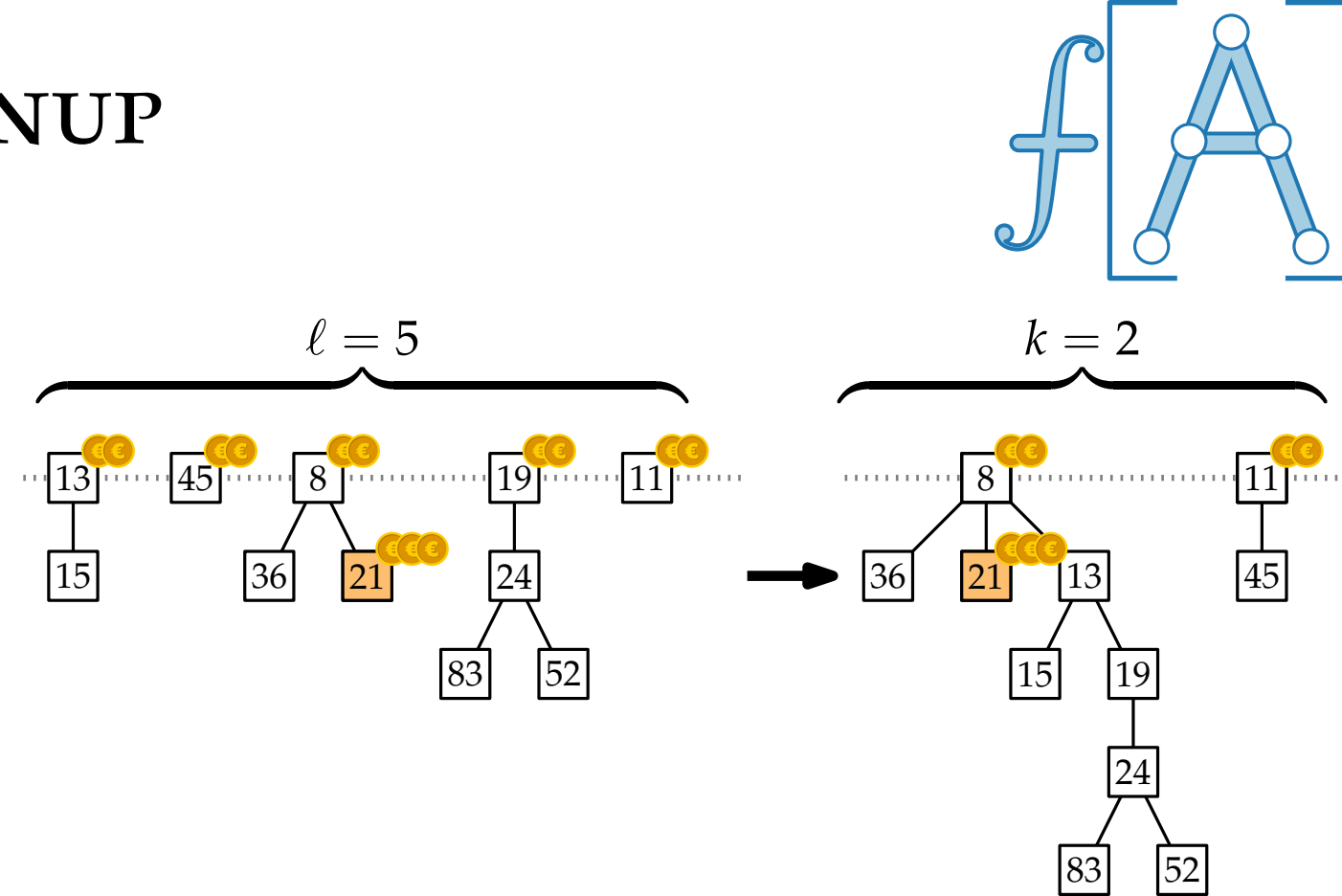
`degArray(deg( $r$ ))` = r

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq$ `nil` do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

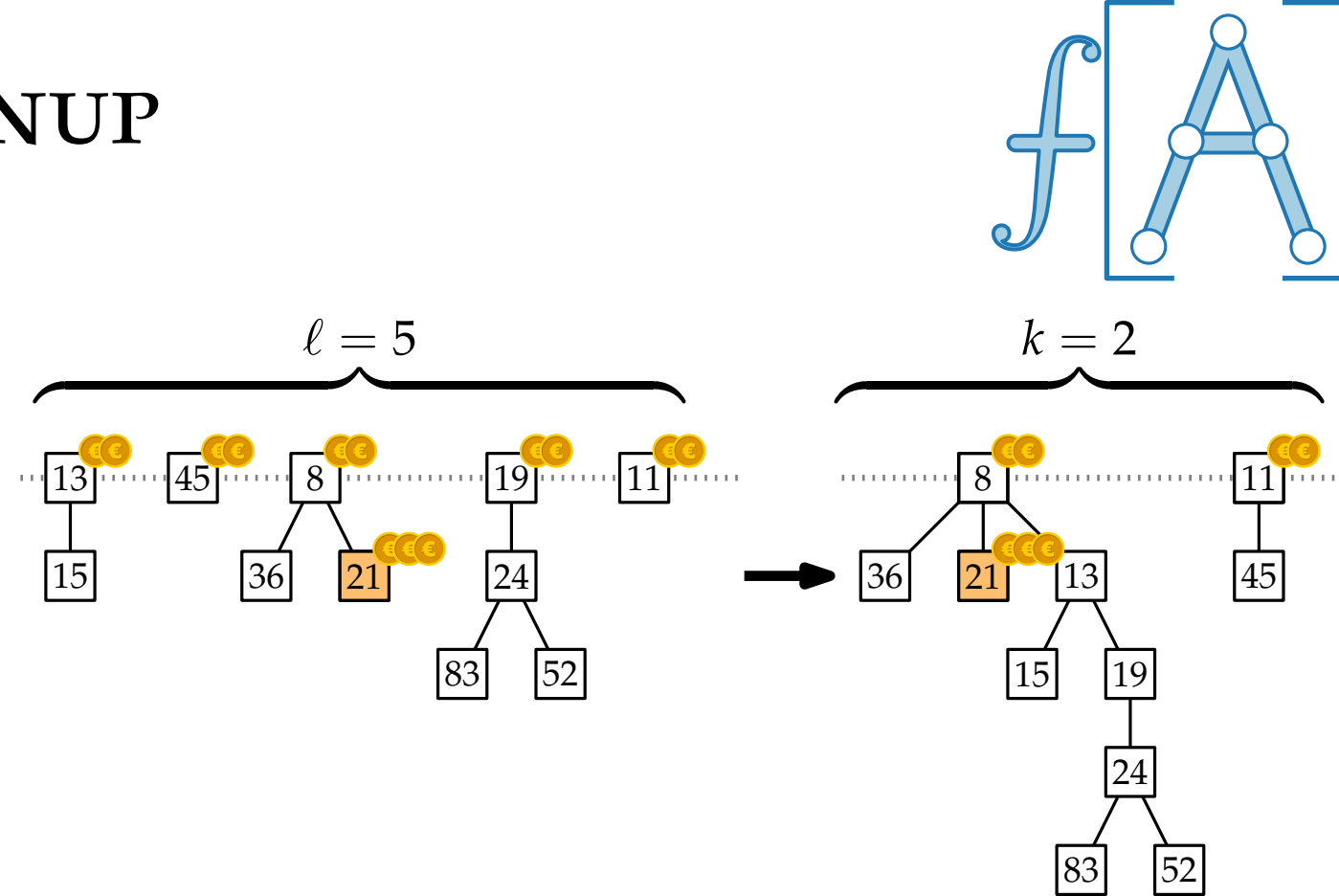
`degArray(deg( $r$ ))` = r

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq$ `nil` do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1) = nil`

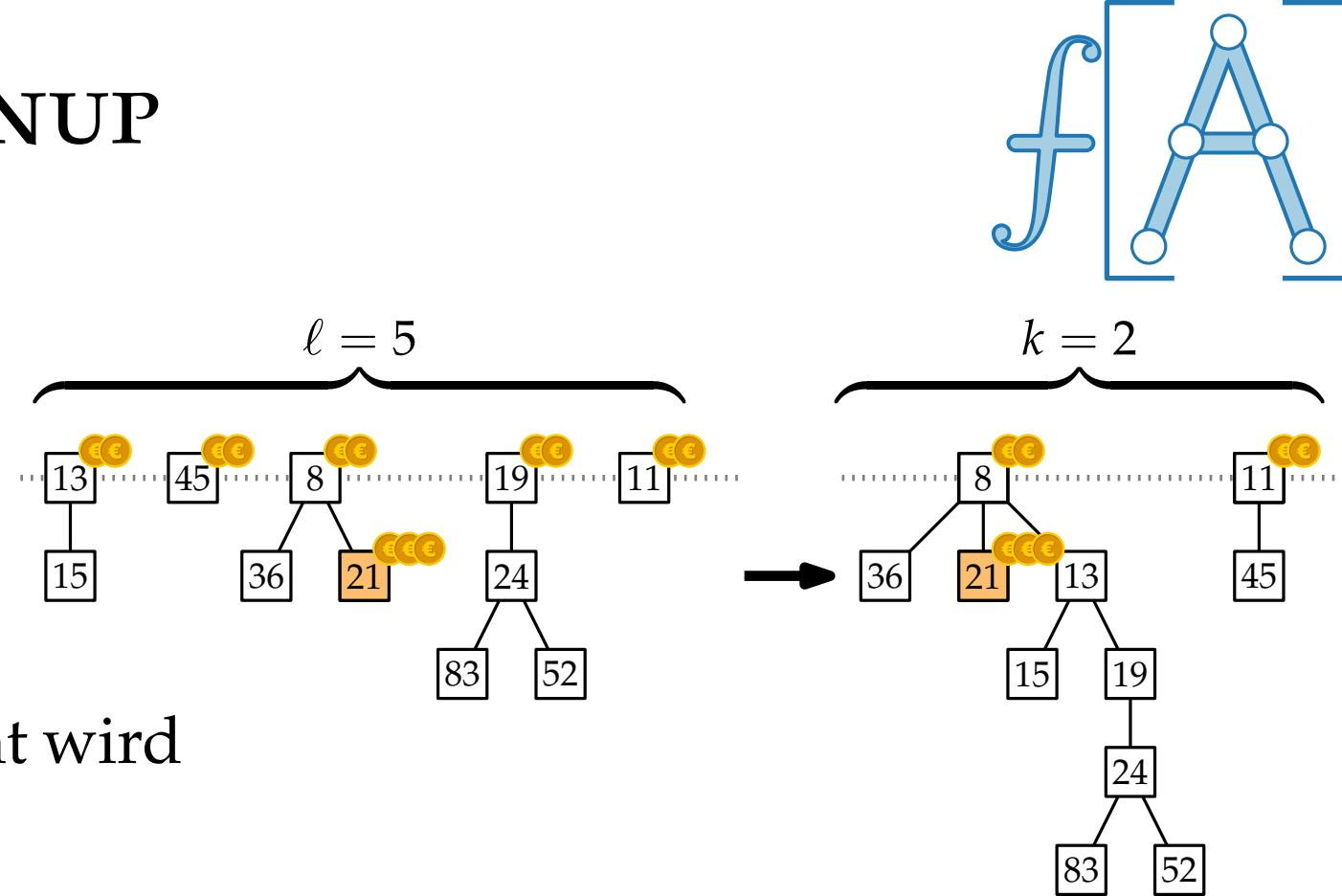
`degArray(deg( $r$ )) =  $r$`

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
- 1 pro Knoten, der aus Wurzelliste entfernt wird



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq$ `nil` do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1) = nil`

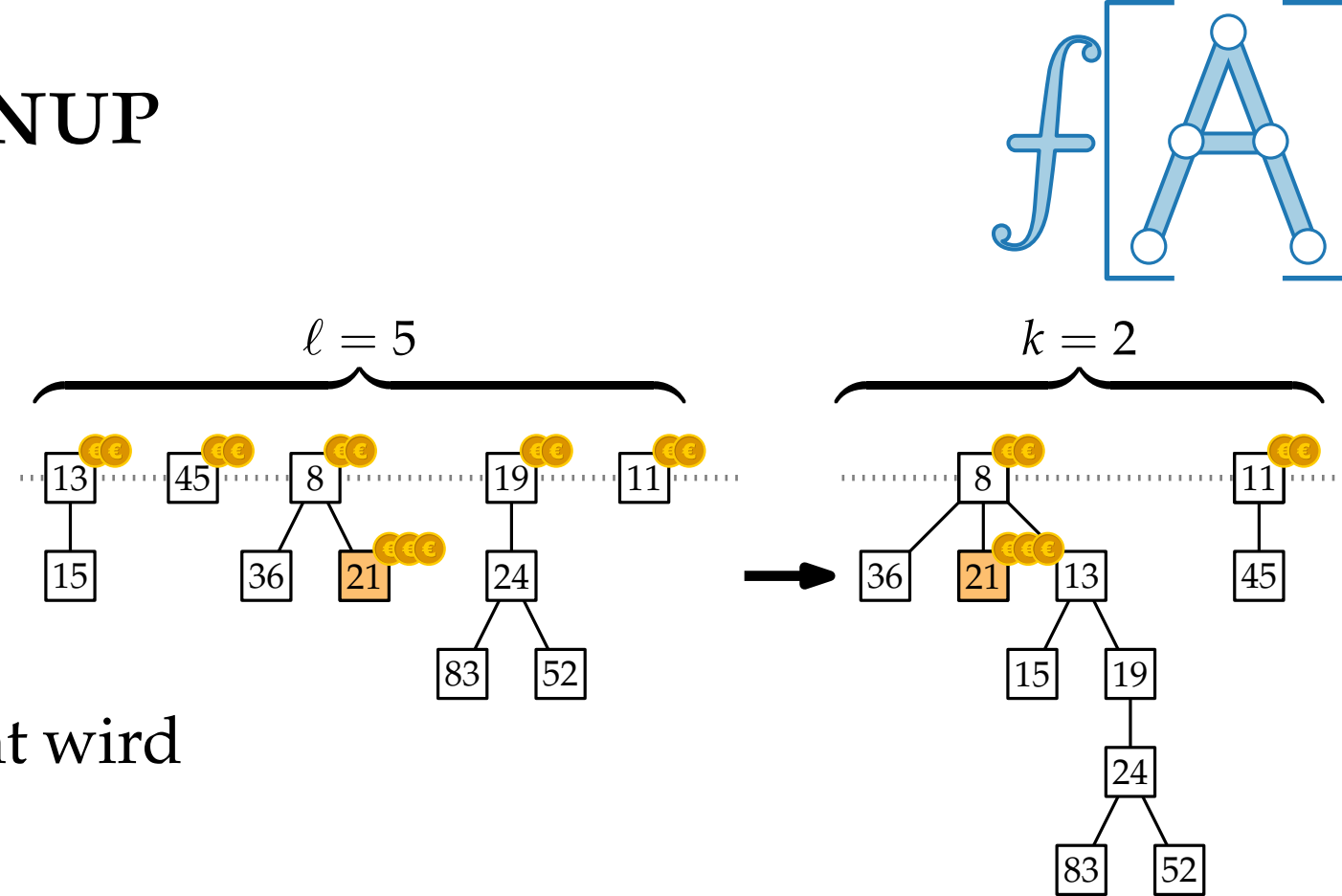
`degArray(deg( $r$ )) =  $r$`

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
- 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq nil$ do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

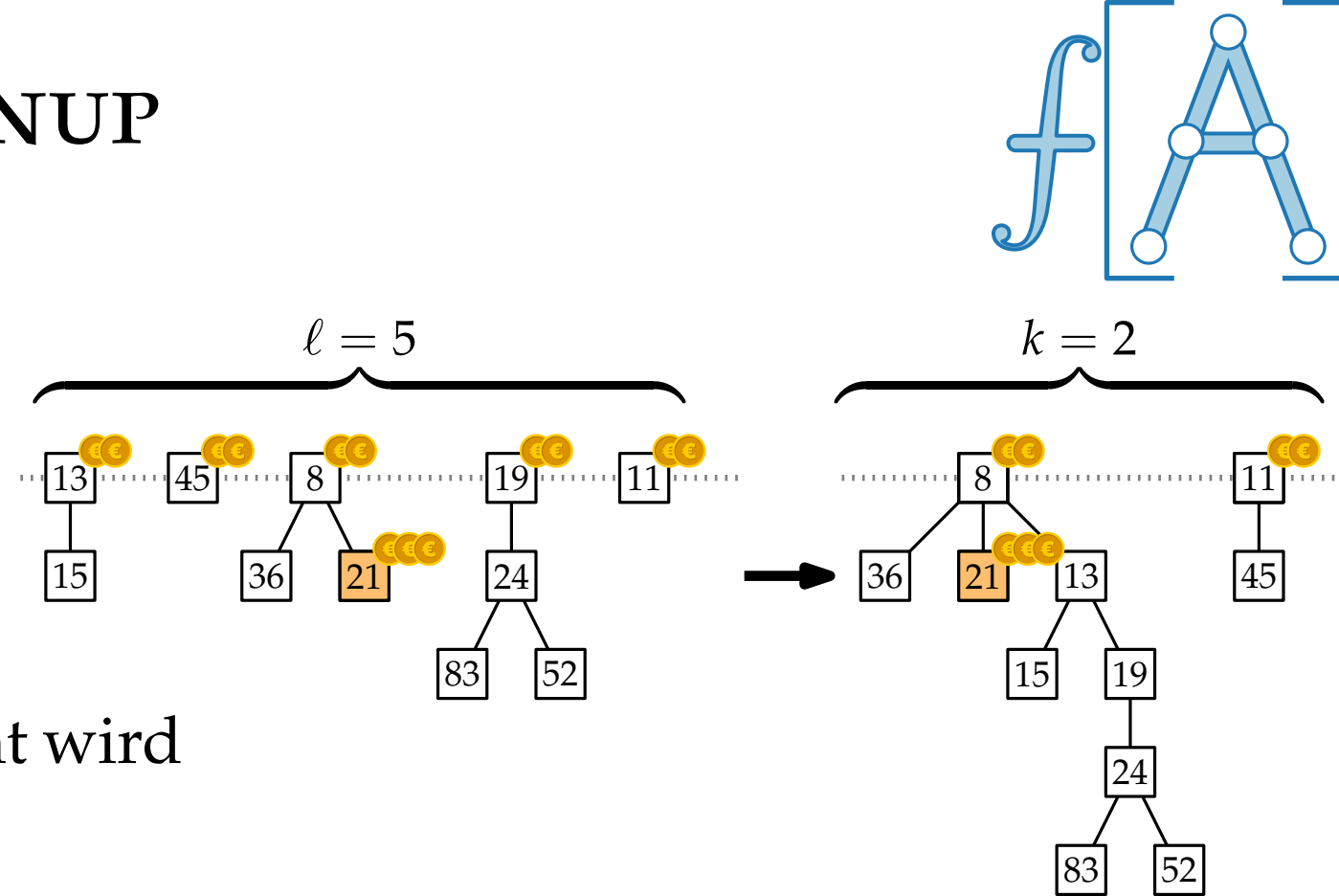
`degArray(deg( $r$ ))` = r

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq$ `nil` do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

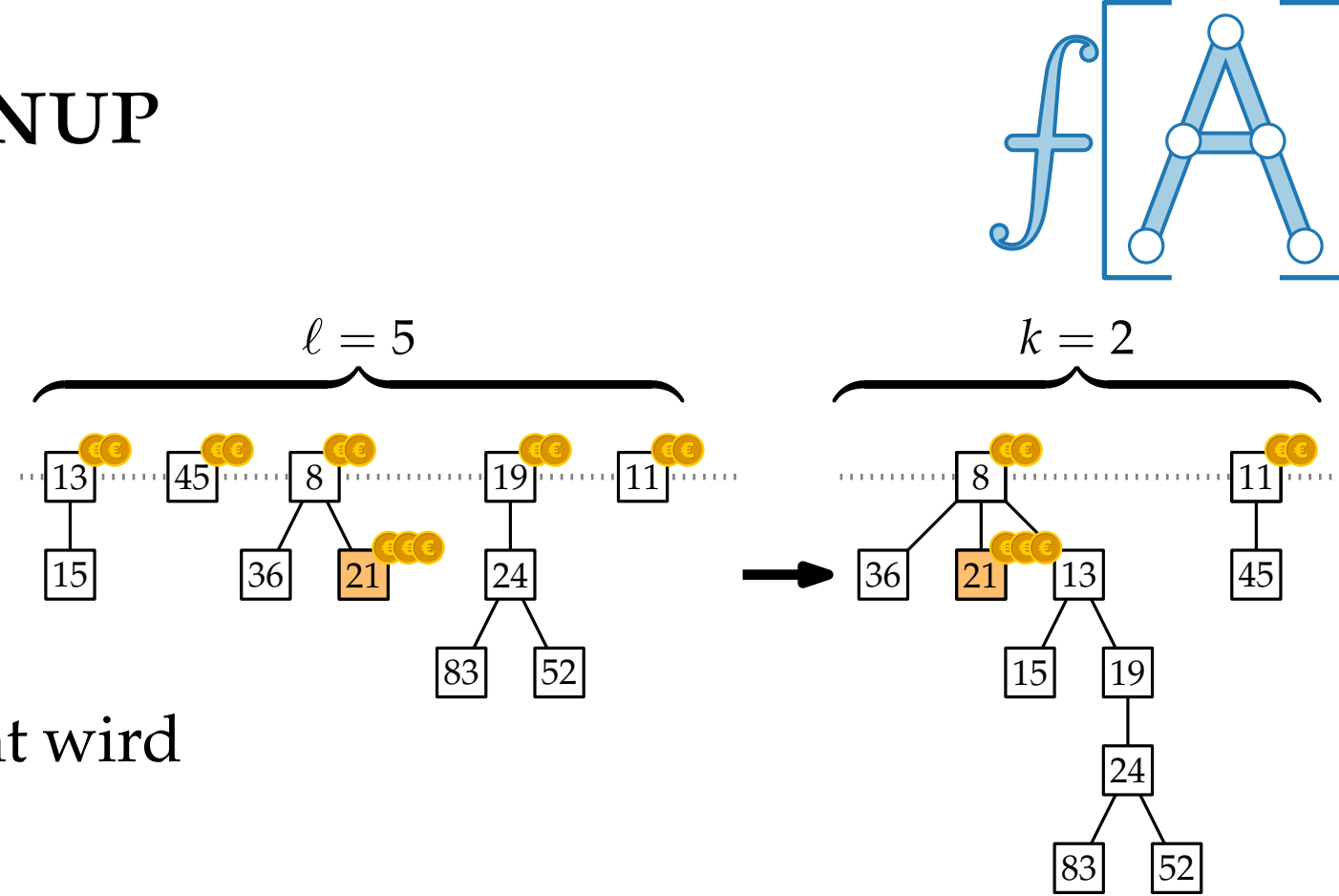
`degArray(deg( $r$ ))` = r

Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ do

 while `degArray(deg( $r$ ))` $\neq$ `nil` do

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1) = nil`

`degArray(deg( $r$ )) =  $r$`

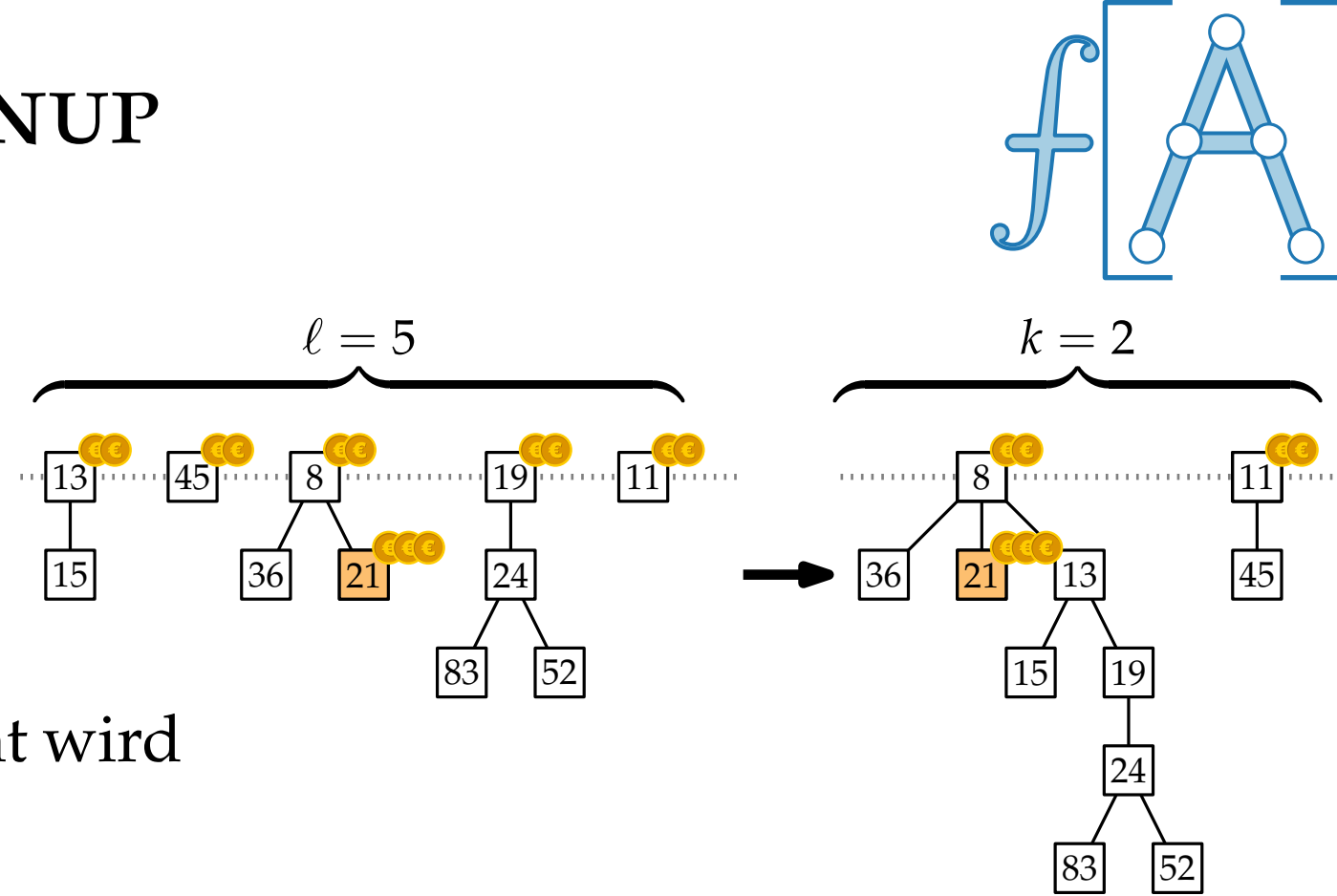
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq$ `nil` **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

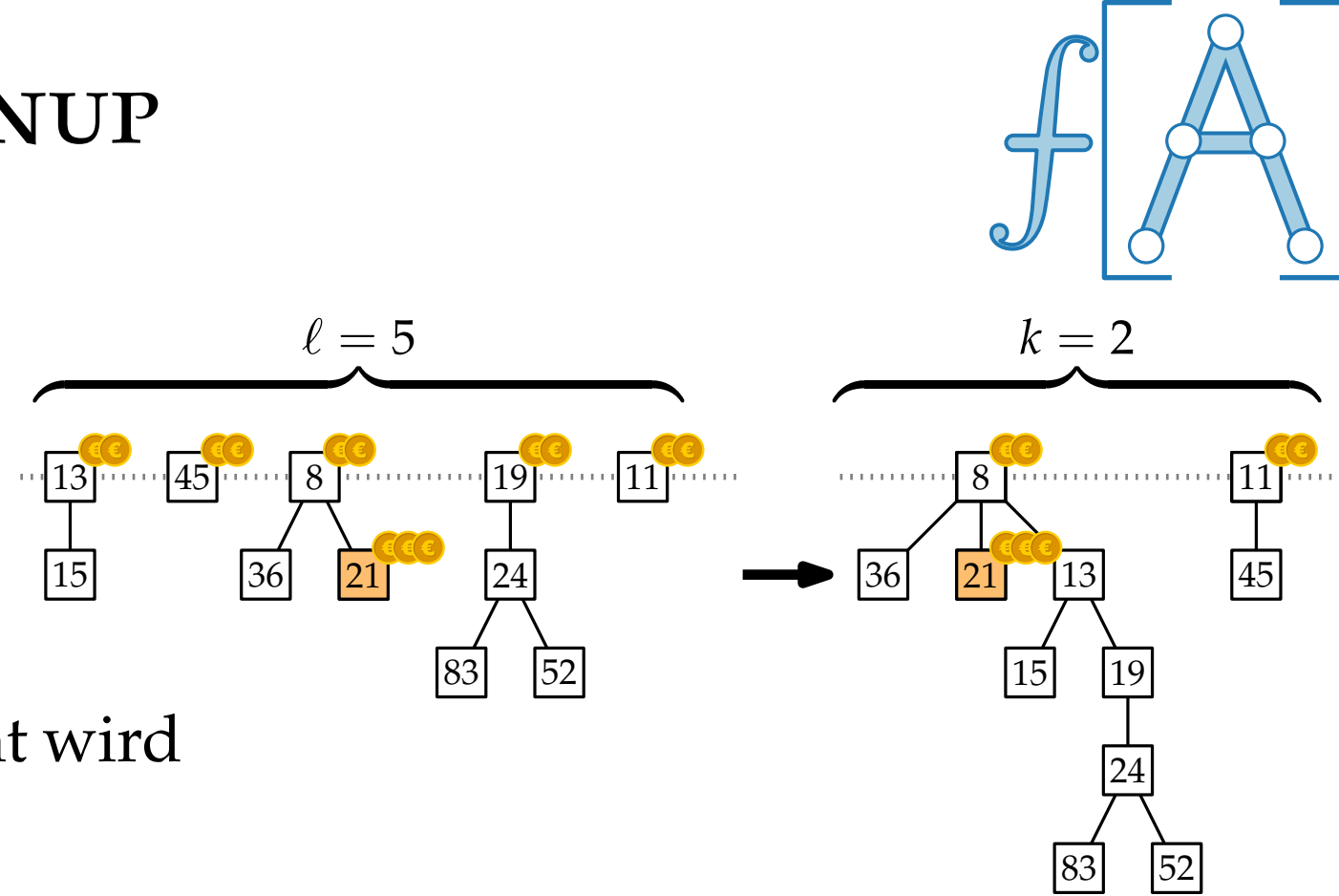
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
- Wir können davon $2(\ell - k)$ Kosten bezahlen



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq$ `nil` **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

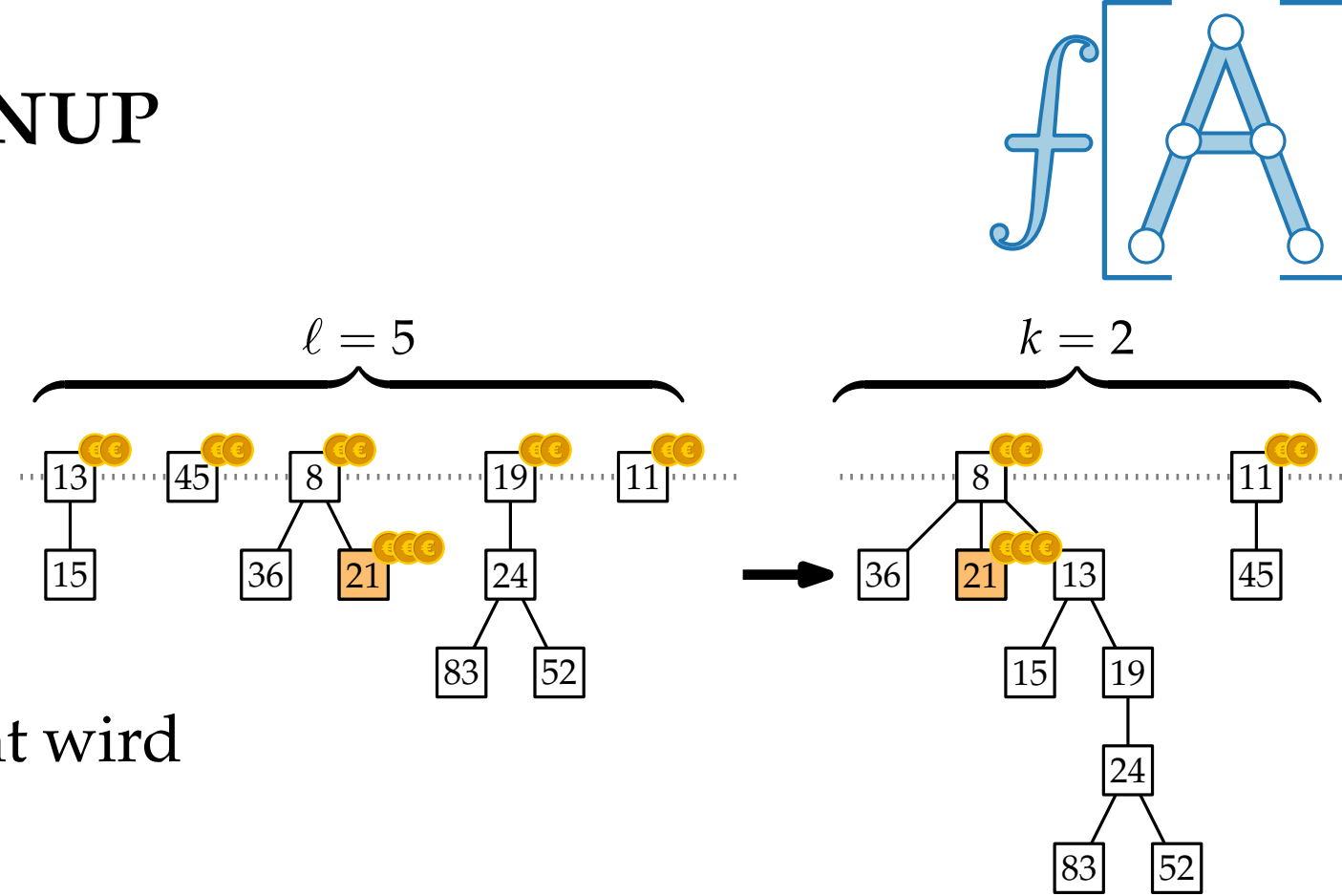
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
 - Wir können davon $2(\ell - k)$ Kosten bezahlen
- ➔ Amortisierte Kosten:



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq$ `nil` **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

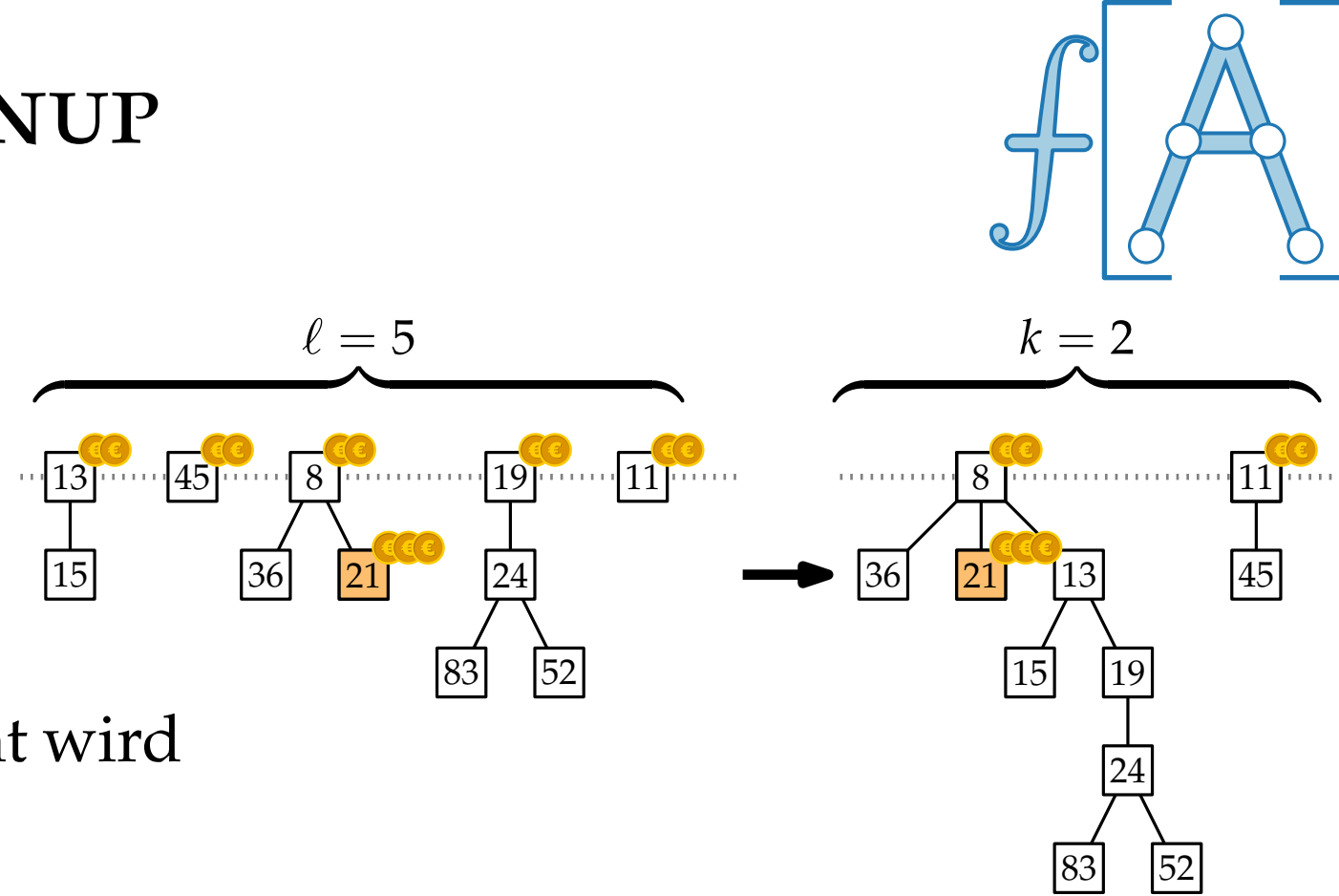
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
 - Wir können davon $2(\ell - k)$ Kosten bezahlen
- ➔ Amortisierte Kosten: $2\ell - k$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq$ `nil` **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

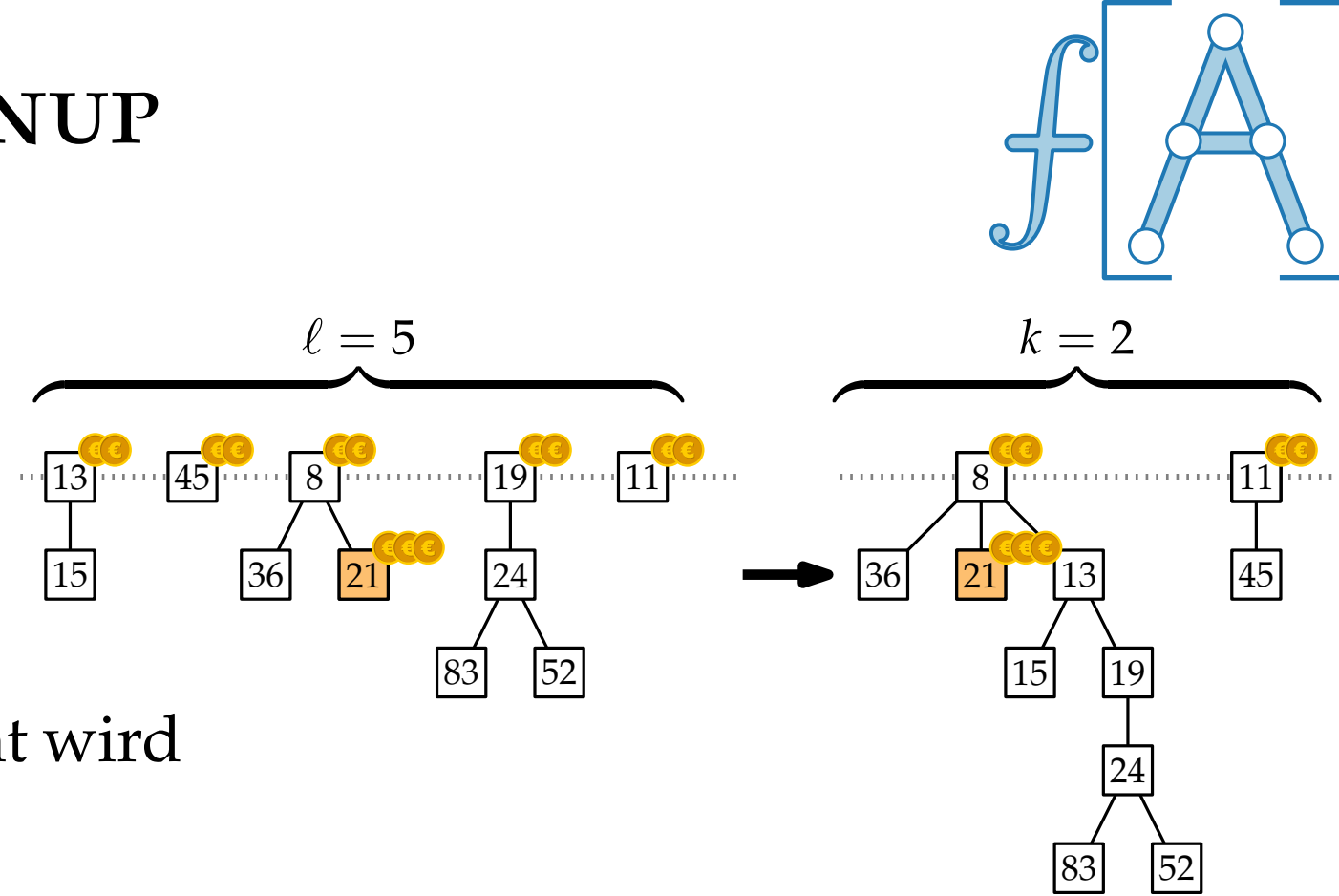
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
 - Wir können davon $2(\ell - k)$ Kosten bezahlen
- ➔ Amortisierte Kosten: $2\ell - k - 2(\ell - k)$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq$ `nil` **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

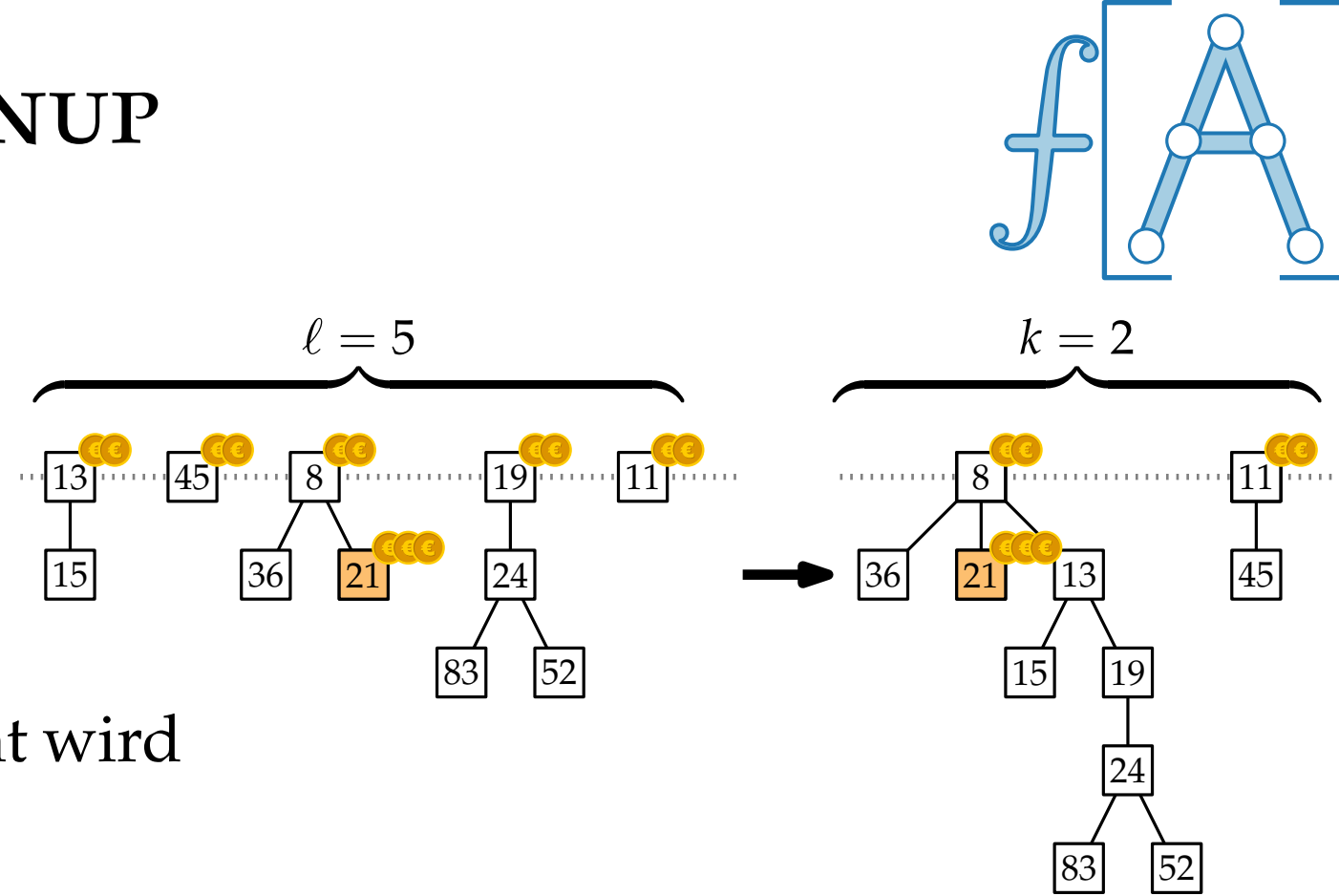
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
 - Wir können davon $2(\ell - k)$ Kosten bezahlen
- ➔ Amortisierte Kosten: $2\ell - k - 2(\ell - k) = k$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq \text{nil}$ **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

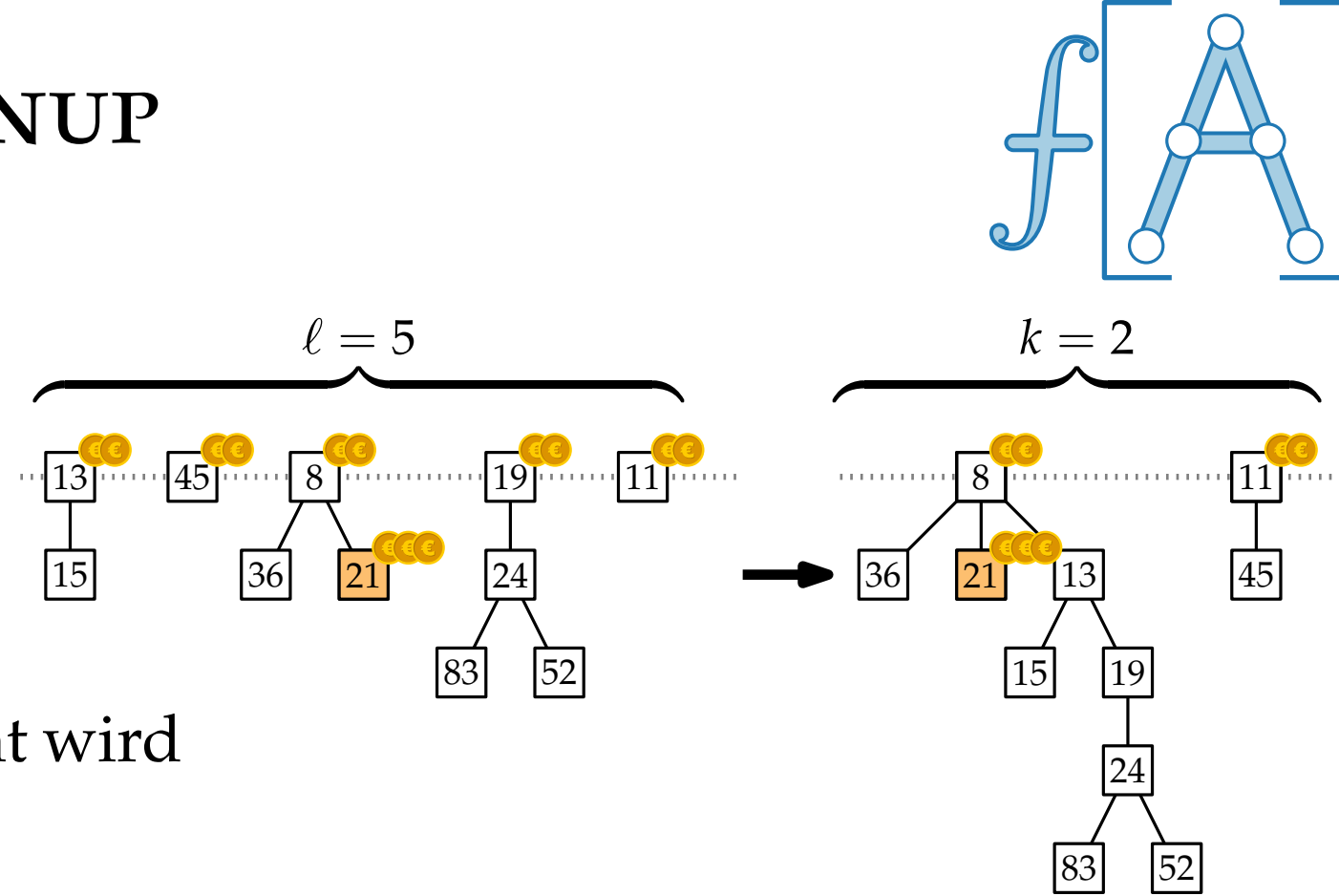
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
 - Wir können davon $2(\ell - k)$ Kosten bezahlen
- ➔ Amortisierte Kosten: $2\ell - k - 2(\ell - k) = k \in \mathcal{O}(k)$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq \text{nil}$ **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

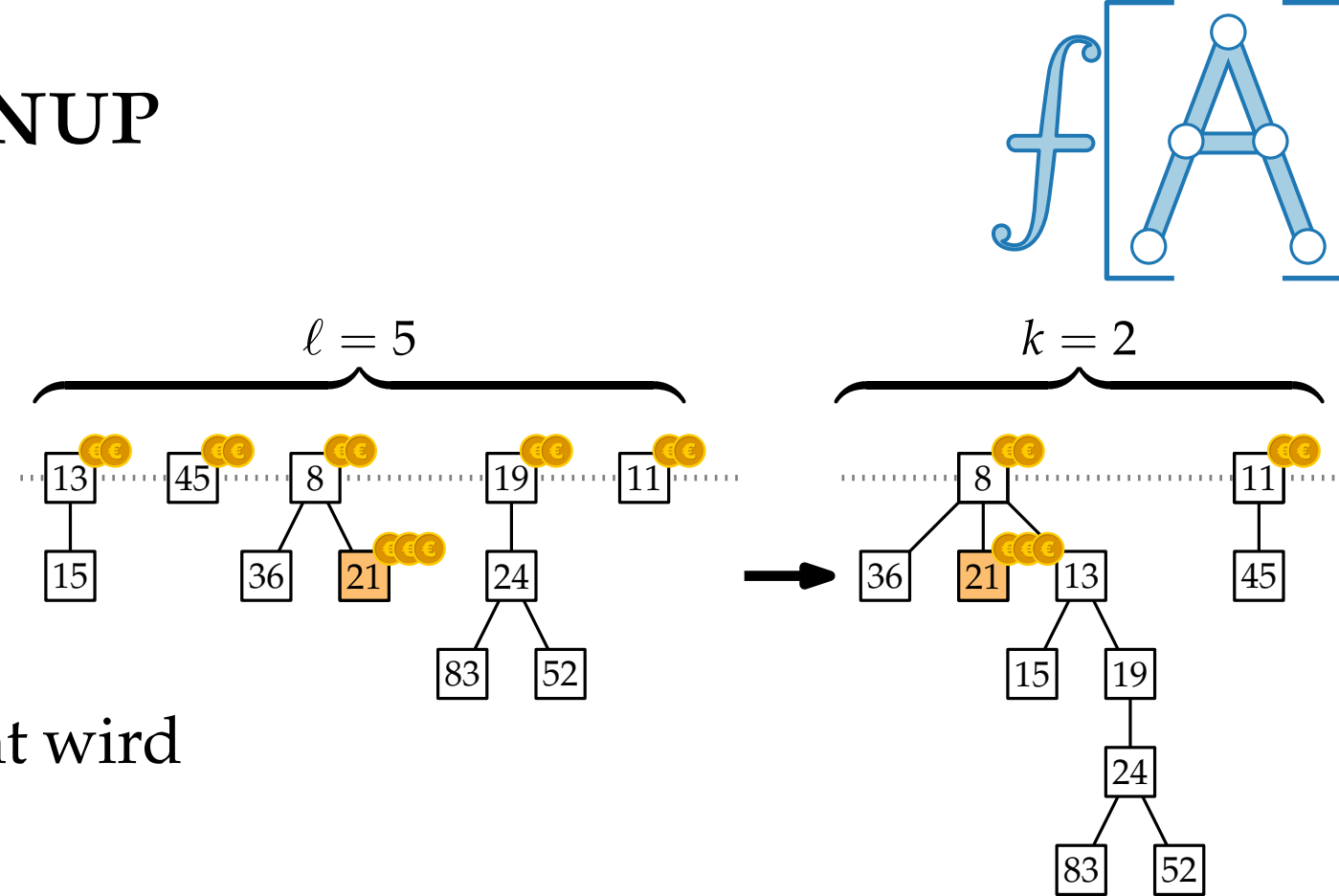
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
 - Wir können davon $2(\ell - k)$ Kosten bezahlen
- ➔ Amortisierte Kosten: $2\ell - k - 2(\ell - k) = k \in \mathcal{O}()$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq \text{nil}$ **do**

$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

Korollar. Die Anzahl der Wurzelknoten in einem normierten Fibonacci-Heap ist höchstens $2 \log n$.

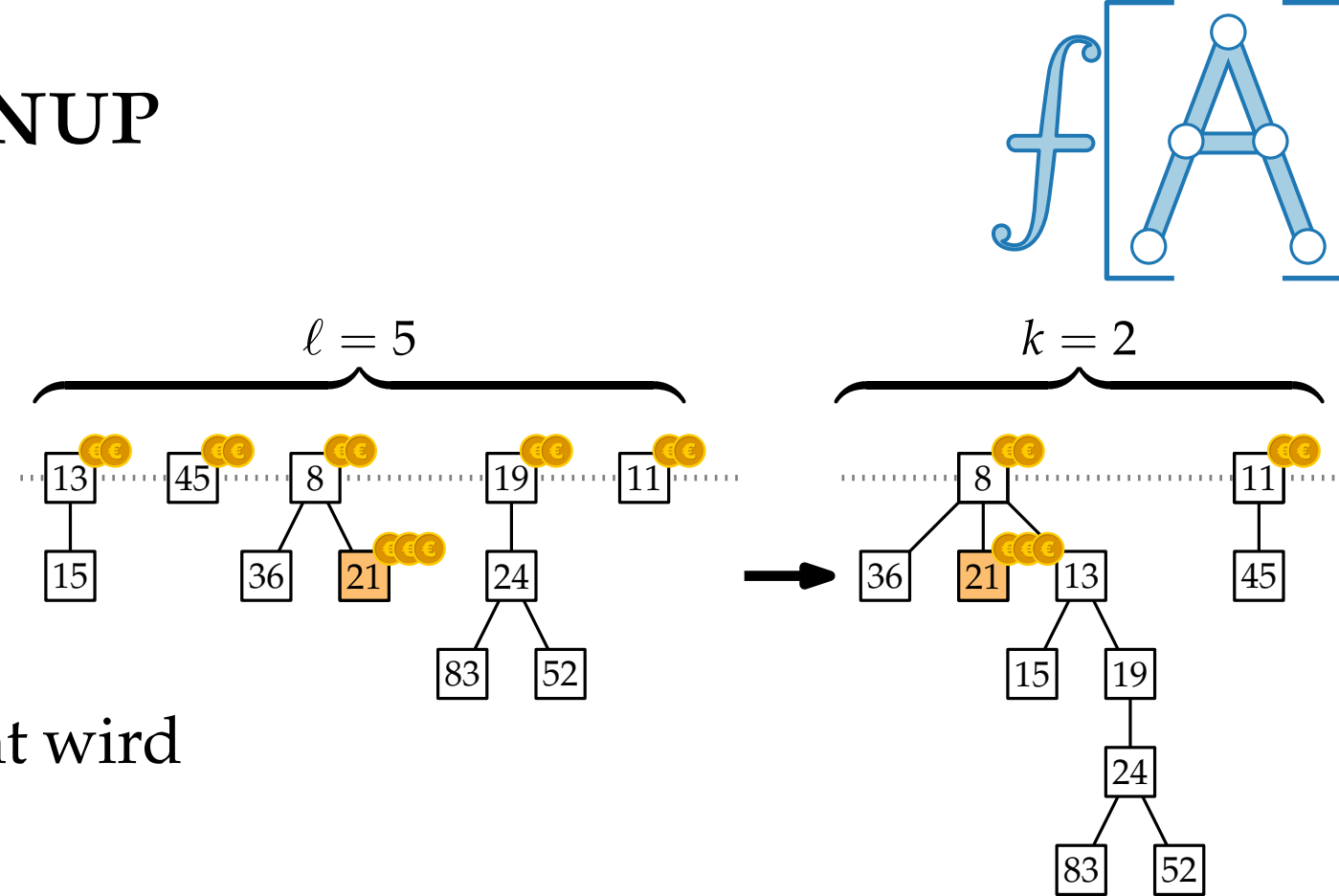
Amortisierte Analyse: CLEANUP

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Wie viele Vergleiche insgesamt?

- 1 pro neuem Knoten in Wurzelliste
 $\leq \ell$
 - 1 pro Knoten, der aus Wurzelliste entfernt wird
 $= \ell - k$
- ➔ Tatsächliche Kosten: $2\ell - k \in \mathcal{O}(n)$

- Wir nehmen uns €€ von jedem Knoten, der aus der Wurzelliste entfernt wird
 - Wir können davon $2(\ell - k)$ Kosten bezahlen
- ➔ Amortisierte Kosten: $2\ell - k - 2(\ell - k) = k \in \mathcal{O}(\log n)$



`Q.CLEANUP()`

R = neue Knoten in Wurzelliste

foreach $r \in R$ **do**

while `degArray(deg( $r$ ))` $\neq \text{nil}$ **do**

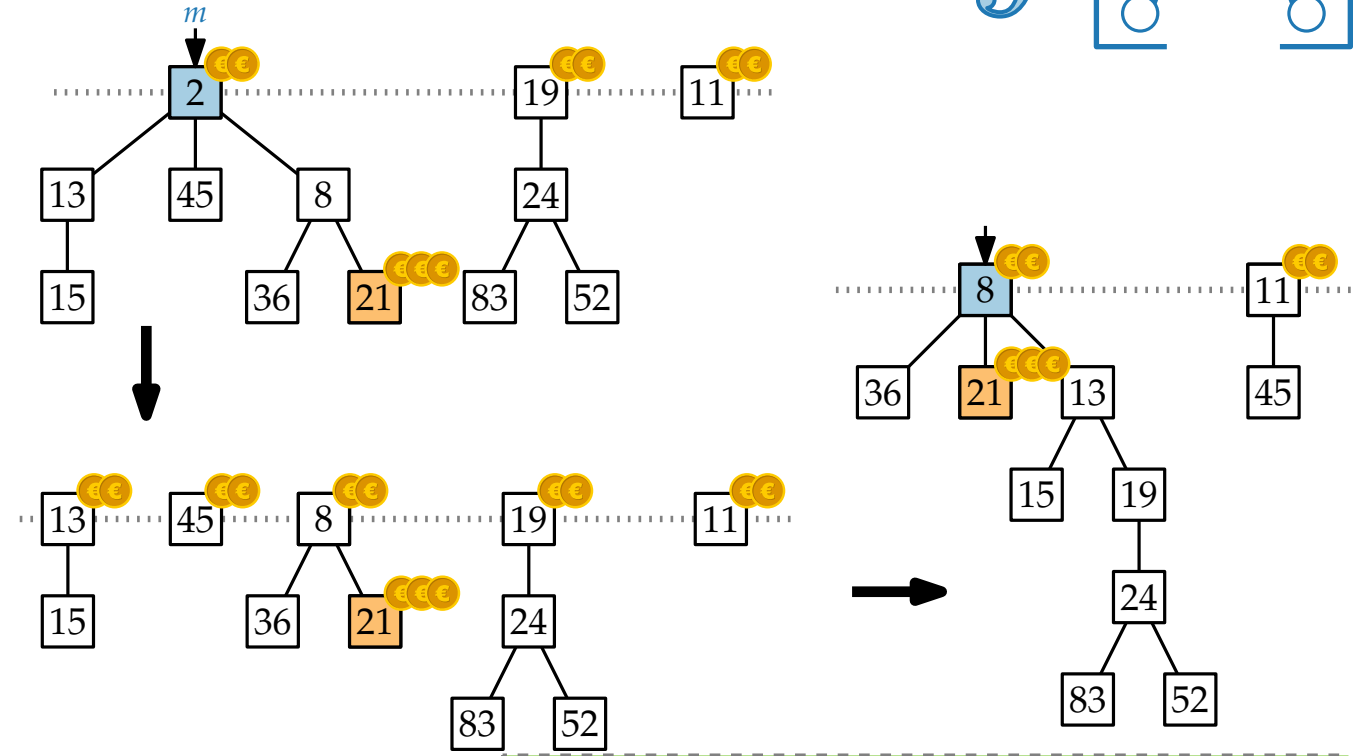
$r = \text{LINK}(r, \text{degArray}(\text{deg}(r)))$

`degArray(deg( $r$ ) - 1)` = `nil`

`degArray(deg( $r$ ))` = r

Korollar. Die Anzahl der Wurzelknoten in einem normierten Fibonacci-Heap ist höchstens $2 \log n$.

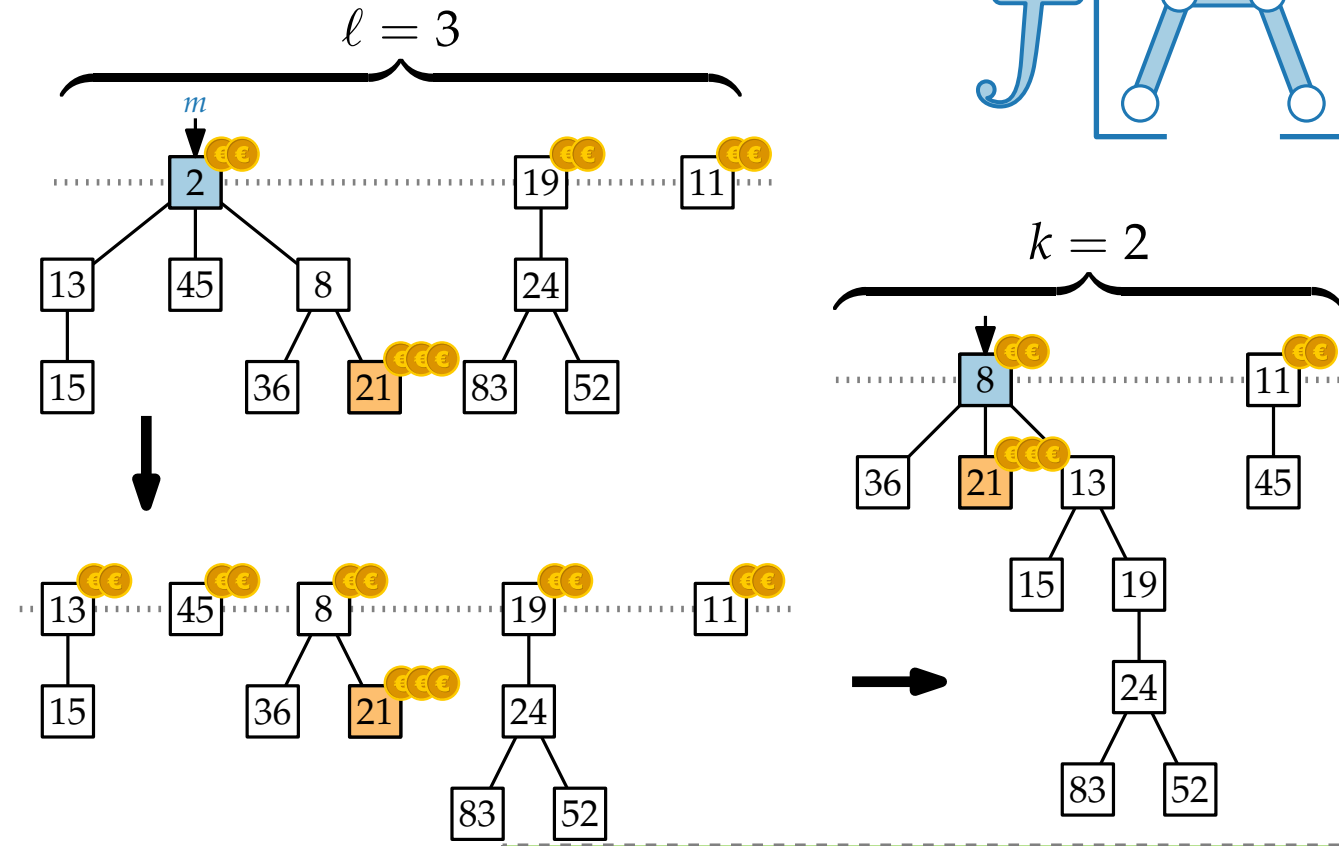
Amortisierte Analyse: EXTRACTMIN



```
Q.EXTRACTMIN()  
   $m = Q.MIN()$   
  Q.REMOVE( $m$ )  
  Q.CLEANUP()  
  Finde neues Minimum,  
  aktualisiere Pointer  
  return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

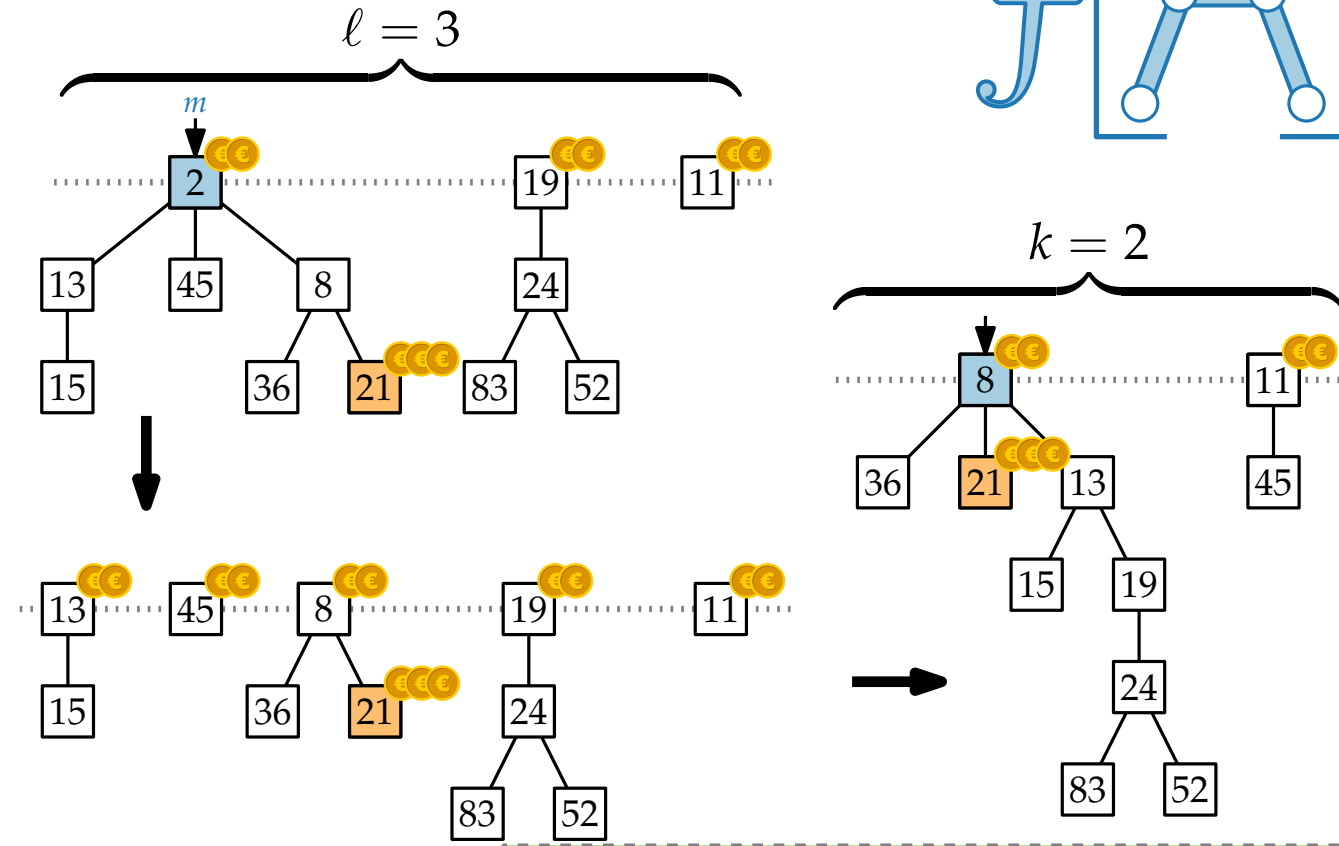


```
Q.EXTRACTMIN()  
   $m = Q.MIN()$   
  Q.REMOVE( $m$ )  
  Q.CLEANUP()  
  Finde neues Minimum,  
  aktualisiere Pointer  
  return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

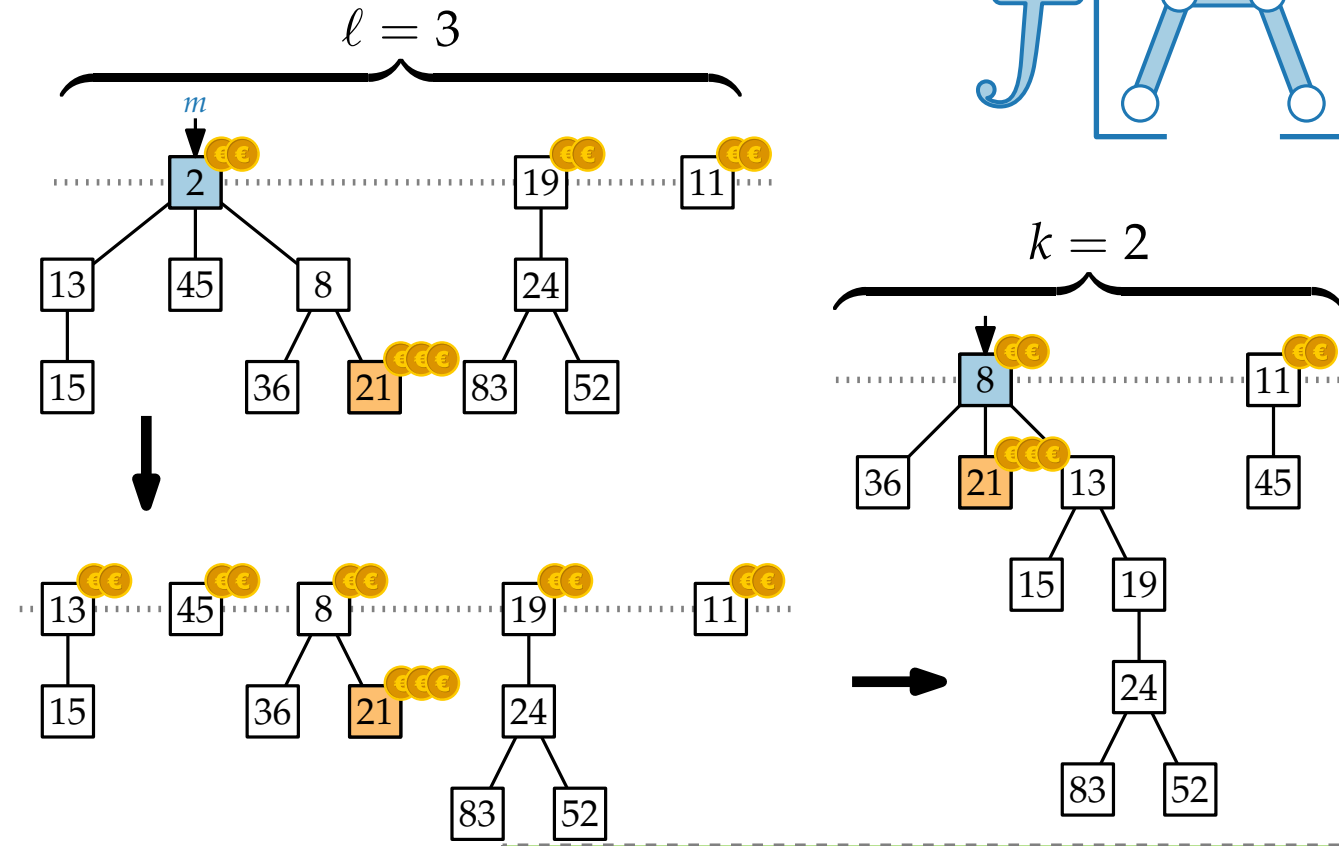


```
Q.EXTRACTMIN()  
 $m = Q.MIN()$   
Q.REMOVE( $m$ )  
Q.CLEANUP()  
Finde neues Minimum,  
aktualisiere Pointer  
return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:



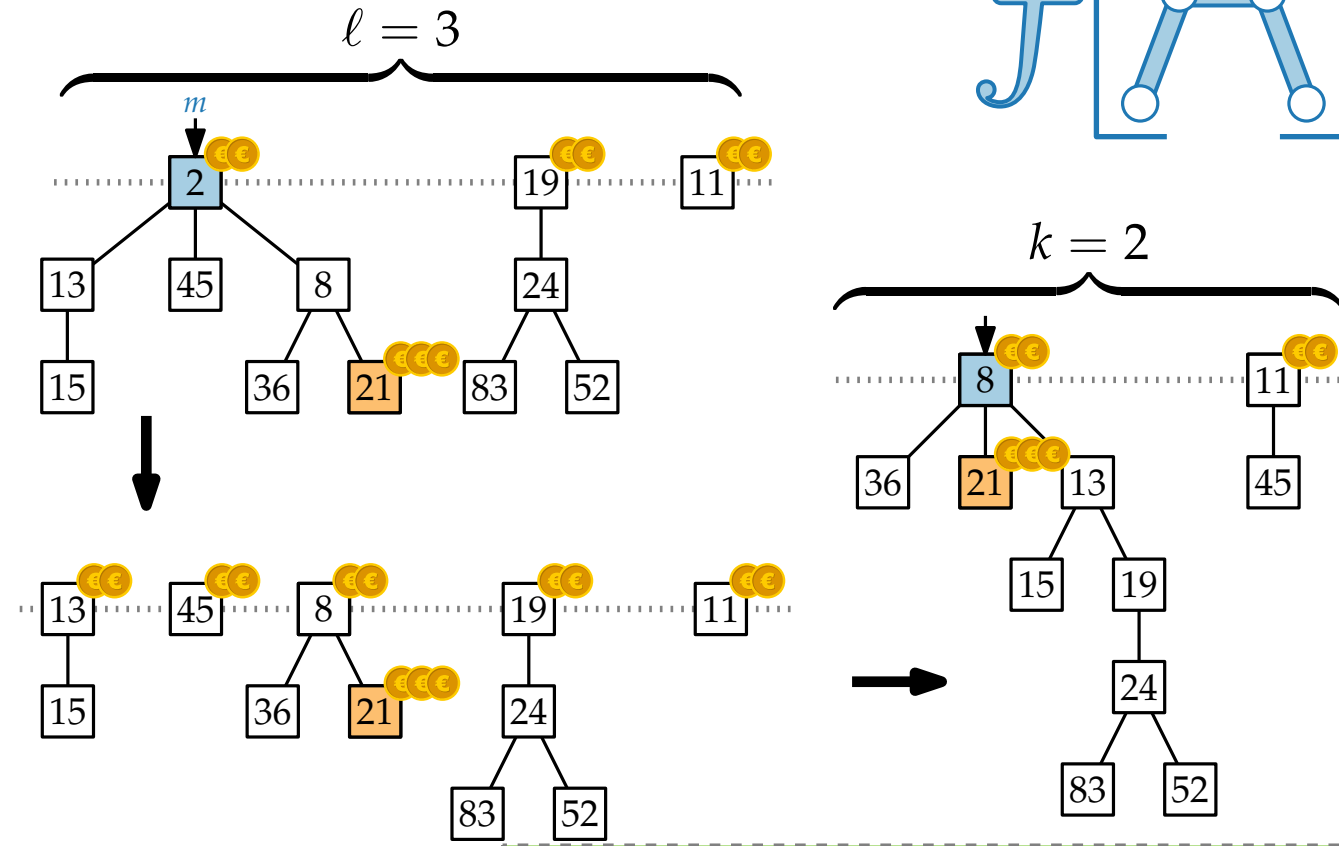
```
Q.EXTRACTMIN()  
   $m = Q.MIN()$   
   $Q.REMOVE(m)$   
   $Q.CLEANUP()$   
  Finde neues Minimum,  
  aktualisiere Pointer  
  return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

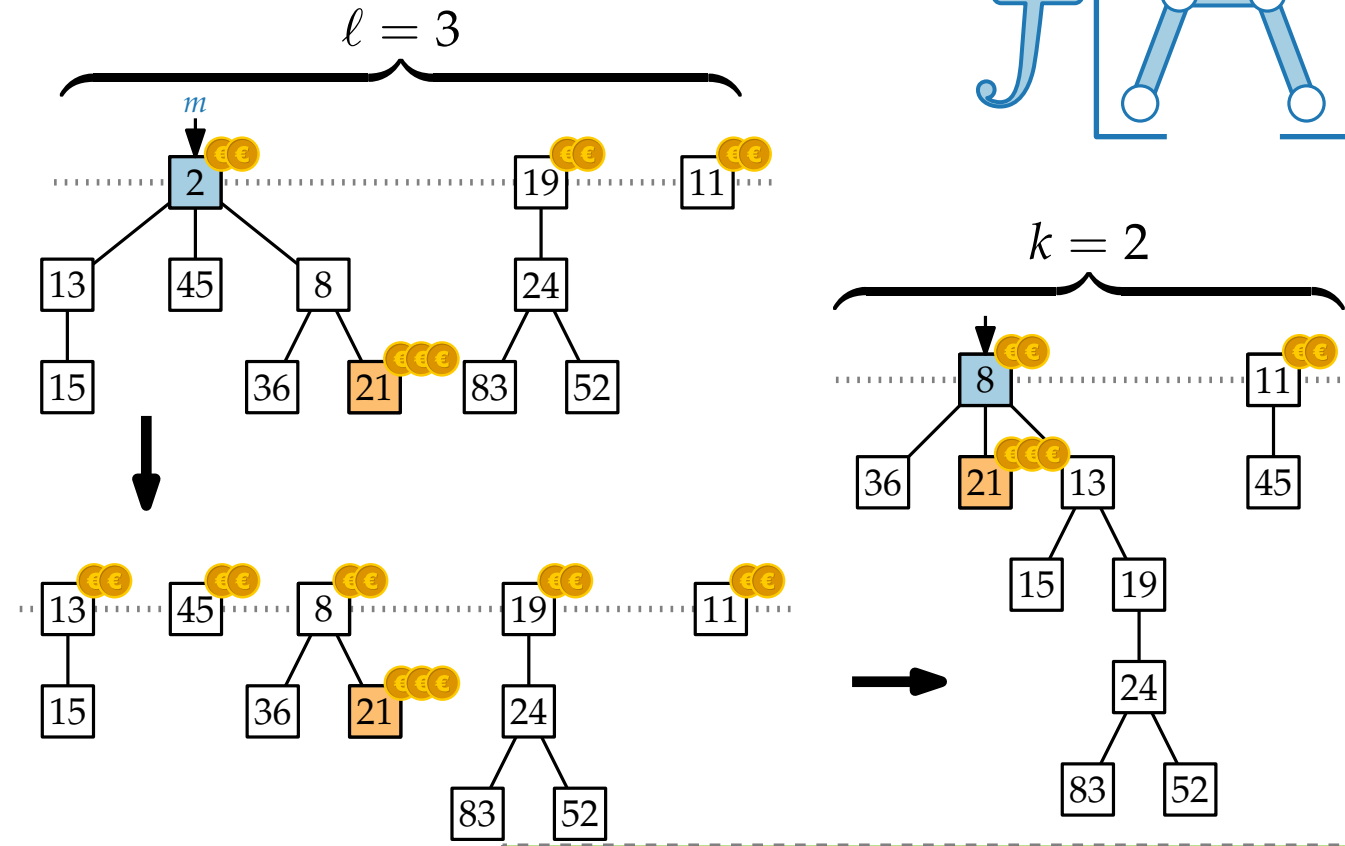
■ 1



```
Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
```

- ## Tatsächliche Kosten:

■ 1


$$m = \text{Q.MIN}()$$

Q.CLEANUP()

Finde neues Minimum, aktualisiere Pointer

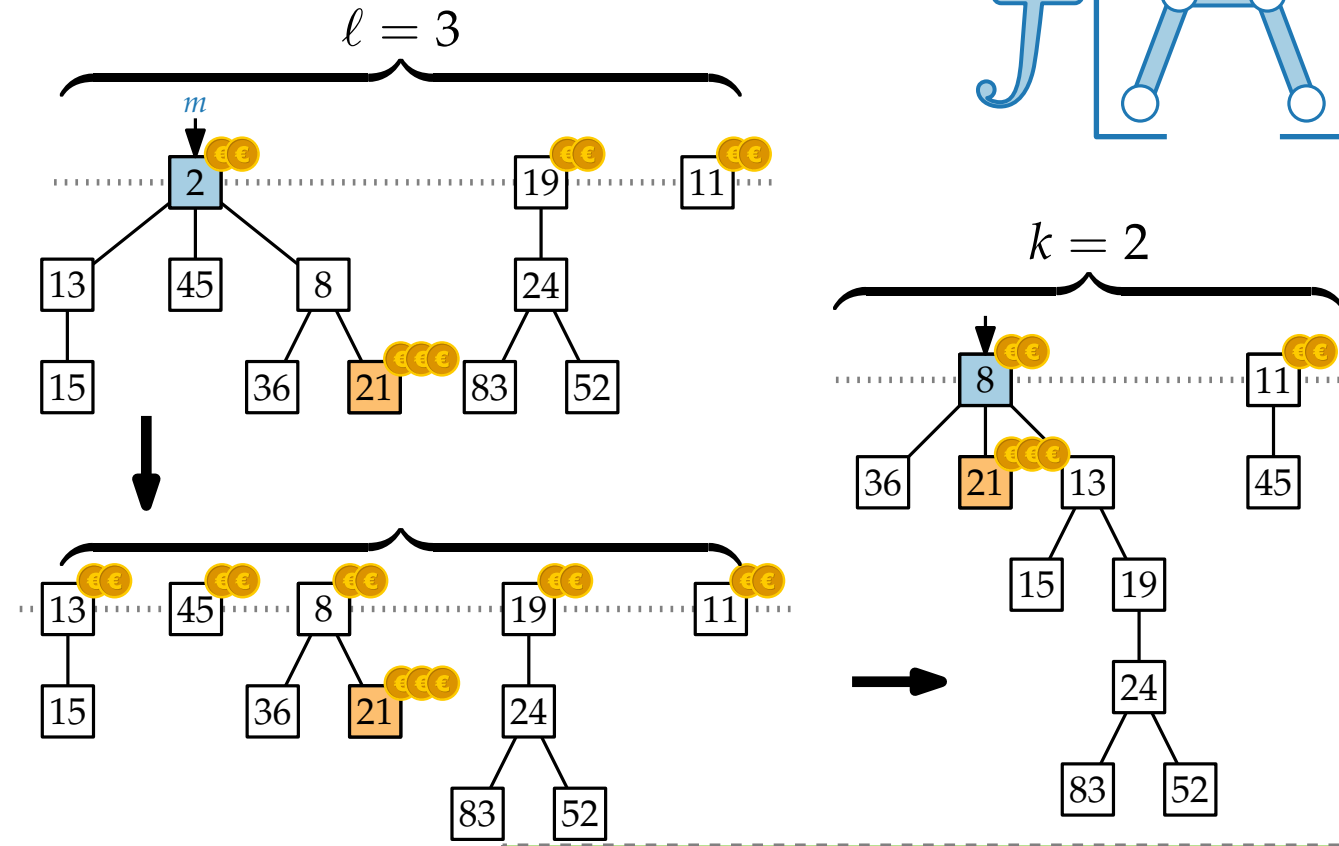
```
return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

■ 1



`Q.EXTRACTMIN()`

$m = Q.MIN()$

`Q.REMOVE( $m$ )`

`Q.CLEANUP()`

Finde neues Minimum,
aktualisiere Pointer

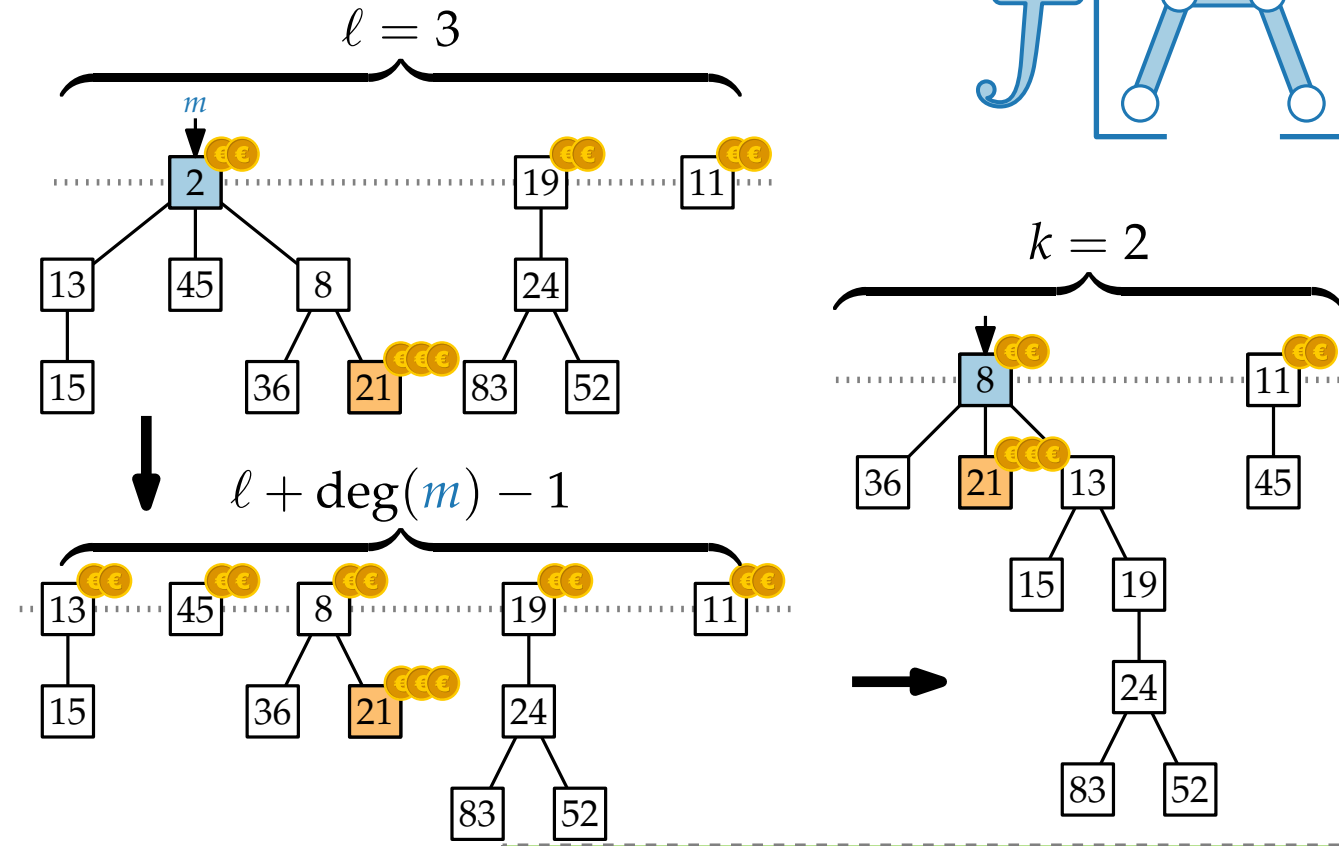
return m

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

■ 1



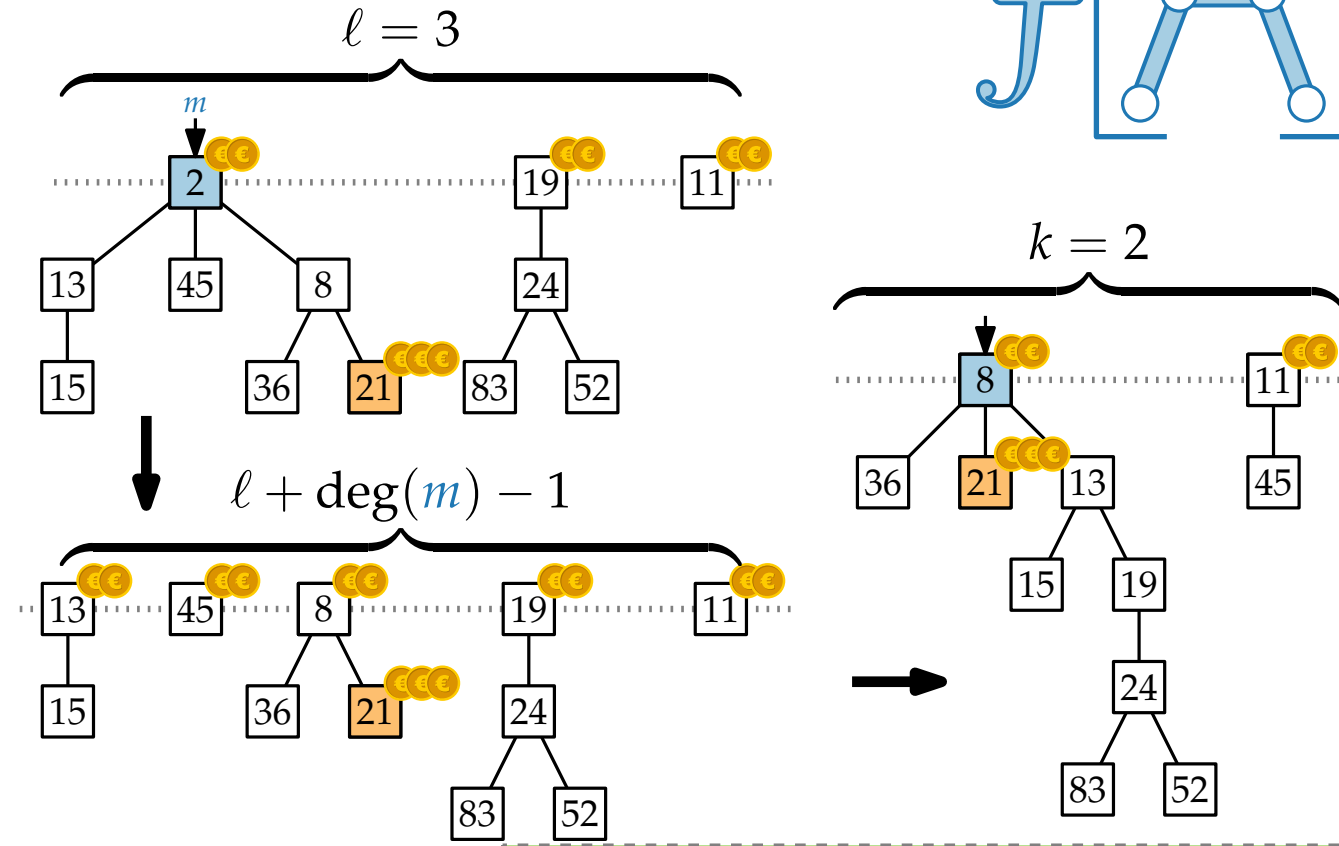
```
Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

- 1
- $2(\ell + \deg(m) - 1) - k$



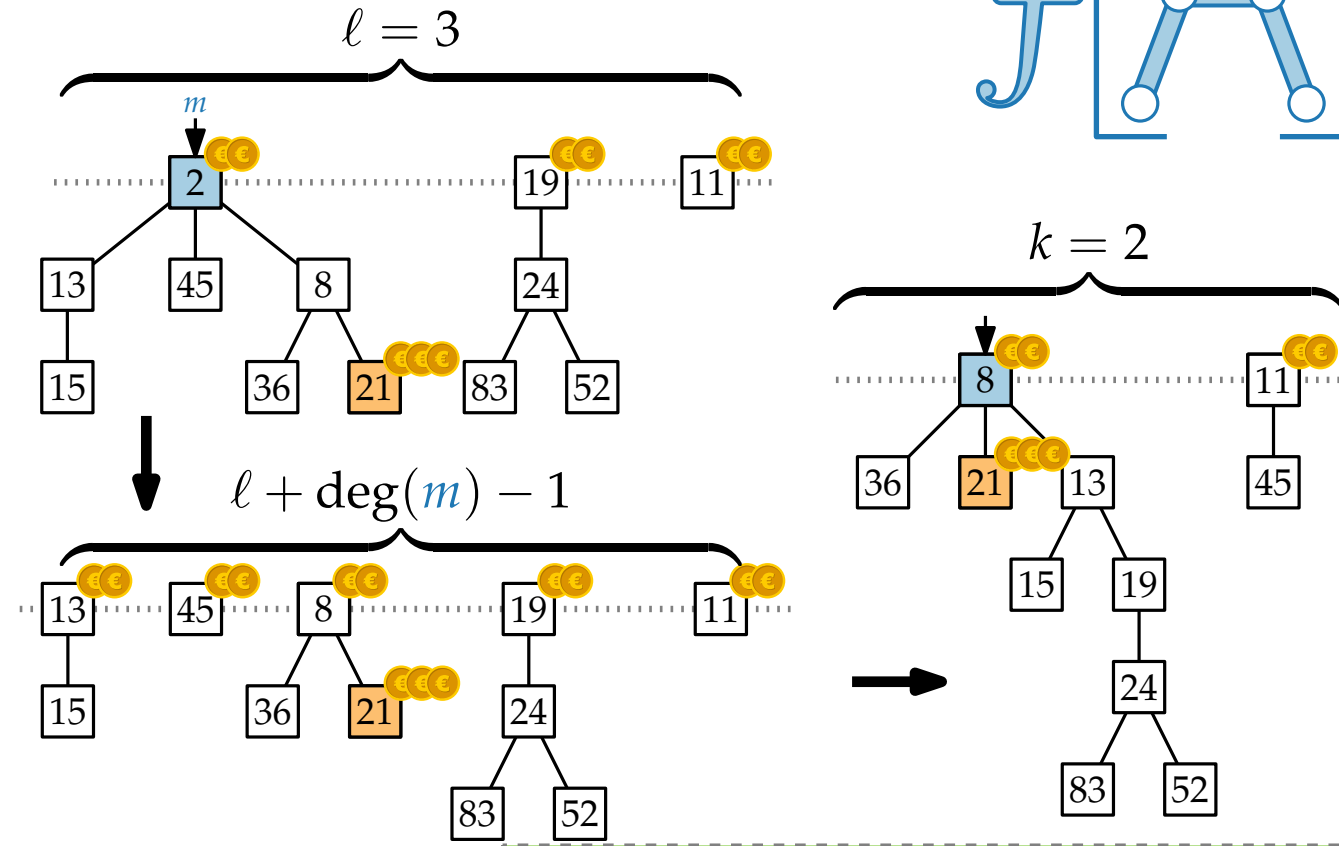
```
Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

- 1
- $2(\ell + \deg(m) - 1) - k$



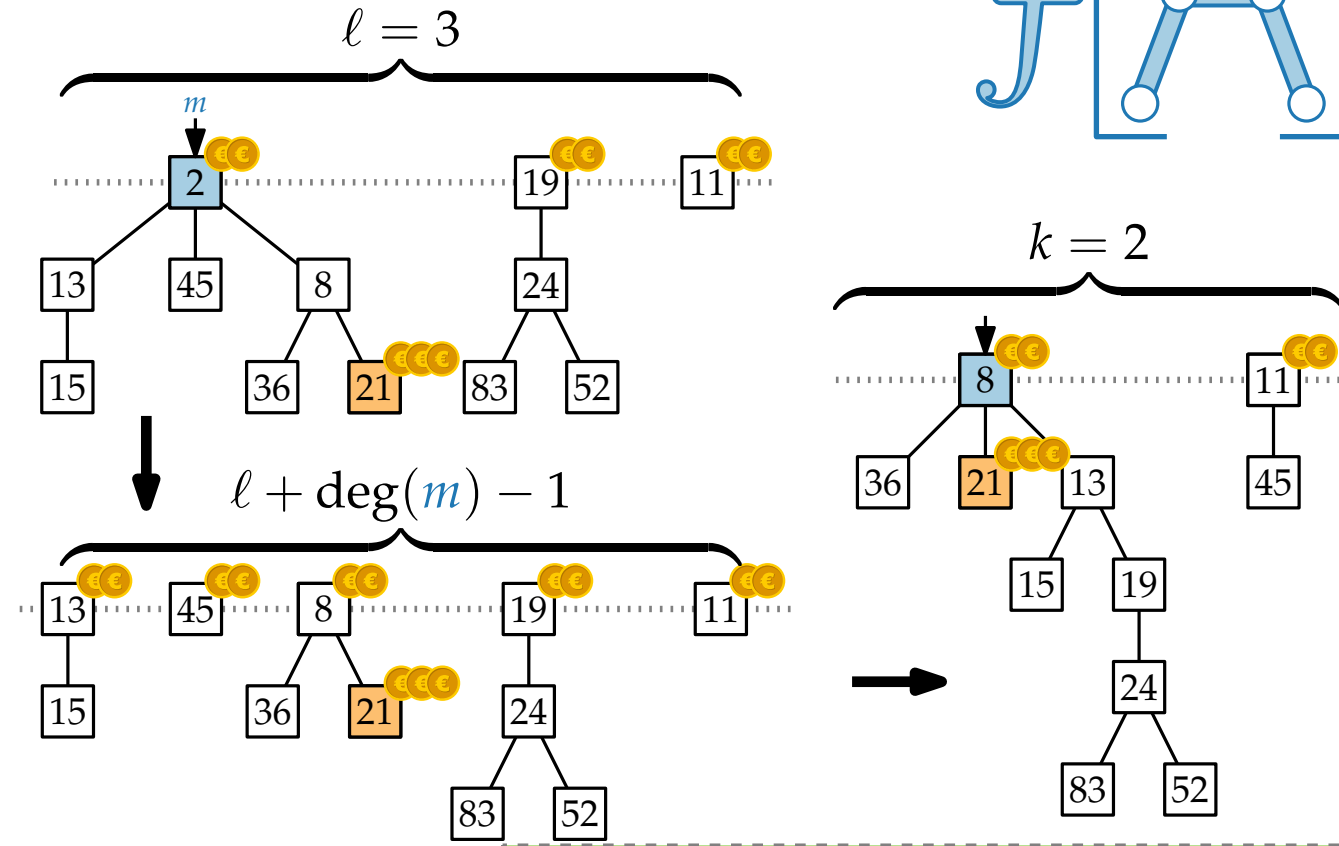
```
Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

- 1
- $2(\ell + \deg(m) - 1) - k$
- k



```
Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
```

Amortisierte Analyse: EXTRACTMIN

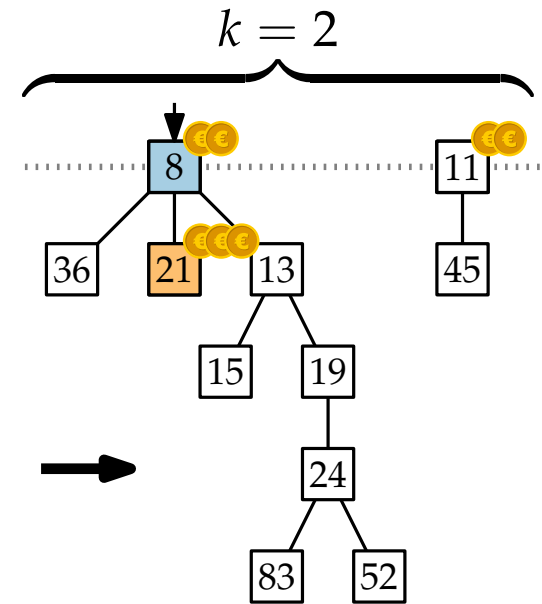
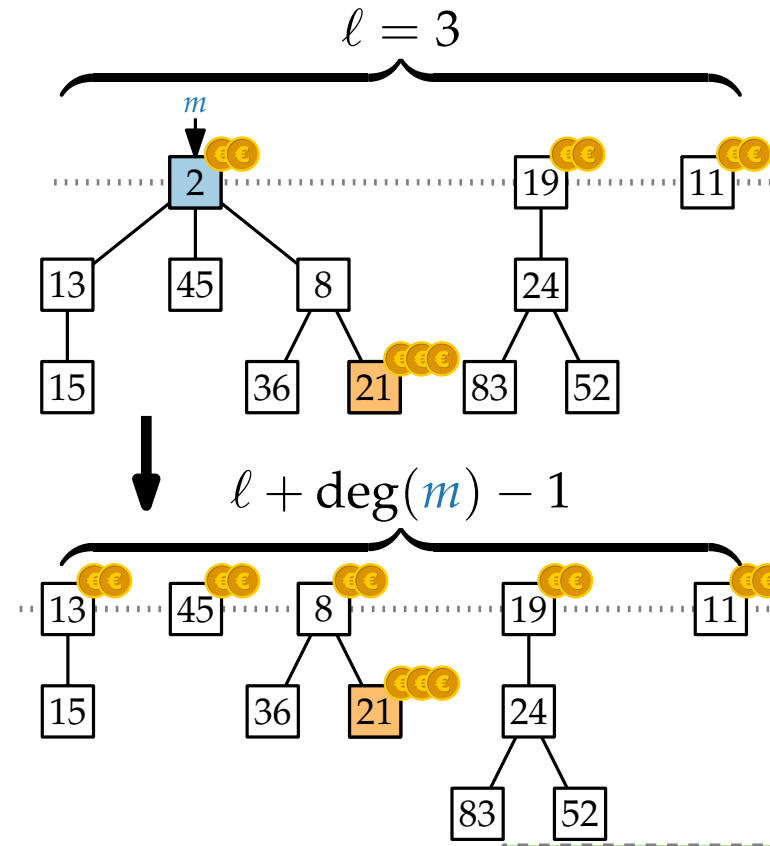
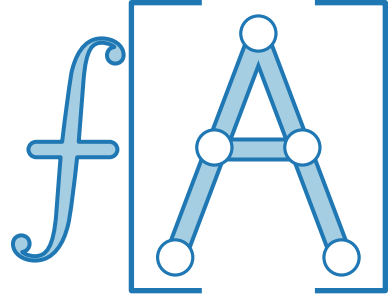
- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

■ 1

■ $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$

■ k



`Q.EXTRACTMIN()`

$m = Q.MIN()$

`Q.REMOVE( $m$ )`

`Q.CLEANUP()`

Finde neues Minimum,
aktualisiere Pointer

return m

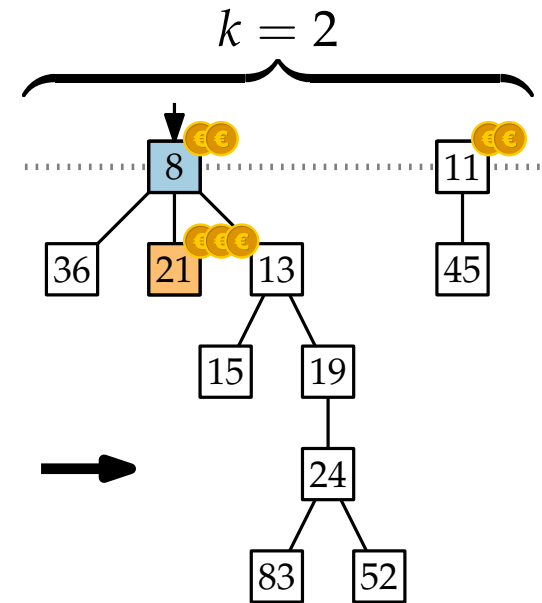
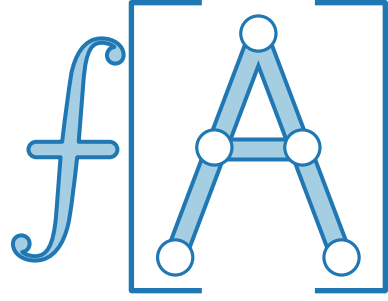
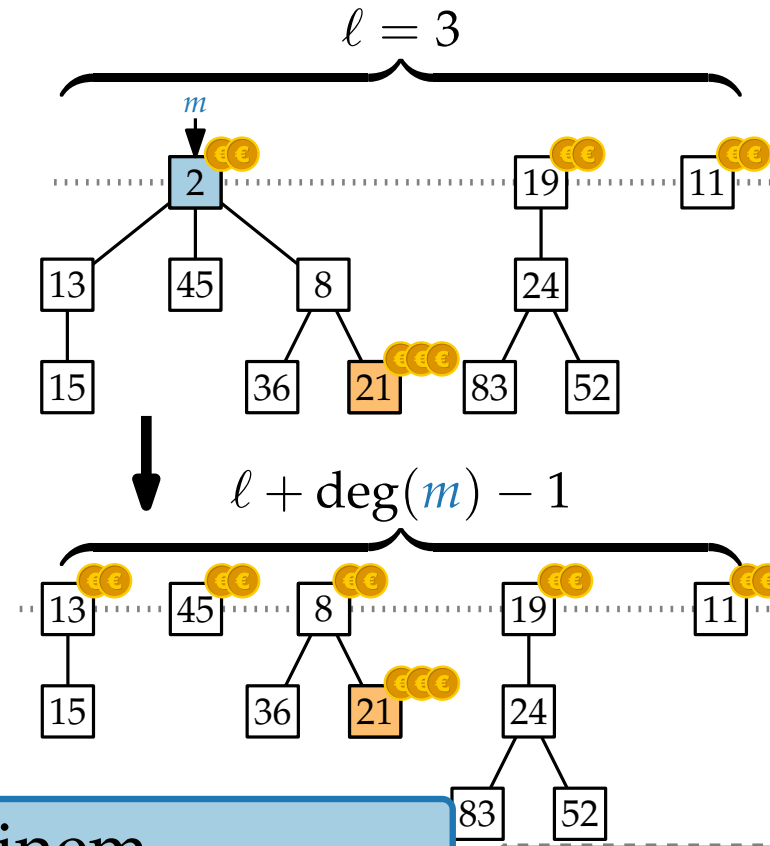
Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

- 1
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- k

Korollar. Die Anzahl der Wurzelknoten in einem **normierten** Fibonacci-Heap ist höchstens $2 \log n$.



```

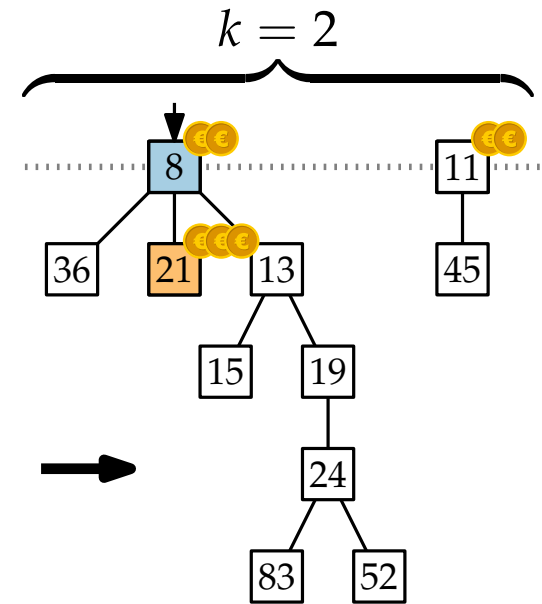
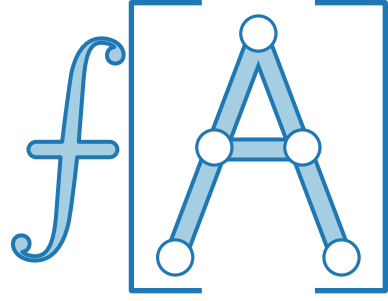
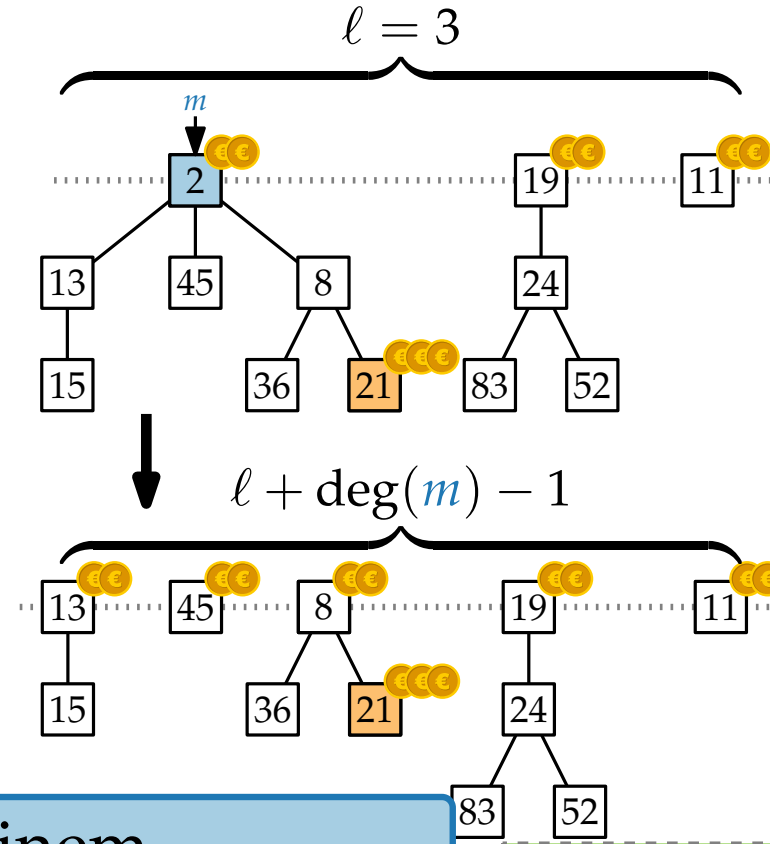
Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten:

- 1
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$



Korollar. Die Anzahl der Wurzelknoten in einem **normierten** Fibonacci-Heap ist höchstens $2 \log n$.

```

Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
    
```

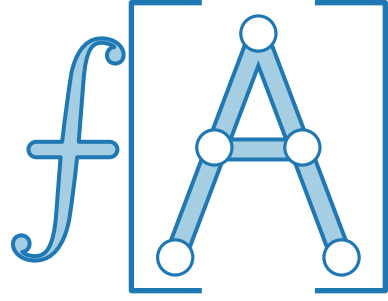
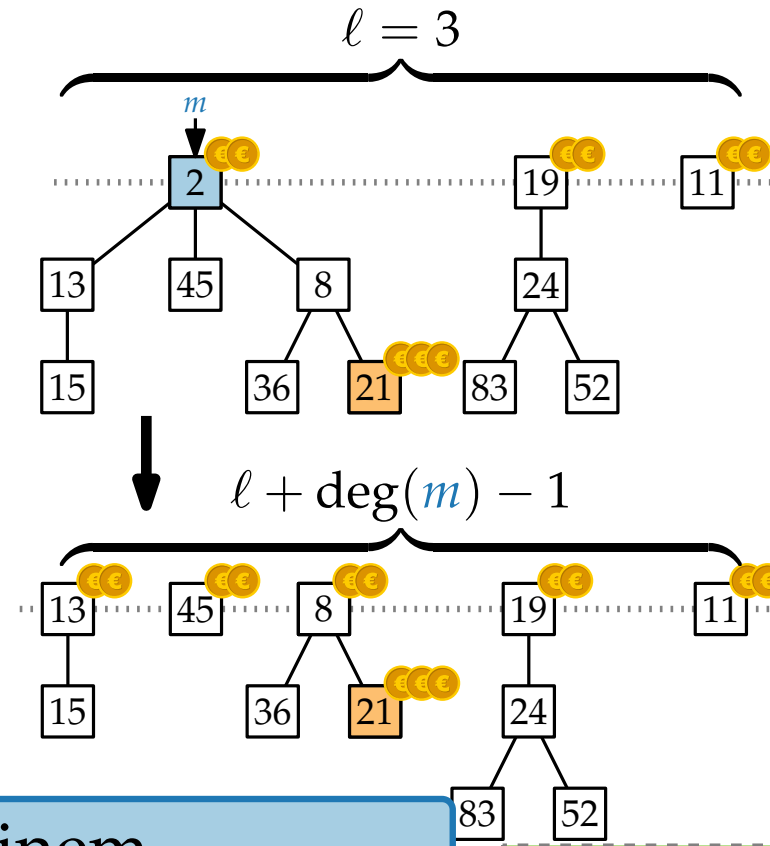
Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Korollar. Die Anzahl der Wurzelknoten in einem **normierten** Fibonacci-Heap ist höchstens $2 \log n$.



```

Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
    
```

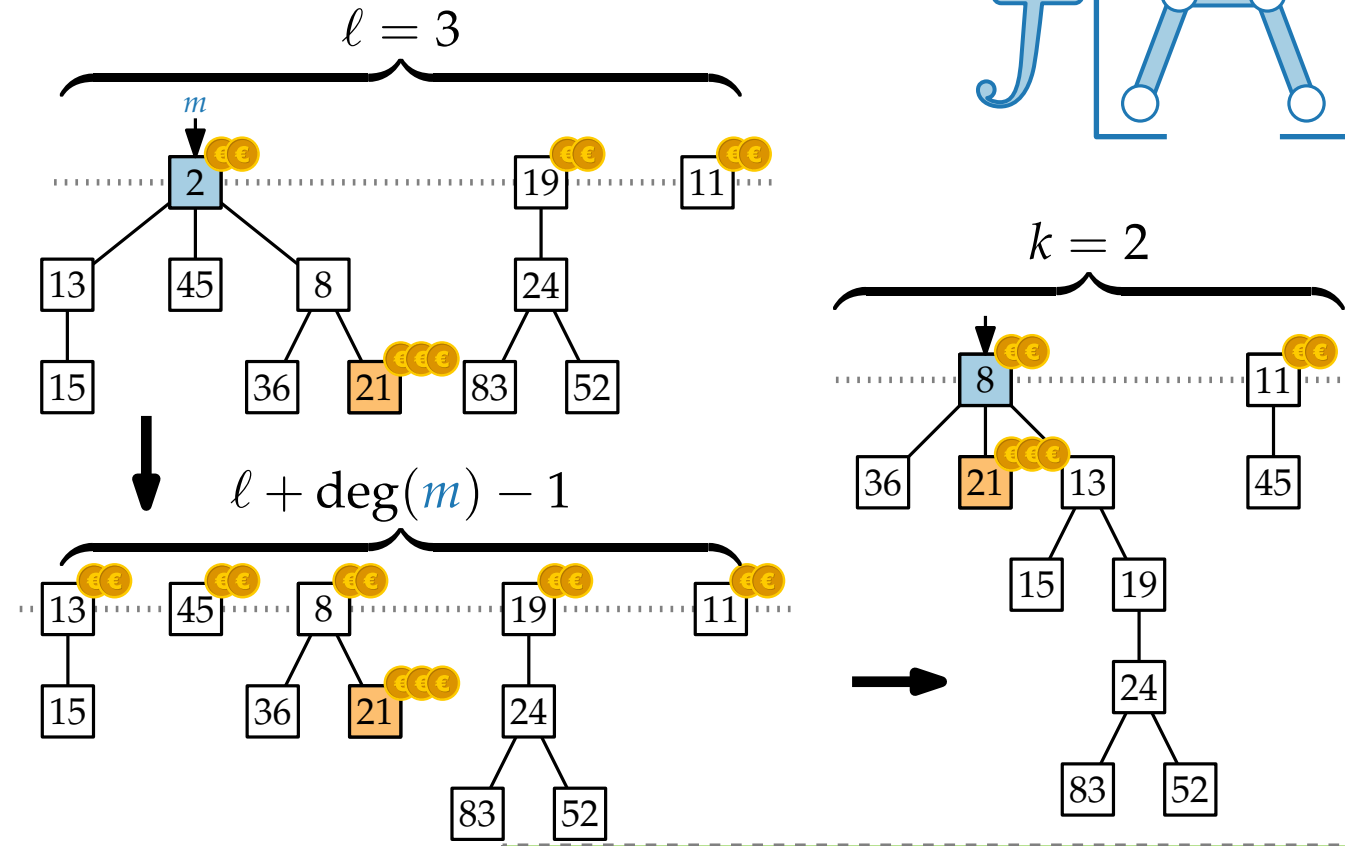

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:



```

Q.EXTRACTMIN()
   $m = Q.MIN()$ 
  Q.REMOVE( $m$ )
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return  $m$ 
    
```

Amortisierte Analyse: EXTRACTMIN

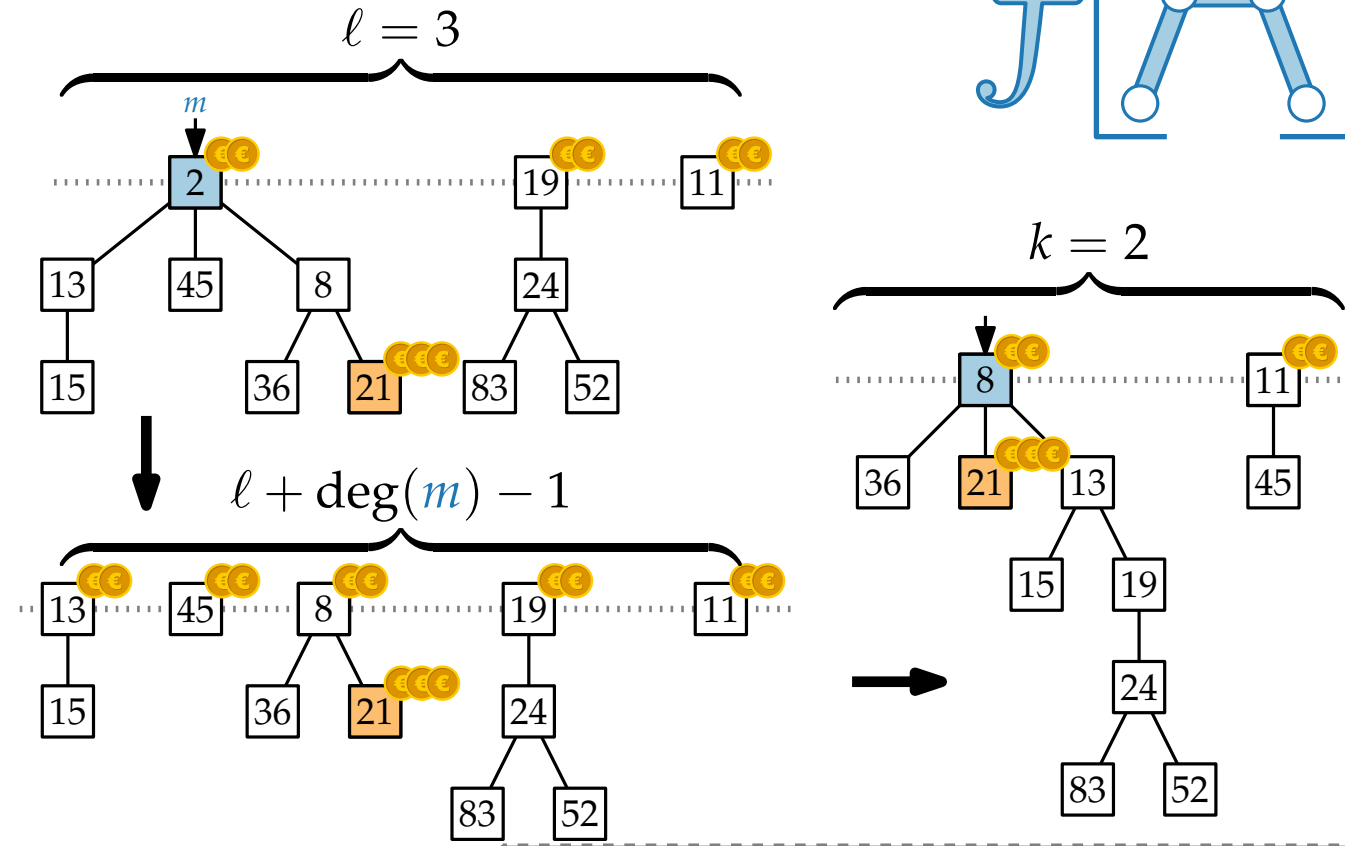
- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$



```

Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```

Amortisierte Analyse: EXTRACTMIN

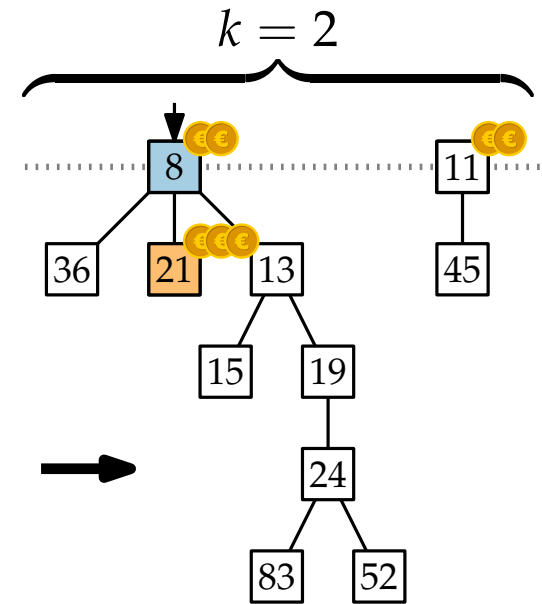
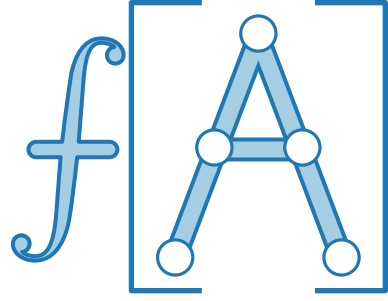
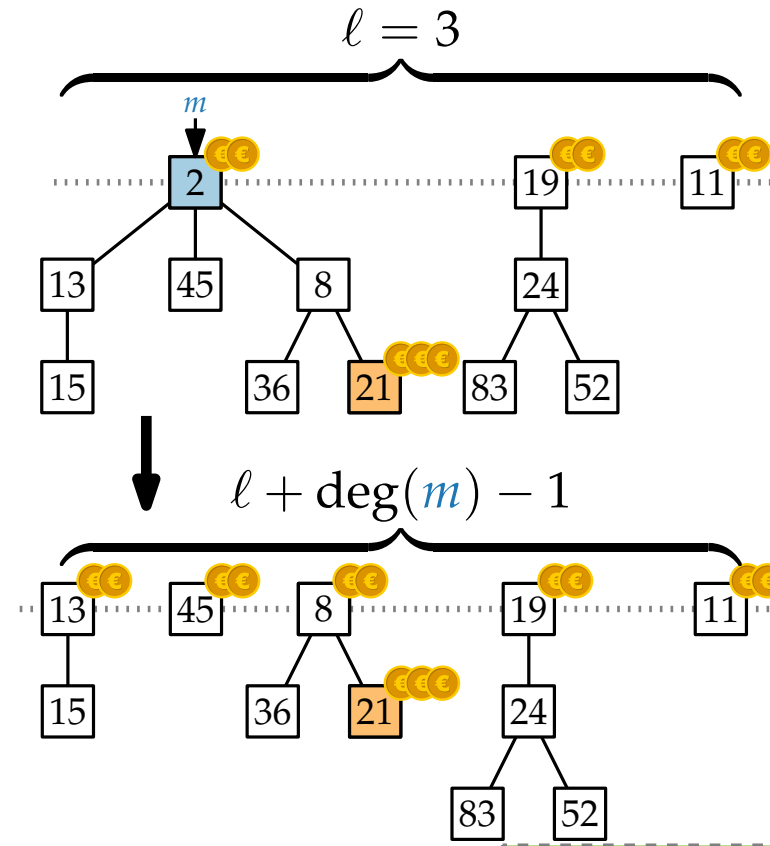
- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1 $\in \mathcal{O}(n)$
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$
- Es kommen $\deg(m)$ neue Knoten in die Wurzelliste und einer (m) wird entfernt



```

Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

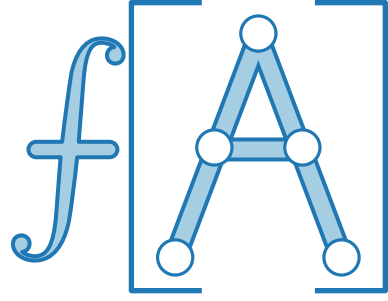
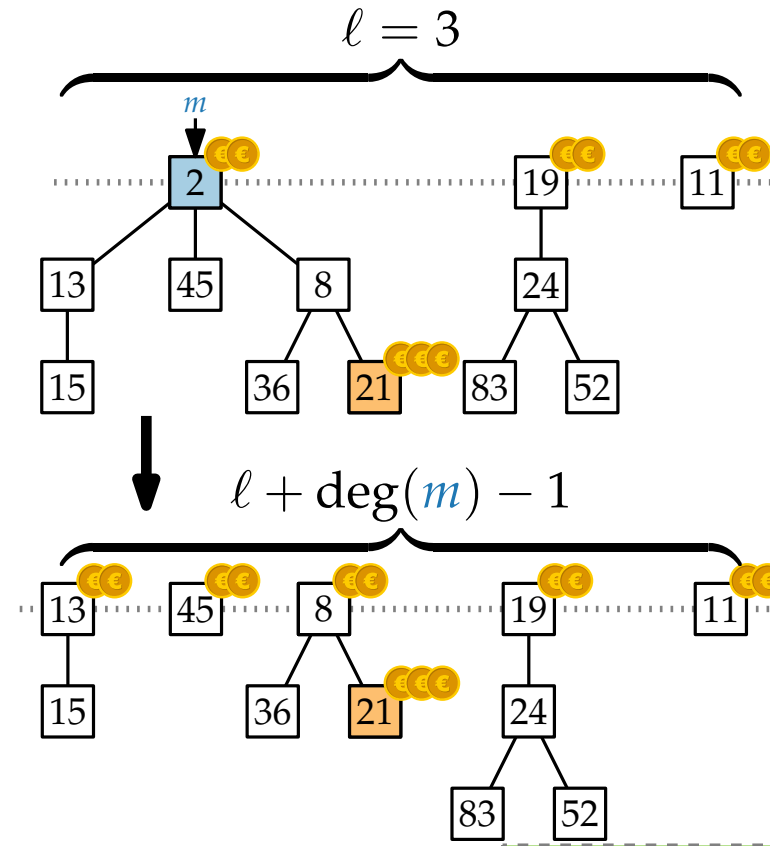
Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1 $\in \mathcal{O}(n)$
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$

- Es kommen $\deg(m)$ neue Knoten in die Wurzelliste und einer (m) wird entfernt
- Wir müssen auf jeden davon €€ legen



```

Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```

Amortisierte Analyse: EXTRACTMIN

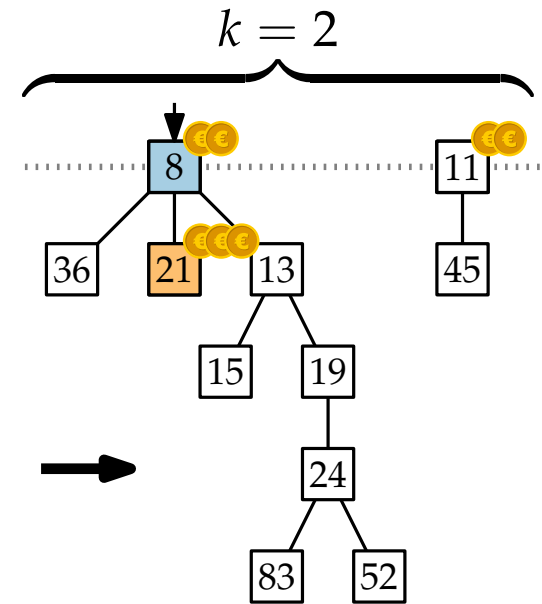
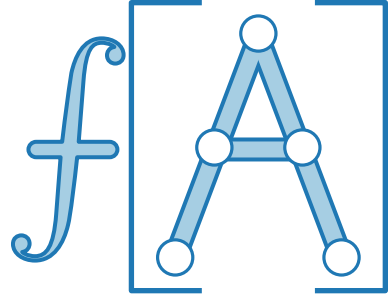
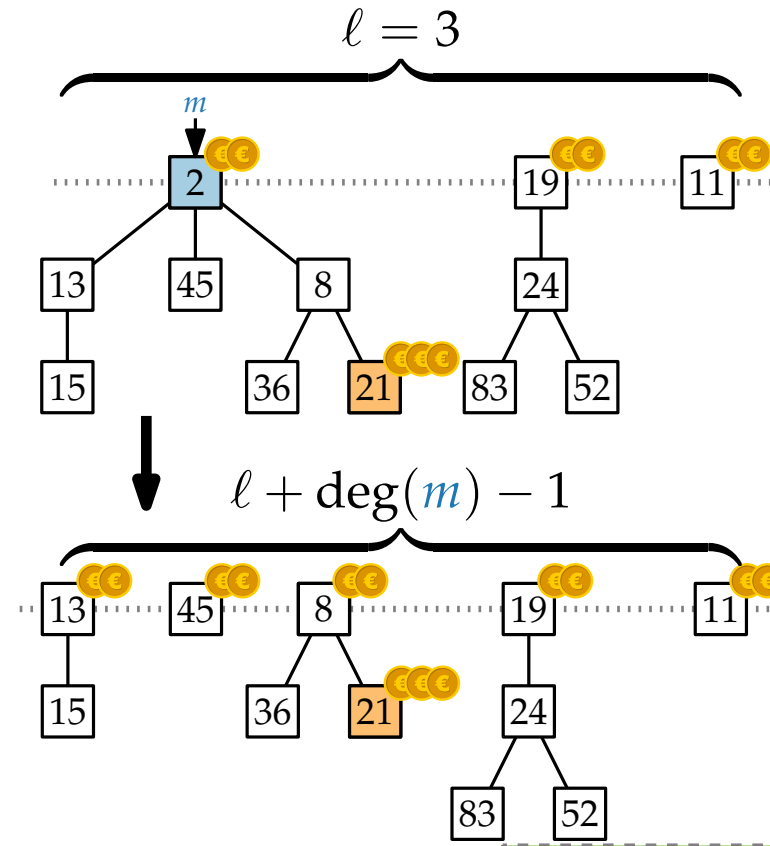
- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$
- Es kommen $\deg(m)$ neue Knoten in die Wurzelliste und einer (m) wird entfernt
- Wir müssen auf jeden davon €€ legen
- ➔ 1 +



```

Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

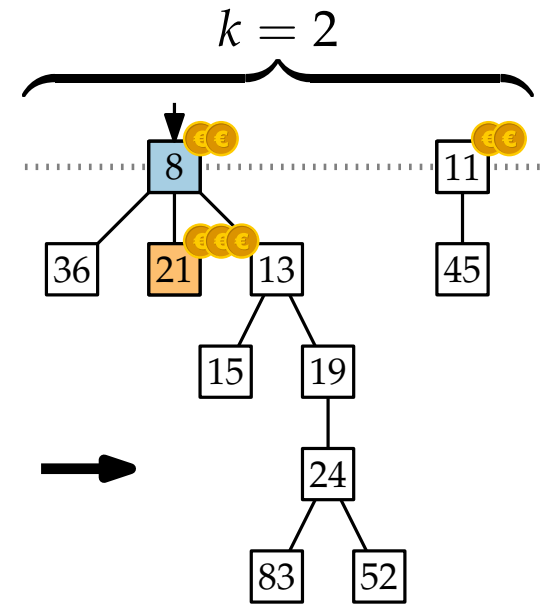
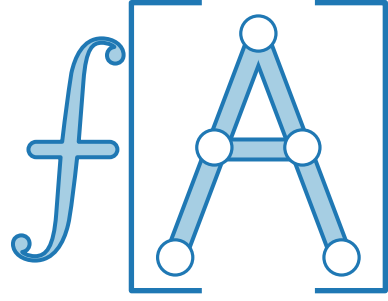
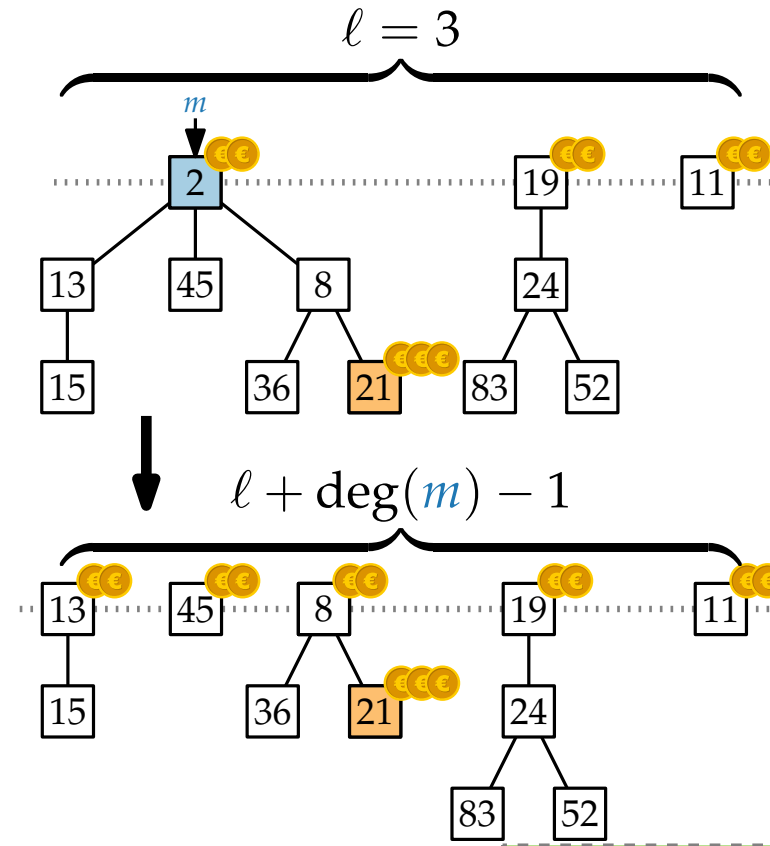
Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$

- Es kommen $\deg(m)$ neue Knoten in die Wurzelliste und einer (m) wird entfernt
- Wir müssen auf jeden davon €€ legen
- ➔ $1 + 2(\deg(m) - 1)$



```
Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

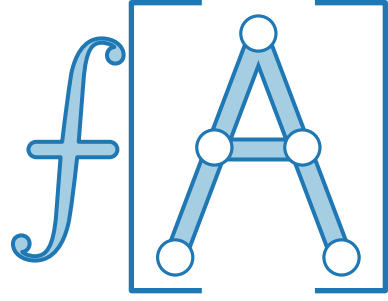
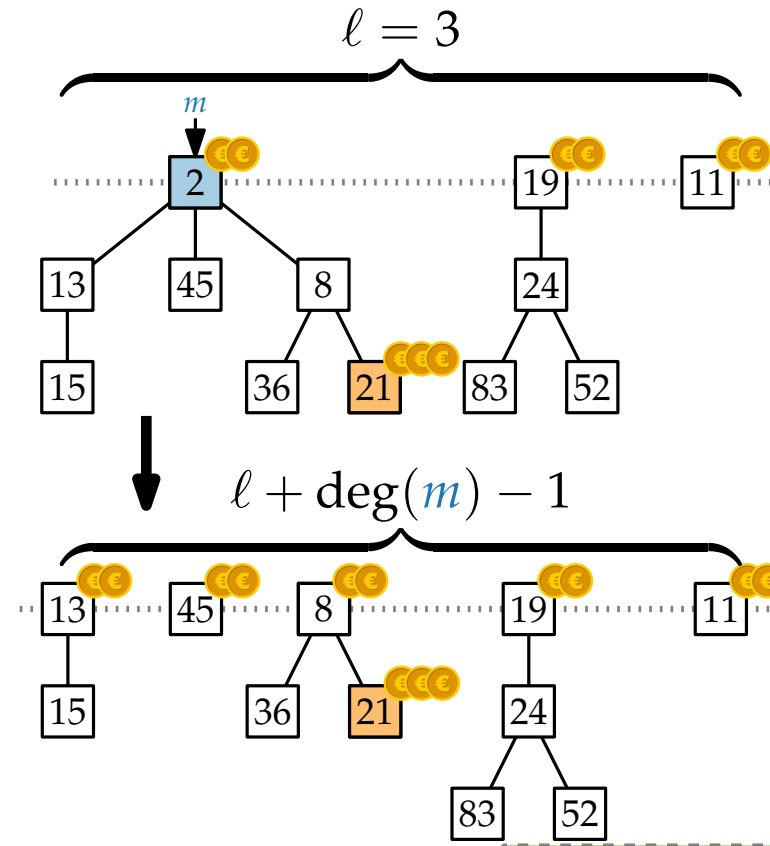
Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1 $\in \mathcal{O}(n)$
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$

- Es kommen $\deg(m)$ neue Knoten in die Wurzelliste und einer (m) wird entfernt
- Wir müssen auf jeden davon €€€ legen
- ➔ $1 + 2(\deg(m) - 1) = 2\deg(m) - 1$



```

Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```

Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

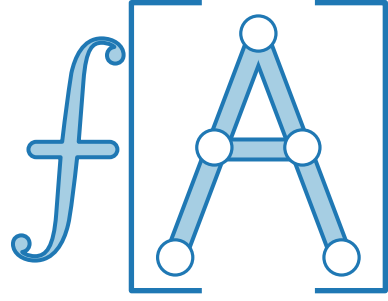
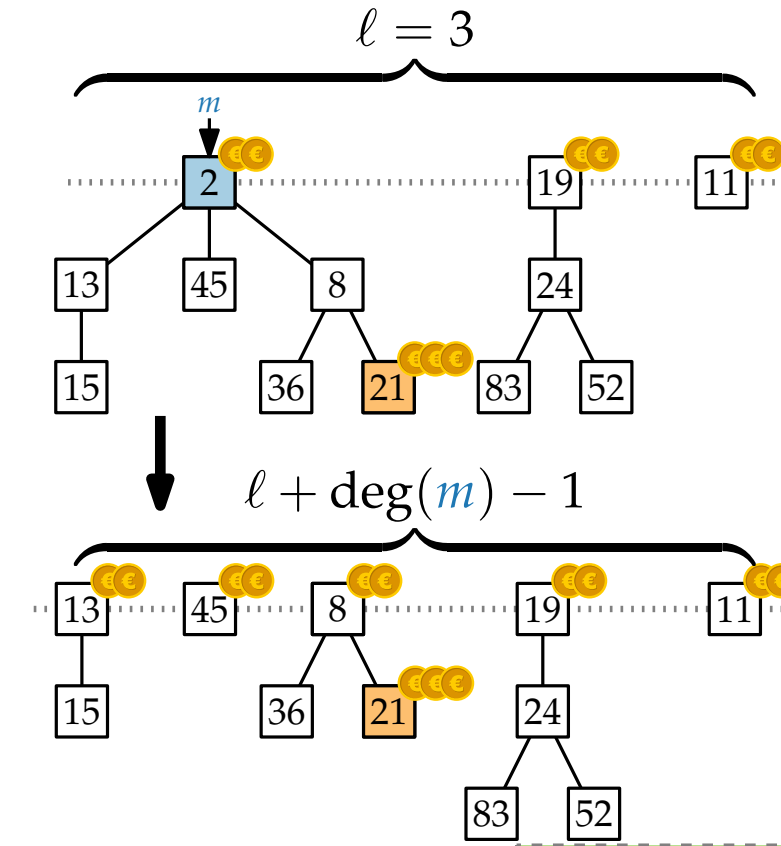
Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1 $\in \mathcal{O}(n)$
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten:

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$

- Es kommen $\deg(m)$ neue Knoten in die Wurzelliste und einer (m) wird entfernt
- Wir müssen auf jeden davon €€€ legen
- ➔ $1 + 2(\deg(m) - 1) = 2\deg(m) - 1 \in \mathcal{O}(\log n)$



```

Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```


Amortisierte Analyse: EXTRACTMIN

- ℓ Knoten zu Beginn in Wurzelliste
- k Knoten nach CLEANUP in Wurzelliste

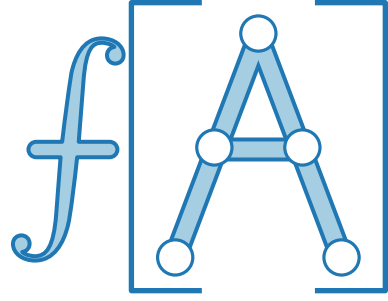
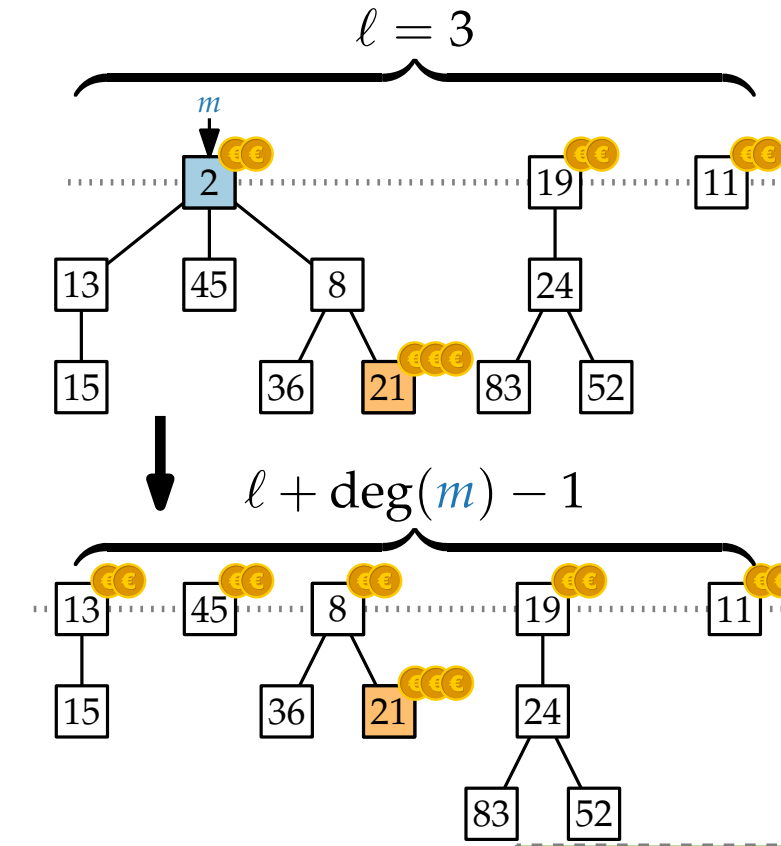
Tatsächliche Kosten: $\in \mathcal{O}(n)$

- 1 $\in \mathcal{O}(n)$
- $2(\ell + \deg(m) - 1) - k \in \mathcal{O}(n)$
- $k \in \mathcal{O}(\log n)$

Amortisierte Kosten: $\in \mathcal{O}(\log n)$

- $k \in \mathcal{O}(\log n)$
- $k \in \mathcal{O}(\log n)$

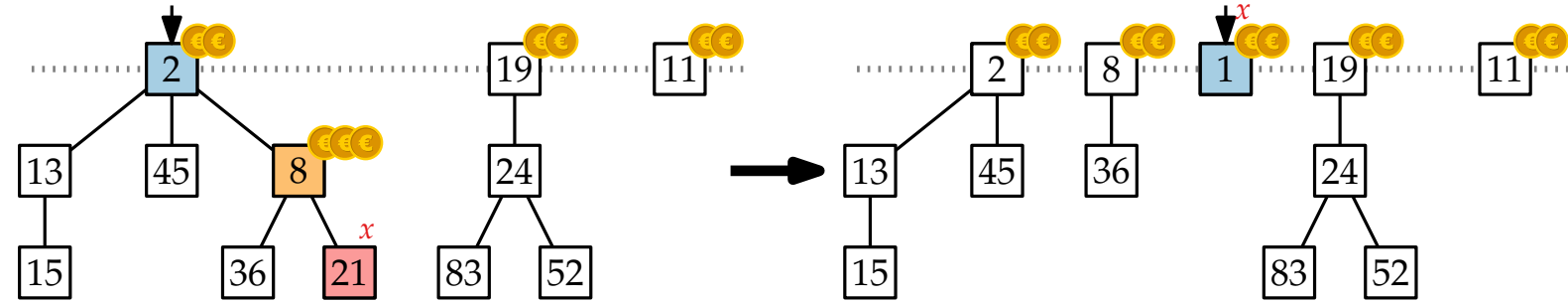
- Es kommen $\deg(m)$ neue Knoten in die Wurzelliste und einer (m) wird entfernt
- Wir müssen auf jeden davon €€€ legen
- ➔ $1 + 2(\deg(m) - 1) = 2\deg(m) - 1 \in \mathcal{O}(\log n)$



```

Q.EXTRACTMIN()
  m = Q.MIN()
  Q.REMOVE(m)
  Q.CLEANUP()
  Finde neues Minimum,
  aktualisiere Pointer
  return m
    
```

Amortisierte Analyse: DECREASEKEY



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

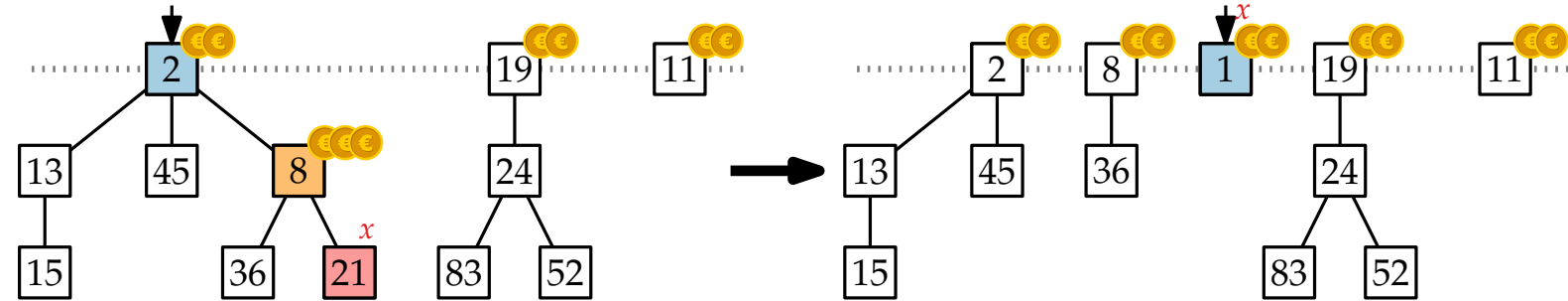
if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten:



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

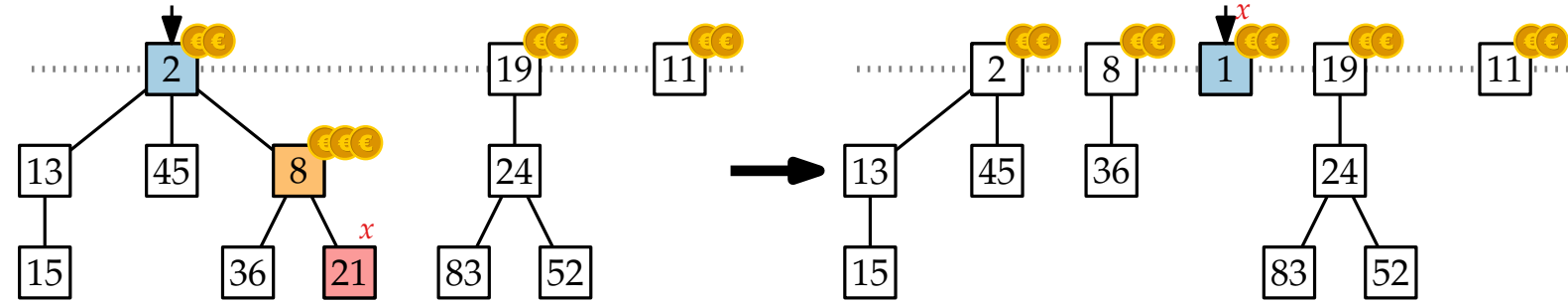
if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten:



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

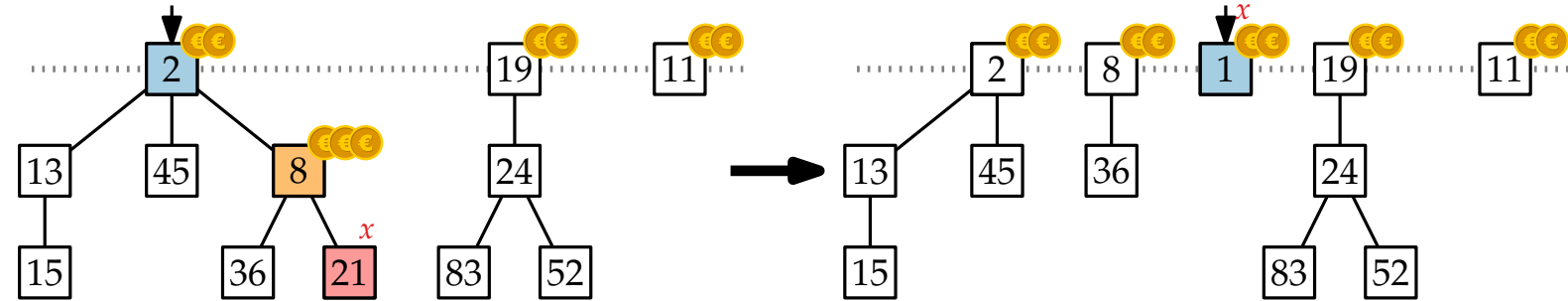
if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten:

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

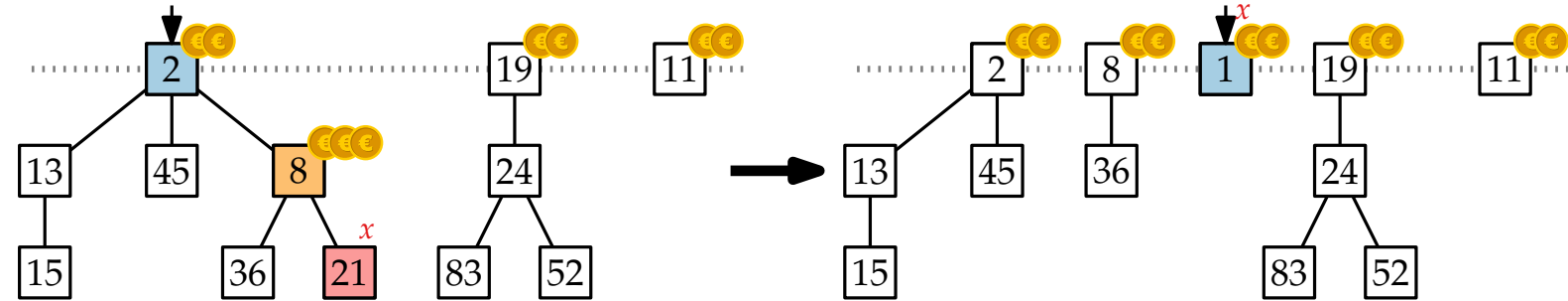
if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten:

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

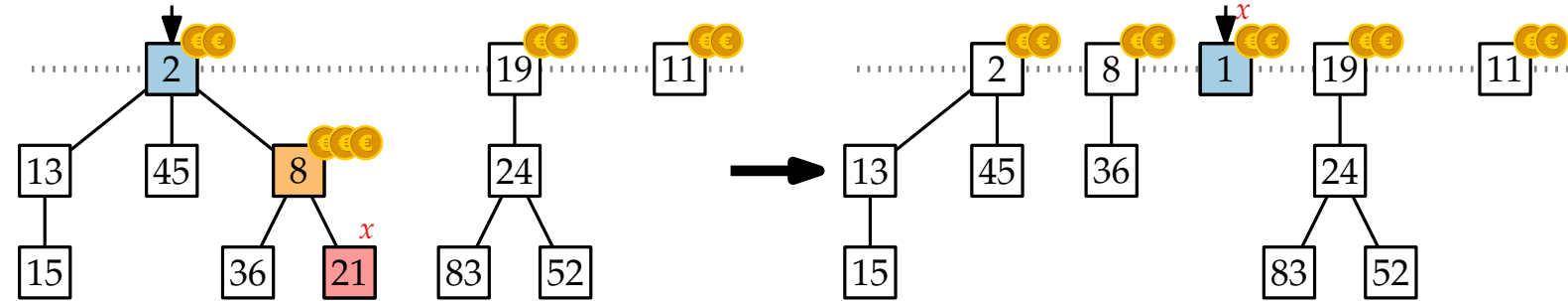
if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten:

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

m mal

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

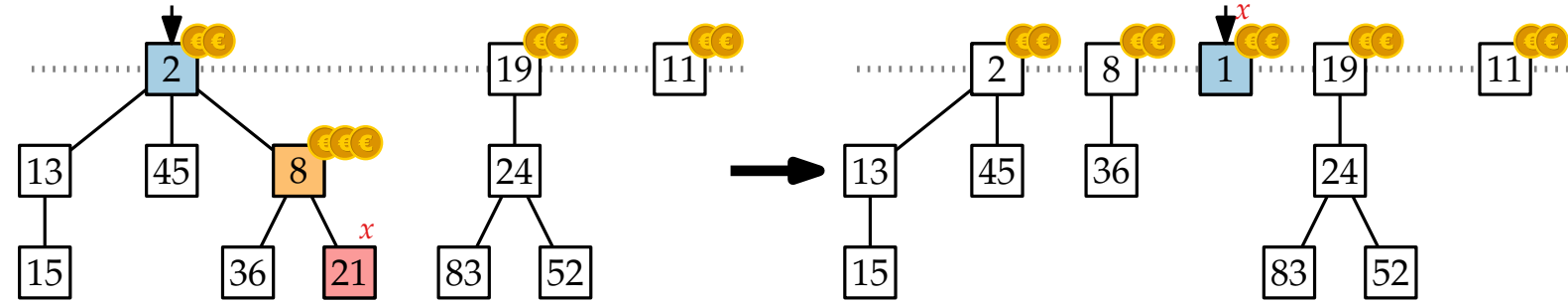
Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten:

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

m mal

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

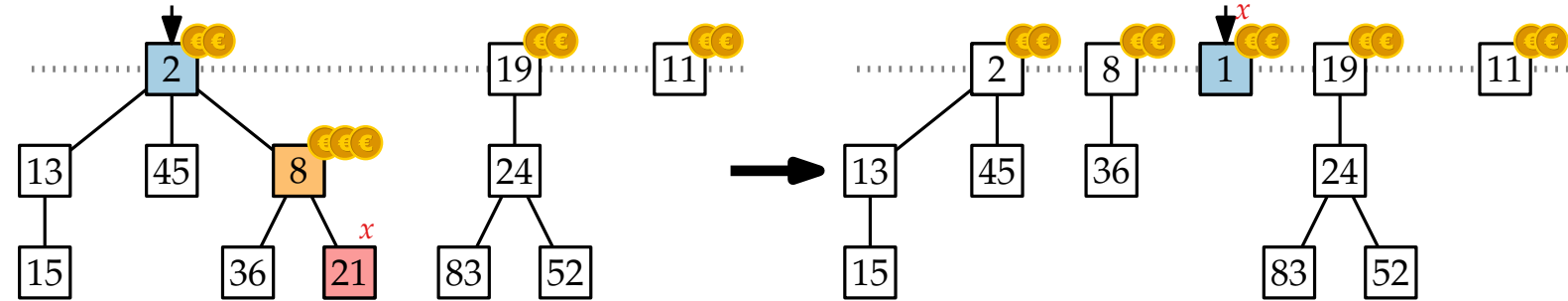
Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten:

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

m mal

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY

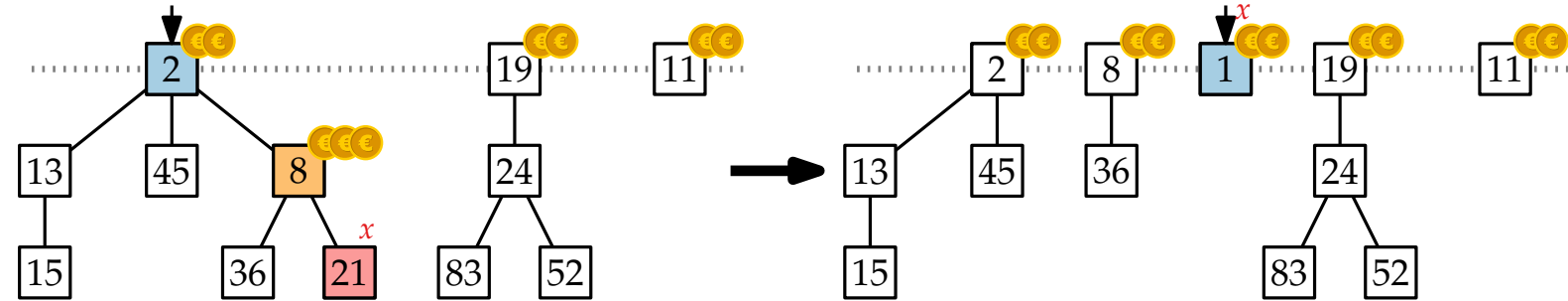


Tatsächliche Kosten:

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

m mal

Amortisierte Analyse: DECREASEKEY

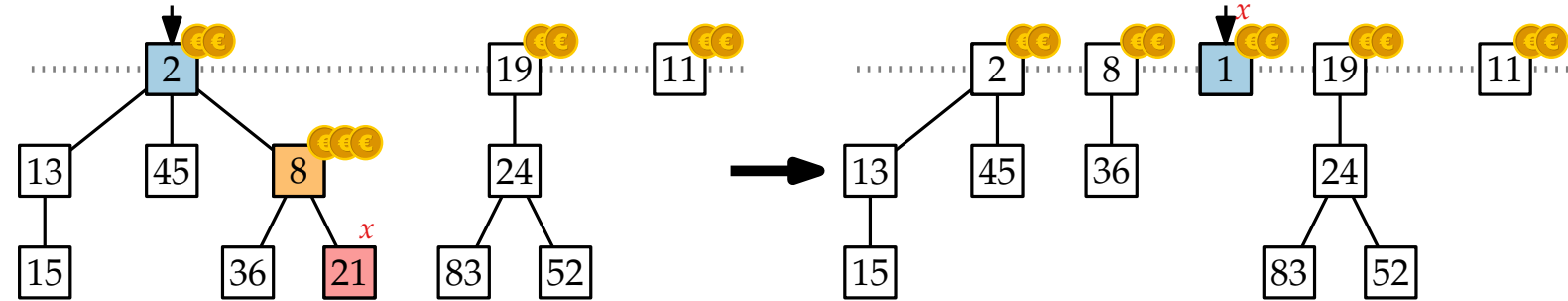


Tatsächliche Kosten:

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

m mal

while y ist **markiert** **do**

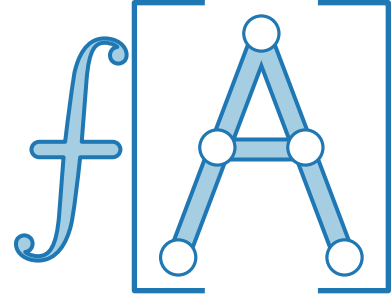
$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



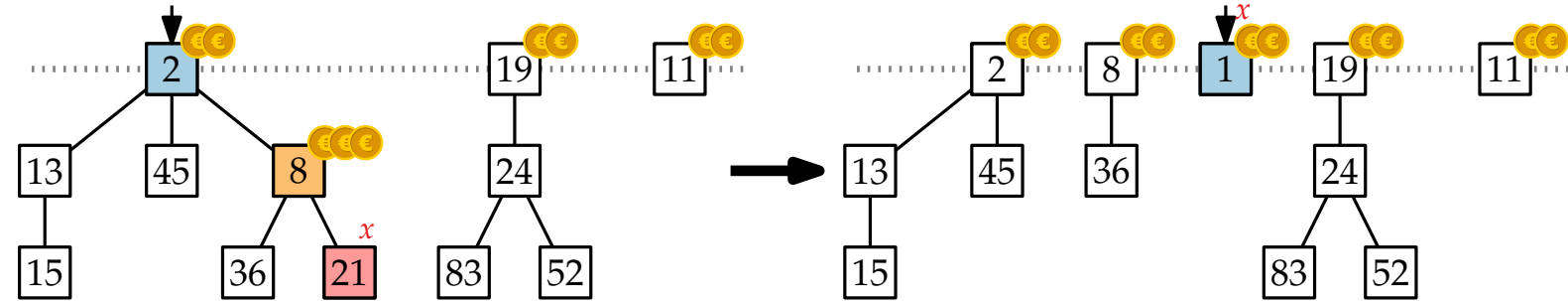
Tatsächliche Kosten:

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

m mal

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



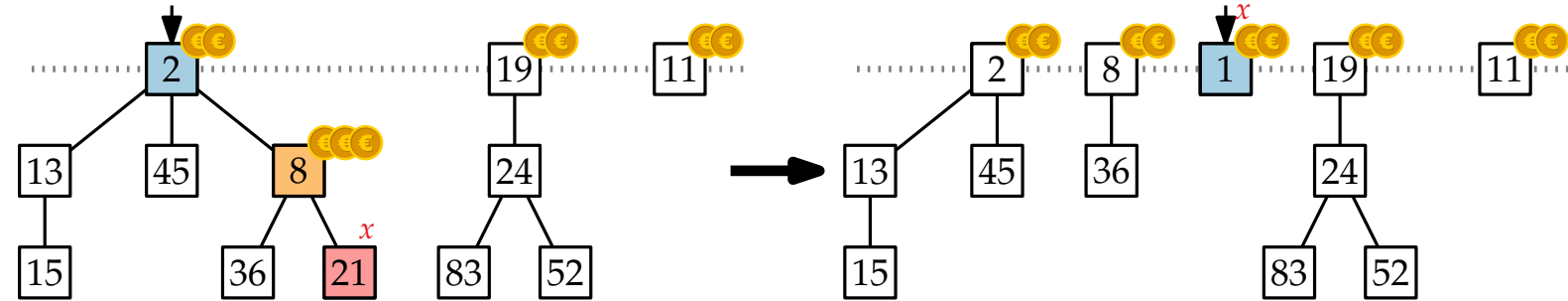
Tatsächliche Kosten: $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

m mal

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



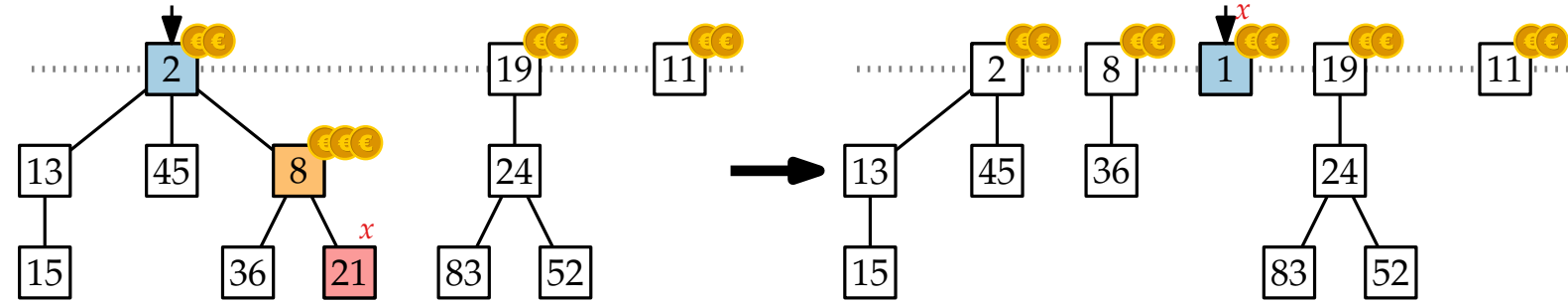
Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

m mal

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

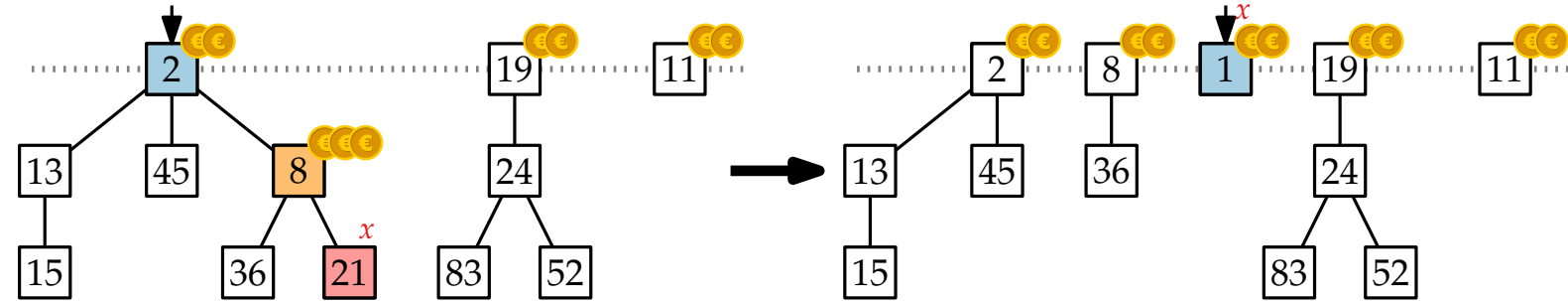
■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

Amortisierte Kosten:



$Q.\text{DECREASEKEY}(x, p)$

$x.\text{key} = p$

$y = x.\text{parent}$

$Q.\text{CUT}(x)$

m mal

while y ist **markiert** **do**

$z = y.\text{parent}; Q.\text{CUT}(y)$

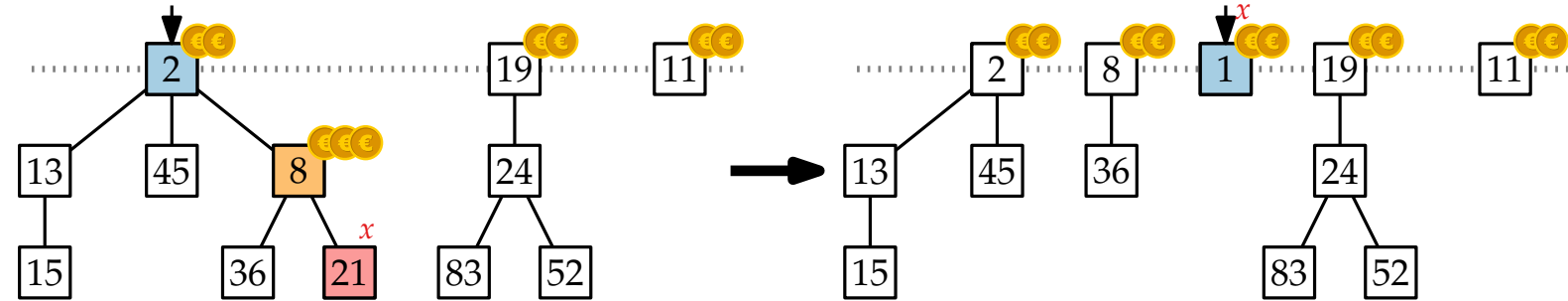
entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.\text{MIN}()$ **then** setze min Pointer auf x

- $\mathcal{O}(1)$
- $\mathcal{O}(m)$
- $\mathcal{O}(1)$
- $\mathcal{O}(1)$

- $\mathcal{O}(1)$



```

x.key = p
y = x.parent
Q.CUT(x)

```

m mal

while y ist markiert do

```
z = y.parent; Q.Cut(y)
entmarkiere y; y = z
```

if y nicht in Wurzelliste then **markiere** y

if $p < Q.\text{MIN}()$ then setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

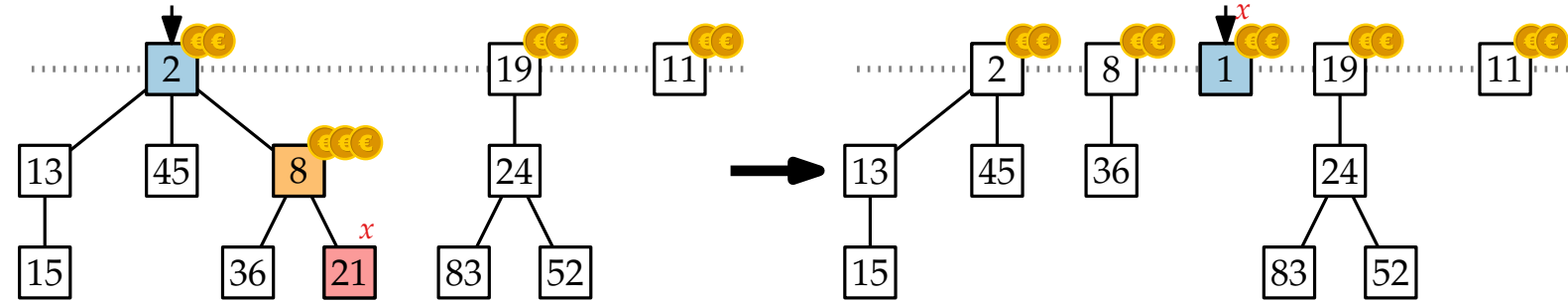
■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

Amortisierte Kosten:

■ $\mathcal{O}(1)$

■ Wir müssen €€ auf x legen



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

while y ist **markiert** **do**

$z = y.parent; Q.CUT(y)$

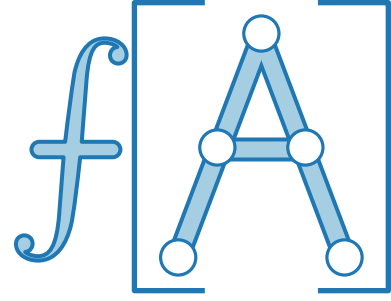
entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.MIN()$ **then** setze min Pointer auf x

m mal

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

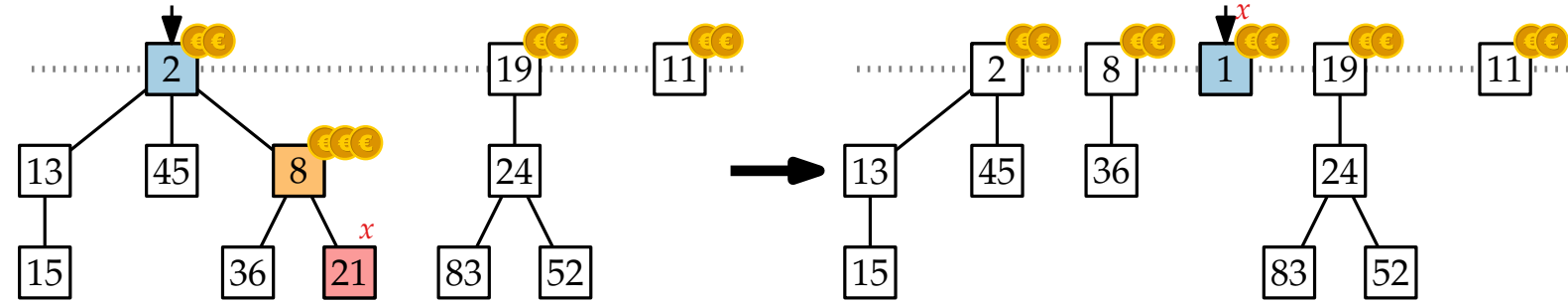
■ $\mathcal{O}(1)$

Amortisierte Kosten:

■ $\mathcal{O}(1)$

■ Wir müssen €€ auf x legen

→ $\mathcal{O}(1)$



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

while y ist **markiert** **do**

$z = y.parent; Q.CUT(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.MIN()$ **then** setze min Pointer auf x

m mal

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

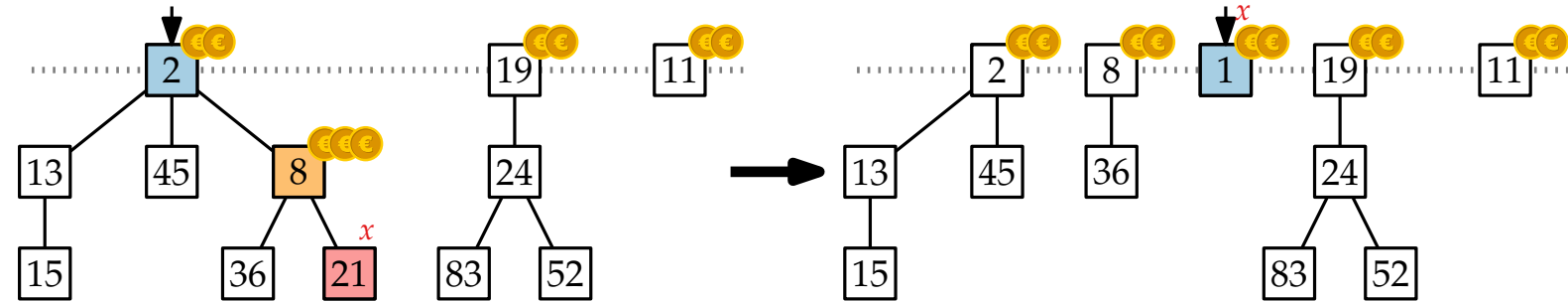
Amortisierte Kosten:

■ $\mathcal{O}(1)$

■ Wir müssen €€ auf x legen

→ $\mathcal{O}(1)$

■ Pro Schleifendurchlauf wandert ein markierter Knoten in die Wurzelliste



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

m mal

while y ist markiert **do**

$z = y.parent; Q.CUT(y)$

 entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** markiere y

if $p < Q.MIN()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

Amortisierte Kosten:

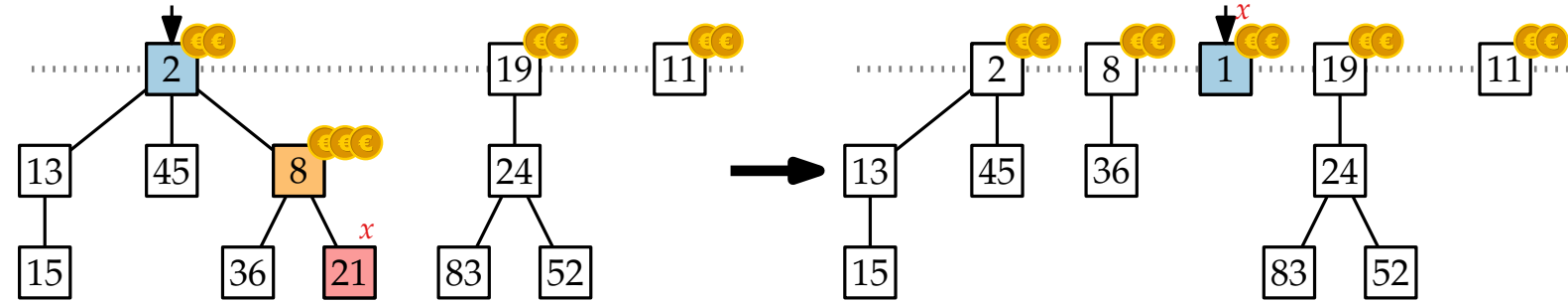
■ $\mathcal{O}(1)$

■ Wir müssen €€ auf x legen

→ $\mathcal{O}(1)$

■ Pro Schleifendurchlauf wandert ein markierter Knoten in die Wurzelliste

■ Der Knoten bezahlt dafür mit €



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

m mal

while y ist markiert **do**

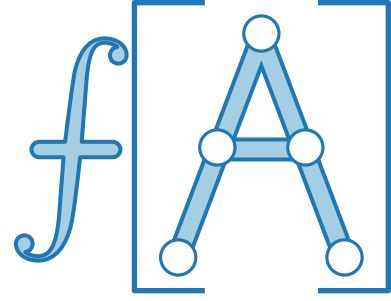
$z = y.parent; Q.CUT(y)$

 entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** markiere y

if $p < Q.MIN()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

Amortisierte Kosten:

■ $\mathcal{O}(1)$

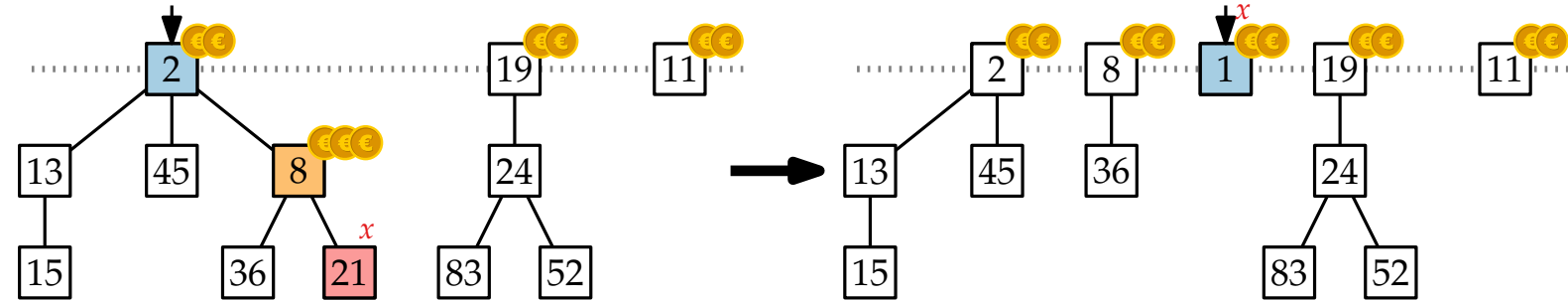
■ Wir müssen €€ auf x legen

→ $\mathcal{O}(1)$

■ Pro Schleifendurchlauf wandert ein markierter Knoten in die Wurzelliste

■ Der Knoten bezahlt dafür mit €

→ $\mathcal{O}(1)$



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

m mal

while y ist markiert **do**

$z = y.parent; Q.CUT(y)$

 entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** markiere y

if $p < Q.MIN()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

Amortisierte Kosten:

■ $\mathcal{O}(1)$

■ Wir müssen €€ auf x legen

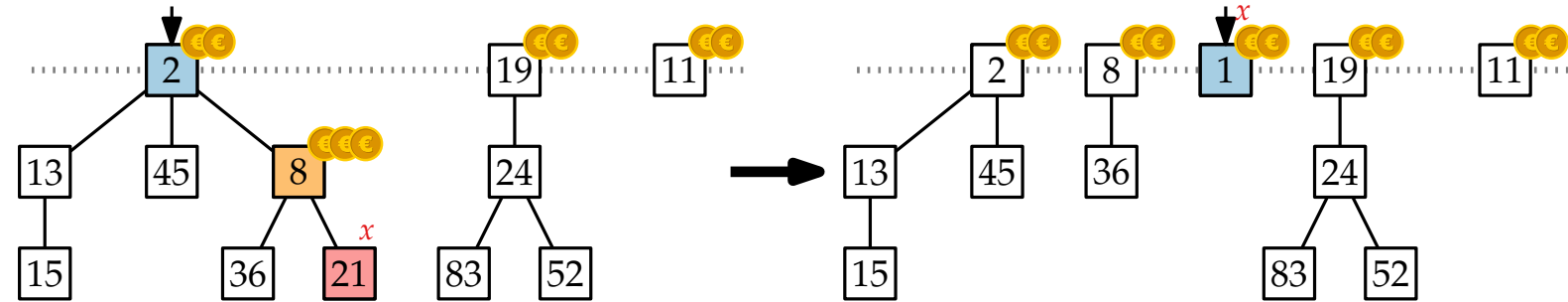
→ $\mathcal{O}(1)$

■ Pro Schleifendurchlauf wandert ein markierter Knoten in die Wurzelliste

■ Der Knoten bezahlt dafür mit €

→ $\mathcal{O}(1)$

■ Wir müssen €€€ auf y legen



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

m mal

while y ist markiert **do**

$z = y.parent; Q.CUT(y)$

 entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** markiere y

if $p < Q.MIN()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

Amortisierte Kosten:

■ $\mathcal{O}(1)$

■ Wir müssen €€ auf x legen

→ $\mathcal{O}(1)$

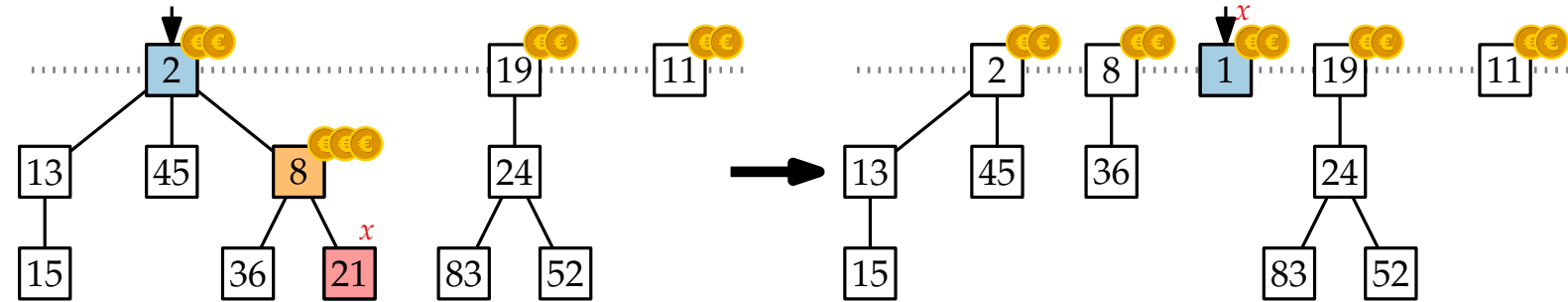
■ Pro Schleifendurchlauf wandert ein markierter Knoten in die Wurzelliste

■ Der Knoten bezahlt dafür mit €

→ $\mathcal{O}(1)$

■ Wir müssen €€€ auf y legen

→ $\mathcal{O}(1)$



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

m mal

while y ist **markiert** **do**

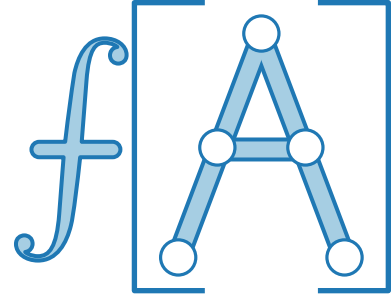
$z = y.parent; Q.CUT(y)$

entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** **markiere** y

if $p < Q.MIN()$ **then** setze min Pointer auf x

Amortisierte Analyse: DECREASEKEY



Tatsächliche Kosten: $\mathcal{O}(m) \in \mathcal{O}(\log n)$ (ohne Beweis)

■ $\mathcal{O}(1)$

■ $\mathcal{O}(m)$

■ $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

Amortisierte Kosten: $\mathcal{O}(1)$

■ $\mathcal{O}(1)$

■ Wir müssen €€ auf x legen

→ $\mathcal{O}(1)$

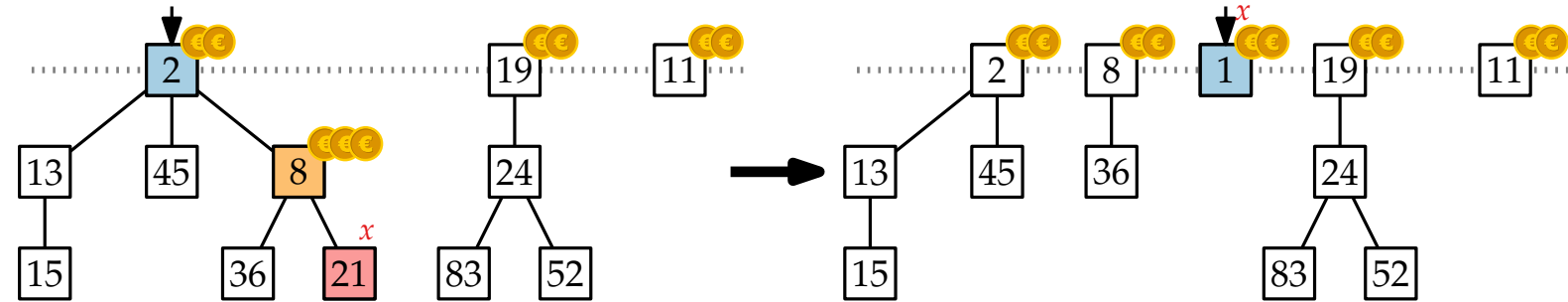
■ Pro Schleifendurchlauf wandert ein markierter Knoten in die Wurzelliste

■ Der Knoten bezahlt dafür mit €

→ $\mathcal{O}(1)$

■ Wir müssen €€€ auf y legen

→ $\mathcal{O}(1)$



$Q.DECREASEKEY(x, p)$

$x.key = p$

$y = x.parent$

$Q.CUT(x)$

m mal

while y ist markiert **do**

$z = y.parent; Q.CUT(y)$

 entmarkiere $y; y = z$

if y nicht in Wurzelliste **then** markiere y

if $p < Q.MIN()$ **then** setze min Pointer auf x

Zusammenfassung



Satz. Gegeben ein Graph $G = (V, E)$, läuft Dijkstras Alg. in $\Theta(n \cdot T_{\text{EXTRACTMIN}}(n) + m \cdot T_{\text{DECREASEKEY}}(n))$ Zeit.

* amortisiert

Operation	ARRAY	LISTE	HEAP	FIBONACCIHEAP
$Q.\text{INSERT}(x)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)^*$
$Q.\text{MIN}()$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
$Q.\text{EXTRACTMIN}()$	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)^*$
$Q.\text{DECREASEKEY}(x, p)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$	$\Theta(1)^*$
$Q.\text{UNION}(Q')$	$\Theta(Q')$	$\Theta(1)$	$\Theta(Q' \log n)$	$\Theta(1)^*$
DIJKSTRA	$\Theta(n^2 + m)$	$\Theta(n^2 + m)$	$\Theta((n + m) \log n)$	$\Theta(n \log n + m)$